

OrgElMorph

An Onboarding Course in Organic-Electronics Morphology Simulation
Physics · Computation · Differentiation

DiffSim project

Abstract

This course teaches the simulation of *morphology* in organic-electronic thin films — the coupled phase separation and crystallization of polymer/small-molecule/solvent blends that decides how organic solar cells, transistors, and bioelectronic devices perform. It is built around **OrgElMorph**, a GPU-native, differentiable phase-field simulator. Three tracks run in parallel: the *physics* of the models, the *computation* that makes them fast and correct, and the *differentiation* that turns the simulator into an instrument for design and inference. Every concept pairs a written explanation with a runnable tutorial; you reproduce the figures and numbers here, then explore the questions on your own.

Contents

1	Using this course	7
I	Physics	8
2	The model hierarchy: order parameters, gradient flows, and scales	9
2.1	Conserved vs non-conserved order parameters	9
2.2	Both dynamics are gradient flows of one free energy	10
2.3	Why Cahn–Hilliard is fourth order, and the mixed formulation	10
2.4	The model hierarchy	11
2.5	Symbol table	12
2.6	Four kinds of parameter — keep them apart	12
2.7	One nondimensionalization, worked through	13
2.8	Two crystallization conventions: p1 vs r14	14
2.9	Three statements about “the energy decreases”	14
2.10	Deliverable	14
2.11	Run it	14
2.12	Explore on your own	15
3	Binary Cahn–Hilliard: two free energies	18
3.1	The model	18
3.2	Two bulk free energies	19
3.3	Linear stability: the selected length scale	20
3.4	Run it	21
3.5	Coarsening: $L(t) \sim t^n$	22
3.6	Code walk-through	24
3.7	Explore on your own	25
4	Adaptive time stepping	26
4.1	The model and the integrator	26
4.2	Step 1 — verify the integrator on a smooth problem	27
4.3	Step 2 — a reference, and the stiffness cliff	28
4.4	Step 3 — the adaptive run, with real cost	28
4.5	Step 4 — convergence, the knee, and the honest speed-up	29
4.6	Run it	30
4.7	Code walk-through	31
4.8	Explore on your own	31

5	Substrate and boundary conditions	33
5.1	The model: two boundary conditions	33
5.2	Coefficients	34
5.3	Conservation: measured by quadrature	34
5.4	The full energy budget	35
5.5	Six boundary conditions	35
5.6	The enrichment boundary layer	36
5.7	Code walk-through	37
5.8	Explore on your own	38
6	Ternary Cahn–Hilliard and the phase diagram	40
6.1	The model, with the N_i factors	40
6.2	The spinodal: where the Hessian loses definiteness	41
6.3	The binodal: a common-tangent prediction	41
6.4	Reading the two phases: cluster, don’t median-split	42
6.5	Admissibility, conservation, and the lever rule	43
6.6	The N_i shift	43
6.7	Code walk-through	45
6.8	Explore on your own	45
7	Evaporation: the drying film	46
7.1	The model	46
7.2	Matched terminal state: rate versus dryness	47
7.3	Run it	47
7.4	Per-solute conservation and the height budget	48
7.5	The Biot regime map	49
7.6	Code walk-through	49
7.7	Explore on your own	50
8	Allen–Cahn crystallization	53
8.1	The model	53
8.2	Interface velocity and the critical radius	54
8.3	Growth kinetics: the Avrami law	54
8.4	Run it	55
8.5	Code walk-through	56
8.6	Explore on your own	57
9	Coupled Cahn–Hilliard + Allen–Cahn	58
9.1	The model: the four-fold χ (p1 vs r14)	58
9.2	Measuring the coupling: commensurate controls	59
9.3	Coupled energy budget	59
9.4	Causality	59
9.5	Run it	59
9.6	Code walk-through	60
9.7	Explore on your own	61

10 Thermal noise and nucleation	62
10.1 The model and the discrete FDT normalization	62
10.2 Nucleation as an ensemble	63
10.3 Run it	63
10.4 Code walk-through	64
10.5 Explore on your own	64
11 Evaporation-conditioned embryo growth	67
11.1 The solubility, derived from the free energy	68
11.2 The arc: wet dissolves, dry grows	68
11.3 Run it	69
11.4 Code walk-through	70
11.5 Explore on your own	70
12 Material-system reproduction (capstone)	72
12.1 The system and its provenance	72
12.2 The reproduced trend	73
12.3 Mesh and time convergence	74
12.4 Parameter-uncertainty sensitivity	75
12.5 The robustness verdict	75
12.6 Explore on your own	76
13 Model-extension (capstone)	79
13.1 The mechanism	79
13.2 The analytic Jacobian	80
13.3 Verification	81
13.4 GPU profile	81
13.5 The scientific result	82
13.6 The contributor checklist	82
13.7 Explore on your own	83
II Computation	85
14 From weak form to CUDA: one scalar problem, every stage	86
14.1 The model	86
14.2 The pipeline, stage by stage	87
14.3 Hands-on: add a coefficient and its analytic Jacobian	88
14.4 Run it: verify the whole pipeline	88
14.5 Code walk-through	89
14.6 Explore on your own	90
15 Convergence: basis order and time integration	93
15.1 Three errors, not one	93
15.2 Spatial order: a steady manufactured solution	94
15.3 Algebraic error: control it, or the plot is worthless	95
15.4 Temporal order: a <i>verified</i> self-convergence reference	95
15.5 Ways to get a wrong answer	96
15.6 Run it	96

15.7	Code walk-through	98
15.8	Explore on your own	98
16	Boundary conditions, numerically	99
16.1	Two boundary conditions live in the weak form	100
16.2	The taxonomy	100
16.3	How a constraint is implemented	101
16.4	The flux balance ties conservation to the wall	101
16.5	Run it	102
16.6	Code walk-through	103
16.7	Explore on your own	104
17	Octree refinement, dynamic AMR, and temporal adaptivity	105
17.1	Octree refinement and hanging-node constraints	106
17.2	What dynamic AMR additionally requires — and the conservative transfer that makes it work	106
17.3	Dynamic AMR: the full solution-adaptive cycle	107
17.4	Does dynamic AMR pay? Error versus dofs	108
17.5	Space-time interaction: a remesh during a BDF2 march	108
17.6	Temporal adaptivity: the LTE step ladder	109
17.7	Why variable-coefficient BDF2	110
17.8	Run it	110
17.9	Code walk-through	111
17.10	Explore on your own	112
18	The solver ecosystem	114
18.1	The real test: march it and look at the solution	114
18.2	Why: an indefinite saddle with no pivoting	115
18.3	The 2-D timing (context only)	116
18.4	The 3-D story, cited	116
18.5	Decision support, with a rationale	116
18.6	Code walk-through	117
18.7	Explore on your own	117
19	Two dimensions versus three	119
19.1	Where the sparsity really comes from	119
19.2	Accounting the memory completely	120
19.3	The device-scale ladder	121
19.4	Physics at equal resolution: 2-D is not cheap 3-D	122
19.5	Code walk-through	123
19.6	Explore on your own	123
20	Nonlinear solver diagnostics	125
20.1	Anatomy of a healthy Newton solve	125
20.2	When have you converged? Three stopping tests	126
20.3	Safeguards: what keeps a hard step alive	126
20.4	Four ways a nonlinear solve fails	127
20.5	Never drop a failed step: checkpoints	129

20.6	Code walk-through	130
20.7	Explore on your own	130
21	CUDA profiling and memory	131
21.1	Time one step — and synchronize, or the timer lies	131
21.2	Data movement rules: bandwidth vs arithmetic intensity	132
21.3	Factor vs solve: the case for reuse	132
21.4	Device memory: measure it, or account for it	133
21.5	Nsight Systems: capture one step	133
21.6	Explore on your own	134
22	Extending DiffSim safely	136
22.1	Step 1 — the term, its equations, and its units	136
22.2	Step 2 — a config knob	137
22.3	Steps 3–4 — residual and <i>analytic</i> Jacobian	137
22.4	Steps 5–6 — verify the derivative, and the limiting case	137
22.5	Step 7 — convergence still holds	138
22.6	Step 8 — conservation and energy still hold	138
22.7	Step 9 — profile the cost	138
22.8	Step 10 — documentation and changelog	139
22.9	Code walk-through	139
22.10	Explore on your own	139
23	Reproducible campaigns	142
23.1	The unit of work is the campaign	142
23.2	The model, and why we sweep χ	143
23.3	Sweep, ensemble, and the two intents	143
23.4	Checkpoint, restart, and failed-run recovery	143
23.5	The no-silent-drop rule	143
23.6	At cluster scale: a SLURM job array	144
23.7	Run it	145
23.8	Code walk-through	146
23.9	Explore on your own	146
24	Validation and uncertainty	147
24.1	Six questions, not one	147
24.2	One material case, one observable	148
24.3	Verification: code, then solution	148
24.4	Model validation (3)	148
24.5	Uncertainty: parametric and stochastic	148
24.6	The uncertainty budget	149
24.7	Run it	149
24.8	Code walk-through	150
24.9	Explore on your own	151

III	Differentiation	152
25	What differentiable simulation is	153
25.1	Why a gradient	153
25.2	The three ways to a gradient	153
25.3	Run it	154
25.4	Code walk-through	154
25.5	Explore on your own	155
26	Morphology sensitivity	157
26.1	A scalar handle on the morphology	157
26.2	Run it	157
26.3	Code walk-through	158
26.4	Explore on your own	158
27	Recovering parameters from a morphology	160
27.1	The inverse problem	160
27.2	Run it	160
27.3	Code walk-through	161
27.4	Explore on your own	162
28	Process gradients	163
28.1	Temperature enters everywhere	163
28.2	Run it	163
28.3	Code walk-through	164
28.4	Explore on your own	165
29	Crystallinity sensitivity	166
29.1	Differentiating the crystalline order	166
29.2	Run it	166
29.3	A frontier, named honestly	167
29.4	Explore on your own	167
30	Learning the free energy from snapshots	169
30.1	A learnable free energy	169
30.2	Run it	169
30.3	Code walk-through	170
30.4	Explore on your own	171
31	Capstone: inverse-designing a process	172
31.1	Design as optimisation	172
31.2	Run it	172
31.3	Code walk-through	173
31.4	Explore on your own	174

Chapter 1

Using this course

Read a concept below, then open its tutorial folder (`tutorials/orgelmorph-course/<track>/<NN_name>/`), run `run.py`, and compare your printed self-check table and figures with the ones in this document. The tutorials call the same production solver the research code uses. The three tracks cross-reference each other; a first pass straight down the Physics track is a good start, dipping into the matching Computational and Differentiable concepts as they are referenced.

Boxes flag the two things you act on:

Key idea. The load-bearing concept.

Run it. The command, and what to compare.

Questions marked **Q** are for you to explore — they have no answers printed here on purpose.

Part I

Physics

Chapter 2

The model hierarchy: order parameters, gradient flows, and scales

Key idea. Every model in this course is a *gradient flow of a free-energy functional*: the film evolves so as to decrease a single free energy F , and the *only* question that distinguishes one model from the next is which fields are conserved, what free energy they minimize, and how fast. Two dynamics cover everything — *conserved* (Cahn–Hilliard, for composition) and *non-conserved* (Allen–Cahn, for crystallinity) — and the eight concepts of the Physics track are the same two dynamics with ingredients added one at a time. This chapter fixes the vocabulary, the symbols, and the *scales*: it derives the two dynamics from F , works one nondimensionalization through in full, and separates the four kinds of parameter (dimensional material data, nondimensional groups, numerical knobs, and deliberately *accelerated* tutorial settings) that the rest of the course keeps apart.

2.1 Conserved vs non-conserved order parameters

An *order parameter* is a coarse-grained field that labels the local state of the film. This course uses two kinds, and the distinction governs the dynamics:

- A **conserved** order parameter is a *composition*: a local volume fraction ϕ (or $\phi_1, \phi_2, \dots$ for several species). Matter is neither created nor destroyed, so the spatial integral $\int_{\Omega} \phi dV$ is fixed — composition can only be *redistributed*. Its dynamics must be a divergence of a flux (a local conservation law).
- A **non-conserved** order parameter is a *degree of internal order*: the local crystallinity $\psi \in [0, 1]$. Crystal can be *created* (an amorphous region orders) or destroyed (it melts), so $\int_{\Omega} \psi dV$ is *not* fixed. Its dynamics is a direct relaxation, with no conservation constraint.

The test. Ask “can the spatial average change on its own?” For composition, no (it takes a flux through the boundary) — *conserved*, Cahn–Hilliard. For crystallinity, yes (it grows in place) — *non-conserved*, Allen–Cahn. The moving-frame evaporation concepts (7, 11) are the one subtlety: the *solvent* leaves through the top surface, so a species’ amount is conserved only in the *moving* film frame, and the bookkeeping is a per-species moving-domain balance, not a fixed integral.

2.2 Both dynamics are gradient flows of one free energy

The organizing principle is a single Ginzburg–Landau free energy. For a composition field it is

$$F[\phi] = \int_{\Omega} \left[f(\phi) + \frac{\kappa}{2} |\nabla \phi|^2 \right] dV, \quad (2.1)$$

a *bulk* (mixing) energy density f that is minimized by the pure phases, plus a *gradient* penalty $\frac{\kappa}{2} |\nabla \phi|^2$ that charges for spatial variation and so sets a finite interface width. The *chemical potential* is the variational derivative $\mu = \delta F / \delta \phi = f'(\phi) - \kappa \nabla^2 \phi$.

Conserved dynamics (Model B, Cahn–Hilliard). Because ϕ is conserved, it can only move down a flux; the natural, thermodynamically consistent choice is a flux proportional to $-\nabla \mu$, so that

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (M \nabla \mu), \quad \mu = f'(\phi) - \kappa \nabla^2 \phi, \quad (2.2)$$

with mobility $M > 0$. This is the **Cahn–Hilliard** equation. It guarantees two things at once: mass conservation (the right-hand side is a divergence, so $\frac{\partial}{\partial t} \int_{\Omega} \phi dV = 0$ under no-flux boundaries) and energy decrease.

Non-conserved dynamics (Model A, Allen–Cahn). Crystallinity has no conservation constraint, so it relaxes *directly* down the energy gradient,

$$\frac{\partial \psi}{\partial t} = -L_{\psi} \frac{\delta F}{\delta \psi} = -L_{\psi} \left[\frac{\partial f_{\text{cr}}}{\partial \psi} - \varepsilon^2 \nabla^2 \psi \right], \quad (2.3)$$

with kinetic coefficient $L_{\psi} > 0$ and crystalline gradient penalty ε^2 . This is the **Allen–Cahn** equation.

Why both decrease F — the gradient-flow guarantee. For either dynamics, the chain rule gives $\frac{\partial F}{\partial t} = \int_{\Omega} \frac{\delta F}{\delta u} \frac{\partial u}{\partial t} dV$. Substituting the two flows:

- Cahn–Hilliard: $\frac{\partial F}{\partial t} = - \int_{\Omega} M |\nabla \mu|^2 dV \leq 0$;
- Allen–Cahn: $\frac{\partial F}{\partial t} = - \int_{\Omega} L_{\psi} \left(\frac{\delta F}{\delta \psi} \right)^2 dV \leq 0$.

F is a *Lyapunov functional* — it can only decrease — for both. The difference is purely the constraint: the conserved flow moves mass *between* regions to lower F ; the non-conserved flow changes the field *in place*. This single fact — “the film descends one free energy” — is the thread through every concept.

The kinetic reading. M and L_{ψ} set only *how fast* the descent proceeds, not *where* it ends: the equilibrium morphology is a property of F alone (the wells of f , the penalties κ, ε^2), while M and L_{ψ} set the time scale. For crystallization the driving force also carries a *sign*: below the melting point the crystal well is lower and ψ grows; above it, the amorphous well is lower and ψ melts (Chapter 8).

2.3 Why Cahn–Hilliard is fourth order, and the mixed formulation

Substituting $\mu = f'(\phi) - \kappa \nabla^2 \phi$ into (2.2) gives a single equation

$$\frac{\partial \phi}{\partial t} = \nabla \cdot \left(M \nabla [f'(\phi) - \kappa \nabla^2 \phi] \right), \quad (2.4)$$

whose highest term is $-M\kappa \nabla^4 \phi$: Cahn–Hilliard is a **fourth-order** PDE (Allen–Cahn, by contrast, is second order). A standard C^0 finite-element basis — continuous, but with discontinuous gradients across element faces — cannot represent a fourth-order operator directly (the weak form would need integrable second derivatives, i.e. a C^1 basis, which is expensive in 2-D/3-D).

The mixed formulation. Rather than a C^1 basis, split the fourth-order equation into *two coupled second-order* equations by keeping μ as an independent unknown:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (M \nabla \mu), \quad \mu = f'(\phi) - \kappa \nabla^2 \phi. \quad (2.5)$$

Now each equation has at most a Laplacian, so a C^0 basis suffices — at the cost of carrying *two* fields per node, (ϕ, μ) , in a node-major 2-dof block. This is the “mixed (ϕ, μ) form” the solver uses everywhere; Chapter 14 traces its element residual and the four 2×2 Jacobian blocks to code, and Chapter 15 verifies it converges at the right order in *both* fields.

2.4 The model hierarchy

Figure 2.1 is the map of the whole Physics track: eight concepts, each adding *one* physical ingredient to the same gradient-flow skeleton. Read top to bottom, it is the arc from the simplest demixing to a full evaporation-conditioned crystallizing blend.

- **Binary CH (3)**: two components demix; the base case. $\rightarrow$ + **substrate BCs (5)**: wetting and no-flux boundaries.
- **Ternary CH (6)**: solvent + two solutes on the Gibbs simplex. $\rightarrow$ + **moving-frame evaporation (7)**: the film shrinks as solvent leaves.
- **Allen–Cahn crystallization (8)**: a non-conserved crystallinity field. $\rightarrow$ **coupled CH+AC (9)**: crystallization drives demixing.
- + **stochastic nucleation (10)**: thermal noise seeds nuclei. $\rightarrow$ **evaporation-conditioned (11)**: drying decides where embryos grow.

2.5 Symbol table

symbol	meaning	type / units
ϕ, ϕ_i	composition (volume fraction)	conserved, dimensionless
μ	chemical potential $\delta F/\delta\phi$	energy/volume
ψ	crystallinity	non-conserved, $\in [0, 1]$
$h(t)$	film height (moving frame)	length
F	Ginzburg–Landau free energy	energy
$f(\phi)$	bulk (mixing) energy density	energy/volume
$f_{\text{cr}}(\psi)$	crystallization energy density	energy/volume
κ	composition gradient penalty	energy/length
ε^2	crystalline gradient penalty	energy-length
M	Cahn–Hilliard mobility	sets composition time scale
L_ψ	Allen–Cahn kinetic coefficient	sets crystal time scale
W	bulk-energy barrier height (well depth)	energy/volume
ℓ	equilibrium interface width $\sim \sqrt{\kappa/W}$	length
χ	Flory interaction parameter (enthalpic)	dimensionless
N, N_i	degree of polymerization (entropic $\sim 1/N$)	dimensionless
A, B	FH entropy ($\sim 1/N$) / enthalpy ($\sim \chi$) scales	dimensionless
Δh	crystallization enthalpy (latent heat)	dimensionless
$\Delta\sigma$	amorphous–crystal barrier	dimensionless
T_m	melting temperature	K
Bi	Biot number $k_e h_0/D_s$ (evaporation rate)	dimensionless
Pe	Péclet number (advection/diffusion)	dimensionless
$\tilde{\kappa}$	Cahn number $(\ell/L)^2 = \kappa/(e_0 L^2)$	dimensionless
σ	BDF time coefficient $b_0/\Delta t$	1/time
L	domain size (length scale)	length
$e_0 = kT/v_0$	bulk free-energy scale (v_0 monomer volume)	energy/volume

2.6 Four kinds of parameter — keep them apart

A recurring source of confusion is mixing four *different* kinds of number. The course keeps them in separate tables, and so should you.

(1) Dimensional material parameters. Physical properties of a real blend, with units and a literature source. These live in `materials/materials.yaml` with provenance.

parameter	example (system)	value	source
χ (polymer:fullerene)	P3HT:PCBM	0.86	Wodo–Gan. CMS 2012
N (polymer)	PDPP5T:PCBM	87	Negi et al.
T_m (melting)	PCBM class	558 K	2310.11844 Table 1
Δh (latent heat)	PCBM class	1.31	2310.11844
Bi (evaporation)	PDPP5T @ 3000 rpm	0.0051	Negi SI

(2) Nondimensional groups. The dimensionless combinations that actually govern the solution — the numbers a nondimensionalization produces. Binary CH has *one*: the Cahn number $\tilde{\kappa} = (\ell/L)^2$. Evaporation adds the Biot and Péclet numbers; crystallization adds the reduced undercooling $T/T_m - 1$ and the barrier and enthalpy ratios.

group	meaning	tutorial value
$\tilde{\kappa}$ (Cahn)	$(\ell/L)^2$; interface-to-domain ratio squared	5.0×10^{-4}
A, B (FH)	entropy $\sim 1/N$, enthalpy $\sim \chi$	1.0, 2.5
$T/T_m - 1$	reduced undercooling (crystal drive sign)	< 0 (grow)
Bi, Pe	evaporation rate, advection/diffusion	Ch. 7

(3) Numerical parameters. Knobs of the discretization, chosen to *resolve* the physics, not part of it: the mesh level (cells per side = 2^{level}), the time step Δt , the time scheme (BDF1/BDF2), and the Newton/linear tolerances.

parameter	role	tutorial value
level	mesh resolution (2^{level} cells/side)	6
Δt	time step	0.02 (P1)
scheme	BDF1 / BDF2	BDF1 (Ch. 15)
Newton / linear tol	algebraic error control	$10^{-8}/10^{-10}$

(4) Accelerated-tutorial parameters. Deliberately *non-physical* settings that make a concept runnable in minutes on a small box, traded against fidelity — and always *labelled* as such. Examples: a deeper quench (B raised) to force fast demixing; a large undercooling (running crystallization at $T = 250$ K far below $T_m = 558$ K, a $5.3\times$ stronger drive than a physical near- T_m anneal) to grow crystals in a short horizon; an inflated L_ψ to speed kinetics. These buy a fast demonstration but must never be quoted as material predictions.

2.7 One nondimensionalization, worked through

Take binary Cahn–Hilliard (2.2) in dimensional form, with $f \sim e_0 = kT/v_0$ (energy/volume), $\kappa = e_0\lambda^2$ (so $\kappa|\nabla\phi|^2$ is an energy density and λ is a microscopic length), and mobility M . Choose three scales: length L (the domain), energy e_0 , and the *diffusive time* $\tau = L^2/(Me_0)$. Writing $\tilde{x} = x/L$, $\tilde{t} = t/\tau$, $\tilde{\mu} = \mu/e_0$, $\tilde{f} = f/e_0$ collapses (2.2) to

$$\frac{\partial \phi}{\partial \tilde{t}} = \tilde{\nabla} \cdot (\tilde{\nabla} \tilde{\mu}), \quad \tilde{\mu} = \tilde{f}'(\phi) - \tilde{\kappa} \tilde{\nabla}^2 \phi, \quad \boxed{\tilde{\kappa} = \frac{\kappa}{e_0 L^2} = \left(\frac{\lambda}{L}\right)^2}. \quad (2.6)$$

Every dimensional constant has been absorbed: the mobility became 1 (it set the time unit τ), and the *only* surviving parameter is the **Cahn number** $\tilde{\kappa}$ — the square of the interface-to-domain length ratio. This is why the tutorials specify a single κ and no separate M or e_0 : at fixed $\tilde{\kappa}$ the morphology is the same blend whatever the physical L and M .

The nondimensional groups fix the resolution — and reproduce the P1 numbers.

With $\tilde{\kappa} = 5.0 \times 10^{-4}$ (taking an interface $\lambda = 11.2$ nm in a domain $L = 500$ nm, so $\lambda/L = 0.0224$), the equilibrium interface widths in nondimensional units are $\tilde{\ell} = \sqrt{2\tilde{\kappa}} = 0.0316$ (polynomial well) and $\tilde{\ell} = \sqrt{\tilde{\kappa}/|f''|}$ (FH). At the P1 mesh level 6 these span 2.02 and 1.43 mesh cells respectively — *exactly* the interface-cell counts the P1 tutorial *measures* (3). The resolution constraint follows directly: to keep ≥ 3 cells across the interface you need at least level 7 (Fig. 2.2). The nondimensional group, the interface physics, and the mesh you must build are one calculation.

2.8 Two crystallization conventions: p1 vs r14

When crystallinity couples to composition (Chapter 9), the Flory interaction χ splits into *four* matrices $\chi_{aa}, \chi_{ac}, \chi_{ca}, \chi_{cc}$ (amorphous/crystalline pairs), and **the two bulk free energies in the code combine them differently** — a convention clash you must know before reading P6/P7. The **p1** bulk uses a *mutually-weighted* sum, so its four χ are *absolute* pair interactions; the **r14** bulk (the 2310.11844 form used by the Allen–Cahn energetics) uses ψ^2 -weighted *increments* on top of χ_{aa} , so the same input means an *increment* $\chi_{aa} + \chi_{ca}$ at the pure limit, not an absolute value. The two also use different interpolation functions for the crystalline state. Chapter 9 gives the exact formulas and `test_chi_limits.py` pins all four pure limits for both conventions against the production evaluator (the P7 finding: quote whether a crystalline χ is absolute or an increment, or the numbers are meaningless).

2.9 Three statements about “the energy decreases”

The gradient-flow guarantee ($\frac{\partial F}{\partial t} \leq 0$) is a statement about the *continuous* model. It does *not* automatically survive discretization, and the course is careful to keep three different claims apart — a distinction you will meet quantitatively in every energy plot:

1. **Thermodynamically motivated (continuous Lyapunov).** In the PDE, $\frac{\partial F}{\partial t} = - \int_{\Omega} M |\nabla \mu|^2 dV \leq 0$ *exactly*. A theorem about the model.
2. **Discretely energy-stable.** Whether the *time scheme* inherits $F^{n+1} \leq F^n$ every step. A property of the discretization, not automatic — it must be *measured* (report the largest positive stepwise increment $\max_n (F^{n+1} - F^n)$).
3. **One monotone run.** The particular trajectory you plotted went downhill. Necessary but not sufficient — report the increment, do not just assert “it looked monotone.”

Chapter 3 makes all three concrete on a real run; the point of naming them here is that “the energy decreased” is three different claims, and only the first is free.

2.10 Deliverable

Your deliverable for this chapter (template in `physics/00_model_hierarchy/deliverable.md`):

1. A **one-page model map**: redraw Figure 2.1 from memory, labelling for each concept (i) the order parameter(s) and whether each is conserved or non-conserved, (ii) the one ingredient it adds, and (iii) the free-energy term that ingredient introduces.
2. A **completed parameter table**: take one real system from `materials/materials.yaml` and fill in all four columns — dimensional value + source, the nondimensional group it maps to, the numerical resolution it demands (minimum mesh level for ≥ 3 interface cells), and any accelerated setting you would use for a quick run, *labelled as accelerated*.

Run `run.py` to reproduce the nondimensionalization numbers and check your table against `outputs/p0/results.json`.

2.11 Run it

Run it.

```
cd physics/00_model_hierarchy
python run.py --config configs/p0.yaml --device cpu \
    --mode reference --output outputs/p0 --overwrite
python gen_figures.py --run-dir outputs/p0
```

The harness computes the worked nondimensionalization (the Cahn number, the interface widths in cells, the minimum mesh level, and the accelerated-vs- physical crystallization drive) from the dimensional inputs in `configs/p0.yaml`, checks them against `baseline.yaml` (the interface-cell counts must match the P1 tutorial), and records provenance in `metadata.json`. `gen_figures.py` renders the hierarchy map and the nondimensionalization figure and writes `numbers/p0.tex`.

2.12 Explore on your own

Q2.12.1 Take a second system from `materials.yaml` (say P3HT:PCBM at $\chi = 0.86$) and compute its FH curvature $f''(\bar{\phi})$ at the symmetric point. Is it spinodally unstable? What mesh level resolves its interface to ≥ 3 cells?

Q2.12.2 Halve the domain L at fixed interface λ . The Cahn number $\tilde{\kappa}$ quadruples — what happens to the interface-in-cells at fixed mesh level, and to the minimum level you need? This is why a bigger physical box is *cheaper* to resolve, not more expensive.

Q2.12.3 For the crystallization drive $\Delta h(T/T_m - 1)$, at what temperature does the accelerated tutorial ($T = 250$ K) overstate the physical ($T = 500$ K) drive, and by how much? When is such an acceleration defensible, and when does it change the physics rather than just the timing?

Q2.12.4 Classify each field in the moving-frame evaporation model (7) as conserved or non-conserved, and explain in which frame each species' amount is actually conserved.

Q2.12.5 Write down which free-energy term is *added* at each step of Figure 2.1. Which additions change the dynamics type (conserved $\leftrightarrow$ non-conserved), and which only add a term to the same F ?

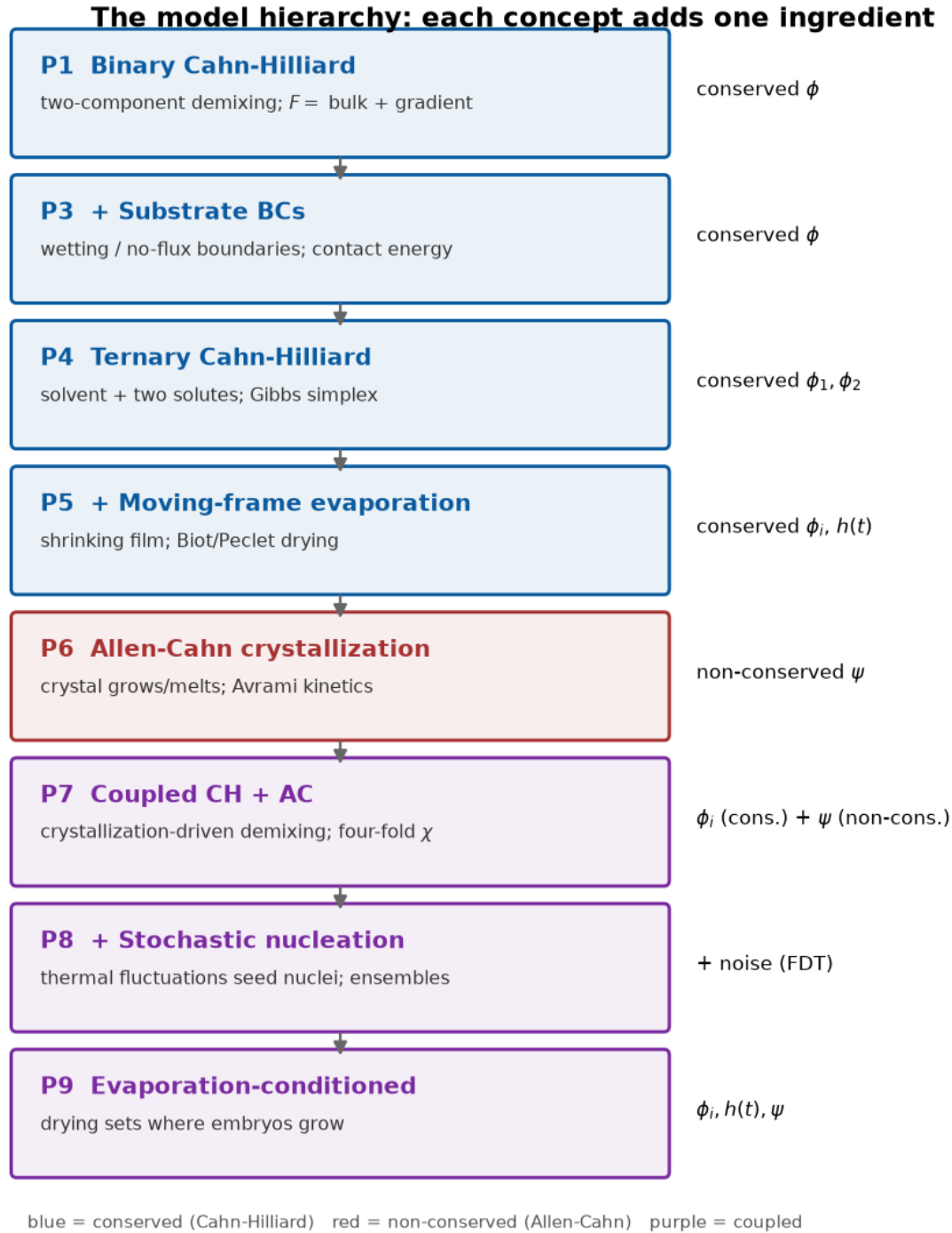


Figure 2.1: The model hierarchy. Blue = conserved (Cahn–Hilliard) fields; red = non-conserved (Allen–Cahn); purple = coupled. Each concept adds one ingredient: substrate boundaries, a third component, a moving (drying) frame, crystallinity, the coupling between composition and crystallinity, stochastic nucleation, and finally the evaporation-conditioned growth that ties them together.

The one worked nondimensionalization (binary Cahn-Hilliard)

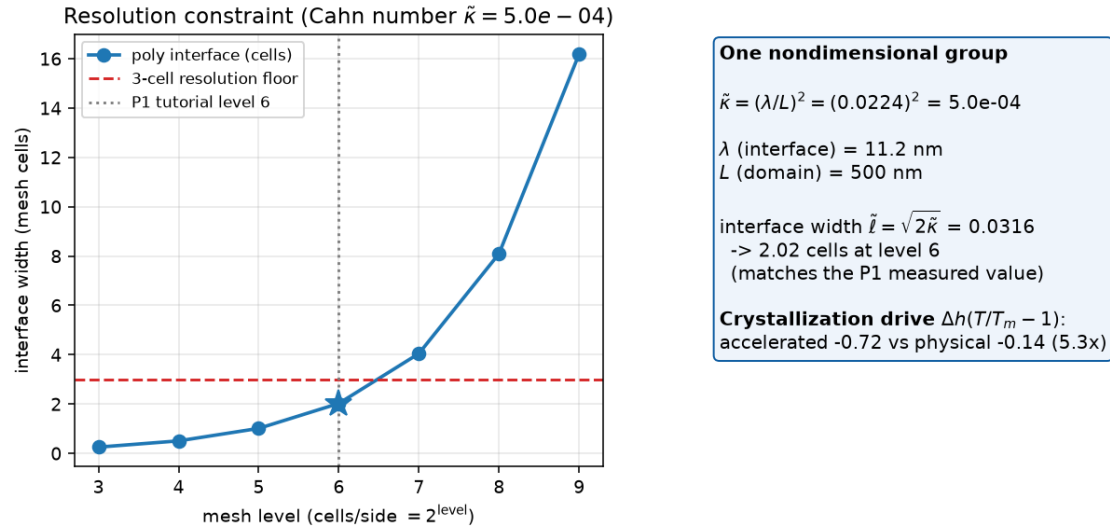


Figure 2.2: The one worked nondimensionalization. *Left*: the interface width in mesh cells grows as 2^{level} ; the P1 tutorial level 6 (star) sits just below the 3-cell resolution floor, so the interface is marginally resolved — a deliberate, documented tutorial trade-off. *Right*: the single nondimensional group $\tilde{\kappa} = (\lambda/L)^2$ and the crystallization drive, computed by the harness and reproducing the P1 measured interface cells.

Chapter 3

Binary Cahn–Hilliard: two free energies

Key idea. A blend of two components lowers its free energy by *demixing*. The Cahn–Hilliard equation is the conserved gradient flow of a Ginzburg–Landau free energy; that free energy can only decrease in time, and it splits into a *bulk* (mixing) part and an *interfacial* (gradient) part whose competition sets the morphology. Two quantitative predictions fall out of the same free energy and we *measure* both: the interface width and the fastest-growing wavelength that the spinodal instability selects.

Learning objectives. By the end you can: derive the conserved Cahn–Hilliard flow from F and split it into bulk and interfacial parts; explain why $\ell \sim \sqrt{\kappa/W}$ (not $\sqrt{\kappa}$) and count the cells it spans; predict $\lambda^* = 2\pi/k^*$ and *verify* it against $S(q)$; keep apart the three “energy decreases” statements and measure the last two; and report the cost of the Flory–Huggins projection.

Prerequisites. Calculus of variations ($\delta F/\delta c$), Fourier modes, thermodynamics of mixing; Chapter 00 (environment green). *First physics chapter*.

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory (an 8 GB card is ample); solver `splu` (the CH saddle is indefinite). Quick mode ≈ 75 s wall (measured, RTX 6000 Ada; mostly kernel compile); reference a few minutes; research (5-seed ensemble) tens of minutes.

3.1 The model

Let $c(\mathbf{x}, t)$ be the local volume fraction of one component (a *conserved* order parameter: the total amount of each species is fixed). The Ginzburg–Landau free energy is

$$F[c] = \int_{\Omega} \left[f(c) + \frac{\kappa}{2} |\nabla c|^2 \right] dV, \quad (3.1)$$

with two contributions:

- the **bulk** (homogeneous) free energy density $f(c)$, which is minimized by the pure phases and *penalized* in between — this is what drives demixing. Write its scale as W , the height of the barrier between the two wells (units of energy/volume);

- the **interfacial** term $\frac{\kappa}{2}|\nabla c|^2$, which penalizes spatial variation of c and therefore *opposes* demixing at short scales.

The interface width is $\ell \sim \sqrt{\kappa/W}$, not $\sqrt{\kappa}$. Balancing the gradient penalty against the bulk barrier in (3.1), the equilibrium interface between the two phases is a tanh profile of width

$$\ell \sim \sqrt{\frac{\kappa}{W}}. \quad (3.2)$$

Only the *ratio* κ/W is a length²: κ alone has units of energy $\times$ length²/volume and cannot by itself set a width. A deeper quench (larger W) sharpens the interface; a larger κ broadens it. For the symmetric polynomial well below ($W = 1$ here) the exact profile gives $\ell = \sqrt{2\kappa/W}$, which at the tutorial's $\kappa = 5 \times 10^{-4}$ is 0.0316 (about 2.0 mesh cells — comfortably resolved). We report the width in physical length *and* in cells so you can judge resolution directly.

Because c is conserved, the dynamics is the *conserved* (Model-B) gradient flow of (3.1): mass moves down gradients of the chemical potential $\mu = \delta F / \delta c$,

$$\frac{\partial c}{\partial t} = \nabla \cdot (M \nabla \mu), \quad \mu = f'(c) - \kappa \nabla^2 c. \quad (3.3)$$

Here $M > 0$ is the mobility (it sets the *time* scale but not the equilibrium morphology). The fourth-order PDE is split into the two second-order equations (3.3) — the *mixed* (c, μ) form the solver uses.

3.2 Two bulk free energies

Polynomial double well (the canonical model). The simplest f with two equal minima,

$$f_{\text{poly}}(c) = \frac{1}{4}(c^2 - 1)^2, \quad f' = c^3 - c, \quad f'' = 3c^2 - 1, \quad c \in [-1, 1]. \quad (3.4)$$

The two phases are $c = \pm 1$; the barrier height is $W = \frac{1}{4}$ (and the curvature scale that enters (3.2) is order one, $|f''(0)| = 1$). The region $f'' < 0$ (i.e. $|c| < 1/\sqrt{3}$) is *spinodally unstable*. This model is dimensionless and symmetric; it isolates the phase-field mechanism from material detail.

Flory–Huggins (the physical model). For a real blend the mixing free energy is entropic + enthalpic,

$$f_{\text{fh}}(c) = A [c \ln c + (1 - c) \ln(1 - c)] + B c(1 - c), \quad c \in (0, 1), \quad (3.5)$$

where the logarithmic *entropy of mixing* (scale $A \sim 1/N$, with N the degree of polymerization) always favors mixing, and the *enthalpic* term $B c(1 - c)$ (with B the Flory interaction parameter χ) favors demixing. The curvature

$$f''_{\text{fh}}(c) = A \left(\frac{1}{c} + \frac{1}{1-c} \right) - 2B \quad (3.6)$$

is negative — unstable — only where the enthalpic $-2B$ beats the entropic term. This is the *spinodal*. The logs diverge at the pure phases, so Flory–Huggins keeps c strictly inside $(0, 1)$, at the cost of a stiff f'' near the walls.

Flory–Huggins needs a regularization *and* a projection — and we report both. Two numerical safeguards keep the logarithm usable, and neither is free:

- *Log regularization.* $\ln x$ is replaced by a C^1 linear extension below $x = \epsilon$ ($\epsilon = 1.0 \times 10^{-4}$), capping f'' at $1/\epsilon$ so a transient Newton overshoot near the wall does not produce an infinite Jacobian.
- *Box projection.* Each Newton iterate is clipped back into $(0, 1)$. This keeps the state physical, but clipping a conserved field *moves mass*: when it fires it breaks exact conservation. We therefore report, for the run, the converged field bounds $\phi \in [0.14, 0.86]$ (strictly interior), the number of projected degrees of freedom, and the largest correction, so the cost of the safeguard is visible rather than hidden.

The distinction that matters: the projection fires on *transient Newton iterates* (2308 dof-clips over the reference run), but each step’s *converged* state stays strictly interior ($\phi \in [0.14, 0.86]$), so no mass is moved on any accepted step and the total mass is conserved to $< 10^{-14}$ (*machine*). On a coarser mesh or a deeper quench, though, Newton no longer resolves the interior state within its iteration budget, the *accepted* step is itself clipped, and mass then drifts (the quick-mode 32^2 run shows a $\sim 2 \times 10^{-4}$ drift for exactly this reason) — a first, concrete example of an admissibility safeguard trading conservation for robustness (the substrate chapter, 5, returns to this with a conservative correction).

Coefficients and their ranges. $M > 0$ (mobility, time scale). $\kappa > 0$ (gradient penalty; interface width $\sqrt{\kappa/W}$ — choose it so the interface spans a few mesh cells). Polynomial: no free parameters, $W = \frac{1}{4}$. Flory–Huggins: $A > 0$ (entropy, $\sim 1/N$), $B = \chi$ (interaction; for $A = 1$ the symmetric point $c = \frac{1}{2}$ is unstable once $B > 2A = 2$). The tutorial uses $A = 1$, $B = 2.5$ (a clear quench); real blends map $A \sim 1/N$, $B \sim \chi$ (see `materials/materials.yaml`).

3.3 Linear stability: the selected length scale

Before any domains form, the nearly uniform blend is a superposition of small Fourier modes. Linearizing (3.3) about the mean $\bar{c}$, a mode $\propto e^{i\mathbf{k}\cdot\mathbf{x}+\sigma t}$ grows at rate

$$\sigma(k) = -M k^2 (f''(\bar{c}) + \kappa k^2). \quad (3.7)$$

When $f''(\bar{c}) < 0$ a band $0 < k < k_c = \sqrt{-f''(\bar{c})/\kappa}$ is unstable, and one wavenumber grows fastest:

$$k^* = \sqrt{\frac{-f''(\bar{c})}{2\kappa}}, \quad \lambda^* = \frac{2\pi}{k^*}. \quad (3.8)$$

This λ^* is the wavelength the spinodal decomposition *selects* — the initial domain spacing. For both free energies at their (unstable) mean, $f'' = -1$, so $k^* = 31.6$ and $\lambda^* = 0.199$.

Prediction vs. measurement. We verify (3.7) directly. The quench is violent: with these coefficients the fastest mode’s growth rate is $\sigma^* \sim M f''^2/(4\kappa) \sim 500$, so at the production time step the linear window is under one step. A dedicated small- Δt *probe* (function `linear_probe`) marches a few tiny steps from the small-amplitude initial field and reads the growth of each Fourier shell, $\sigma_{\text{meas}}(k) = \frac{1}{2} \ln[S(k, \tau)/S(k, 0)]/\tau$ (the factor $\frac{1}{2}$ because $S \sim \text{amplitude}^2$). Its

peak lands on the predicted k^* : the measured fastest wavelength is 0.182 versus the predicted 0.199 (ratio 0.92), well within one FFT shell. Figure 3.1 overlays the two.

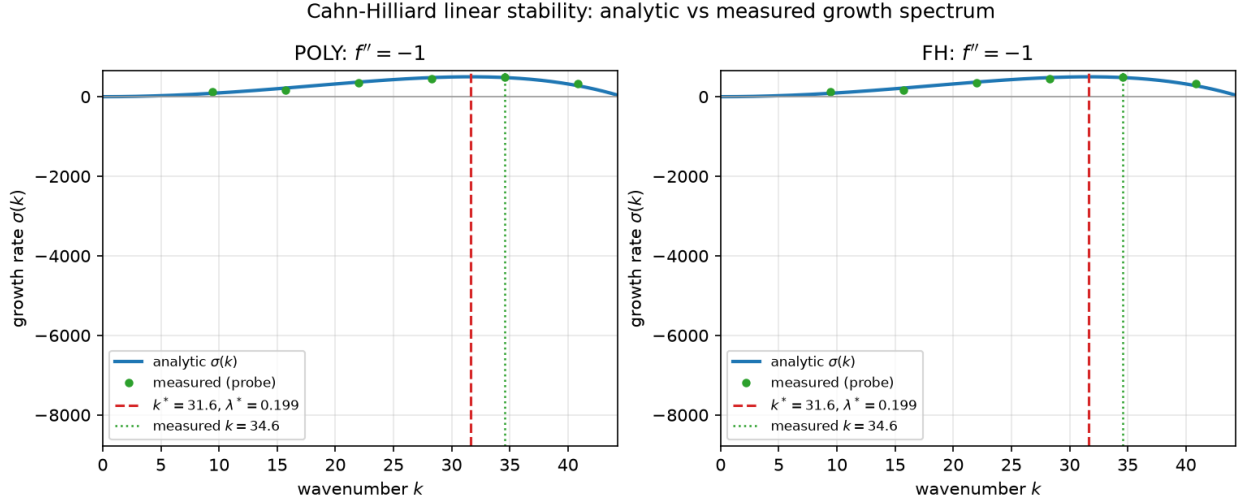


Figure 3.1: Linear-stability dispersion. The analytic growth rate $\sigma(k)$ of (3.7) (line) against the measured growth spectrum from the small- Δt probe (points), for the polynomial (left) and Flory–Huggins (right) free energies. The measured peak sits on the predicted fastest mode k^* (dashed), confirming that the spinodal selects $\lambda^* = 2\pi/k^*$.

3.4 Run it

Run it.

```
cd physics/01_ch_binary_energies
python run_harness.py --config configs/p1.yaml \
    --mode reference --output outputs/p1 --overwrite
python gen_figures.py --run-dir outputs/p1
```

The harness marches a spinodal decomposition for both free energies, runs the linear-stability probe, tracks the energy budget and the coarsening length scale, and checks the results against `baseline.yaml` (tolerance-based invariants, not bit-identity). `gen_figures.py` renders every figure below from the saved `results.json/history.npz` — the document’s numbers *are* the run’s output. `quick` mode is a fast smoke; the 5-seed research ensemble is `--modererearch`.

What you should see. From a nearly uniform, slightly noisy blend, fluctuations in the unstable band grow at the rate (3.7), the pattern sharpens into domains of spacing $\sim \lambda^*$ separated by interfaces of width $\ell \sim \sqrt{\kappa/W}$, and then *coarsens*. The total free energy drops (after a one-step start-up transient), mass is conserved to solver tolerance, and the field saturates at the two coexisting phases.

Three statements about “the energy decreases” — keep them apart.

1. *Continuous Lyapunov.* In the PDE, $dF/dt = -\int_{\Omega} M|\nabla\mu|^2 dV \leq 0$ exactly. A theorem

about the model.

2. *Discrete energy-stability.* Whether the *time scheme* inherits $F^{n+1} \leq F^n$ every step. This is a property of the discretization, not automatic. We measure it: the largest positive stepwise increment $\max_n(F^{n+1} - F^n)$ after the start-up step is 0 (polynomial) and 0 (Flory–Huggins) — non-positive to round-off, so this scheme *is* energy-stable here.
3. *One monotone run.* The particular trajectory you plotted went downhill. Necessary, not sufficient: report the increment, not just “it looked monotone”.

The step-0 $\rightarrow$ 1 increment is separately reported (-0.00645 for the polynomial): the `mu_init="consistent"` projection sets a matching initial chemical potential so this start-up transient is small and bounded, not an instability.

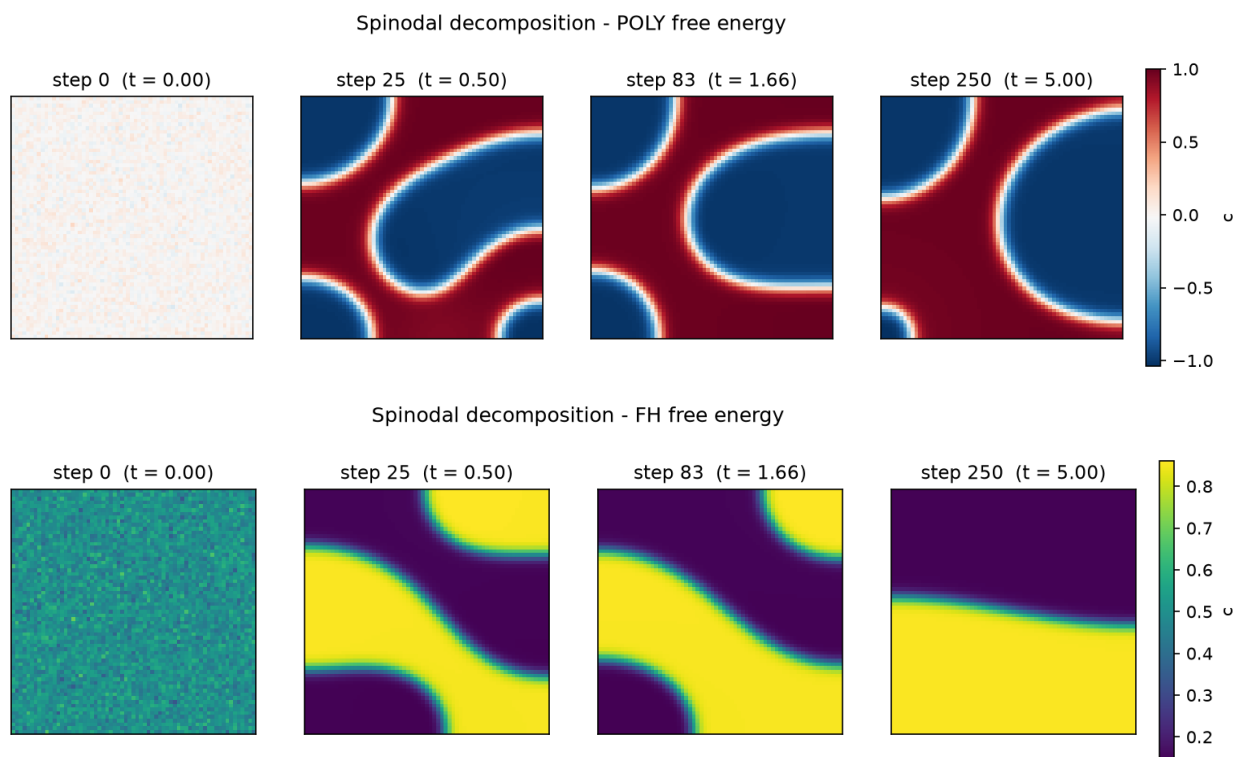


Figure 3.2: Spinodal decomposition and coarsening for the polynomial (top, $c \in [-1, 1]$) and Flory–Huggins (bottom, $c \in (0, 1)$) free energies. Same mixing physics, different order-parameter range and interface structure.

3.5 Coarsening: $L(t) \sim t^n$

Once domains exist, they grow: interfacial energy is paid down by merging domains, so the characteristic length $L(t)$ increases. We measure $L(t)$ *three* ways (all from the shared `diffsim.diagnostics.morphology`) — and the choice matters:

- *peak* wavelength $2\pi/q_{\text{peak}}$ of the structure factor $S(q)$: the intuitive domain spacing, but on a coarse mesh it jumps between discrete q -shells (a quantization that shows up as a near-zero, poorly-fit exponent);

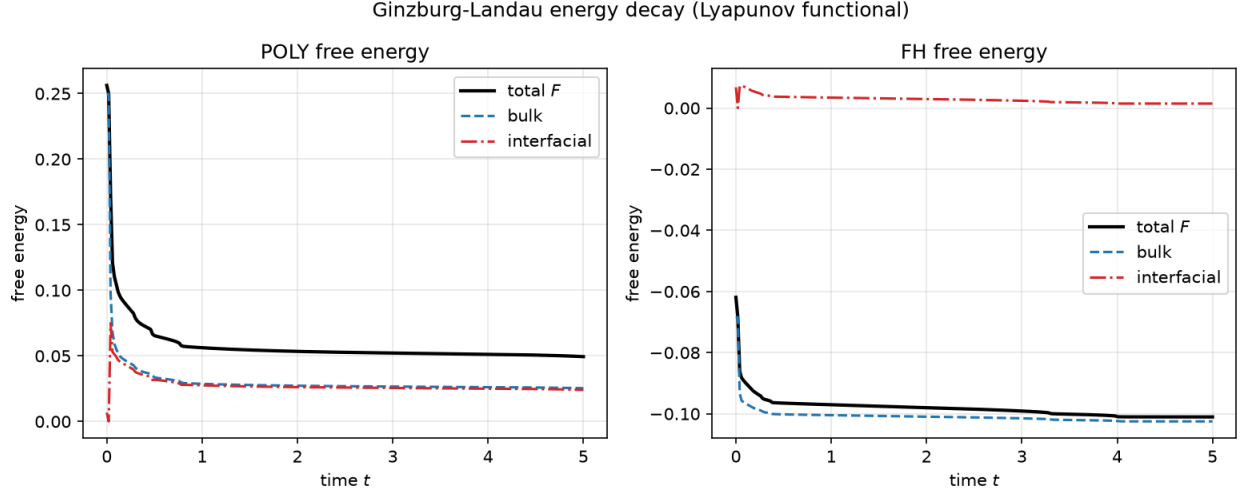


Figure 3.3: The Ginzburg–Landau energy budget: total free energy (black) and its bulk (blue) and interfacial (red) parts versus time. Early on, forming interfaces *raises* the interfacial energy while the bulk energy falls faster; during coarsening, domains merge and the interfacial energy falls as total interface area shrinks.

- *first-moment* wavelength $2\pi/\langle q \rangle$: smoother, but biased low by the sharp-interface high- k tail (it partly measures the interface width, not the domain size);
- *interfacial-area* length $L = A_{\text{domain}}/P_{\text{int}}$ (total area over total interface length): robust and monotone, because the interface shrinks as $\sim 1/L$ during coarsening. This is the headline.

Fitting $L \sim t^n$ by least squares in log–log on the *pre-saturation* window (skip the onset, cap at $L < \text{box}/3$ where finite-size saturation sets in — the fit range is reported with the number) gives, from the interfacial-area length, $n = 0.30 \pm 0.02$ ($R^2 = 0.99$) for the polynomial well and 0.20 ± 0.02 for Flory–Huggins — of order the diffusive Lifshitz–Slyozov value $n = \frac{1}{3}$ for a conserved order parameter, though the precise value is sensitive to the length definition, the fit window, and finite-size saturation on the small reference box (the research mode, on a larger box and longer horizon, is where a clean asymptotic $1/3$ can be resolved). The peak ($n \approx -0.00$, quantization-limited) and first-moment ($n \approx 0.20$, tail-biased) definitions on the *same* run give very different slopes: a concrete lesson that a coarsening exponent is only as good as the length definition, the fit window, and the reported uncertainty — never a single slope asserted to be $1/3$.

On the random seed — and why the morphology is *not* reproducible. The initial perturbation is drawn from a *fixed* seed so the figures here are reproducible *on this machine*. But the morphology is not a bit-reproducible quantity: which domain forms where is chaotically sensitive to the seed, the mesh, and floating-point order across GPUs. What *is* robust is the physics — the energy decay, the selected wavelength λ^* , the coarsening exponent, the phase values $c_{\min}, c_{\max}$. The research mode makes this explicit: it runs a **5-seed ensemble** and reports the mean $\pm$ standard deviation of the seed-independent quantities (final energy, final length scale, exponent). The single fixed seed is for the *figures*; the science is the ensemble.

Measured self-check (reference mode; your run reproduces these within `baseline.yaml` tol-

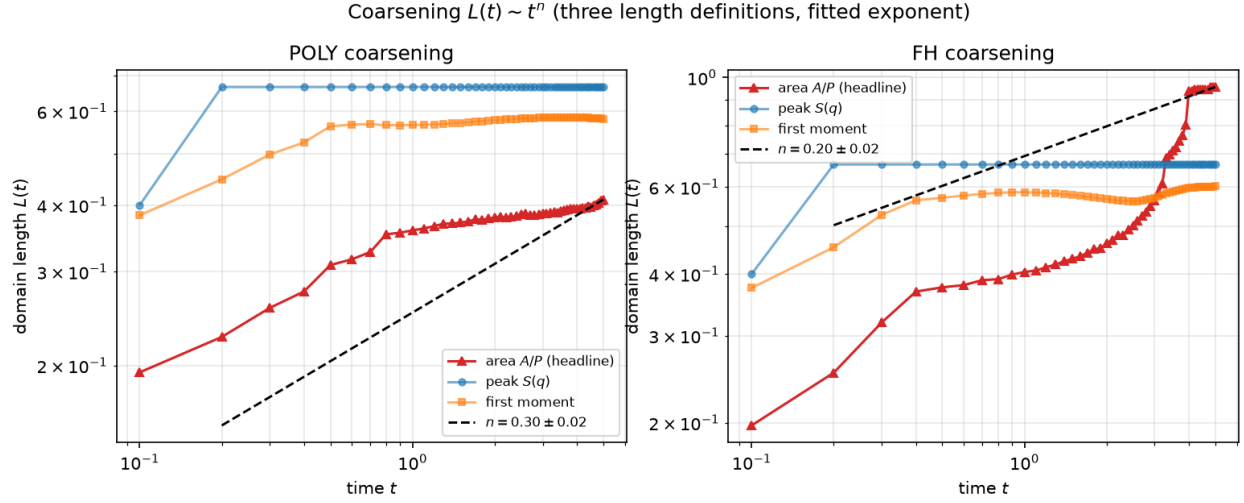


Figure 3.4: Coarsening length $L(t)$ on log–log for the two free energies, from the peak (circles) and first-moment (squares) definitions of $S(q)$, with the fitted power law (dashed) and its uncertainty. The fit is taken on the coarsening window only.

erances, not bit-for-bit).

	polynomial	Flory–Huggins
total F : start $\rightarrow$ end	0.256 $\rightarrow$ 0.0494	-0.0619 $\rightarrow$ -0.101
interfacial energy: peak $\rightarrow$ end	0.0749 $\rightarrow$ 0.024	0.00741 $\rightarrow$ 0.00146
interface width ℓ (cells)	0.0316 (2.0)	0.0224 (1.4)
λ^* : predicted / measured	0.199 / 0.182	0.199 / 0.182
coarsening n (first moment)	0.30 $\pm$ 0.02	0.20 $\pm$ 0.02
phase values $[c_{\min}, c_{\max}]$	$[-1.04, 0.99]$	$[0.14, 0.86]$
largest + energy increment	0	0
mass drift $ \Delta m $	$< 10^{-14}$ (machine)	$< 10^{-14}$ (machine)
projected dofs	0	2308

3.6 Code walk-through

The tutorial’s physics lives in `spinodal.py`, which drives the production brick `src/diffsim/physics/cahn_hilliard.py`; the harness `run_harness.py` turns it into a repeatable workflow, and the measured quantities all come from the shared, unit-tested `src/diffsim/diagnostics/` library (energy budget, structure factor, admissibility, ensemble statistics).

1. Build a mesh and the stepper.

```
dm, mesh, cons = build_mesh_dm(level=6, p=1, device="cuda:0") # 64x64
st = CahnHilliardStepper(dm, M, kappa, dt, order=1,
                          energy="poly") # or energy="fh"
```

energy selects $f(c)$; order is the time scheme (1 = backward Euler); M , $kappa$ are the coefficients above.

2. Seed the initial blend.

```
st.set_initial(lambda x: c_avg + amp*rng.standard_normal(len(x)),
               mu_init="consistent")
```

A nearly uniform field with a small random perturbation — the seed of the spinodal instability. `mu_init="consistent"` projects a matching initial chemical potential so the first step sees no artificial μ -transient.

3. March, measure, verify. `run_spinodal` advances the stepper, integrating the two energy densities of (3.1) by the same quadrature the solver uses and sampling the length scale $L(t)$ as it goes; `linear_probe` does the small- Δt dispersion verification; and the harness writes `results.json`, checks it against `baseline.yaml`, and records provenance in `metadata.json`.

3.7 Explore on your own

Q3.7.1 Vary κ over $\{2, 5, 10\} \times 10^{-4}$. Confirm the interface width scales as $\sqrt{\kappa/W}$ (not $\sqrt{\kappa}$) and count the mesh cells it spans in each case. At what κ does the interface fall below ~ 3 cells, and why should you stop trusting the result there?

Q3.7.2 Verify the dispersion relation yourself: change $\bar{c}$ (for Flory–Huggins) or κ and check that the probe’s measured fastest mode tracks $k^* = \sqrt{-f''/2\kappa}$. Where does $\bar{c}$ leave the spinodal band ($f'' > 0$) and growth stop?

Q3.7.3 Run the 5-seed research ensemble. How large is the seed-to-seed spread in the final energy and length scale compared with the difference between the two free energies? Is the coarsening exponent resolved within its uncertainty?

Q3.7.4 Extend the horizon (research mode, more steps). Does the fitted coarsening exponent climb toward the Lifshitz–Slyozov $1/3$? Plot n versus fit-window start to see the asymptotic range emerge.

Q3.7.5 Drive Flory–Huggins into the wall: lower the mesh resolution or deepen the quench (B up) until the box projection fires. Watch `proj_dofs` and the mass drift rise together — the safeguard trading conservation for robustness.

Chapter 4

Adaptive time stepping

Key idea. Coarsening is multi-scale *in time*: the spinodal quench is stiff (interfaces form on a fast timescale that a time integrator must resolve) and then coarsening slows down geometrically. A single fixed time step cannot serve both regimes efficiently. An *error-controlled* step—small during the quench, large through the tail—delivers a target accuracy at a fraction of the cost. This chapter treats that claim as a *measurement*: we verify the integrator’s order, build a reference solution, and report the *real* cost and the *honest* speed-up.

Learning objectives. By the end you can: verify an integrator’s observed order (BDF1 ≈ 1 , BDF2 ≈ 2) on a smooth relaxation before trusting it on the quench; explain why accuracy is measured against a tight- Δt reference and never read off the free energy; locate the quench’s stiffness cliff and explain why a fixed step safe for it is wasteful for coarsening; account for step-doubling’s real cost (one full plus two half solves per attempt); and report a matched-accuracy speed-up honestly in both steps *and* solves, distinguishing it from the naive worst-case bound.

Prerequisites. Implicit (BDF) time integration and Newton solves, local-truncation-error step control, reading an observed convergence order off a log-log slope; Chapter 00 (environment green) *and* Chapter 3 (same model and free energy, only the time integration changes).

Expected cost. Any CUDA GPU, 2-D meshes (16^2 – 64^2), < 1 GB device memory; solver `splu` (the CH saddle is indefinite, as in Chapter 3). Quick mode comparable to Chapter 3’s measured ≈ 75 s wall; reference mode (the cited numbers) ≈ 10 – 20 min; research mode tens of minutes.

Accuracy is measured against a reference, not read off the energy. A lower free energy does *not* mean a more accurate solution — an integrator can reach a lower energy simply by taking a different (even worse) trajectory. Throughout this chapter accuracy means the error of $c(T)$ (and of the physics diagnostics: energy, structure- factor length scale, phase fractions) against a tight small- Δt *reference*.

4.1 The model and the integrator

The equation is unchanged from Chapter 3: the conserved gradient flow (3.3) of the Ginzburg–Landau energy (3.1). What changes is the time integration. A backward-difference formula (BDF)

of order p writes the time derivative at t_{n+1} as

$$\left. \frac{\partial c}{\partial t} \right|_{t_{n+1}} \approx \frac{1}{\Delta t_n} \left(c_0 c^{n+1} - \sum_{j \geq 1} c_j c^{n+1-j} \right), \quad (4.1)$$

turning each step into a nonlinear (Newton) solve. Here we use $p = 2$.

The variable-step BDF2, mapped to code. A constant-coefficient BDF2 ($c_0 = \frac{3}{2}$, weights $[2, -\frac{1}{2}]$) is only second-order when the step is *constant*. Under an adaptive Δt the step ratio $r = \Delta t_n / \Delta t_{n-1}$ varies, so the coefficients must be rebuilt from the actual $(\Delta t_n, \Delta t_{n-1})$ every step. In `cahn_hilliard.step` this is exactly

$$c_0 = \frac{1 + 2r}{1 + r}, \quad [c_1, c_2] = \left[1 + r, -\frac{r^2}{1 + r} \right], \quad \sigma = \frac{c_0}{\Delta t_n}, \quad (4.2)$$

which for $r = 1$ collapses to the constant-step $\frac{3}{2}, [2, -\frac{1}{2}]$ scheme bit-for-bit (so fixed- Δt trajectories are unchanged). The **startup** is handled by the same code: the first step (and any step taken before a previous step exists) falls back to BDF1 ($c_0 = 1$, $[c_1] = [1]$), whose single $O(\Delta t^2)$ local error does not spoil the global second order. A **rejected** step rewinds the state *including* Δt_{n-1} , so the replayed, halved step rebuilds (4.2) consistently. We *verify* this claim below rather than asserting it.

Step-doubling error control. From a state c^n , take one step of size Δt to get $c_{\Delta t}$, and *separately* two steps of $\Delta t/2$ to get $c_{\Delta t/2}$. Their difference estimates the local truncation error (relative L_2 form, robust to the sharpest interface node),

$$\text{LTE} \approx \frac{\|c_{\Delta t} - c_{\Delta t/2}\|_2}{(2^p - 1) \|c_{\Delta t/2}\|_2}. \quad (4.3)$$

Accept the (more accurate) half-step solution if $\text{LTE} < \text{tol}$, and propose the next step from the standard controller

$$\Delta t_{\text{new}} = \Delta t \cdot \text{clamp}\left(\text{safety}(\text{tol}/\text{LTE})^{1/(p+1)}, \frac{1}{2}, 2\right); \quad (4.4)$$

otherwise *reject*, halve Δt , and retry (Wodo & Ganapathysubramanian, JCP 230 (2011) 6037). Each *attempt* therefore costs **one full plus two half solves**; this is the cost we account for honestly below.

4.2 Step 1 — verify the integrator on a smooth problem

Before trusting an integrator on the chaotic quench, verify its *observed order* on a problem whose exact solution is smooth in time. We use a single low- k cosine, $c = a \cos(2\pi x)$ with small amplitude a — a Neumann eigenmode ($k = 2\pi$) — with $\kappa k^2 > 1$, so it *relaxes* smoothly (exponential single-mode decay). Refining Δt against a fine- Δt reference on the *same* mesh (so spatial error cancels) and reading the slope with the shared convergence diagnostic gives

$$\text{BDF1: observed order 1.00,} \quad \text{BDF2: observed order 2.02,}$$

i.e. first and second order as designed (Fig. 4.1). This is the evidence that the variable-step BDF2 of (4.2) is implemented correctly; only now do we use it on the quench.

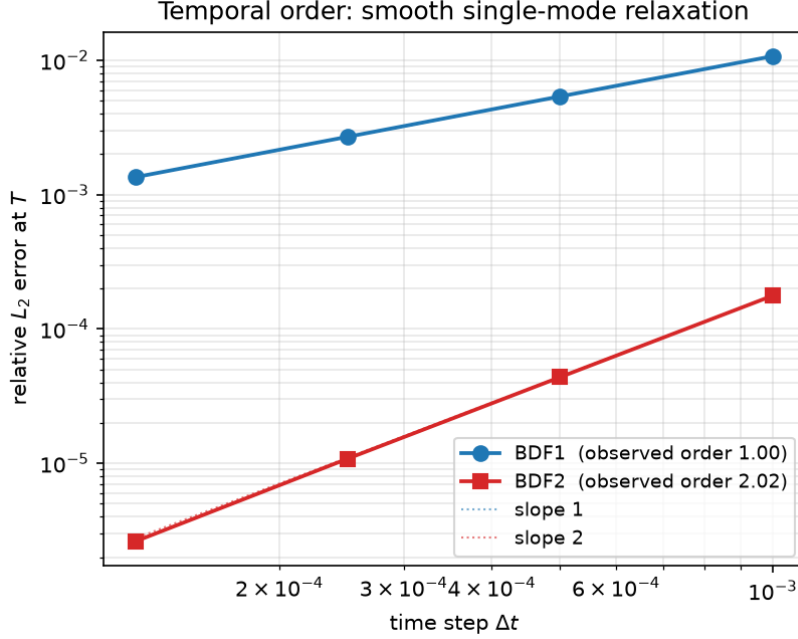


Figure 4.1: Temporal-order verification on the smooth single-mode relaxation: the relative L_2 error at T versus Δt , with reference slopes 1 and 2. BDF1 tracks slope 1, BDF2 tracks slope 2.

4.3 Step 2 — a reference, and the stiffness cliff

We march the binary quench (33×33 , sharp interface $\kappa = 2.0 \times 10^{-3}$, to $t = 0.6$) and build a *reference* by integrating with a tight fixed step $\Delta t = 2.0 \times 10^{-4}$ (3000 steps). We check the reference against itself by halving Δt : the change in $c(T)$ is 5.6×10^{-4} (relative L_2), so the reference is converged to well below the errors we compare against.

Refining a *fixed* step toward the reference, the error falls at the expected order (observed order 2.05) — *until* the step is too large to resolve the quench, at which point the morphology is destroyed (the “stiffness cliff”). A comfortable fixed step $\Delta t = 2.0 \times 10^{-3}$ (300 steps) sits safely below the cliff (relative L_2 error 1.7×10^{-2}), but pushing to $\Delta t = 5.0 \times 10^{-3}$ lands *above* it and the error jumps to 0.79 — more than an order of magnitude worse. **This is the real reason a fixed step is awkward:** you must know the quench’s stiffness in advance to choose it, and a step safe for the quench is wastefully small for the coarsening tail.

4.4 Step 3 — the adaptive run, with real cost

The controller finds the stiff phase on its own: the accepted step collapses to $\Delta t_{\min} = 1.3 \times 10^{-7}$ through the spinodal onset and grows to the ceiling $\Delta t_{\max} = 1.0 \times 10^{-2}$ through coarsening — several orders of magnitude (Fig. 4.2, top). We report the *real* cost, not the accepted-step count alone (Table 4.1): at $\text{tol} = 5.0 \times 10^{-4}$ the run takes 138 accepted and 18 rejected steps; with one full and two half solves per attempt that is 156 full and 312 half solves, 1796 Newton/linear solves in total, and 52.9s of wall time. Its accuracy against the reference is relative $L_2 = 1.8 \times 10^{-3}$, free-energy error 3.8×10^{-5} , and a structure-factor length scale of 19.9 cells, matching the reference’s 19.9. The energy trajectory is indistinguishable from the reference at a fraction of the steps (Fig. 4.5).

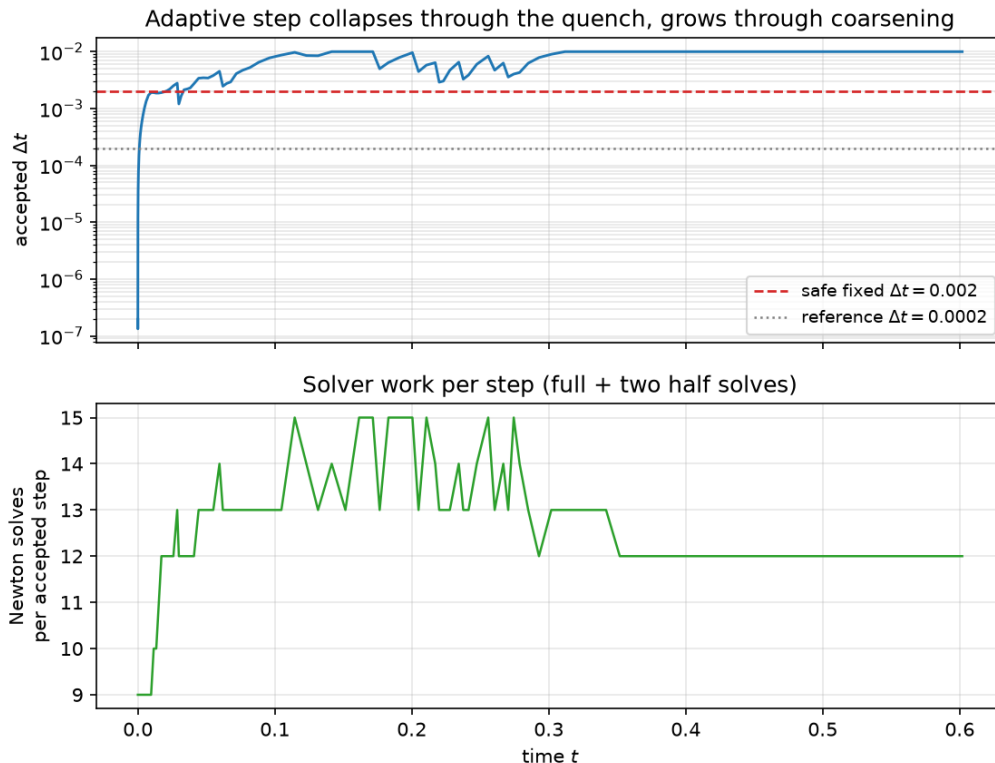


Figure 4.2: Top: the accepted step size collapses through the quench (below even the reference step) and grows by orders of magnitude through coarsening; the safe fixed step (dashed) must stay small the whole time. Bottom: Newton solves per accepted step (full + two half solves) — the solver works hardest through the stiff onset.

Robust statistics vs the pixel field. Two integrators that take different step *sequences* produce different *coarsened patterns* (which domain merges when is chaotically sensitive), exactly as changing the RNG seed does in Chapter 3. So the pixelwise $c(T)$ error has a floor set by this sensitivity, while the *physics* — free energy, structure-factor length scale, phase fractions — agrees to much tighter tolerance (Fig. 4.4). Report both, and lean on the robust statistics.

4.5 Step 4 — convergence, the knee, and the honest speed-up

Sweeping the tolerance over 4 values, the error against the reference reaches its minimum near $\text{tol} = 5.0 \times 10^{-4}$ and does *not* improve for tighter tolerances — indeed at this level it is non-monotone, sitting on a trajectory-sensitivity floor of order 10^{-3} in relative L_2 — even as the cost keeps climbing (Fig. 4.3). That minimum is the **accuracy–cost knee**: the tolerance worth using; tightening further is wasted work.

Matched-accuracy speed-up. The honest comparison holds *accuracy* fixed and compares cost. To reach the adaptive run’s $c(T)$ accuracy with a *fixed* step requires 902 steps ($\approx 3\,564$ solves), versus the adaptive run’s 138 accepted steps (1 796 solves): a matched-accuracy saving of $6.5\times$ in steps and $2.0\times$ in solves (the step-doubling overhead of three solves per step is why the solve saving

is smaller than the step saving). For contrast, the naive worst-case bound — a fixed step *pinned* at the quench’s smallest adaptive step for the whole horizon — would need 4 501 631 steps; that number overstates the case and we do *not* claim it as the speed-up.

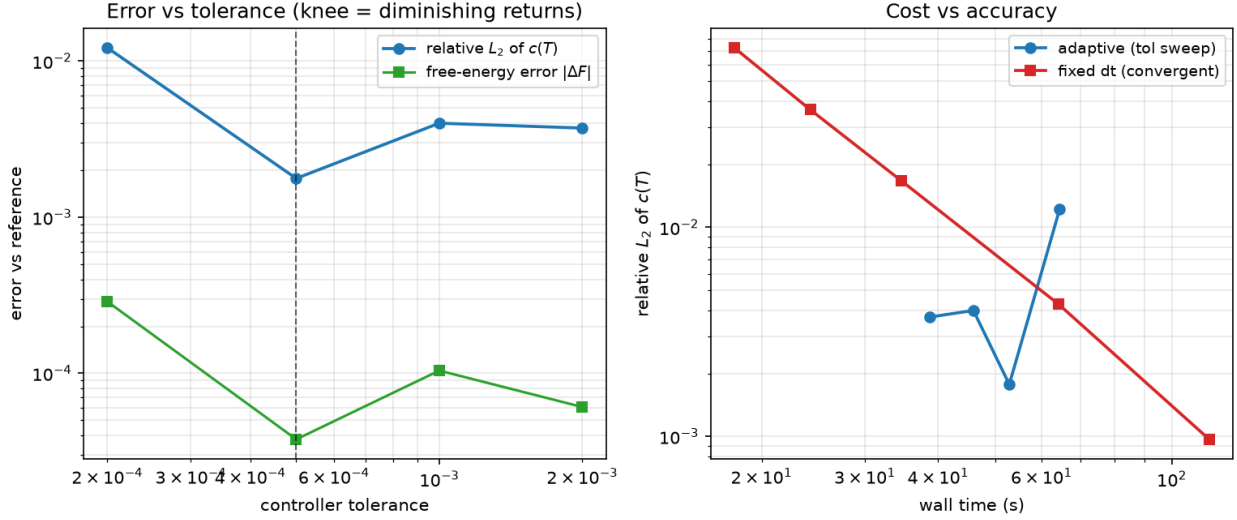


Figure 4.3: Left: error vs controller tolerance — the pixelwise L_2 error and the free-energy error both fall, then flatten at the knee (dashed). Right: accuracy vs wall time for the adaptive tolerance sweep and the convergent fixed- Δt sweep.

Table 4.1: Real cost of the adaptive run ($\text{tol} = 5.0 \times 10^{-4}$) and the matched-accuracy fixed alternative. Solves are total Newton/linear solves; the adaptive attempt cost is one full plus two half solves.

	adaptive	matched fixed
accepted steps	138	902
rejected steps	18	—
full / half solves	156 / 312	—
total Newton/linear solves	1 796	$\approx 3\,564$
Δt range	$1.3 \times 10^{-7} \rightarrow 1.0 \times 10^{-2}$	fixed
wall time (s)	52.9	—
rel. L_2 of $c(T)$	1.8×10^{-3} (matched)	
free-energy error	3.8×10^{-5}	

4.6 Run it

Run it (the workflow).

```
cd physics/02_ch_adaptive_stepping
python run_harness.py --config configs/p2.yaml --mode quick \
    --output outputs/p2 --overwrite # <2 min smoke
python run_harness.py --config configs/p2.yaml --mode reference \
    --output outputs/p2 --overwrite # the cited numbers
```

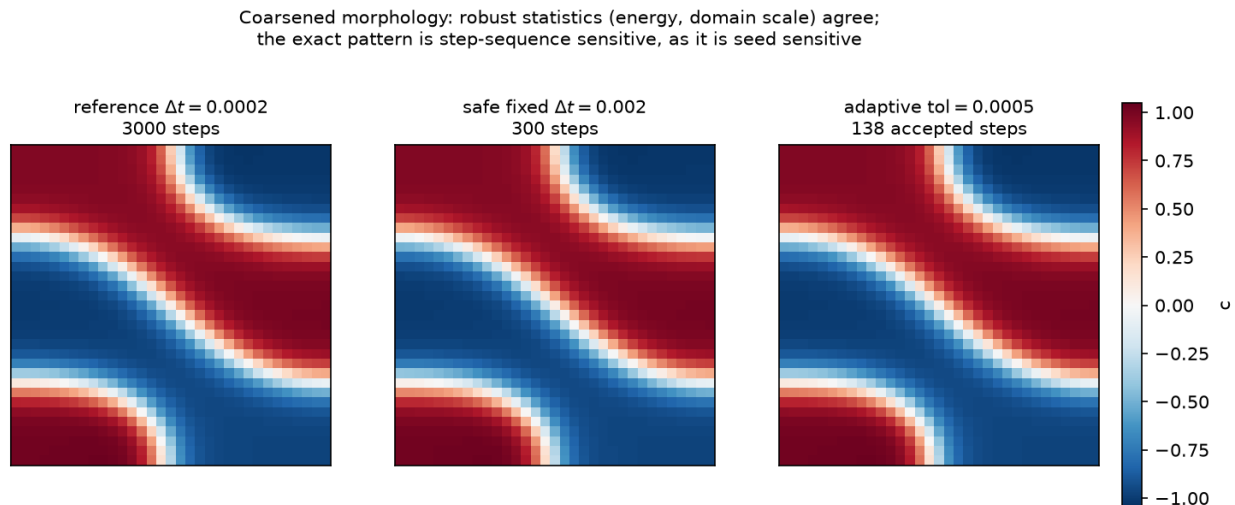


Figure 4.4: Final coarsened morphology from the reference, the safe fixed step, and the adaptive run. The robust statistics (energy, domain scale) agree; the exact pattern is step-sequence sensitive, as it is seed sensitive.

```
python gen_figures.py --run outputs/p2 # figures + numbers/p2.tex
```

The harness writes `results.json` (checked against `baseline.yaml`), `history.npz`, `metadata.json` and the resolved config; `gen_figures.py` renders every figure and the number macros above *purely* from that saved run.

4.7 Code walk-through

1. Verify, then use. `order_study` marches the smooth cosine mode at several Δt and reads the observed order with `diagnostics.convergence.observed_order`; only then does the harness run the quench.

2. The adaptive march, instrumented.

```
from diffsim.physics.cahn_hilliard import adaptive_march
ts, dts = adaptive_march(steeper, t_end, tol=5e-4,
                        dt_max=1e-2, dt_min=1e-7)
```

`adaptive_march` owns the accept/reject ladder and returns the accepted $(t, \Delta t)$ pairs. The harness wraps `stepper.step` to count *every* solve, so it can reconstruct accepted vs. rejected attempts and the per-step Newton work (each attempt is one full + two half solves) — the honest cost in Table 4.1.

3. Accuracy from a reference. `_metrics` compares each run’s $c(T)$ to the tight reference by relative L_2 and by the physics diagnostics (energy, structure-factor length scale, phase fractions) — never by “whose energy is lower”.

4.8 Explore on your own

Q4.8.1 Move the tolerance across the knee (`work_tol`). Below the knee the pixelwise error stops falling while the solve count keeps rising — confirm this, and explain it in terms of the trajectory-

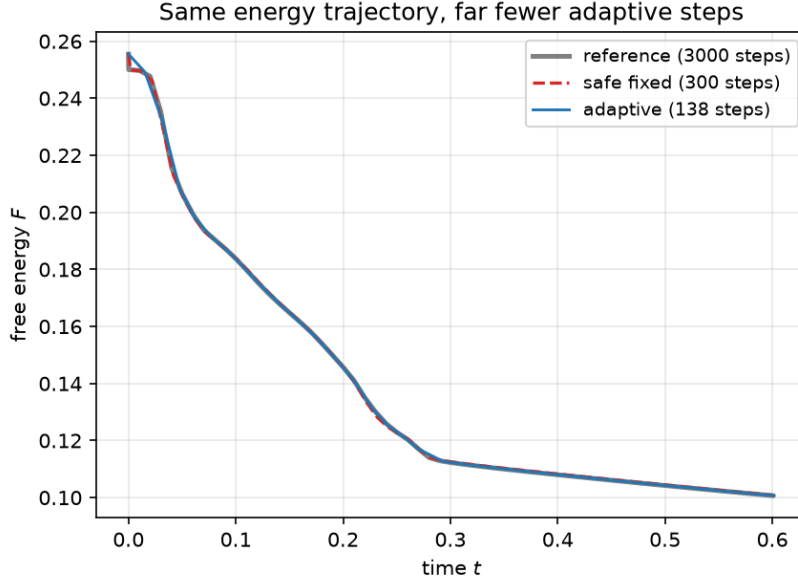


Figure 4.5: Free-energy history: the adaptive run (few accepted steps) traces the same $F(t)$ as the tight reference and the safe fixed step. Energy alone cannot rank accuracy — that is why we compare against the reference, not by “whose energy is lower”.

sensitivity floor and the reference.

Q4.8.2 Widen the interface (increase κ). Does the stiffness cliff move to a *larger* safe fixed Δt ? Relate the cliff location to the fastest spinodal growth rate $\lambda_{\max} \sim M/(4\kappa)$ and hence to the smallest step the controller must take.

Q4.8.3 Lengthen the horizon (t_{end}) and raise dt_{max} . The coarsening tail tolerates ever-larger steps; does the matched-accuracy speed-up grow? Why does adaptivity pay off more the longer (and more scale-separated) the run?

Q4.8.4 Change the RNG seed and rerun. Which reported quantities move (pixel field, domain scale, energy, phase values) and which do not? Relate this to the seed discussion in Chapter 3 and to the pixelwise-error floor.

Q4.8.5 Turn the controller off by using the largest fixed step above the cliff. What does the morphology look like, and which diagnostic (energy, length scale, phase fraction) flags the failure most clearly?

Chapter 5

Substrate and boundary conditions

Key idea. A cast film is bounded by a *substrate* and by *air*, and those interfaces are usually not neutral: one component prefers the wall. That preference is imposed through the *boundary condition* and sets the *vertical stratification* of the morphology—which in a device decides whether the right material meets the right electrode. Mixed Cahn–Hilliard carries *two* natural boundary integrals; this chapter maps both to code, shows the wall energy conserves mass to machine precision, and measures the enrichment boundary layer.

Learning objectives. By the end you can: derive both natural boundary integrals of mixed Cahn–Hilliard from the weak form and explain why only the wall energy $f_w(\phi) = g\phi + h\phi^2$ needs a face term; read off the preferred surface composition $\phi^* = -g/(2h)$ and predict enrichment vs depletion before running anything; explain why the wall condition conserves mass exactly while a Dirichlet condition does not, and why the *quadrature* mass $\int \phi dV$ is the correct check near a boundary; and predict $\delta \sim \sqrt{\kappa}$ and *verify* it by fitting the measured decay length across several κ .

Prerequisites. Natural boundary conditions from integration by parts, the mixed (ϕ, μ) weak form, quadrature vs nodal quantities; Chapter 3 (bulk Cahn–Hilliard and quadrature mass) and Chapter 00 (environment green).

Expected cost. Any CUDA GPU, 2-D (64×64 at reference), < 1 GB device memory; solver `splu` (the CH saddle is indefinite, as in Chapter 3). Quick mode comparable to Chapter 3’s measured ≈ 75 s wall (RTX 6000 Ada, similarly-sized mesh); reference a few minutes; research (finer mesh, denser κ sweep) tens of minutes.

5.1 The model: two boundary conditions

The bulk equations are binary Cahn–Hilliard (Chapter 3) in the mixed (ϕ, μ) form,

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (\Lambda \nabla \mu), \quad \mu = f'(\phi) - \kappa \nabla^2 \phi, \quad (5.1)$$

with f the Flory–Huggins bulk of Chapter 3. The weak form of (5.1) produces *one boundary integral per equation*, and the physics of a wall lives entirely in what we do with them.

(a) The mass-flux condition, $\nabla_\mu n = 0$. Testing the ϕ (mass-balance) equation and integrating $\nabla \cdot (\Lambda \nabla \mu)$ by parts leaves a boundary term $\int_{\partial\Omega} N_a (\Lambda \nabla \mu) \cdot n dS$. The *natural* (“do-nothing”) choice

drops it: no mass crosses the boundary. In code this is simply the *absence* of any face term on the ϕ rows—which is exactly why total mass is conserved (Sec. 5.3).

(b) The wall condition, $\kappa \nabla \phi \cdot n + f'_w(\phi) = 0$. Add a *substrate surface energy* on the bottom edge Γ_w ,

$$F_w = \int_{\Gamma_w} f_w(\phi) dS, \quad f_w(\phi) = g\phi + h\phi^2, \quad (5.2)$$

a surface density minimized at the preferred surface composition $\phi^* = -g/(2h)$. Testing the μ equation and integrating the $-\kappa \nabla^2 \phi$ term by parts gives a boundary term $\int_{\partial\Omega} N_a \kappa \nabla \phi \cdot n dS$; the variation of F_w contributes $+\int_{\Gamma_w} N_a f'_w(\phi) dS$. Setting the sum to zero gives the *natural* wall condition

$$\kappa \frac{\partial \phi}{\partial n} \Big|_{\Gamma_w} = -f'_w(\phi) = -(g + 2h\phi), \quad (5.3)$$

so the μ residual gains $-\int_{\Gamma_w} N_a (g + 2h\phi) dS$ (the consistent, basis-generic face term). Crucially this is a condition on ϕ , *not* on the mass flux: the flux condition (a) is unchanged, so **mass is still conserved**—the wall changes *where* material wants to be, not how much there is.

Boundary-condition taxonomy. A phase-field boundary can carry: (i) *natural / no-flux* ($\nabla \mu \cdot n = \nabla \phi \cdot n = 0$): neutral, nothing crosses, mass conserved—the default; (ii) *wall energy* (this chapter): a Neumann-type condition on $\nabla \phi$ from the surface energy f'_w ; still no mass flux, but the equilibrium is stratified; (iii) *Dirichlet*: pin ϕ or μ to a prescribed value (a reservoir / fixed-composition contact)—this *does* exchange mass and is used mainly for manufactured-solution verification (Computational track). This chapter demonstrates (i) and (ii).

5.2 Coefficients

Wall coefficients and their meaning. g is a linear surface field and $h > 0$ its curvature; together they set the preferred surface composition $\phi^* = -g/(2h)$. With $h > 0$, $g < 0$ puts ϕ^* *above* the blend mean (the wall *enriches* the component); a shallow well with ϕ^* below the mean *depletes* it. ϕ^* must lie strictly inside $(0, 1)$: a bounded well then keeps ϕ admissible for free (Sec. 5.3). The tutorial uses a weakly demixing blend ($\chi = 2.2$, just above the spinodal at $\chi = 2$) with $\phi^* = 0.75$ (attracting) and $\phi^* = 0.25$ (depleting) around a $\bar{\phi} = 0.5$ blend. *These are a surface field and its curvature, not a contact angle*: the brick implements no wetting-angle boundary condition, so we describe g, h as the surface-energy asymmetry and read ϕ^* directly.

5.3 Conservation: measured by quadrature

The conserved quantity is the *quadrature* content $m = \int_{\Omega} \phi dV$, evaluated at the same Gauss points the assembly integrates with (`diffsim.diagnostics.conservation.quadrature_mass`) —*not* a nodal average against a nominal value. Measured that way, every one of the six cases conserves mass to machine precision: $|\Delta m| \leq 10^{-13}(\text{machine})$. Two facts make this exact:

- the wall energy is a natural condition on μ , *not* a mass flux, so the discrete ϕ -balance integrates to $d \int \phi dV / dt = 0$;
- with ϕ^* strictly interior the admissibility projection (the $[10^{-3}, 1 - 10^{-3}]$ box clip) stays inactive for the single-phase wells: the number of projected DOFs is 0. It fires only transiently in the demixing quench (16 projections, as the phases sharpen toward 0 and 1)—and even

there the *converged*-state mass is untouched, because BDF1 with the no-flux flux condition conserves $\int \phi dV$ regardless of the guard (the projection acts on intermediate iterates, not the answer). Nothing moves mass.

Making the projection fire—honestly. The projection is the *only* thing that could break conservation, and a bounded well never triggers it (we checked up to $\phi^* = 0.9975$: `projected_dofs=0`). You *can* force it to engage by quenching from an *admissibility-violating* initial condition (fluctuation amplitude 0.3, so $\phi(0)$ overshoots $(0,1)$): the box clip then acts on the early Newton iterates—102 projections, largest correction 2.5×10^{-1} . *Yet the quadrature mass still holds to $< 10^{-13}$ (machine)*: BDF1 with the no-flux flux condition conserves $\int \phi dV$ at every *converged* state, so the projection guards the iteration without corrupting the answer. Genuine non-conservation would need the *equilibrium* to leave $(0,1)$ —a *linear* wall $f_w = g\phi$ with no interior minimum—but that also drives the Flory–Huggins log-barrier stiff and collapses the step to non-termination. The practical lesson: keep ϕ^* interior and conservation is unconditional (Chapter 4 for the stiffness).

5.4 The full energy budget

The total free energy splits into three quadrature integrals,

$$F = \underbrace{\int_{\Omega} f(\phi) dV}_{F_{\text{bulk}}} + \underbrace{\int_{\Omega} \frac{\kappa}{2} |\nabla \phi|^2 dV}_{F_{\text{grad}}} + \underbrace{\int_{\Gamma_w} f_w(\phi) dS}_{F_{\text{wall}}}, \quad (5.4)$$

the first two by volume quadrature, the wall term by *surface* quadrature on the substrate face (the same consistent face mass the residual uses). For the attracting case $F_{\text{bulk}} = -0.145$, $F_{\text{grad}} = 0.0015$, $F_{\text{wall}} = -0.562$. The wall term is the diagnostic: it is 0.000 for the neutral boundary and falls (becomes more negative) as the preferred component wets the wall—most strongly in the demixing case ($F_{\text{wall}} = -0.562$), where a whole phase coats the substrate (Fig. 5.1).

5.5 Six boundary conditions

Run it.

```
cd physics/03_ch_substrate_bcs
python run_harness.py --config configs/p3.yaml --mode reference \
    --output outputs/p3 --overwrite
python gen_figures.py --run outputs/p3
```

The harness marches six boundary conditions, the boundary-layer $\delta(\kappa)$ study, and the clipping demonstration, writing `results.json`; `gen_figures.py` renders every figure and the cited numbers from that saved run. (`run.py` is the quick student self-check for the first three cases.)

Measured self-check (your harness run should reproduce these).

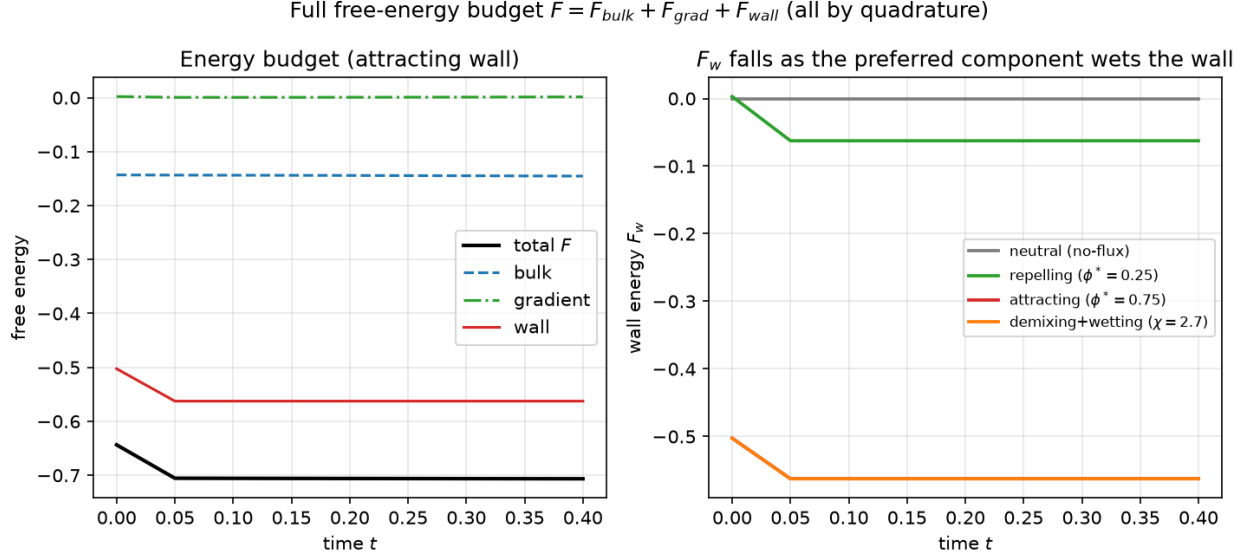


Figure 5.1: Left: the quadrature energy budget $F = F_{\text{bulk}} + F_{\text{grad}} + F_{\text{wall}}$ versus time for the attracting wall. Right: the wall energy $F_{\text{wall}}(t)$ across cases—it decreases as the preferred component enriches the substrate, the surface-energy signature of wetting.

case	substrate ϕ	enrichment $\Delta\phi$
neutral (no-flux)	0.493	-0.008
attracting $\phi^*=0.75$	0.750	+0.248
repelling $\phi^*=0.25$	0.250	-0.249
opposing walls	0.750	air $\phi = 0.250$
confined lateral	0.750	+0.248
demixing+wetting ($\chi=2.7$)	0.750	+0.247

Mass drift $\leq 10^{-13}(\text{machine})$ in *every* case (conservation survives the wall energy); the substrate composition tracks the preferred $\phi^* = -g/(2h)$. The opposing-wall case enriches the substrate *and* depletes the air face; the confined case (no-flux lateral sides instead of the periodic film) reaches essentially the same substrate enrichment—the vertical wall physics is set by the substrate, not the lateral boundary.

5.6 The enrichment boundary layer

The wall preference decays into the film over a length δ . Near the substrate the excess $\phi(y) - \phi_{\text{bulk}}$ relaxes exponentially, and linearizing (5.1) about the bulk gives a decay length $\delta \sim \sqrt{\kappa/f''}$ —so, at fixed f'' , $\delta \propto \sqrt{\kappa}$. Measured on a *stable* (sub-spinodal) bulk—so the only structure in $\phi(y)$ is the wall layer—across four κ values, the $1/e$ decay length fits $\delta \sim \kappa^{0.49}$ ($\kappa^{0.47}$ from an exponential fit), both within a few percent of the predicted $\frac{1}{2}$ (Fig. 5.4): the layer thickens from $\delta_{1/e} = 0.030$ at $\kappa = 0.002$ to 0.083 at $\kappa = 0.016$. This is a *measured* scaling, not an assertion: to make the substrate layer thinner, lower κ .

Substrate/air wall energy stratifies the morphology (substrate = bottom white line)

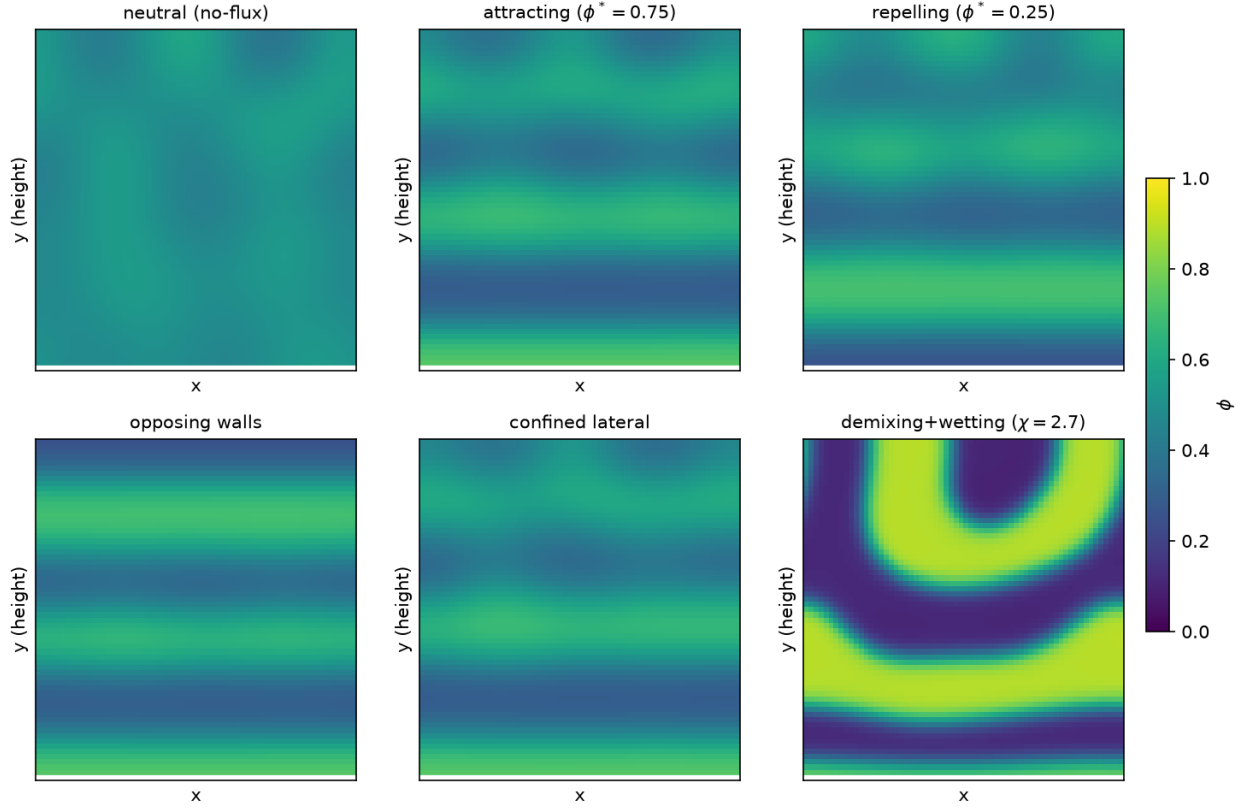


Figure 5.2: The composition field under the six boundary conditions (substrate = the bottom white line). Attracting/demixing walls enrich the bottom layer; the repelling wall depletes it; the opposing walls enrich the substrate while depleting the air side; the confined case replaces the periodic sides with no-flux walls; the neutral wall does neither.

5.7 Code walk-through

1. Turn on the wall. The substrate energy is three constructor arguments on the production brick:

```
st = MultiPhaseStepper(dm, M=1, K=0, chi_aa=..., onsager=[[0.2]],
                        wall_g=[-1.5], wall_h=[1.0], wall_face=(1, 0))
```

`wall_face=(1,0)` selects the $y = 0$ (axis 1, min side) substrate edge; `wall_g`, `wall_h` are g, h of (5.2). Setting both to zero recovers the neutral no-flux boundary bit-for-bit. The opposing-wall case adds a second face on the air side; the confined case builds the mesh with no-flux (non-periodic) lateral sides.

2. Where the two boundary integrals live. In `src/diffsim/physics/multiphase.py` (`_assemble_host`) the wall term is the only face integral added—on the μ rows:

```
# r_mu_i(a) += - INT_w N_a (g_i + 2 h_i phi_i) dS
fw = g_i + 2*h_i * phi_i[wall_faces]
```

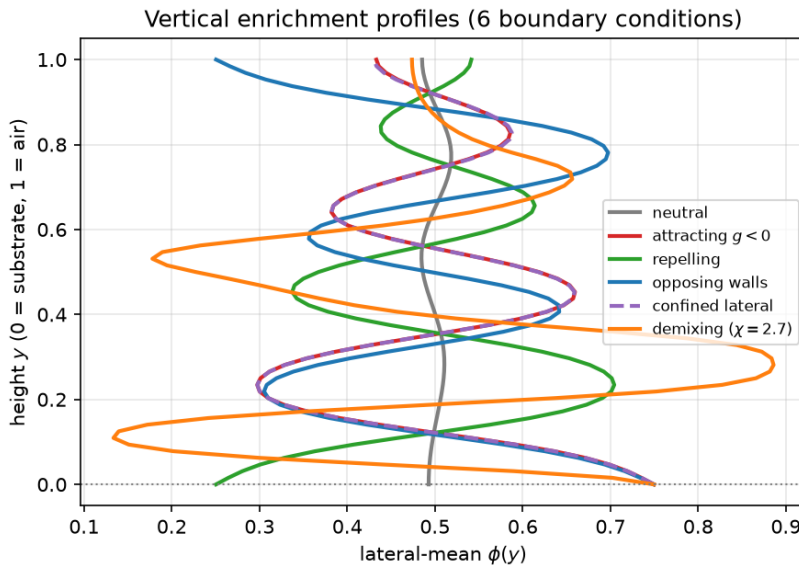


Figure 5.3: Vertical enrichment profiles (lateral-mean ϕ vs height, 0 = substrate, 1 = air). The attracting/demixing walls bend the profile up at the substrate; the repelling wall bends it down; the opposing walls tilt it both ways; the neutral profile is flat. This boundary layer is the substrate’s fingerprint on the morphology.

```
F_full[mu_rows] += einsum("fab,fb->fa", wall_face_M, fw)
# Jacobian: -2 h_i * wall_face_M on the (mu_i, phi_i) block
```

The mass-flux condition (a) needs *no* code: leaving the ϕ rows without a face term *is* the natural $\nabla\mu \cdot n = 0$.

3. Every number by quadrature. `substrate.py`’s `WallDiagnostics` evaluates ϕ and $\nabla\phi$ at the Gauss points and integrates the content, the bulk/gradient energies (volume quadrature) and the wall energy (surface quadrature on the face mass), and reports the projection status—all from the shared `diffsim.diagnostics` library, so the document’s numbers are the unit-tested metrics, not a bespoke re-computation.

5.8 Explore on your own

Q5.8.1 Move ϕ^* toward 0 or 1 (increase $|g|$ at fixed h). The bounded well never clips—verify `projected_dofs=0` even at $\phi^* = 0.99$. Now quench from a violating IC (raise the amplitude) and watch the projection fire while the converged-state mass stays fixed. Finally switch to a *linear* wall ($h = 0$) and observe the time step collapse: why is that equilibrium inadmissible? Relate to Chapter 4.

Q5.8.2 Put a wall on the *air* side (`wall_face=(1,1)`) with the opposite preference—the opposing-wall case. Does the morphology invert top-to-bottom? What does this imply for which component ends up at the top vs bottom electrode?

Q5.8.3 Raise χ above the spinodal (the demixing case) so the blend also phase-separates. Does the attracting wall bias *which* phase wets the substrate? Compare F_{wall} with the single-phase cases.

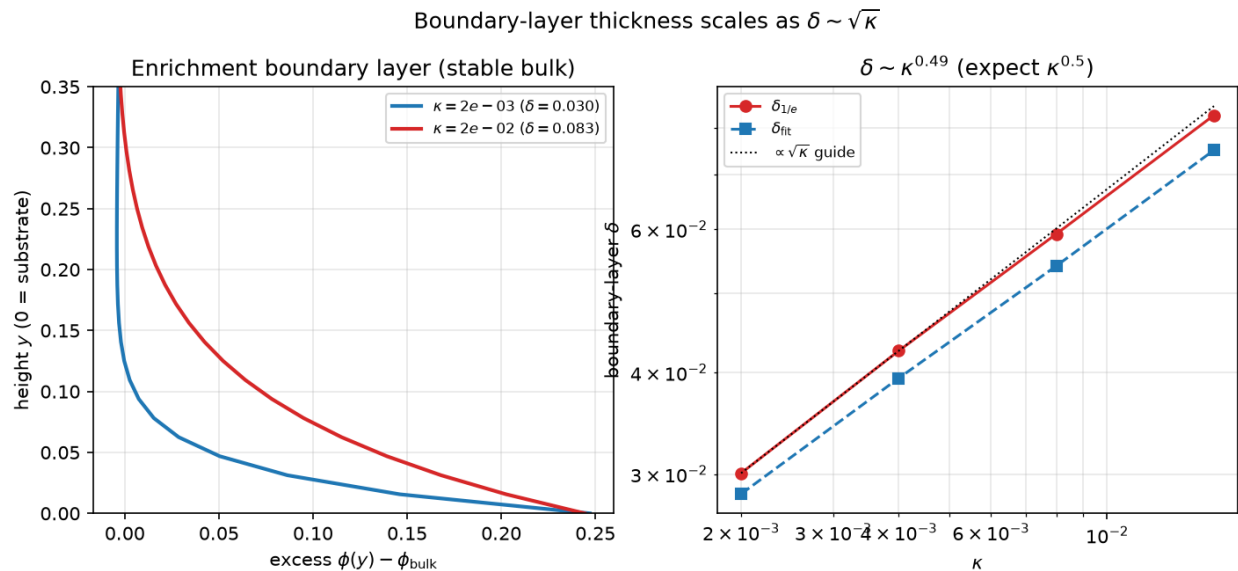


Figure 5.4: Left: the near-wall excess $\phi(y) - \phi_{\text{bulk}}$ and its $1/e$ depth. Right: the boundary-layer thickness δ versus κ (log-log); both the $1/e$ and exponential-fit lengths track the $\sqrt{\kappa}$ guide.

Q5.8.4 Verify conservation explicitly: track the quadrature mass $\int \phi dV$ over the run for each boundary condition. Why does the wall energy, unlike a Dirichlet condition, leave the mass exactly fixed—and why is the nodal-mean a misleading proxy near the wall?

Q5.8.5 Measure the boundary-layer thickness at three more κ values and confirm $\delta \sim \sqrt{\kappa}$. Why did the scaling study use a sub-spinodal bulk rather than the demixing $\chi = 2.2$?

Chapter 6

Ternary Cahn–Hilliard and the phase diagram

Key idea. A real cast blend has (at least) three components — two solutes and a solvent. With two independent conserved compositions the free energy is a *surface over the Gibbs triangle*, and demixing follows *tie-lines* on that triangle. We do not eyeball the tie-line: we *derive* the spinodal from the free-energy Hessian, *predict* the binodal by a common-tangent construction, and then *measure* the two coexisting phases from the simulation by clustering the composition cloud — theory and simulation compared on one triangle.

Learning objectives. By the end you can: derive the ternary exchange chemical potentials μ_i from the Flory–Huggins free energy (6.1) with the solvent eliminated; derive the spinodal from the sign of $\det H$ and solve for the critical χ_{12}^* at a composition; predict the binodal by the common-tangent (equal μ_i , equal ω) construction pinned by the lever rule; and explain why a two-component Gaussian-mixture clustering of the composition cloud replaces a median split, reporting the interface-excluded sensitivity as its uncertainty.

Prerequisites. Chapter 3 (this chapter promotes its Flory–Huggins bulk term and stepper to two composition fields, `bulk="p1"`); Chapter 00 (environment green). The 2×2 Hessian / eigenvalue sign test, Gaussian-mixture (EM) clustering, common-tangent constructions.

Expected cost. Any CUDA GPU; the production (M=2,K=0) `MultiPhaseStepper` exposes the fast `cuDSS` solver, with a documented `splu` fallback for small runs without `cuDSS`. Quick mode is comparable to Chapter 3’s measured ≈ 75 s wall for a similarly-sized 2-D mesh; reference a few minutes (5–30 min); research mode is a single finer/longer run (129×129), not a multi-seed ensemble.

6.1 The model, with the N_i factors

Let ϕ_1, ϕ_2 be the volume fractions of the two solutes; the solvent is eliminated by $\phi_s = 1 - \phi_1 - \phi_2$. The Flory–Huggins free-energy density (in kT/v_0 units) is

$$f(\phi_1, \phi_2) = \sum_{i \in \{1,2,s\}} \frac{\phi_i}{N_i} \ln \phi_i + \sum_{i < j} \chi_{ij} \phi_i \phi_j, \quad (6.1)$$

where N_i is the degree of polymerization of species i (entropic: a long chain has $N_i \gg 1$ and a *weak* mixing entropy $\propto 1/N_i$) and χ_{ij} the pairwise interaction (enthalpic). The exchange chemical potentials, eliminating the solvent, are

$$\mu_i = \frac{\partial f}{\partial \phi_i} = \frac{1}{N_i} (\ln \phi_i + 1) - \frac{1}{N_s} (\ln \phi_s + 1) + \chi_{12} \phi_j + \chi_{is} (\phi_s - \phi_i) - \chi_{js} \phi_j, \quad j \neq i, \quad (6.2)$$

and the coupled conserved dynamics is

$$\frac{\partial \phi_i}{\partial t} = \nabla \cdot \left(\sum_j M_{ij} \nabla \mu_j \right), \quad \mu_i = \mu_i^{\text{bulk}} - \kappa_i \nabla^2 \phi_i, \quad (6.3)$$

with M a symmetric positive-definite Onsager mobility (its off-diagonal couples the two solute fluxes) and κ_i the gradient penalties. This is Chapter 3 promoted to two composition fields; in the code it is the ($M=2, K=0$) case of the production `MultiPhaseStepper`, with the same (6.1) bulk (bulk="p1").

Coefficients come from a materials file, not from code. The run reads $(\chi_{12}, \chi_{1s}, \chi_{2s})$ and (N_1, N_2, N_s) from `materials/ternary_p4.yaml` through the validated loader `materials/loader.py` — each parameter carries `value/units/definition/source/uncertainty`, so no number is hand-copied. The default `synthetic_demix` blend is $\chi_{12} = 3.5$, $\chi_{1s} = 1$, $\chi_{2s} = 0.6$, $N = (1, 1, 1)$ — a deliberately strong quench so the symmetric blend sits well inside the spinodal.

6.2 The spinodal: where the Hessian loses definiteness

A blend demixes spontaneously (spinodal decomposition) where f is locally concave in some direction — i.e. where the 2×2 exchange Hessian $H_{ij} = \partial^2 f / \partial \phi_i \partial \phi_j$ stops being positive definite. Differentiating (6.2),

$$\begin{aligned} H_{11} &= \frac{1}{N_1 \phi_1} + \frac{1}{N_s \phi_s} - 2\chi_{1s}, & H_{22} &= \frac{1}{N_2 \phi_2} + \frac{1}{N_s \phi_s} - 2\chi_{2s}, \\ H_{12} &= \frac{1}{N_s \phi_s} + \chi_{12} - \chi_{1s} - \chi_{2s}. \end{aligned} \quad (6.4)$$

The blend is unstable where $\det H = H_{11}H_{22} - H_{12}^2 < 0$; the curve $\det H = 0$ is the *spinodal*. At the initial blend (0.35, 0.35) the code evaluates (6.4) and finds $\det H = -6.475$ (smaller eigenvalue $\lambda_{\min} = -0.658 < 0$): the quench is unstable, as it must be for the noise to grow rather than decay. Because H_{12} is the only entry carrying χ_{12} , we can solve $\det H = 0$ for the *critical* interaction at this composition, $\chi_{12}^* = \sqrt{H_{11}H_{22}} - (1/N_s \phi_s - \chi_{1s} - \chi_{2s}) = 2.840$; any $\chi_{12} > \chi_{12}^*$ demixes.

6.3 The binodal: a common-tangent prediction

The spinodal bounds the *unstable* region; the two compositions the system actually separates *into* lie on the *binodal*, defined by equal chemical potentials and equal osmotic (grand) potential in the two phases, $\mu_i(\vec{\phi}^A) = \mu_i(\vec{\phi}^B)$ and $\omega(\vec{\phi}^A) = \omega(\vec{\phi}^B)$ with $\omega = f - \sum_i \mu_i \phi_i$ — the common-tangent-plane construction. Pinning the tie-line to pass through the conserved mean (the lever rule) closes the system; the code solves it by a small bounded least-squares step, giving the predicted endpoints (0.122, 0.546) and (0.653, 0.089) (in (ϕ_1, ϕ_2)). These are the theory the simulation is checked against.

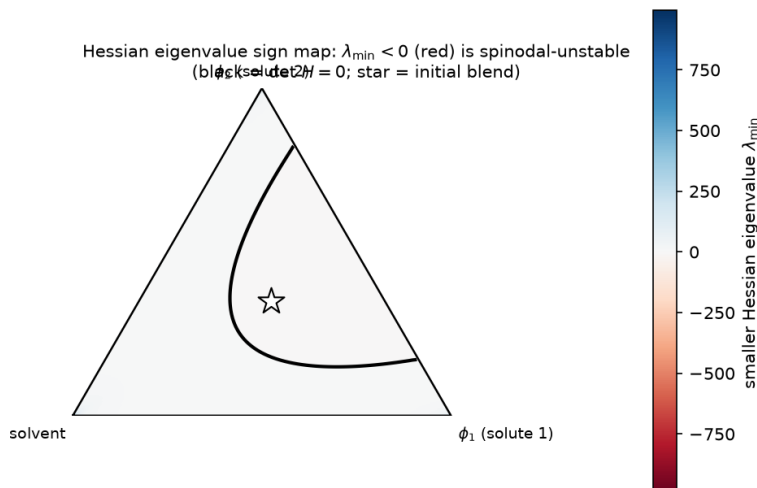


Figure 6.1: Hessian eigenvalue sign map on the Gibbs triangle: the smaller eigenvalue $\lambda_{\min}$ ($\lambda_{\min} < 0$) marks the spinodal-unstable region, bounded by the $\det H = 0$ curve (black). The star is the initial blend — inside the unstable region.

6.4 Reading the two phases: cluster, don't median-split

The naive tie-line readout — split the final field at the median ϕ_1 and average each half — is biased: it forces a 50/50 split and lets interfacial (mixing-transient) nodes contaminate both means. Instead we treat the cloud of local compositions $\{(\phi_1, \phi_2)\}_{\text{nodes}}$ as a two-population sample and fit a **2-component Gaussian mixture** (EM; no `sklearn` needed). Each phase gets a *mean*, a 2×2 *covariance*, and a *population fraction*. As a method check we refit on the *interface-excluded* bulk nodes only (dropping the high- $|\nabla\phi|$ half, whose composition is not a bulk-phase value) and report the endpoint shift as a *sensitivity*.

Run it.

```
cd physics/04_ternary_ch_phase_diagram
python run.py                                # quick look, prints the
    phases
# the full checked workflow (results.json + baseline gate):
PYTHONPATH=<repo>/src python run_harness.py \
    --config configs/p4.yaml --mode reference --output outputs/p4
python gen_figures.py --run outputs/p4        # figures + numbers from the
    run
```

What you should see. The composition spread grows from 0.0201 (a tight blob) to 0.2080 as the blend separates. The GMM finds two coexisting phases at (0.162, 0.523) (population 0.49) and (0.529, 0.186) (population 0.51) — one ϕ_1 -poor/ ϕ_2 -rich, the other the mirror. The interface-excluded means shift by only 0.130, so the endpoints are robust to the clustering choice. The simulated endpoints sit *inside* the predicted binodal (Fig. 6.2): a short quench has not yet reached full coexistence, which is exactly what the tie-line comparison reveals.

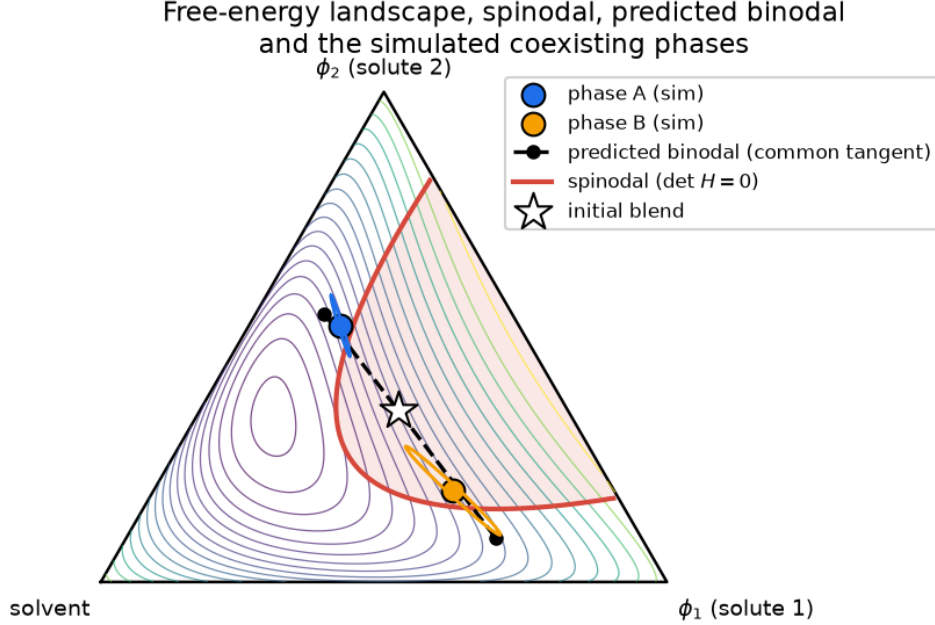


Figure 6.2: The signature figure: free-energy contours of (6.1), the spinodal ($\det H = 0$, red) enclosing the unstable region, the initial blend (star), the predicted binodal tie-line (common tangent, dashed), and the *simulated* coexisting phases (GMM means with 1σ covariance ellipses). Theory and simulation on one triangle.

6.5 Admissibility, conservation, and the lever rule

A trustworthy multi-component run must stay on the simplex, conserve each solute, and respect the lever rule. The harness reports all three from the shared `diffsim.diagnostics` library:

- **Simplex residual.** $\max |\phi_1 + \phi_2 + \phi_s - 1| = < 10^{-14}$ — machine zero, because the solvent is eliminated exactly ($\phi_s \equiv 1 - \phi_1 - \phi_2$). Field bounds stay in-range ($\phi_1 \in [0.104, 0.713]$) with clipped fraction 0 (the log barrier keeps the fractions admissible).
- **Quadrature conservation.** Integrating each solute by the *assembly quadrature* (Gauss points $\times$ Jacobian, not a nodal mean), the per-solute content drifts are $|\Delta C_1| = < 10^{-14}$ and $|\Delta C_2| = < 10^{-14}$ — Model-B no-flux dynamics conserves each solute to round-off.
- **Lever rule.** The conserved mean must equal the population-weighted average of the two phase compositions, $\vec{\phi} = w_A \vec{\phi}^A + w_B \vec{\phi}^B$. The residual is 6.13×10^{-4} — small, and a genuine cross-check that the clustering and the conservation agree.

6.6 The N_i shift

Unequal degrees of polymerization move the spinodal. Because the entropic Hessian terms in (6.4) scale as $1/N_i$, a longer solute-1 chain weakens the entropy that resists demixing and *enlarges* the unstable region. Running the asymmetric blend $N = (2, 1, 1)$ at the same composition, the critical interaction drops from 2.840 to 1.979 (a shift of -0.860) and $\det H$ deepens from -6.475 to -13.605 .

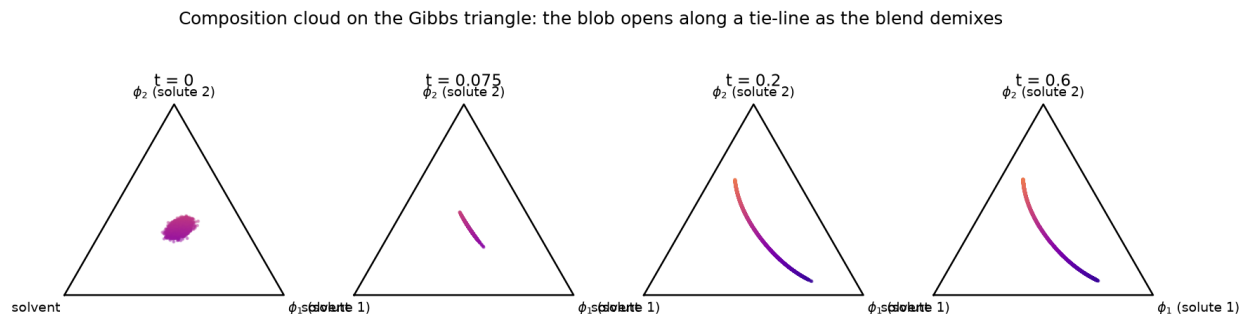


Figure 6.3: Composition-cloud evolution on the Gibbs triangle: the tight initial blob opens along a tie-line as the blend demixes.

The stronger instability drives a more complete separation — the simulated phases move out to (0.096, 0.902) and (0.954, 0.045), near the triangle edges (Fig. 6.4).

The N_i shift: unequal degrees of polymerization enlarge the spinodal (critical χ_{12} drops 2.84 $\rightarrow$ 1.98)

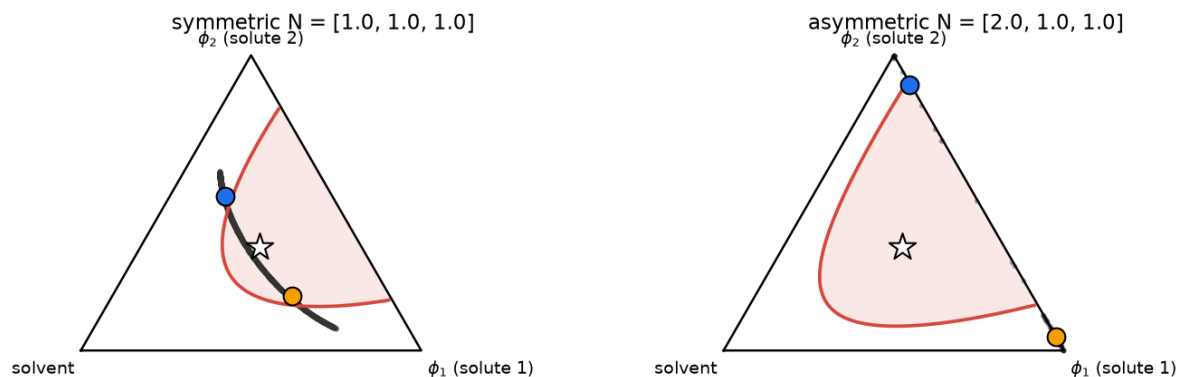


Figure 6.4: The N_i shift: the asymmetric- N spinodal (right) is much larger than the symmetric one (left); the asymmetric blend demixes almost completely (endpoints near the edges).

Measured self-check (reference mode).

spinodal $\det H$ @ IC	-6.475 (< 0 : unstable)
critical χ_{12}^* @ IC	2.840
coexisting phase A (ϕ_1, ϕ_2) / pop.	$(0.162, 0.523)$ / 0.49
coexisting phase B (ϕ_1, ϕ_2) / pop.	$(0.529, 0.186)$ / 0.51
clustering sensitivity (GMM vs. bulk-only)	0.130
predicted binodal A / B	$(0.122, 0.546)$ / $(0.653, 0.089)$
simplex residual $\max \sum \phi - 1 $	$< 10^{-14}$
solute content drift $ \Delta C_1 , \Delta C_2 $	$< 10^{-14}, < 10^{-14}$
lever-rule residual	6.13×10^{-4}
N_i shift $\Delta \chi_{12}^*$ ($N_1:1 \rightarrow 2$)	-0.860

6.7 Code walk-through

1. Materials. `load_blend("synthetic_demix")` (`materials/loader.py`) returns a validated (χ, N) ; the run never hard-codes them.

2. The stepper. `MultiPhaseStepper(dm, M=2, K=0, chi_aa=..., N=...)` builds the coupled $(\phi_1, \mu_1, \phi_2, \mu_2)$ system; `set_initial([f1, f2])` seeds the two compositions.

3. Analysis (in `ternary.py`, not `src/`). `spinodal_hessian` / `spinodal_maps` give (6.4); `gmm2` + `cluster_phases` read off the phases with the interface-excluded sensitivity; `predict_binodal` solves the common-tangent tie-line; the harness adds the quadrature conservation and lever-rule residual.

6.8 Explore on your own

Q6.8.1 Increase χ_{12} past $\chi_{12}^* = 2.840$ and below it. Confirm from `spinodal_hessian` that $\det H$ changes sign, and that below χ_{12}^* the noise decays ($\text{spread} \rightarrow 0$) instead of growing.

Q6.8.2 Make the solvent selective, $\chi_{1s} \neq \chi_{2s}$. Which solute segregates preferentially, and how does the predicted binodal *tilt* on the triangle? Does the GMM sensitivity grow?

Q6.8.3 Push the asymmetry to $N_1 = 5$ or 10 . The spinodal enlarges further — but the quench also gets stiffer (weaker log barrier). Watch the Newton iteration count and the reject ladder; this is why the tutorial demonstrates $N_1 = 2$ on a small mesh.

Q6.8.4 Turn off the off-diagonal mobility M_{12} . Does the tie-line (the endpoints) change, or only the coarsening rate?

Q6.8.5 Map a real system from `materials/ternary_p4.yaml` (say P3HT_PCBM) and predict, from its spinodal, whether the blend at a chosen composition should demix. Then run it and check against the GMM endpoints.

Chapter 7

Evaporation: the drying film

Key idea. A film is cast from solution and *dries*: solvent leaves the top surface, the film thins, and the solutes concentrate until they demix. Evaporation is the *clock* of morphology formation. But the honest question is subtle: does the drying *rate* reshape the morphology, or does it merely decide *when* you look? We compare morphologies at a *matched dryness* (the same mean solvent fraction ϕ_s , not the same time) and find that the *degree* of demixing is set by dryness alone — essentially rate-independent — while the domain *size* collapses onto a single Biot number $\text{Bi} = k_e h_0 / D_s$.

Learning objectives. By the end you can: write the moving-frame film equations ($K = \sum_i k_{e,i} \overline{\phi_i^{\text{top}}}$, $\frac{\partial h}{\partial t} = -K$) atop the ternary bulk dynamics of Chapter 6; explain why a matched-*time* rate comparison confounds rate with dryness and design the matched- ϕ_s fix; define $\text{Bi} = k_e h_0 / D_s$ and *verify* that different (k_e, D_s) sharing a Bi collapse onto one morphology; keep apart “dryness sets the demixing degree” from “ Bi sets the domain size”; and report per-solute conservation and the height budget as a resolution-limited invariant.

Prerequisites. The ternary free energy, spinodal, and mobility machinery of Chapter 6 (reused verbatim here); the interface width/resolution intuition of Chapter 3; Chapter 00 (environment green).

Expected cost. Any CUDA GPU, 2-D (level 5–7). Solver: cuDSS, with `splu` a documented fallback for small runs. Quick mode target < 2 min (comparable to Chapter 3’s measured ≈ 75 s); reference (7-run sweep) 5–30 min; research widens to a finer (k_e, D_s) regime grid.

7.1 The model

We add a moving, evaporating top surface to the ternary system of Chapter 6 (the Landau-mapped moving frame of the Wodo–Ganapathysubramanian film formulation). The computational box $[0, 1]^2$ represents a physical strip of shrinking height $h(t)$; the vertical coordinate maps as physical $z = h(t) \xi_y$. The solvent flux at the top drives the frame,

$$K = \sum_i k_{e,i} \overline{\phi_i^{\text{top}}} \geq 0, \quad \frac{\partial h}{\partial t} = -K, \quad (7.1)$$

and each conserved solute gains a frame-advection term plus a surface- enrichment flux, arranged so the solute *content* $h \int \phi_i$ is conserved exactly per BDF1 step (only the solvent leaves). The bulk

dynamics is the ternary Cahn–Hilliard of Chapter 6.

The Biot number $Bi = k_e h_0 / D_s$. k_e is the nondimensional evaporation rate of the solvent; D_s is the solute Onsager mobility (its diffusivity scale). Their ratio is the *drying Biot number* — equivalently the drying Péclet number in this unit-aspect geometry — comparing how fast the surface recedes with how fast the solutes diffuse. $Bi > 1$ is *drying-limited* (the film skins before it can homogenise); $Bi < 1$ is *diffusion-limited*. Because Bi mixes k_e and D_s , different solvents/temperatures (k_e) and different blends (D_s) that share a Bi dry to the *same* morphology — we test that collapse directly.

7.2 Matched terminal state: rate versus dryness

A naive sweep compares each rate at the end of a fixed time horizon. That *confounds* two different things: a slow film may only have reached $\phi_s = 0.30$ while a fast one has dried to $\phi_s = 0.10$, so any morphology difference could be entirely the different *dryness*, not the rate. The fix is to drive *every* rate past the same dryness and read each morphology metric at a *matched* $\phi_s \in \{0.30, 0.20, 0.10\}$, interpolated along the run’s own $\phi_s(t)$ trajectory (Fig. 7.2). What is confounded in the naive comparison, and de-confounded here, is exactly this rate-versus-state distinction.

The morphology metric is a real length. We take the first spectral moment of the *lateral* (periodic- x) structure factor of the polymer field — a mean cycles-per-box $\bar{k}$ — and report the wavelength both as a fraction of the box width, $\lambda/L_x = 1/\bar{k}$, and in cells, $\lambda/\Delta x = n_x/\bar{k}$. A resolved length is always > 1 cell; the earlier code reported a bare “cells” number below one, which conflated the box-width fraction with a cell count. We report ≥ 2 metrics: the lateral wavelength, the interfacial length, and the phase contrast.

7.3 Run it

Run it.

```
cd physics/05_evaporation
python run_harness.py --config configs/p5.yaml \
    --mode reference --output outputs/p5 --overwrite
python gen_figures.py --run-dir outputs/p5    # figures + numbers
```

The harness dries the film over a (k_e, D_s) sweep, records each morphology metric along the whole $\phi_s(t)$ trajectory, interpolates to the matched dryness levels, checks per-solute conservation, and writes a tolerance-checked `results.json`. `run.py` is a lighter student driver of the same core.

What you should see. Every film dries and thins (Fig. 7.1); the march reports *why* it stopped, and we never call a horizon-truncated run “dried”. At $k_e = 0.6$ ($Bi = 3$) the film reaches the dryness cutoff (`phis_stop`) by $t \approx 3.06$; at $k_e = 0.3$ ($Bi = 1.5$) it takes until $t \approx 6.05$ (`phis_stop`). Reading morphology at *matched* dryness (Fig. 7.2) tells the real story:

- **Dryness sets the *degree* of demixing.** The phase contrast rises monotonically as the film dries ($0.316 \rightarrow 0.367 \rightarrow 0.418$ at $\phi_s = 0.30, 0.20, 0.10$) — and does so *identically for every rate*: at a matched $\phi_s = 0.20$ the contrast agrees across all runs to 0.9%. How far the blend has separated is fixed by how *dry* it is, not by how fast it got there.

- **The domain *size* collapses onto Bi.** At matched dryness the lateral wavelength depends on the Biot number (λ at $\phi_s = 0.20$: 33.4 cells for slow ($Bi = 1.5$), 26.1 for fast ($Bi = 3$)), and runs that *share* a Bi from different (k_e, D_s) agree to 6.1% (Fig. 7.4). With only ~ 2 lateral domains across the box the wavelength is statistically noisy, so we report the collapse, not a sharp size-versus-Bi law.
- **The old “faster = finer” was the confound.** It compared a *dried* fast film against a *still-wet* slow one — a dryness difference misread as a rate effect. De-confounded, the robust, rate-independent quantity is the demixing *degree* (contrast), and the size is governed by Bi.

Evaporation sets the morphology clock — every rate driven to the same dryness

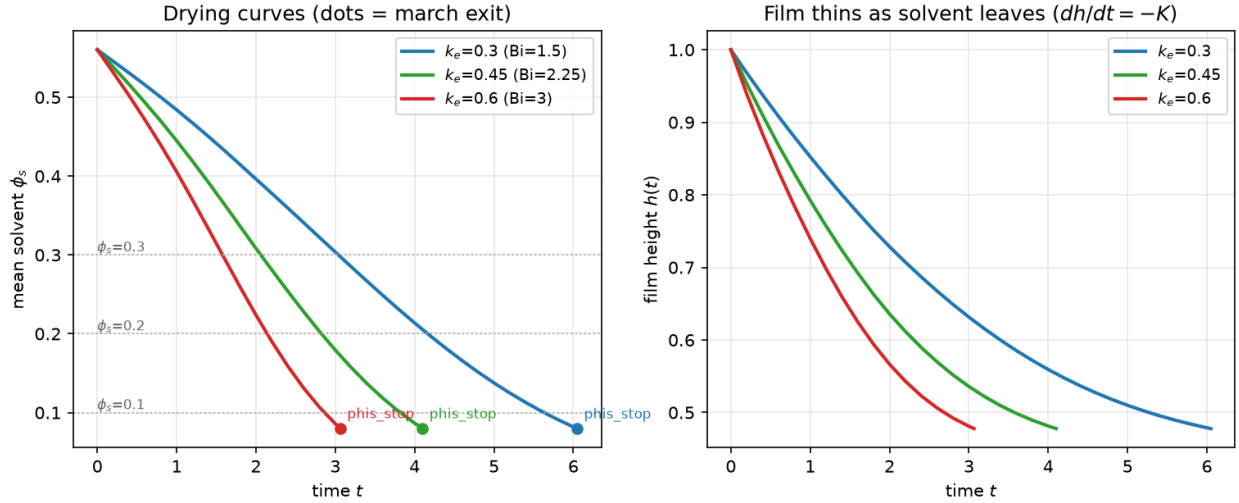


Figure 7.1: Drying curves $\phi_s(t)$ (left) and film height $h(t)$ (right). Dots mark the march exit; the label is the exit reason (**phis_stop** = dried to the cutoff, not the time horizon). Dotted lines are the matched dryness levels.

7.4 Per-solute conservation and the height budget

Only the solvent evaporates, so each solute’s content $h(t) \int \phi_i$ must be constant. We integrate $\int \phi_i$ by quadrature on the fixed reference domain (the P1 lumped-mass weights) and multiply by $h(t)$; the relative drift over the whole run is $< 10^{-13}$ (polymer) and $< 10^{-13}$ (fullerene) — at solver tolerance, confirming the moving-frame scheme conserves solute. The height budget then closes identically: since the solutes are conserved, the solvent lost equals the height drop, $h_0 - h(t)$, to $< 10^{-13}$. Over the slow run the film loses 93% of its solvent and thins by $\Delta h = 0.522$.

Conservation is resolution-limited, and we say so. This exact per-step conservation holds *once the demixed interfaces and the surface enrichment layer are resolved*. On a too-coarse mesh (level 4, 16×16) the same run leaks $\sim 10\%$ of a solute; at level 5–6 the drift falls to machine precision ($\sim 10^{-15}$). The tutorial’s **quick** tier therefore uses level 5, the coarsest mesh that still conserves — a concrete lesson that a conservation law in the continuous model is only inherited by the discretisation you can afford to resolve.

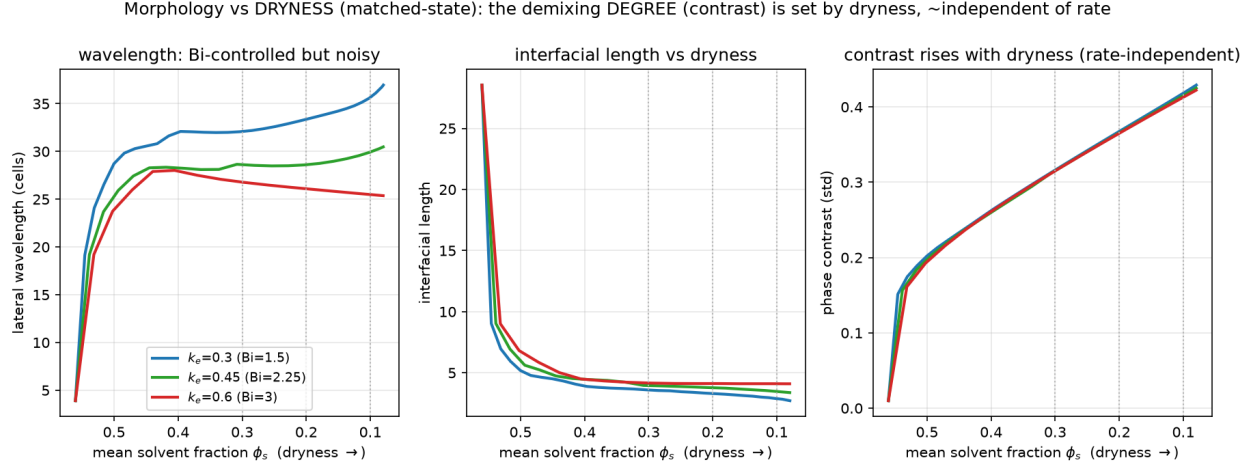


Figure 7.2: Morphology metrics versus *dryness* ϕ_s (the un-confounded axis, drying to the right), all rates on one plot. The phase contrast (right) rises with dryness *identically* for every rate — the demixing degree is rate-independent; the wavelength and interfacial length (left, centre) separate by Biot number. Vertical lines: the matched ϕ_s .

7.5 The Biot regime map

Sweeping *both* k_e and the solute mobility D_s places each run in the (k_e, D_s) plane (Fig. 7.4), with $\text{Bi} = k_e h_0 / D_s$ iso-contours. Two observations:

- **Morphology collapses onto Bi.** Runs that share a Biot number — reached by different combinations of a faster solvent (larger k_e) or a slower blend (smaller D_s) — land on the same matched-dryness wavelength, to 6.1%. The pair (k_e, D_s) does not matter; only their ratio does.
- **The crossover is $\text{Bi} = 1$.** Our sweep spans $\text{Bi} \in [0.75, 6]$, straddling the drying-limited/diffusion-limited boundary. This is the map a process engineer reads: pick a solvent and a blend, compute Bi, and you know which regime you are casting in.

Measured self-check (matched dryness, $\phi_s = 0.20$).

	slow (Bi = 1.5)	fast (Bi = 3)
exit reason	<code>phis_stop</code>	<code>phis_stop</code>
drying time t	6.05	3.06
final height h	0.478	0.478
wavelength (cells)	33.4	26.1
interfacial length	3.30	4.12
phase contrast	0.367	(same, $\pm 0.9\%$)

Per-solute drift $\leq 10^{-13}$; Bi-collapse spread 6.1%; contrast rate-independent to 0.9%.

7.6 Code walk-through

1. Film mode. A single constructor argument turns on the moving frame:

```
st = MultiPhaseStepper(dm, M=2, K=0, ...,
```

```
film=dict(k_e=0.3, h0=1.0))
```

`k_e` is the solvent evaporation rate, `h0` the initial height. The stepper carries `h_curr`, updated $-\Delta t K$ on each accepted step.

2. The drying march. `march(..., phis_stop=0.08, h_min=0.12)` adds film stop criteria and returns the *exit reason* (`phis_stop`, `h_min`, `dt_underflow`, `t_end`); the harness records it per run.

3. The matched-state readout. `run_film` records the morphology metrics along the whole $\phi_s(t)$ trajectory; `morphology_metrics` returns the lateral wavelength (fraction *and* cells), the interfacial length, and the contrast. The harness interpolates each to the matched ϕ_s and computes $\text{Bi} = k_e h_0 / D_s$.

7.7 Explore on your own

Q7.7.1 Widen the Biot range (push D_s down and k_e up). Does the matched-dryness wavelength ever break its collapse onto Bi, or is Bi the only group that matters here?

Q7.7.2 Change the solvent selectivity (χ_{1s} vs χ_{2s}). Does a solvent that prefers one solute change which component enriches at the drying surface — and does that show up more in the *vertical* (physical-frame) profile than in the lateral wavelength?

Q7.7.3 The drying curve $\phi_s(t)$ is roughly linear early (constant-rate period) then bends. Explain the bend from the surface composition feeding back into $K = \sum_i k_{e,i} \phi_i^{\text{top}}$.

Q7.7.4 Combine this with Chapter 5: add a substrate wall energy and dry the film. Does evaporation-driven enrichment at the top compete with substrate enrichment at the bottom? Which wins, and at what Bi?

Q7.7.5 Re-run at level 4 and watch the per-solute drift blow up. At what mesh does conservation recover? Relate the answer to the interface width $\ell \sim \sqrt{\kappa/W}$ of Chapter 3.

Polymer field at matched dryness — fixed computational frame vs physical (thinning) frame

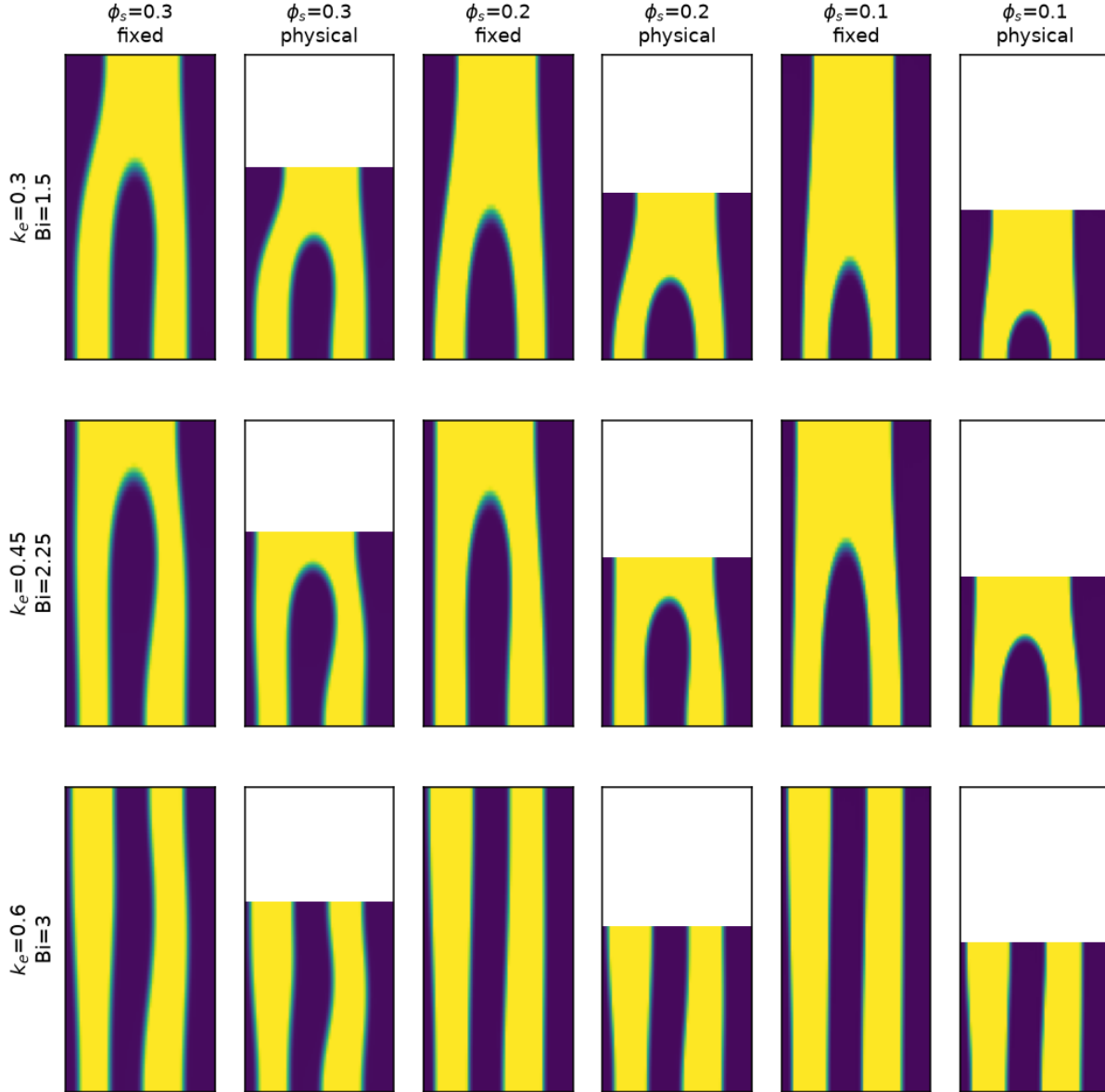


Figure 7.3: Polymer field at the matched dryness levels, shown in *both* coordinate frames: the fixed computational box (unit height) and the *physical* frame with the vertical extent scaled by $h(t)$ — the actual thinning film, not just the mapped box.

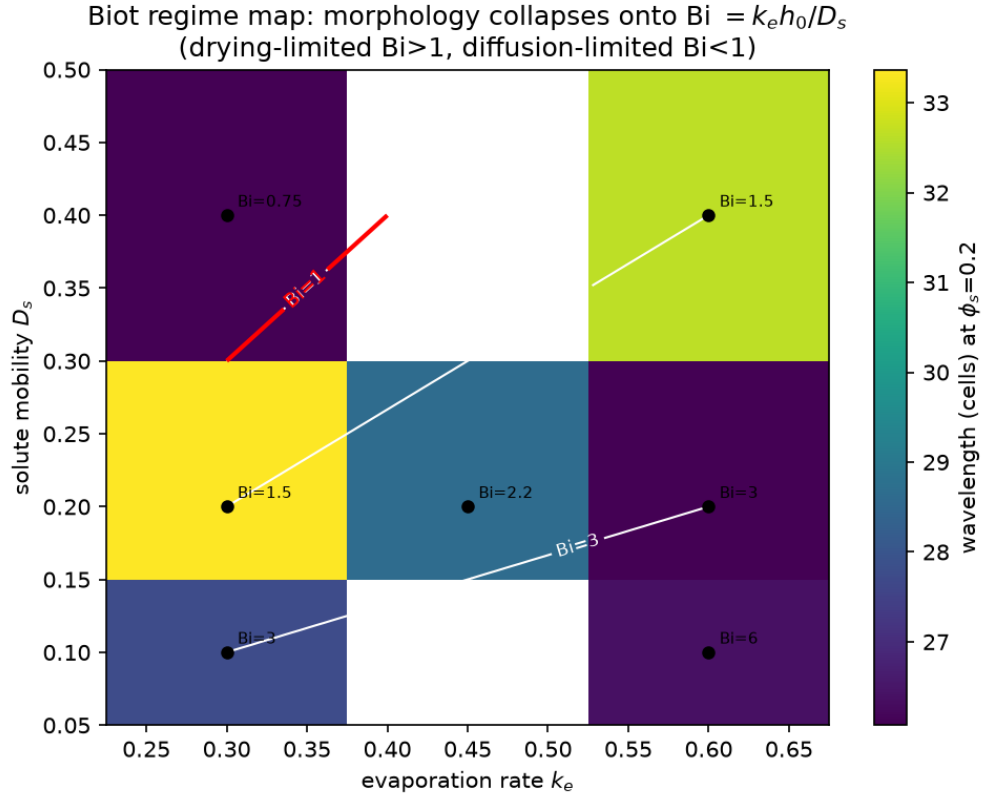


Figure 7.4: Biot regime map over the (k_e, D_s) sweep (7 runs), coloured by the matched-dryness wavelength, with $Bi = k_e h_0 / D_s$ contours and the $Bi = 1$ drying/diffusion crossover in red.

Chapter 8

Allen–Cahn crystallization

Key idea. Crystallinity is a *non-conserved* order parameter — crystal can be created, unlike composition — so it obeys *Allen–Cahn* (non-conserved gradient flow), not Cahn–Hilliard. A crystal grows below the melting point and melts above it; the growth-front *velocity* rises with undercooling; a *critical radius* separates growing from redissolving seeds; and the way the crystalline fraction advances in time is the Avrami (JMAK) law.

Learning objectives. By the end you can: write the non-conserved Allen–Cahn flow for ψ and explain why crystal need not be conserved; predict the sign of drive $= \Delta h(T/T_m - 1)$ and hence grow vs melt; measure $v(\Delta T)$ and explain its monotone rise (constant mobility, no transport maximum); derive and bracket the critical radius $r^* \propto \sigma/|\text{drive}|$; and fit the Avrami exponent $n \pm \text{CI}$, explaining why finite pre-placed seeds give n below the ideal value.

Prerequisites. Non-conserved gradient flows, the Gibbs–Thomson balance, least-squares fitting with an uncertainty; Chapter 3 (free-energy/gradient-flow language and the measure-with-uncertainty habit).

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory; solver cuDSS (documented splu fallback). Quick mode comparable to P1’s measured ≈ 75 s wall (RTX 6000 Ada, mostly kernel compile); reference (four-experiment battery) a few minutes; research (`--level 6`) tens of minutes.

8.1 The model

Let $\psi \in [0, 1]$ be the local crystallinity of a crystallizable species. Its non-conserved gradient flow is

$$\frac{\partial \psi}{\partial t} = -L_\psi \left[\frac{\partial f}{\partial \psi} - \varepsilon^2 \nabla^2 \psi \right], \quad (8.1)$$

with the crystallization free energy (the 2310.11844 “r14” form, PCBM-class energetics)

$$f_{\text{cr}} = \phi[q(\psi) \Delta \sigma + p(\psi) \text{drive}], \quad q = \psi^2(1 - \psi)^2, \quad p = \psi^2(3 - 2\psi), \quad (8.2)$$

and the Turnbull undercooling driving force

$$\text{drive} = \Delta h (T/T_m - 1). \quad (8.3)$$

$q(\psi)$ is a double-well barrier separating amorphous ($\psi = 0$) from crystalline ($\psi = 1$); $p(\psi)$ interpolates the driving; L_ψ sets the growth rate; ε^2 the crystal-interface width.

The melting point sets the sign. $\text{drive} \propto (T/T_m - 1)$ is *negative* for $T < T_m$ (so $\psi \rightarrow 1$ lowers f : the crystal *grows*) and *positive* for $T > T_m$ (the crystal *melts*). $T_m = 558$ K here (materials.yaml, PCBM class). Δh is the crystallization enthalpy, $\Delta\sigma$ the barrier height, L_ψ the kinetic prefactor, ε^2 the gradient penalty (crystal interface width $\sim \sqrt{\varepsilon^2}$).

Measuring crystallinity: quadrature, not a threshold. The primary crystalline content is the *quadrature* average $\langle\psi\rangle = \int \psi dV / \int dV$, evaluated at the assembly Gauss points (`diffsim.diagnostics.conservation`). It counts partial (diffuse-interface) crystallinity honestly. A thresholded area fraction ($\psi > 0.5$) is reported *secondarily*, together with its sensitivity to the cut (0.4/0.5/0.6), because a hard cut discards the interface and depends on where you place it.

8.2 Interface velocity and the critical radius

Below T_m a crystal front advances at a roughly constant velocity v ; we measure it as $v = dr_{\text{eff}}/dt$ with $r_{\text{eff}} = \sqrt{\langle\psi\rangle_{\text{area}}/\pi}$ over the pre-fill window. Because the mobility L_ψ here is *constant* (no thermally-activated hopping), v rises *monotonically* with the undercooling $\Delta T = T_m - T$ (measured $v = 0.007 \rightarrow 0.122$ as $\Delta T = 138 \rightarrow 258$). Real crystallization has a *maximum* at intermediate ΔT because molecular transport freezes out on deep cooling — reproducing that requires an activated $L_\psi(T)$, a labelled modelling extension (Exercise 1), not the constant-mobility physics implemented here.

A curved seed pays interface energy $\sim 2\pi r \sigma$ while gaining bulk energy $\sim \pi r^2 |\text{drive}|$, so $dF/dr = 0$ at a *critical radius*

$$r^* \sim \frac{\sigma}{|\text{drive}|} \propto \frac{1}{|\Delta h (T/T_m - 1)|}. \quad (8.4)$$

Seeds larger than r^* grow; smaller ones redissolve (the Gibbs–Thomson floor). Deeper undercooling raises $|\text{drive}|$ and so *lowers* r^* . We bracket r^* by seeding a range of radii and classifying grow/shrink at two undercoolings: at $T = 333$ K (deep) $r^* \approx 0.045$, at $T = 400$ K (mild) $r^* \approx 0.095$. The measured ratio 2.11 agrees in sense (and order of magnitude) with the Gibbs–Thomson prediction $|\text{drive}_{\text{deep}}|/|\text{drive}_{\text{mild}}| = 1.42$; the gap is honest — finite-time classification, a weakly T -dependent effective σ , and the ϕ -coupling all enter.

8.3 Growth kinetics: the Avrami law

With athermal nuclei placed at $t = 0$ growing at velocity v , the crystalline area of each grain grows like $(vt)^2$ in 2-D before grains impinge, so the crystalline *fraction* follows the Johnson–Mehl–Avrami–Kolmogorov (JMAK) law

$$X(t) = 1 - \exp[-(kt)^n]. \quad (8.5)$$

Because our nuclei are *finite pre-placed* seeds (point seeds are sub-critical and redissolve — exactly the r^* floor above), the film starts with a crystalline baseline X_0 . We remove it with the *generalized-JMAK* conversion $X^* = (X - X_0)/(1 - X_0)$ and fit

$$\ln[-\ln(1 - X^*)] = n \ln t + n \ln k \quad (8.6)$$

by least squares over the pre-impingement window $0.05 < X^* < 0.9$, reporting the window, the number of points, R^2 , and a 95% CI on n . The ideal exponents are $n \approx 3$ for point nuclei appearing

over time and $n \approx 2$ for pre-existing 2-D nuclei; finite pre-placed seeds give an effective n *below* 2 because the seeds start with area and the early t^2 regime is foreshortened.

Accelerated parameters. The Avrami run uses $T = 250$ K and $L_\psi = 11$, which are *pedagogical accelerated* values (not the physical PCBM rate): they let X sweep the full $(0, 1)$ range inside a short run so the double-log fit has a real window. Likewise **orientation θ is a fixed grain label, not an evolved field** (`theta_mode="frozen"`, $\alpha_\theta = \beta_\theta = 0$): there is *no* grain-boundary energy and *no* θ dynamics here. `grain_labels` merely reads the frozen θ plateaus to count crystals. Evolving θ (Kobayashi–Warren–Carter grain boundaries) is the coupled concept of Chapter 9.

8.4 Run it

Run it.

```
cd physics/06_allen_cahn_crystallization
python run.py                # grow/melt + v(dT) + r* + Avrami, then checks
```

It runs the four-experiment battery, writes `outputs/results.json`, and checks it against `baseline.yaml` (tolerance-based). Compare with `EXPECTED.md` and the figures.

What you should see. Below the melting point ($T = 333$ K) the seeded disc *grows* (quadrature crystallinity $0.0681 \rightarrow 0.1168$, a factor 1.72, drive -0.527); above it ($T = 700$ K, drive $+0.333$) it *melts* ($\rightarrow 0.0034$). The growth-front velocity rises with undercooling, the critical radius shrinks with it, and the multi-nucleus run reaches $\langle \psi \rangle_{\text{end}} = 0.820$ ($X_{\psi>0.5} = 0.856$) from 4 nuclei (3 grains resolved), with a baseline-corrected Avrami exponent $n = 1.30 \pm 0.07$ ($R^2 = 0.976$, $X_0 = 0.024$).

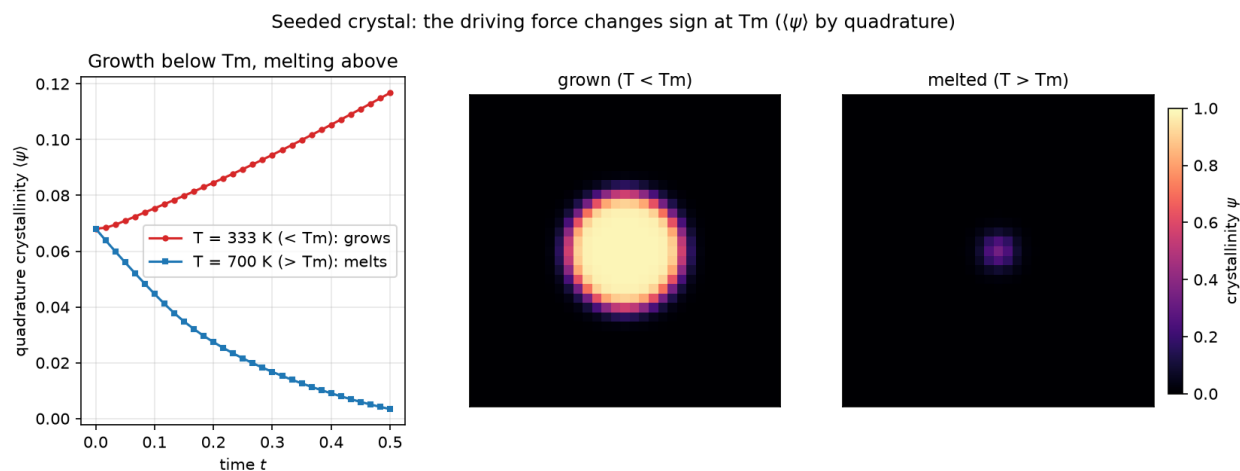


Figure 8.1: A seeded crystal grows below T_m and melts above it (left: quadrature crystallinity $\langle \psi \rangle$ vs time; right: the final ψ fields). The driving force changes sign at the melting point.

Measured self-check (level 5, cuDSS; tolerance-checked in `baseline.yaml`).

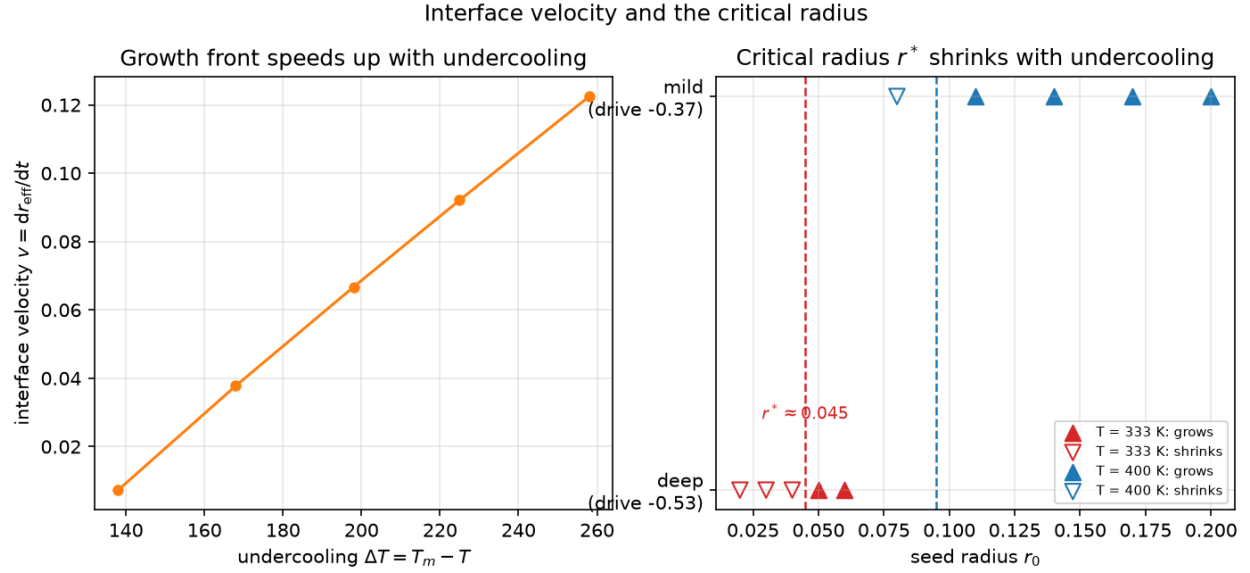


Figure 8.2: Left: the growth-front interface velocity v rises monotonically with undercooling $\Delta T = T_m - T$ (constant mobility — no thermal-transport maximum). Right: the critical radius r^* brackets — seeds above r^* grow ($\triangle$), below it shrink (∇) — and r^* shrinks as the undercooling deepens.

grow ($T < T_m$): $\langle \psi \rangle$ start $\rightarrow$ end	0.0681 $\rightarrow$ 0.1168 (1.72 $\times$)
melt ($T > T_m$): $\langle \psi \rangle$ end	0.0034
critical radius r^* (deep $\rightarrow$ mild)	0.045 $\rightarrow$ 0.095
Avrami exponent n	1.30 ± 0.07 ($R^2 = 0.976$)
crystalline fraction $\langle \psi \rangle_{\text{end}}$	0.820
nuclei $\rightarrow$ grains resolved	4 $\rightarrow$ 3

8.5 Code walk-through

1. The crystallizing stepper. `MultiPhaseStepper(dm, M=1, K=1, ..., bulk="r14")` adds one crystallinity field ψ (and its frozen orientation label θ) to one composition. `dsig`, `dh`, `Tm`, `eps2`, `L_psi` are the coefficients above.

2. Quadrature crystallinity. `quad_mean_psi` evaluates ψ at the Gauss points and forms $\int \psi dV / \int dV$ with the assembly quadrature weights — the primary crystallinity measure.

3. Interface velocity and r^* . `run_interface_velocity` fits $v = dr_{\text{eff}}/dt$; `run_critical_radius` classifies a radius sweep into grow/shrink and brackets r^* .

4. The Avrami fit. `_fit_avrami` baseline-corrects $X \rightarrow X^*$, linearizes over the window, and returns n , k , R^2 and the CI; the grains come from `grain_labels` (a θ -plateau watershed on the frozen label).

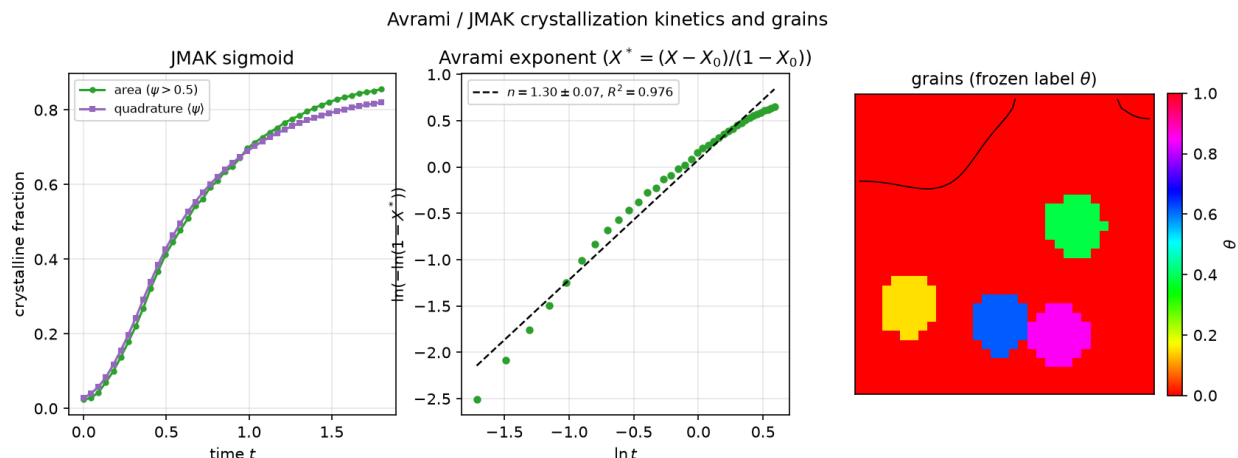


Figure 8.3: Avrami/JMAK kinetics: the sigmoidal crystalline fraction (left, quadrature and thresholded), the baseline-corrected double-log linearization whose slope is the Avrami exponent $n \pm \text{CI}$ with its R^2 (middle), and the grains coloured by the frozen orientation label θ with the crystal outline (right).

8.6 Explore on your own

Q8.6.1 Replace the constant L_ψ with an activated mobility $L_\psi(T) = L_0 \exp[-E_a/T]$. Does $v(\Delta T)$ now develop a maximum at intermediate undercooling (transport-limited on deep cooling)? Where does the peak sit relative to the constant-mobility curve of Figure 8.2?

Q8.6.2 Change the number of pre-placed nuclei and re-fit n (with its CI). Does n stay near its finite-seed value, or move toward $2/3$ as you shrink the seeds toward point nuclei? What does n tell you about the nucleation mode?

Q8.6.3 Sweep ε^2 , measure the crystal interface width, and confirm the $\sqrt{\varepsilon^2}$ scaling. Relate the width to the mesh spacing — when is the crystal interface under-resolved, and how does that bias r^* ?

Q8.6.4 Give two seeds *different* orientation labels θ and let them grow into contact. With the frozen θ here they simply merge into one ψ blob; adding grain-boundary energy (an evolving θ) is the coupled orientation dynamics of Chapter 9.

Q8.6.5 The crystal expels the amorphous component as it grows (the ϕ field). Measure the composition inside vs outside the crystal — this is the crystallization-driven demixing that Chapter 9 makes central.

Chapter 9

Coupled Cahn–Hilliard + Allen–Cahn

Key idea. The real morphology problem is *coupled*: composition (Cahn–Hilliard) and crystallinity (Allen–Cahn) evolve together and feed back. The signature of the coupling is *crystallization-driven demixing* — as a species crystallizes it enriches its own phase and expels the others, sharpening a composition pattern that phase separation alone would not produce.

Learning objectives. By the end you can: explain why crystallinity needs a four-fold $\chi_{aa/ac/ca/cc}$ and convert between the p1 (absolute) and r14 (increment) conventions, verified against `test_chi_limits.py`; design commensurate controls (same mask, same metric, no divide-by-floor) to isolate a coupling; decompose crystallization-driven demixing into its crystal-bulk and χ -expulsion channels; verify the coupled energy budget decreases monotonically; and run causality controls (frozen geometry; fixed coupling, varied kinetics).

Prerequisites. Chapter 8 (crystallinity model, driving force sign); the ternary/ χ extension (Chapter 4); Chapter 3 (free-energy language, measure-with-uncertainty habit).

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory; solver cuDSS (documented splu fallback). Quick mode comparable to P1’s measured ≈ 75 s wall (RTX 6000 Ada, mostly kernel compile); reference (controls + budget + causality) a few minutes; research (`--level 6`) tens of minutes.

9.1 The model: the four-fold χ (p1 vs r14)

With crystallinity, the interaction between two species depends on whether each is amorphous or crystalline, so the single Flory χ becomes *four* matrices $\chi_{aa}, \chi_{ac}, \chi_{ca}, \chi_{cc}$ (with $\chi_{ca} = \chi_{ac}^\top$). **The convention differs between the two bulk free energies**, and getting this right is the crux of the chapter.

This chapter runs the **p1** bulk, whose χ_{eff} is the *mutually-weighted* combination of all four terms:

$$\chi_{\text{eff}}^{\text{p1}}(i, j) = (1 - \psi_i)(1 - \psi_j) \chi_{aa} + (1 - \psi_i)\psi_j \chi_{ac} + \psi_i(1 - \psi_j) \chi_{ca} + \psi_i\psi_j \chi_{cc}. \quad (9.1)$$

At the four *pure* limits this collapses to a single matrix each, so in p1 the four χ are **absolute** pair interactions: $\chi_{\text{eff}}(0, 0) = \chi_{aa}$, $(1, 0) = \chi_{ca}$, $(0, 1) = \chi_{ac}$, $(1, 1) = \chi_{cc}$ (measured here 1.20, 2.60, 2.10, 3.00). By contrast the **r14** bulk uses ψ^2 -weighted *increments* on top of χ_{aa} ,

$$\chi_{\text{eff}}^{\text{r14}}(i, j) = \chi_{aa} + \psi_i^2 \chi_{ca} + \psi_j^2 \chi_{ac} + \psi_i^2 \psi_j^2 \chi_{cc}, \quad (9.2)$$

so there the same crystalline-amorphous input χ_{ca} means an *increment* $\chi_{aa} + \chi_{ca}$ at the pure limit (e.g. 3.80), *not* an absolute value. `test_chi_limits.py` verifies all four pure limits for both conventions against the production evaluator.

Making the crystal-contact interaction *larger* than χ_{aa} means “a crystal of species i dislikes amorphous species j more than two amorphous phases dislike each other” — crystallizing i pushes j out. Together with the crystal-bulk energy (which enriches i where it crystallizes) this is the thermodynamic origin of crystallization-driven demixing.

9.2 Measuring the coupling: commensurate controls

No divide-by-floor. We do *not* compare a crystal run against a differently-masked no-crystal run (that amplified a near-zero denominator). We run **three commensurate controls** — `ch_only` ($K=0$), `coupled_nochi` (crystal present but $\chi_{ca}=\chi_{aa}$), and `full` ($\chi_{ca} > \chi_{aa}$) — and score all three with the *same* mask (the full run’s $\psi > 0.5$ footprint) and the *same* metric (the species-0 composition contrast $\langle \phi_0 \rangle_{\text{in}} - \langle \phi_0 \rangle_{\text{out}}$). We report *absolute* contrasts and the attributable increments; a relative amplification is quoted *only* when the denominator is resolved.

The controls decompose the demixing into two physical channels:

$$\text{crystal-bulk channel} = \text{coupled_nochi} - \text{ch_only} = +0.376, \quad (9.3)$$

$$\chi\text{-expulsion channel} = \text{full} - \text{coupled_nochi} = +0.044. \quad (9.4)$$

On the crystal footprint ordinary Cahn–Hilliard produces essentially no contrast (`ch_only` = -0.000). *Most* of the demixing is the **crystal-bulk** channel — the $\phi_k^2 W(\psi_k)$ term enriches the crystallizing species where it orders, present even with $\chi_{ca}=\chi_{aa}$. The χ **expulsion** adds a further, smaller increment (full = $+0.420$; relative to `coupled_nochi` that is $1.12\times$, denominator resolved). The contrast is reproducible across seeds (Sec. 9.5).

9.3 Coupled energy budget

The coupled film is a gradient flow of a single free energy $F = F_{\text{entropy}} + F_{\chi} + F_{\text{cryst}} + F_{\text{grad } \phi} + F_{\text{grad } \psi}$, evaluated by quadrature with the exact production densities (`coupled_energy`). Over the quench the total decreases monotonically ($F : -0.6592 \rightarrow -0.6708$, largest positive step $0.0e+00$) — the coupling redistributes energy between the mixing, χ and crystal terms but cannot increase the total (Fig. 9.2, left).

9.4 Causality

Correlation is not cause; we perturb the coupling to check its direction. **Frozen geometry:** freeze the crystal ($L_{\psi}=0$) and run the composition with vs without the χ coupling — the coupling still adds demixing at *fixed* geometry (increment $+0.190$), so the coupling, not the *act* of crystallizing, drives it. **Kinetics at fixed coupling:** a fast ($L_{\psi}=6$) and a slow ($L_{\psi}=1$) run both reach a large contrast (0.420 vs 0.371) — the *coupling* sets the demixing magnitude while the kinetic rate sets only the timing.

9.5 Run it

Run it.

```
cd physics/07_coupled_ch_ac
python test_chi_limits.py    # the four-fold chi convention (p1 & r14)
python run.py                # controls + budget + causality + checks
```

`run.py` runs the three commensurate controls, the seed-sensitivity sweep, the coupled energy budget and the causality experiments, writes `outputs/results.json`, and checks it against `baseline.yaml` (tolerance-based). Compare with `EXPECTED.md` and the figures.

What you should see. On the shared crystal-footprint mask the composition contrast is -0.000 (CH only), $+0.376$ (crystal-bulk channel) and $+0.420$ (full) — so crystallization-driven demixing is dominated by the crystal-bulk channel with a smaller χ -expulsion increment. The total free energy decreases monotonically, and the causality experiments confirm the coupling (not the kinetics) causes the demixing.

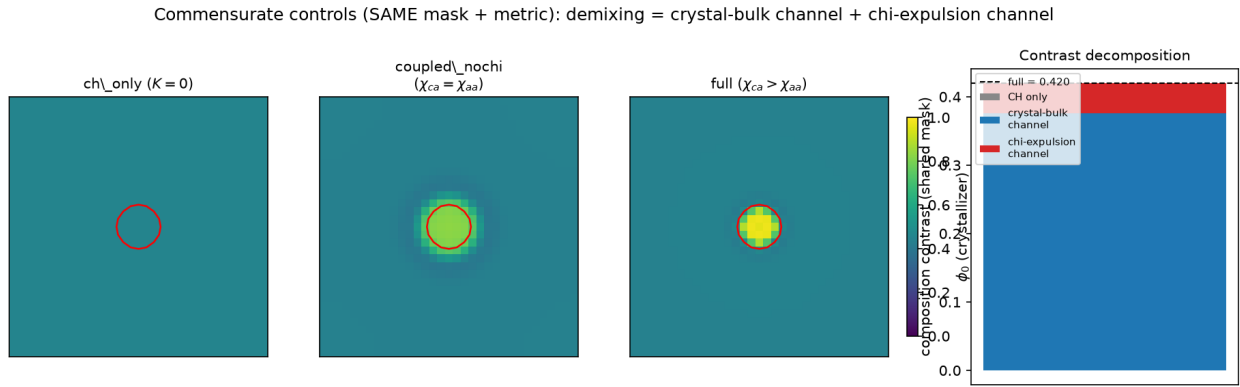


Figure 9.1: Three commensurate controls scored on the *same* mask (the full crystal footprint, red outline) with the *same* contrast metric: `ch_only`, `coupled_nochi` and full ϕ_0 fields, and the contrast decomposition into the crystal-bulk and χ -expulsion channels (right).

Measured self-check (level 5, cuDSS; tolerance-checked in `baseline.yaml`).

χ_{eff} p1 pure limits <i>aa/ca/ac/cc</i>	1.20/2.60/2.10/3.00
contrast: <code>ch_only</code> / <code>coupled_nochi</code> / full	-0.000 / $+0.376$ / $+0.420$
crystal-bulk / χ -expulsion channel	$+0.376$ / $+0.044$
total F (start $\rightarrow$ end)	$-0.6592 \rightarrow -0.6708$
frozen-geometry χ -channel	$+0.190$

9.6 Code walk-through

1. Four χ matrices + the convention test. `_chis` builds χ_{aa} (all pairs) and the crystal-contact χ_{ca} ; `chi_eff_limits` and `test_chi_limits.py` pin the p1 (absolute) vs r14 (increment) pure limits against the production evaluator `_np_chi_eff`.

2. Commensurate controls. The `run` entry point takes mode one of `ch_only`, `coupled_nochi`, or `full`; the `contrast_on_mask` routine scores every control on the one shared mask.

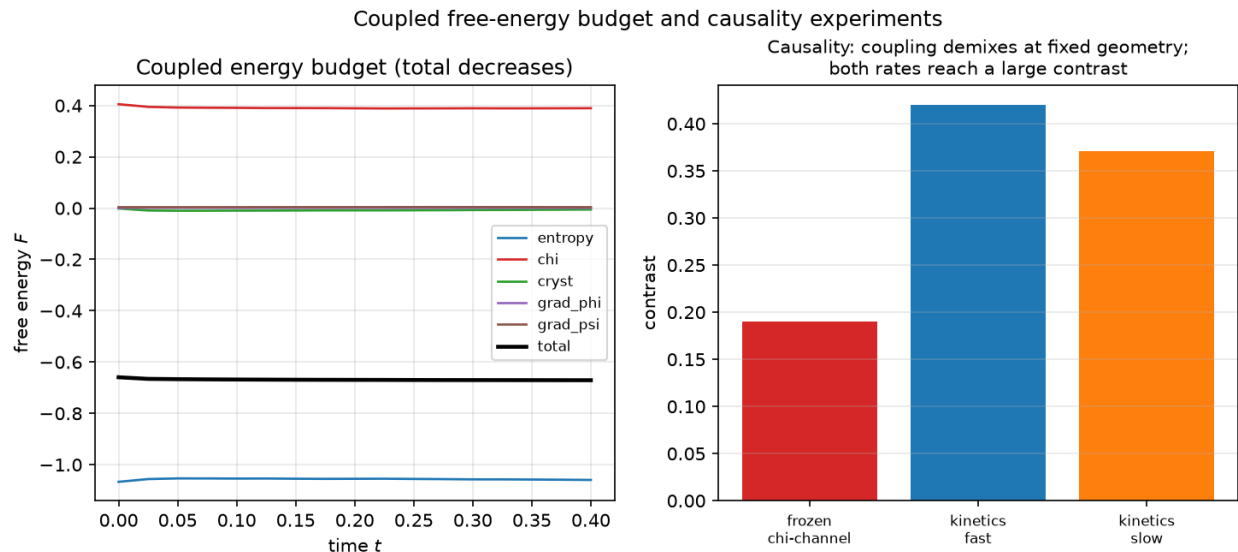


Figure 9.2: Left: the coupled free-energy budget — entropy, χ , crystal and gradient terms, whose *total* decreases monotonically. Right: causality — the χ coupling demixes at frozen geometry, and fast/slow crystallization both reach a large contrast (coupling sets the magnitude, kinetics the timing).

3. Energy + causality. `coupled_energy` integrates the exact free-energy split; `freeze_psi=True` ($L_\psi=0$) gives the frozen-geometry control, and varying `L_psi` gives the kinetics control.

9.7 Explore on your own

Q9.7.1 Sweep χ_{ca} from χ_{aa} upward. How does the χ -expulsion channel (full — `coupled_nochi`) scale with $\chi_{ca} - \chi_{aa}$, and does it ever overtake the crystal-bulk channel?

Q9.7.2 Switch the bulk to `r14`. Re-derive the pure-limit χ you must input to match the `p1 absolute` interactions of this chapter (hint: subtract χ_{aa}). Verify with `test_chi_limits.py`.

Q9.7.3 Turn crystallization on *after* the blend has partly demixed (delay the seed). Does the crystal template on the existing composition pattern or reshape it? Relate to processing order (phase-separate then crystallize vs simultaneously).

Q9.7.4 At $(M, K) = (3, 2)$ two species crystallize. Do their crystals compete for the same regions or segregate? Re-run the commensurate controls and decompose each crystallizer's demixing.

Q9.7.5 The contrast is a purity measure. Relate it to what a solar cell wants (pure domains for transport, a bicontinuous network for charge separation). Is more contrast always better?

Chapter 10

Thermal noise and nucleation

Key idea. Where do the crystal seeds of Chapter 8 come from? From thermal *fluctuations*. An undercooled melt is *metastable*: $\psi = 0$ sits behind a free-energy barrier, and the crystal cannot form until a fluctuation pushes a small region over that barrier — classical nucleation. Here the noise is *physics*, and its amplitude ($k_B T$) sets the nucleation rate. Getting the DISCRETE fluctuation–dissipation normalization right — and *verifying* it numerically — is the heart of this chapter.

Learning objectives. By the end you can: derive the discrete FDT per-Gauss-point noise normalization $\sqrt{2L/(\Delta t w^J)}$ and explain why the $1/\Delta t$ and $1/w^J$ factors are both required for mesh/ Δt -independence; verify equipartition (variance Δt -independent, $\propto 1/V_{\text{cell}}$) in a clip-free stable well; report nucleation as an **ensemble** (probability, X distribution, nuclei density, induction time with a 95% CI), not a single seed; and state precisely why this is a noise-amplitude sweep, not a temperature sweep.

Prerequisites. Chapter 8 (crystallization model, drive sign); fluctuation–dissipation theorems and equipartition; ensemble statistics as in Chapter 3’s seed discussion (here the seed is *physical*, not numerical).

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory. **Solver split:** the FDT-well verification runs on a *coarse CPU/splu* mesh (fast, no cuDSS re-plan); the nucleation *ensemble* runs at level 5 on the *GPU*. **BDF1 only** (the brick asserts noise off under BDF2). Quick mode comparable to P1’s measured ≈ 75 s wall; the ensemble multiplies wall time by the realization count.

10.1 The model and the discrete FDT normalization

We add the physical (fluctuation–dissipation) noise to the Allen–Cahn crystallinity equation:

$$\frac{\partial \psi}{\partial t} = -L_\psi \left[\frac{\partial f}{\partial \psi} - \varepsilon^2 \nabla^2 \psi \right] + \xi, \quad \langle \xi(\mathbf{x}, t) \xi(\mathbf{x}', t') \rangle = 2L_\psi k_B T \delta(\mathbf{x} - \mathbf{x}') \delta(t - t'). \quad (10.1)$$

On the finite-element mesh the load-only forcing is drawn per Gauss point with standard deviation

$$\text{std}(\xi_{gp}) = \text{noise_psi} \sqrt{\frac{2L_\psi}{\Delta t w_{gp}^J}}, \quad \text{noise_psi}^2 = k_B T, \quad (10.2)$$

where w_{gp}^J is the Gauss-point quadrature weight. The two factors are not cosmetic: the $1/\Delta t$ makes the white-noise time increment consistent, and the $1/w_{gp}^J$ is the *spatial* white-noise quadrature normalization, so the assembled nodal force $b_a = \int N_a \xi \, dV$ has variance

$$\text{Var}(b_a) = 2 \text{noise_psi}^2 L_\psi \int N_a^2 \, dV / \Delta t \quad (10.3)$$

exactly — independent of how finely the mesh samples space. A mesh-blind noise would make this scale with the element size and the equilibrium fluctuations would *not* converge under refinement.

Central verification (not a footnote). Fix the physical noise $k_B T$, put ψ in a STABLE quadratic well (a superheated, clip-free *r14* well so fluctuations are two-sided and un-rectified), and measure the equilibrium spatial variance $\langle \text{Var } \psi \rangle$. Equipartition predicts a value set by $k_B T$ and the well/mesh, *independent* of Δt . Refining Δt the measured variance converges to a Δt -independent plateau (measured coefficient of variation across Δt : 0.022); refining the mesh, $\text{Var } \psi \cdot V_{\text{cell}}$ stays constant (CoV 0.111, i.e. $\text{Var } \psi \propto 1/V_{\text{cell}}$ as equipartition demands). The physical statistic converges — the Eq. (10.2) normalization is correct.

10.2 Nucleation as an ensemble

Nucleation is a random, rare-event, barrier-crossing process (rate $\sim \exp(-\Delta F^*/k_B T)$), so a single seed is one *sample*, not a result. We run several noise realizations per amplitude and report, via `diffsim.diagnostics.stochastic`, the nucleation *probability*, the crystalline-fraction distribution, the nuclei *density*, and the induction time (a first-passage crossing of $X = 0.02$), each as a mean with a 95% interval.

Noise-amplitude sweep, NOT a temperature sweep. We vary *only* `noise_psi` ($= \sqrt{k_B T}$ in the forcing) at *fixed* undercooling T/T_m . A genuine temperature change would also move the driving force and the barrier (drive $= \Delta h(1 - T/T_m)$, the entropy, the mobility); read this as a noise-amplitude / FDT-calibration sensitivity. **Clipping is quantified:** the undercooled runs clip $\psi \in [0, 1]$, and we report the saturated ($\psi > 0.99$) fraction; the $\psi = 0$ floor rectifies sub-barrier fluctuations — exactly why the quantitative FDT test above uses the clip-free well. (BDF1 only: the brick asserts noise off under BDF2.)

10.3 Run it

Run it.

```
cd physics/08_noise_nucleation
python run.py                # FDT verification + nucleation ensemble +
                             checks
```

It runs the central FDT verification and the noise-amplitude ensemble sweep, writes `outputs/results.json`, and checks it against `baseline.yaml`. Compare with `EXPECTED.md` and the figures.

What you should see. The equilibrium variance is Δt -independent (CoV 0.022) and scales as $1/V_{\text{cell}}$ (CoV 0.111) — the FDT normalization checks out. With zero noise the melt stays amorphous ($X = 0.000$: no nucleation). At the largest amplitude (`noise_psi` = 0.06) the ensemble (4 seeds)

nucleates with probability 1.00, reaching $X = 0.505 \pm 0.011$ in 19.2 nuclei (a saturated fraction 0.205). More noise nucleates more grains.

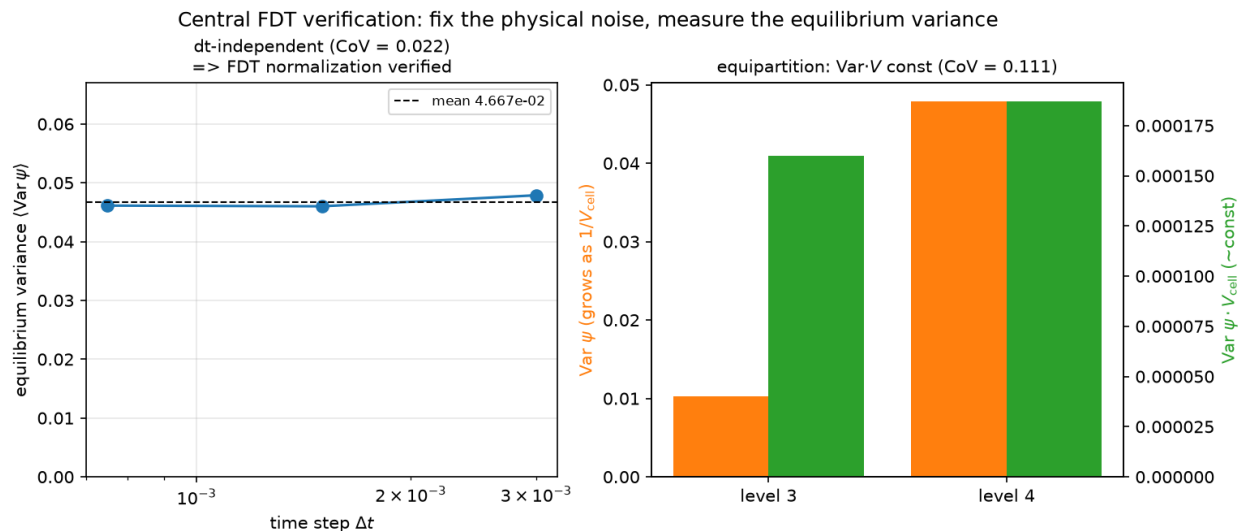


Figure 10.1: The central FDT verification. Left: the equilibrium variance is Δt -independent (a wrong $1/\Delta t$ normalization would tilt the line). Right: across meshes $\text{Var } \psi$ grows as $1/V_{\text{cell}}$ while $\text{Var } \psi \cdot V_{\text{cell}}$ stays constant — equipartition.

Measured self-check (tolerance-checked in `baseline.yaml1`).

FDT variance Δt -CoV / mesh-CoV	0.022 / 0.111
zero-noise X	0.000
X at <code>noise_psi</code> = 0.06 (4 seeds)	0.505 ± 0.011
nuclei density / nucleation prob.	19.2 / 1.00

10.4 Code walk-through

1. The FDT verification. `verify_fdt` builds a stable clip-free $r14$ well on a coarse CPU/splu mesh, sweeps Δt (and the mesh), and returns the variance CoVs — the Eq. (10.2) check.

2. Turn on the FDT noise. `MultiPhaseStepper(..., timestep="bdf1", noise_psi=..., noise_seed=...)`; `noise_psi` is $\sqrt{k_B T}$, `noise_seed` fixes the realization.

3. The ensemble sweep. `run_ensemble` runs many seeds per amplitude and aggregates (probability, X , nuclei density, first-passage induction) with CIs via `diffsim.diagnostics.stochastic`; the saturated- ψ fraction quantifies the clipping.

10.5 Explore on your own

Q10.5.1 The nucleation rate should scale like $\exp(-\Delta F^*/k_B T)$. Lower the amplitudes toward the onset where the nucleation *probability* drops below one; plot $\ln(\text{nuclei density})$ against

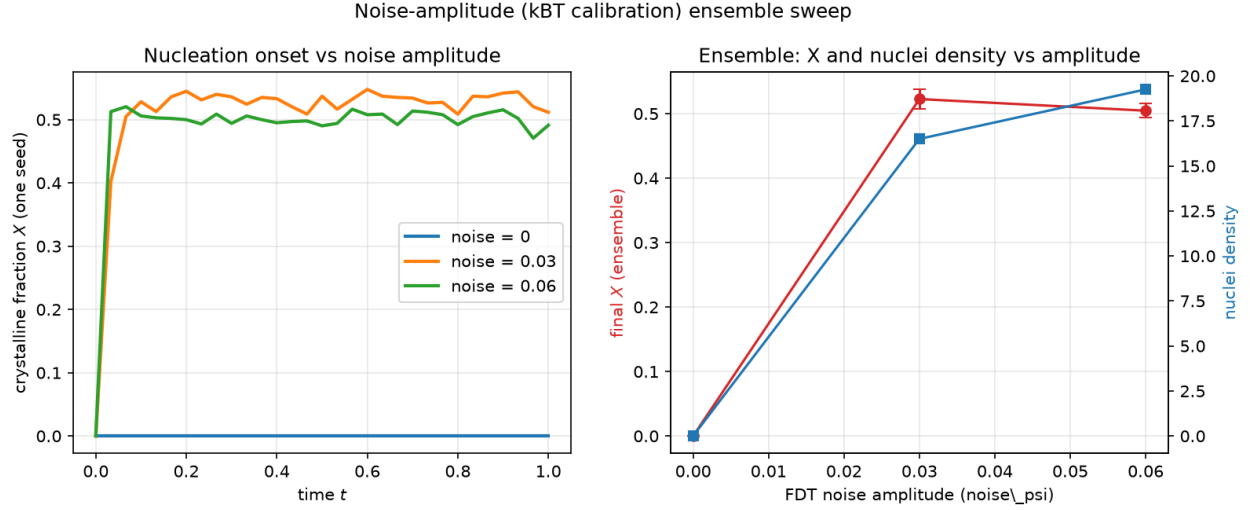


Figure 10.2: The noise-amplitude ensemble sweep. Left: $X(t)$ for a representative seed at each amplitude (zero noise never nucleates). Right: the ensemble final X (mean $\pm$ sd) and the nuclei density vs amplitude.

$1/\text{noise_psi}^2$ and estimate the barrier from the slope.

Q10.5.2 Re-run `verify_fdt` with the $1/w^J$ factor removed from Eq. (10.2) (a mesh-blind noise). Does the equilibrium variance now **DEPEND** on the mesh (fail the equipartition check)? This is the point of the normalization.

Q10.5.3 Deepen the undercooling (lower T) at fixed noise. Does the induction-time distribution shift earlier? Relate to how ΔF^* depends on the driving force. (Remember: that IS a temperature change — it moves the barrier, unlike the noise-amplitude sweep.)

Q10.5.4 Change only the `noise_seed`. Which is reproducible — the grain *positions*, or the ensemble statistics? Connect to the seed discussion of Chapter 3: here the seed is physical, so a different seed is a different valid sample.

Q10.5.5 Quantify the clipping vs amplitude. At what noise level does the $\psi = 0$ floor start to bias the pre-nucleation statistics, and how would you correct for it (hint: the clip-free well)?

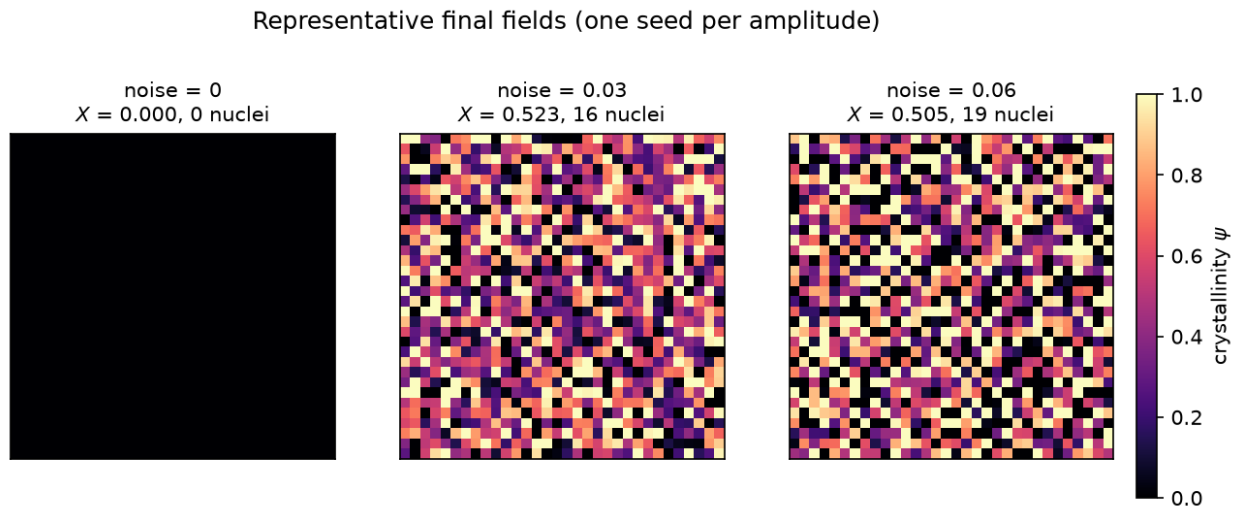


Figure 10.3: Representative final crystallinity fields (one seed per amplitude): more noise nucleates more grains at fixed undercooling.

Chapter 11

Evaporation-conditioned embryo growth

Key idea. This is the whole story, at tutorial scale: a *wet* film *dries* (Ch. 7), the concentrating blend *phase-separates* (Ch. 3–6), and once the crystallizable species is concentrated enough an *implanted* crystal embryo *grows* (Ch. 8) coupled back to composition (Ch. 9). The arc *wet film* $\rightarrow$ *phase separation* $\rightarrow$ *embryo growth* $\rightarrow$ *crystalline film* is how a real solution-cast organic solar cell forms.

Naming: embryo growth, not nucleation. The default runs do *not* nucleate crystals from thermal noise — they *implant* a supercritical embryo and ask whether the drying-conditioned local composition lets it *grow* or forces it to *dissolve*. So this is *evaporation-conditioned embryo growth*. Genuine noise-driven nucleation (Ch. 10) is available as an advanced mode (`make_stepper(noise_psi=...)`); it is slower and stochastic, so the checked tutorial uses the deterministic implant.

Learning objectives. By the end you can: derive the solubility $\phi^* = 1 - |\text{drive}|/\chi_{ca}$ from the homogeneous free energy and explain why it is an *upper bound*; validate ϕ^* against an independent embryo-composition sweep and interpret why the measured crossover sits below it; explain why the default runs *implant* rather than nucleate; run and interpret the wet-dissolves/dry-grows arc; and distinguish a sub- from a supercritical embryo at fixed composition (the Gibbs–Thomson floor), reading the honest **TERMINATION** status of a march.

Prerequisites. Chapter 7 (evaporation/drying); Chapter 9 (coupled CH+AC, the crystal-bulk self-enrichment channel); Chapter 8 (crystallization model, Gibbs–Thomson radius); the χ /ternary-blend language of Chapter 4.

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory; solver cuDSS (documented `splu` fallback); *tutorial simplification* — constant Onsager mobility with cuDSS, not the production Vignes mobility. Quick mode comparable to P1’s measured ≈ 75 s wall; reference (solubility + sweep + control + arc) a few minutes; research (`--level 6`) tens of minutes.

11.1 The solubility, derived from the free energy

We run $(M, K) = (2, 1)$ in film mode: a crystallizable small molecule (species 0), a polymer (species 1), and an evaporating solvent. The r14 crystallization bulk carries a crystal-contact penalty χ_{ca} . Compare the homogeneous free energy at $\psi = 1$ versus $\psi = 0$ at fixed composition: the ψ -dependent part changes by

$$\Delta f = \phi_0 \text{ drive} + \phi_0 \chi_{ca} (1 - \phi_0), \quad \text{drive} = \Delta h (T/T_m - 1) < 0, \quad (11.1)$$

where the bulk drive lowers the crystal energy and the crystal-contact penalty against the $(1 - \phi_0)$ non-crystallizing surroundings raises it. The crystal is favoured ($\Delta f < 0$) iff

$$\phi_0 > \phi^* = 1 - \frac{|\text{drive}|}{\chi_{ca}}. \quad (11.2)$$

This is a genuine *solubility*: below the local small-molecule fraction ϕ^* an embryo redissolves; above it, it grows. For the tutorial energetics (drive = -0.527 , $\chi_{ca} = 1.6$) this gives the *homogeneous* solubility $\phi^* = 0.671$.

Validated, with an honest caveat. `embryo_composition_sweep` implants the *same* supercritical embryo into uniform blends of increasing ϕ_0 with *no drying*, and brackets the grow/dissolve crossover in $[0.30, 0.45]$. The measured crossover lies *below* the derived $\phi^* = 0.671$: ϕ^* is the *homogeneous* solubility, but a supercritical embryo *enriches* ϕ_0 in its neighbourhood as it orders (the Chapter 9 crystal-bulk channel), lowering its effective growth threshold. So the derivation is an *upper bound*, and the sweep confirms a composition threshold exists while lying below ϕ^* — the honest, physically-grounded result.

Seeding is physical and delicate (the S3b campaign lessons). The implanted embryo radius r_0 must clear the Gibbs–Thomson critical radius r^* with margin (sub-critical embryos redissolve), and the amplitude must be near 1. At a fixed super-solubility composition, a large embryo ($r_0 = 0.20$) grows but a small one ($r_0 = 0.05$) redissolves — the critical radius, not just the composition. We read the *terminal* crystalline state, not a mid-growth snapshot.

11.2 The arc: wet dissolves, dry grows

Drying *raises* the local ϕ_0 , so the *same* embryo behaves oppositely depending on *when* it is introduced:

- implanted in the **wet** film ($\phi_s = 0.836$, local ϕ_0 below ϕ^*) it *dissolves* (terminal area 0.000, $\psi_{\max} = 0.00$);
- implanted **mid-drying** ($\phi_s = 0.633$, ϕ_0 above ϕ^*) it *grows* to a crystalline film (terminal area 1.000, $\psi_{\max} = 0.99$).

The fate is set by the drying-*conditioned local composition* relative to ϕ^* — not by time or evaporation *per se*, which is exactly what the no-drying composition sweep isolates.

Termination, honestly. A march that stops at the requested time horizon is a `time_horizon` stop — *not* a “drying time”. The honest status enum is `time_horizon / solvent_target` (dryness) / `height_floor / min_dt` (Newton/stiffness) / `step_budget`; the wet and dry legs terminate `[solvent_target]` and `[solvent_target]`. **Recorded simplification:** constant

Onsager mobility with cuDSS instead of the production Vignes composition-singular mobility (tests/test_multiphase_s3.py); the qualitative arc is unchanged, the drying-front sharpness softened.

11.3 Run it

Run it.

```
cd physics/09_evaporation_crystallization
python run.py # solubility + controls + arc + checks
```

It derives ϕ^* , validates it by the composition sweep, runs the sub/supercritical embryo control and the wet-vs-dry arc, and checks outputs/results.json against baseline.yaml. Compare with EXPECTED.md and the figures.

Evaporation-conditioned embryo growth: wet film -> phase separation -> embryo growth -> crystalline film

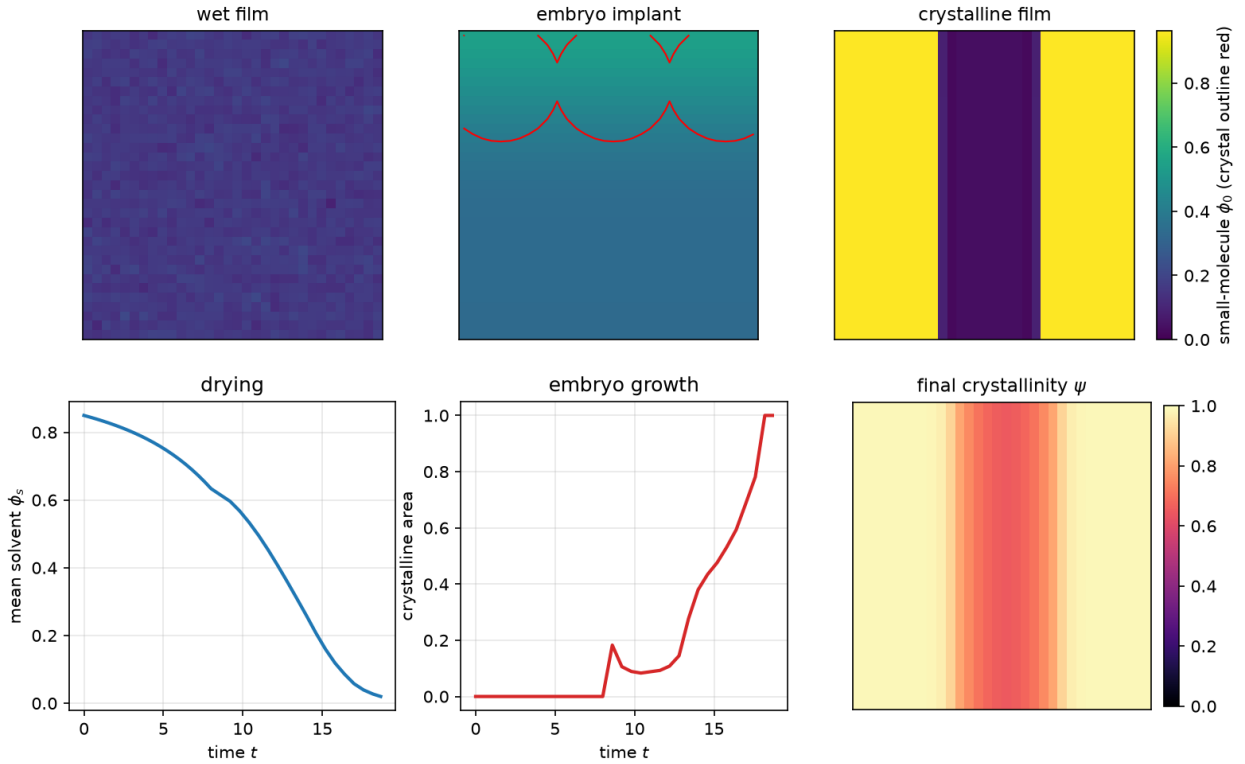


Figure 11.1: The arc (dry-implant run): top — wet film, embryo implant, and the final crystalline film (ϕ_0 with crystal outline in red); bottom — the drying curve $\phi_s(t)$, the crystalline-area growth, and the final crystallinity ψ .

Measured self-check (level 5, cuDSS; tolerance-checked in baseline.yaml).

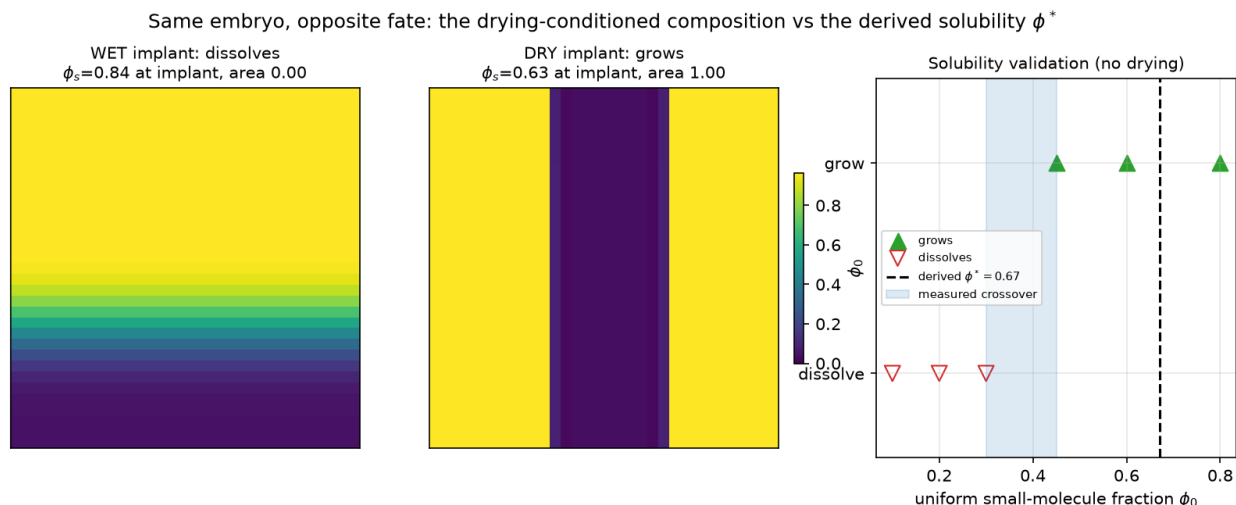


Figure 11.2: Left/middle: the same embryo, implanted wet (dissolves) vs mid-drying (grows). Right: the solubility validation — implanting into uniform blends of varying ϕ_0 (no drying) brackets the derived ϕ^* .

	wet implant	dry implant
ϕ_s at implant	0.836	0.633
crystalline area (terminal)	0.000	1.000
$\psi_{\max}$ (terminal)	0.00	0.99
derived $\phi^* = 0.671$; measured crossover $[0.30, 0.45]$.		

11.4 Code walk-through

1. The derived solubility. `solubility_threshold` returns $\phi^* = 1 - |\text{drive}|/\chi_{ca}$ from the r14 free energy; `embryo_composition_sweep` validates it with no-drying static-blend implants.

2. The drying, crystallizing film. `make_stepper` builds the (2,1) film-mode stepper (r14, cuDSS) and asserts the crystal energetics reached it — the missing-splat guard from the S3b campaign (a dropped keyword silently defaulted $T_m = 1$, a huge *melting* force).

3. Implant, march, read the terminal state. `run_arc` marches the drying film to the implant time, calls `implant`, marches on, and reports the terminal area, $\psi_{\max}$ and the honest termination status.

11.5 Explore on your own

Q11.5.1 Sweep the implant time from wet to dry. Is there a sharp crossover in ϕ_s between dissolution and growth, and does the LOCAL ϕ_0 at that crossover match the derived ϕ^* ?

Q11.5.2 Vary χ_{ca} and re-derive ϕ^* . Re-run the composition sweep — does the measured crossover track $1 - |\text{drive}|/\chi_{ca}$? Where does ϕ^* hit 0 (no solubility barrier: the embryo grows even wet)?

Q11.5.3 Vary the implant radius r_0 around the critical radius at a super-solubility composition. Find the margin below which even a mid-drying embryo redissolves — the Gibbs–Thomson r^* .

Q11.5.4 Switch on the advanced FDT-noise mode (`make_stepper(noise_psi=...)`) and let the drying film nucleate on its own. Does crystallization still wait for the solvent to leave? What sets the number of crystals?

Q11.5.5 This is the design problem: given a target (fine, pure, bicontinuous crystalline domains), which knobs — k_e , χ_{ca} , undercooling, blend ratio — would you turn, and which fight each other? (The Differentiable track answers this with gradients.)

Chapter 12

Material-system reproduction (capstone)

Key idea. This is the “put it all together on a real system” capstone. You pick ONE documented blend from the materials database, load *every* parameter through the validated loader (never hand-copied), reproduce a published morphology trend, and then *stress* that conclusion — mesh convergence, time convergence, and a parameter-uncertainty sweep — so you can state honestly which conclusions are ROBUST and which are artefacts of the tutorial scale.

Learning objectives. Load a real material with its full provenance and audit which numbers are measured, fitted, or assumed; reproduce a rate-dependent morphology trend on the de-confounded matched-dryness axis; run mesh and time convergence on a production system; propagate an order-of-magnitude parameter uncertainty; and separate robust physics from tutorial-scale noise.

Prerequisites. Chapter 7 (the drying film and matched terminal state), Chapter 6 (ternary Flory–Huggins), and Chapter 00 (environment green).

Expected cost. Quick mode < 2 min (a two-rung smoke on a 16×16 mesh); reference mode a few to ~ 20 min (4-rung ladder plus one finer mesh, two time steps, and two chi values on a 32×32 mesh); research mode a 64×64 ladder with a 128×128 convergence rung.

12.1 The system and its provenance

We reproduce a trend for **PDPP5T:PCBM**, a diketopyrrolopyrrole donor blended with a PCBM-class acceptor and spin-coated from chloroform, after the drying study of Negi et al. (recorded in `materials.yaml`). It is the natural choice here because it carries a concrete Flory interaction triple, concrete degrees of polymerization, AND an explicit spin-speed $\rightarrow$ Biot ladder — literally “the real chi, N , and Biot.” Every parameter is loaded through `materials/loader.py`; the run archives the resolved values and their provenance to `materials.resolved.json`.

The capstone is built around an honest audit: which of these numbers can you trust? The loader exposes each parameter’s *status*, *source*, and *uncertainty*. For PDPP5T:PCBM every parameter is `accelerated_tutorial` / `fitted` — *representative* values, NOT measured table entries — and the polymer–fullerene interaction $\chi_{pf} = 1$ carries an *order-of-magnitude* uncertainty of 0.5. Fig. 12.1

is that audit, generated directly from the loaded provenance.

P10 provenance audit — PDPP5T_PCBM (loaded via the validated materials loader)
every parameter is accelerated_tutorial / fitted — representative, NOT measured table values

parameter	status	source	uncertainty	value known?
chi_polymer_fullerene	accelerated_tutorial	fitted	order_of_magnitude 0.5	yes
chi_polymer_solvent	accelerated_tutorial	fitted	order_of_magnitude 0.1	yes
chi_fullerene_solvent	accelerated_tutorial	fitted	order_of_magnitude 0.3	yes
N	accelerated_tutorial	fitted	range [50, 200]	yes
biot	accelerated_tutorial	fitted	order_of_magnitude 0.00	yes
diffusivity_ratio	accelerated_tutorial	fitted	order_of_magnitude	yes
noise_b_reg	accelerated_tutorial	assumed	none	yes

Figure 12.1: Provenance audit of every loaded PDPP5T:PCBM parameter: status (production vs. accelerated-tutorial), source type (measured / fitted / assumed), uncertainty, and whether the value is even known. This governs how much weight any downstream conclusion can bear.

An honest negative on the archetype. The archetypal organic-solar-cell blend P3HT:PCBM is the more famous evaporation-morphology benchmark, but at its recorded weak $\chi_{pf} \approx 0.86$ with a large polymer N it barely demixes at tutorial scale (almost no lateral contrast within the horizon). We document that as a genuine finding and choose PDPP5T:PCBM, which demixes robustly, so the trend study has signal to measure.

12.2 The reproduced trend

Negi et al. vary the *spin speed*, which sets the drying *rate*: higher rpm removes solvent faster. Evaporation is the clock of morphology formation (Chapter 7): faster drying gives the blend less time to phase-separate before it is quenched, so the classical expectation is a *finer* frozen morphology at higher rpm. The database records the spin speed as a Biot ladder Bi(rpm).

Reproducing only what is reproducible. Two facts about those recorded Biot numbers govern how we use them, and both are honoured in the run provenance. First, their absolute magnitudes are illustrative and carry an order-of-magnitude uncertainty, so neither the absolute Biot nor even the exact ratio between rungs is meaningful — only the *ordering* (higher rpm = faster drying) is. Second, as recorded they are $\sim 10^{-3}$, far too small to dry a film within the tutorial horizon. We therefore reproduce the physically robust content — the monotone spin-speed $\rightarrow$ drying-rate ordering — by mapping the 4 recorded rungs, in rank order, onto a moderate, resolvable drying-rate window in k_e (the Chapter 7 drying regime, $\text{Bi} \in [1.5, 3]$). Morphology is then read at MATCHED dryness — the same mean solvent fraction ϕ_s for every rung — so drying RATE is never confounded with final STATE.

Run it.

```
cd physics/10_material_case_study
python run_harness.py --config configs/p10.yaml --mode reference \
  --output outputs/p10 --overwrite
python gen_figures.py --run-dir outputs/p10 # figures + numbers
```

The harness loads PDPP5T:PCBM through the validated loader, marches the 4-rung drying ladder, reads morphology at matched dryness, runs the mesh + time convergence and the χ_{pf} sensitivity sweep, and gates results.json against baseline.yaml.

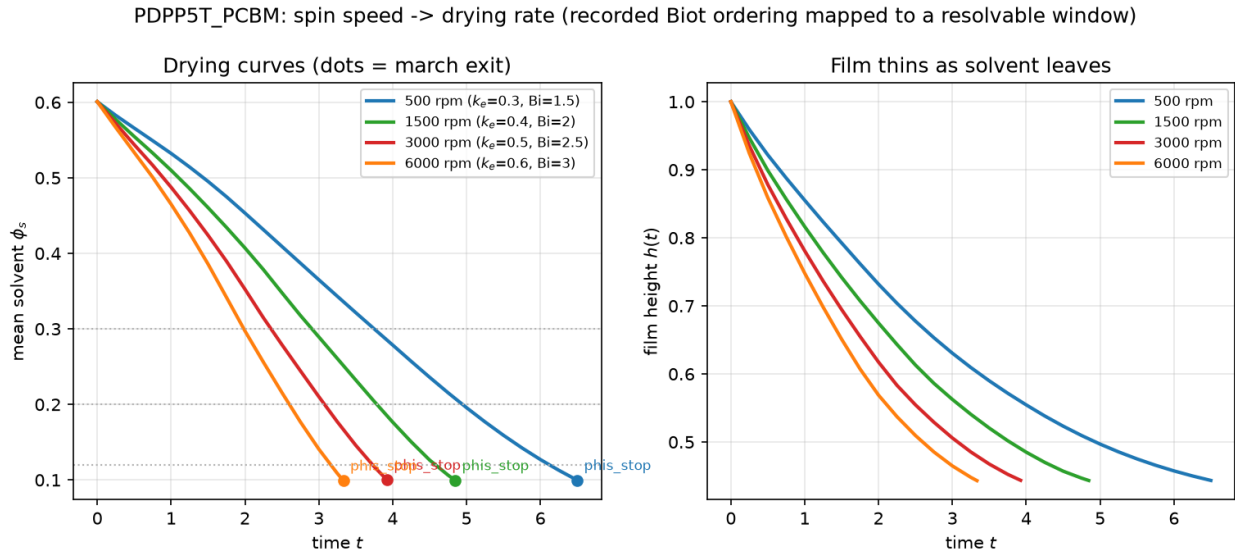


Figure 12.2: Drying curves $\phi_s(t)$ and film height $h(t)$ for the spin-speed ladder. Every rung is driven past the smallest matched dryness (dotted lines); dots mark the march exit, so a truncated run is never mistaken for a dried one.

What you should see. At matched dryness the “faster = finer” wavelength ordering is *weak and noisy* at tutorial scale (single seed, only a few lateral domains): the slowest rung (500 rpm) and fastest rung (6000 rpm) give matched- ϕ_s wavelengths of 18.1 and 18.3 cells, and whether the faster film is strictly finer comes out as “no.” What *is* robust is visible in Fig. 12.3: the DEGREE of demixing — the phase contrast — is set by DRYNESS. It rises monotonically as the film dries (contrast 0.282 $\rightarrow$ 0.339 $\rightarrow$ 0.384 from wet to dry on the slow rung) and is nearly rate-independent at matched dryness (spread 1.6% across the whole ladder). The matched-state wavelength collapses onto the Biot number to within 1.6%.

12.3 Mesh and time convergence

A reference case is only worth maintaining if its conclusions survive refinement. The harness repeats one ladder rung on a finer mesh and at smaller time steps and reports the matched-dryness metrics in mesh-independent form — the wavelength as a fraction of the box width, and the phase contrast. Fig. 12.5 shows the demixing-degree conclusion is robust: the phase contrast changes by only 1.4%

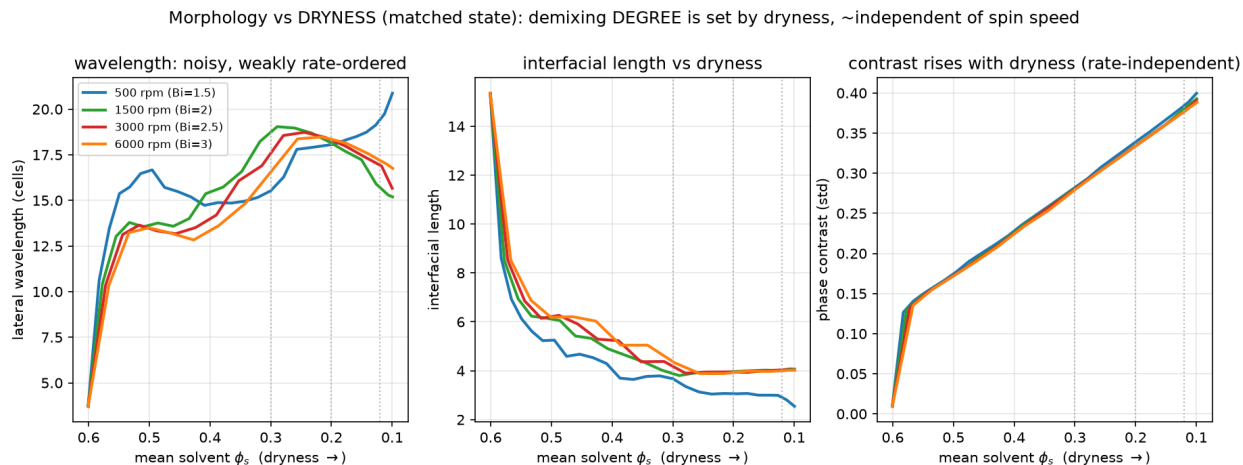


Figure 12.3: Morphology metrics as a continuous function of dryness ϕ_s (the de-confounded axis). Wavelength (left) is Biot-ordered but noisy; interfacial length (centre) and phase contrast (right) rise cleanly with dryness and are nearly independent of spin speed at matched ϕ_s .

between the coarse and fine mesh and by 0.1% between the largest and smallest time step (the box-fraction wavelength moves 7.2% under mesh refinement, consistent with its noisier character).

A conservation caveat, stated honestly. With a large polymer $N = 87$ the polymer segregates into a sharp demixed surface layer that the 32×32 reference mesh only partly resolves, so the per-solute moving-frame balance for the polymer is 1.8×10^{-4} (relative) — looser than the fullerene’s machine-precision $< 10^{-13}$ — and it tightens as the mesh refines. The fullerene conservation is the hard, resolution-robust gate; the polymer residual is reported and tracked, not hidden.

12.4 Parameter-uncertainty sensitivity

The recorded χ_{pf} carries an order-of-magnitude uncertainty, so a robust conclusion must survive changing it by a factor of a few. The harness sweeps χ_{pf} across that band at a fixed rung and dryness (Fig. 12.6). The demixing DEGREE tracks χ_{pf} cleanly and monotonically (monotone: yes; contrast spans 21.2% across the band), exactly as Flory–Huggins predicts — a stronger enthalpic penalty demixes harder. The wavelength, by contrast, stays noisy and only weakly χ -dependent.

12.5 The robustness verdict

Which conclusions are robust?

- **Robust** (survives mesh + time refinement and the χ_{pf} uncertainty): the DEGREE of demixing (phase contrast) is set by DRYNESS, rises monotonically as the film dries, is nearly rate-independent at matched ϕ_s , and increases monotonically with χ_{pf} . Morphology collapses onto the Biot number.
- **Not robust at tutorial scale:** the “faster spin = finer morphology” *wavelength* ordering. With a single seed and only a few lateral domains it is noisy, moves under mesh

refinement, and is a weak function of χ_{pf} . Reproducing it quantitatively needs seed ensembles and a larger box — a research-mode campaign (Chapter 23), not a tutorial run.

This is the scientific habit the capstone is teaching: a real reference case reports what the data support at the resolution actually run, separates the robust invariant (demixing degree vs. dryness and χ) from the scale-limited claim (the wavelength ordering), and hands the reader a tolerance-checked artefact they can re-run and extend.

12.6 Explore on your own

Q12.6.1 Swap the system to P3HT_PCBM in `configs/p10.yaml`. Confirm the honest negative: measure the phase contrast at matched dryness and show it is far smaller than for PDPP5T:PCBM. What does that say about using contrast as a screening metric for candidate blends?

Q12.6.2 Widen the drying-rate window (`k_e_slow`, `k_e_fast`) and re-run. Does a larger Biot span sharpen the “faster = finer” wavelength ordering, or does the noise still dominate at a single seed?

Q12.6.3 Run several seeds at one rung and report the wavelength as a mean $\pm$ spread. How many seeds would you need for the “faster = finer” claim to clear its own error bar? Connect this to the reproducible-campaign machinery of Chapter 23.

Q12.6.4 Push the mesh-convergence rung to level 7 (research mode). Does the polymer per-solute conservation residual tighten toward the fullerene’s machine-precision value as you predicted from the surface-layer argument?

Polymer field at matched dryness (fixed computational frame)

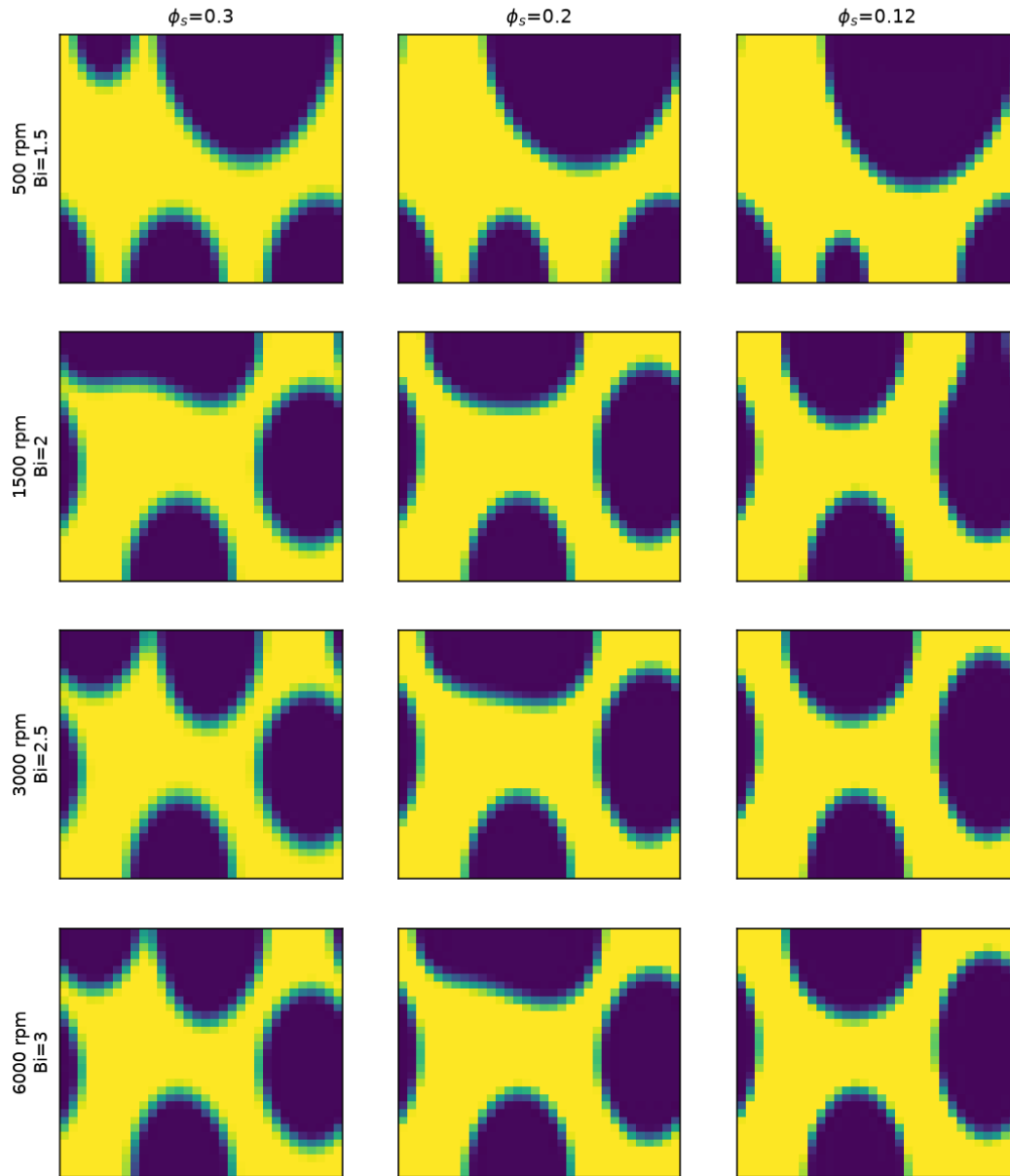


Figure 12.4: Polymer field at the matched dryness levels for each ladder rung (fixed computational frame). The pattern coarsens/sharpens with dryness far more than it changes with spin speed.

The demixing-degree conclusion is mesh- and time-robust (small relative change under refinement)

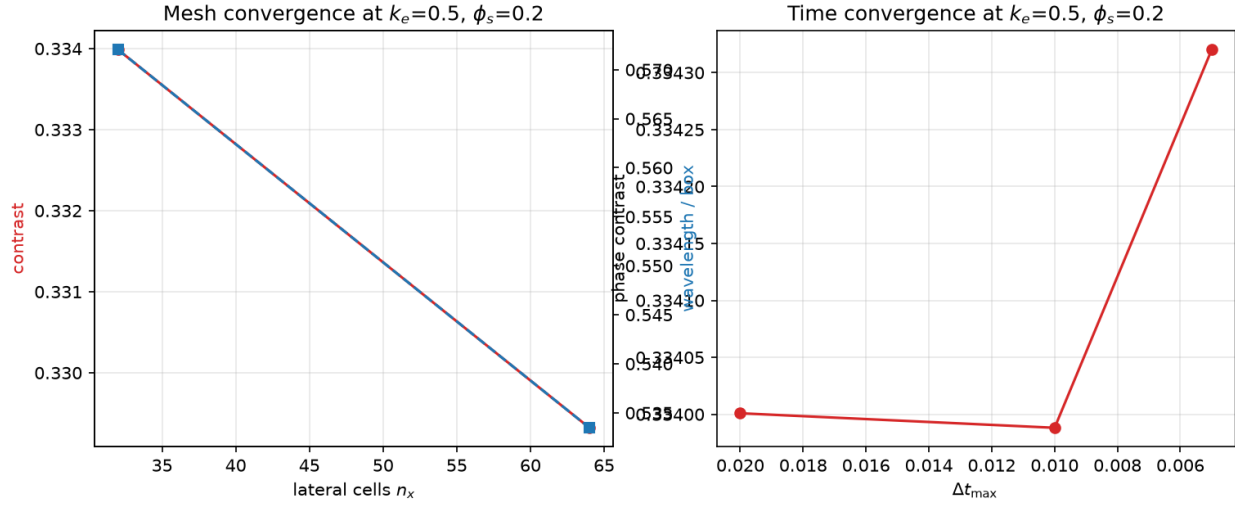


Figure 12.5: Mesh (left) and time (right) convergence of the matched-state metrics at one ladder rung. The phase contrast — the robust conclusion — is nearly flat under refinement.

χ_{pf} uncertainty sweep (order_of_magnitude 0.5): which conclusions survive an order-of-magnitude change in χ

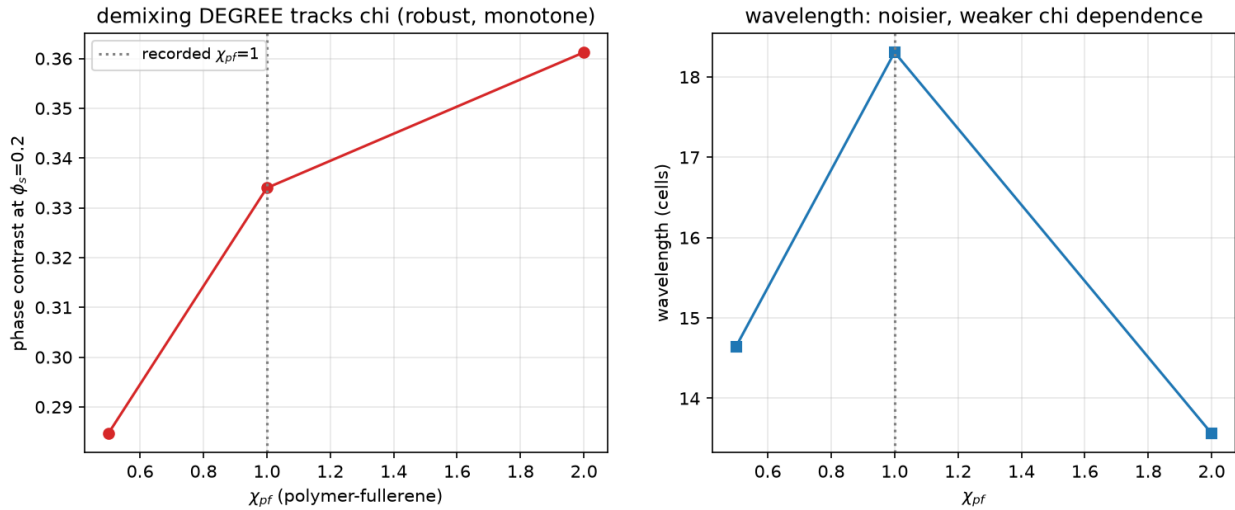


Figure 12.6: Sensitivity to the order-of-magnitude χ_{pf} uncertainty. The demixing degree (left) is a robust, monotone function of χ_{pf} ; the wavelength (right) is not a reliable discriminator at this scale.

Chapter 13

Model-extension (capstone)

Key idea. This is the contributor capstone: take ONE new physical mechanism all the way through the workflow a real DiffSim pull request must follow — derivation, implementation, an *analytic* Jacobian, a finite-difference derivative test, a limiting-case check, a manufactured-solution (MMS) convergence study, a conservation check, a GPU profile, and a scientific result — and ship it with PR-quality docs. The mechanism here is **vitrification**: a composition-dependent mobility that *arrests* as the film vitrifies, which is exactly what freezes the drying morphology of Chapter 12.

Learning objectives. Add a mechanism as a self-contained, tutorial-local brick (no core edit); derive an analytic Jacobian by hand and prove it correct against finite differences; verify a solver by its limiting-case dispersion, by MMS spatial convergence and by temporal order; profile it on the GPU; and report a scientific result with honest verification.

Prerequisites. Chapter 7 (Cahn–Hilliard mobility and the drying film), Chapter 12 (the frozen morphology), and Chapter 22 (extending DiffSim — the same term $\rightarrow$ residual $\rightarrow$ Jacobian $\rightarrow$ tests workflow).

Expected cost. The verification + scientific run is seconds to a couple of minutes; the test suite runs in well under a minute. Quick mode is a sub-two-minute smoke; reference is the gated verification.

13.1 The mechanism

As an organic film dries (Chapter 12) the polymer-rich phase concentrates and its molecular mobility collapses at a glass composition: the morphology stops evolving even though it is not yet at equilibrium. We model this as a one-dimensional Cahn–Hilliard system with a composition-dependent mobility carrying a smooth *arrest* factor. With order parameter $\phi \in [0, 1]$, double-well free energy $f(\phi) = \phi^2(1 - \phi)^2$, and gradient penalty $\kappa = 3.0 \times 10^{-4}$,

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (M(\phi) \nabla \mu), \quad \mu = f'(\phi) - \kappa \nabla^2 \phi, \quad (13.1)$$

$$M(\phi) = M_0 a(\phi), \quad a(\phi) = \frac{1}{1 + e^{(\phi - \phi_g)/w}} = \sigma\left(\frac{\phi_g - \phi}{w}\right), \quad (13.2)$$

so the mobility is $\sim M_0$ for $\phi < \phi_g$ (mobile) and collapses to zero for $\phi > \phi_g$ (vitrified), over a transition width $w = 0.03$ about the glass composition $\phi_g = 0.7$ (Fig. 13.1). The arrest derivative is

analytic, $a'(\phi) = -\frac{1}{w}a(1-a)$, so $M'(\phi) = -\frac{M_0}{w}a(1-a)$. In the limit $\phi_g \rightarrow \infty$, $a \rightarrow 1$ and $M \rightarrow M_0$: the brick reduces to standard constant-mobility Cahn–Hilliard, which we exploit for verification.

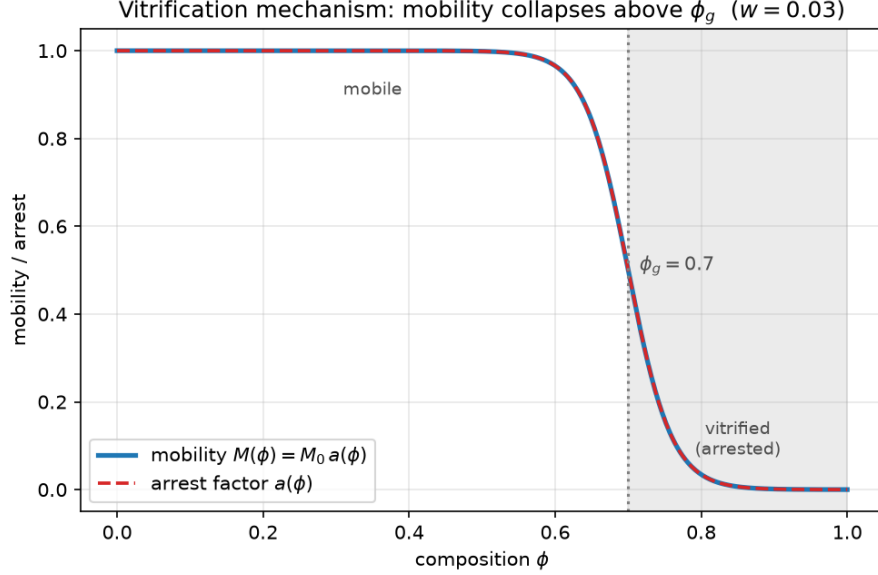


Figure 13.1: The added term: a composition-dependent mobility $M(\phi) = M_0 a(\phi)$ whose vitrification arrest factor $a(\phi)$ collapses the mobility above the glass composition ϕ_g .

Discretisation. On a periodic grid of N nodes (spacing h), the Laplacian is the central second difference and the flux divergence uses an arithmetic face-averaged mobility $M_{i+1/2} = \frac{1}{2}(M_i + M_{i+1})$, which keeps the analytic Jacobian clean. Time stepping is backward Euler, giving the residual

$$R(\phi) = \frac{\phi - \phi^n}{\Delta t} - \nabla \cdot (M(\phi) \nabla \mu(\phi)). \quad (13.3)$$

13.2 The analytic Jacobian

The heart of a solver contribution is a correct analytic Jacobian. Writing the discrete flux divergence as $D(\phi)$, the Jacobian of (13.3) is

$$J = \frac{1}{\Delta t} \mathbf{I} - \frac{\partial D}{\partial \phi}, \quad \frac{\partial D}{\partial \phi} = A_{\text{mob}} + A_{\text{op}} J_\mu, \quad (13.4)$$

where $J_\mu = \text{diag } f''(\phi) - \kappa \mathbf{L}$ is $\frac{\partial \mu}{\partial \phi}$, A_{op} is the flux-divergence operator with the face mobilities frozen, and A_{mob} is the extra term from differentiating the face mobilities through $M'(\phi)$. All three blocks are assembled by hand (never by autograd) in `jacobian`; the derivation is written out in the module docstring.

Derivative test (the gate). The hand-derived Jacobian is checked column-by-column against a central-difference Jacobian of the residual on a random smooth state: the maximum absolute error is 3.6×10^{-8} . As an independent cross-check, the same analytic matrix matches PyTorch autograd to 2.3×10^{-13} — confirming the by-hand chain rule is exact, not merely consistent with its own finite differences. The free-energy derivatives f' , f'' are likewise checked against central differences (1.3×10^{-11} , 5.4×10^{-11}).

13.3 Verification

Limiting case: the Cahn–Hilliard dispersion. With arrest off ($\phi_g \rightarrow \infty$, constant mobility) the linearised dynamics about $\bar{\phi} = \frac{1}{2}$ has the exact growth rate $\sigma(k) = -M_0 k^2 (f''(\bar{\phi}) + \kappa k^2)$. Measuring the single-mode amplitude change over one backward-Euler step reproduces it: at the fundamental mode $\sigma_{\text{measured}} = 39.06$ against $\sigma_{\text{analytic}} = 39.01$ (relative error 1.2×10^{-3}), and the whole $\sigma(k)$ curve tracks the analytic one (Fig. 13.2).

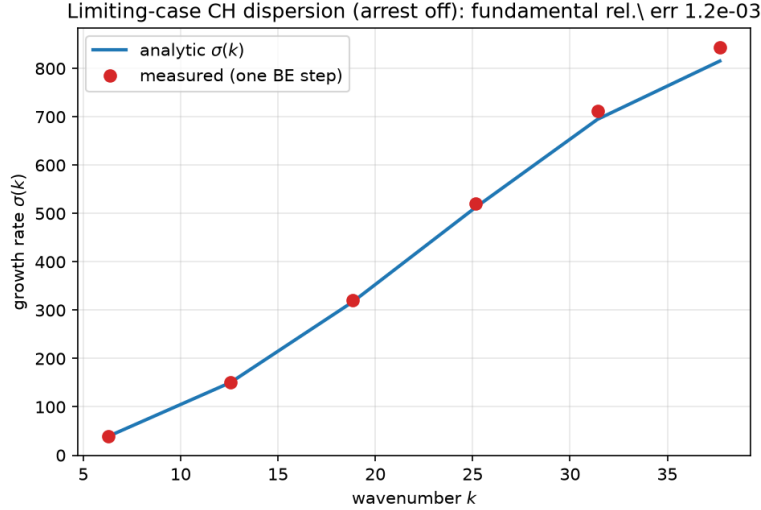


Figure 13.2: Limiting-case verification: with the arrest switched off the measured single-mode growth rate matches the analytic Cahn–Hilliard dispersion $\sigma(k)$.

Manufactured solution and orders of accuracy. A manufactured solution $\phi^*(x, t) = \bar{\phi} + A \sin(qx) e^{-\lambda t}$ with its exact analytic source added to the residual gives a clean spatial convergence study: the L_2 error against ϕ^* falls from 4.7×10^{-7} to 3.0×10^{-8} under mesh doubling, an observed order of 1.996 (minimum 1.994) — the expected second order of the central/face-averaged discretisation. Refining the time step instead recovers the first-order backward-Euler rate 1.023 (Fig. 13.3). Mass is conserved to $< 10^{-13}$ over the scientific run, and Newton converges in at most 5 iterations.

13.4 GPU profile

A new mechanism ships with a *measured* cost. `profile_gpu.py` times one full Newton step (residual + analytic-Jacobian assembly + dense solve) against grid size N on both devices (Fig. 13.4). At $N = 1024$ the step costs 8.90 ms on the cuda:0 device at a peak of 75.8 MB, a $1.7\times$ speedup over the CPU. Honestly, at small N the CPU wins: this brick forms a *dense* $N \times N$ Jacobian and the GPU only repays its launch overhead once N is large. That is the right lesson — profile before assuming the GPU helps, and know that the production path (Chapter 21) uses a sparse assembly for exactly this reason.

Verification: hand-derived Jacobian FD-error 3.6e-08 (autograd match 2e-13)

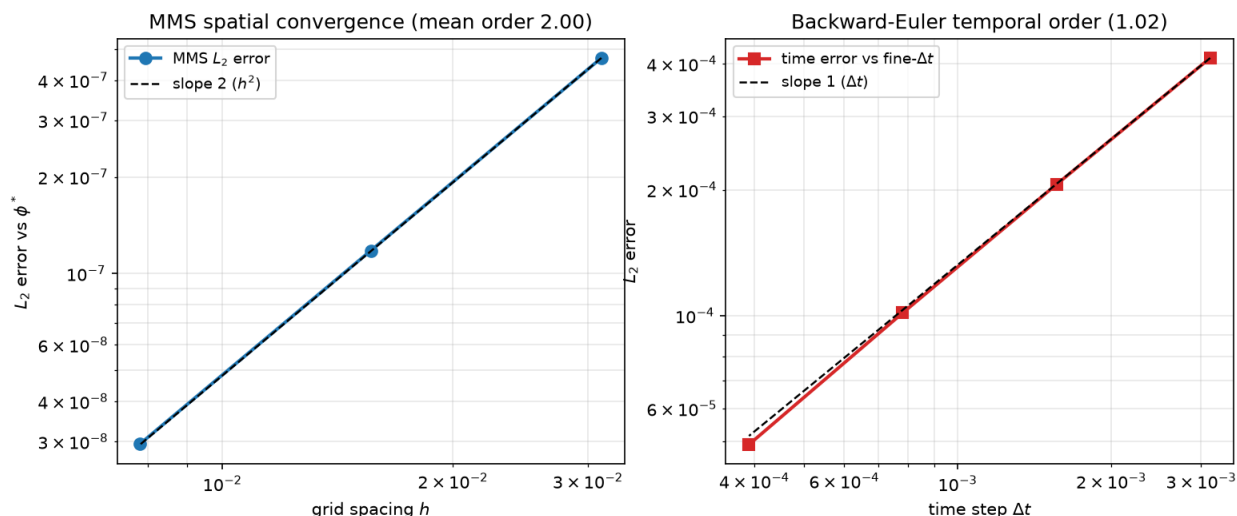


Figure 13.3: MMS spatial convergence (left, second order) and temporal convergence (right, first order for backward Euler), each against its theoretical reference slope.

13.5 The scientific result

With every check green we can trust the physics. Starting from a random initial condition at the glass composition (so the polymer-rich matrix vitrifies), a run *without* arrest coarsens — the domain length $L(t)$ grows to 0.384 — while the vitrified run *arrests*: $L(t)$ plateaus at 0.113 (a last-half growth of only 0.8%), a factor 3.40 finer (Fig. 13.5). This is the mechanism that stops the clock in Chapter 12: vitrification freezes a fine, non-equilibrium morphology instead of letting it coarsen to a coarse equilibrium — precisely what makes a fast-dried organic film’s morphology kinetically selectable.

Run it.

```
cd physics/11_model_extension
python -m pytest test_vitrification.py -q           # the derivative + MMS
    gates
python run_harness.py --config configs/p11.yaml --mode reference \
    --output outputs/p11 --overwrite                # results.json vs
    baseline.yaml
python profile_gpu.py --run-dir outputs/p11         # the GPU profile
python gen_figures.py --run-dir outputs/p11         # figures + numbers
```

The brick is entirely tutorial-local (`vitrification.py`); it does not touch the core `src/` solver, so it is a template for a first contribution.

13.6 The contributor checklist

This is the workflow a real mechanism PR follows (spec C8), all exercised here:

Newton step cost (residual + analytic Jacobian + dense solve) vs. peak GPU memory (dense $N \times N$ Jacobian; 1.7x vs CPU at $N=1024$)

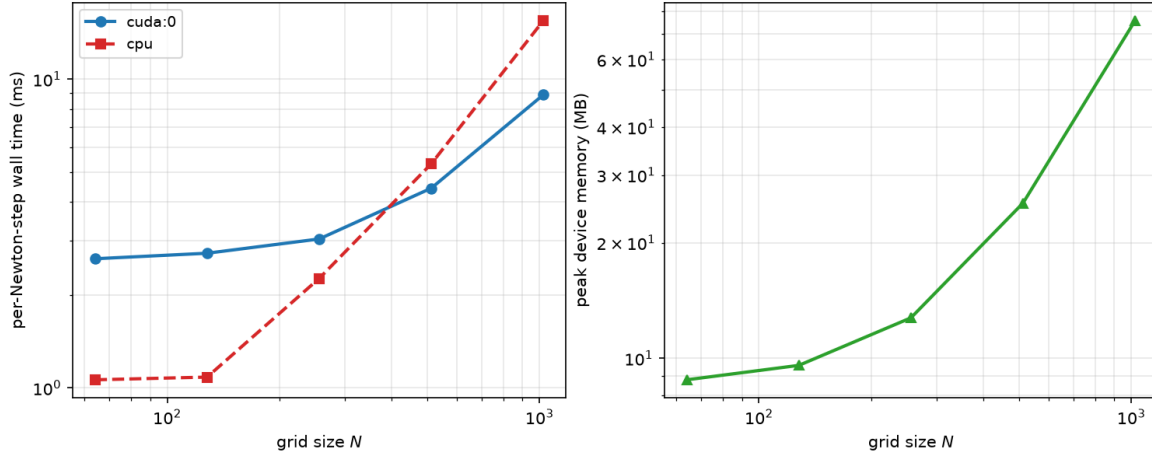


Figure 13.4: GPU profile of the Newton step: wall time vs grid size N (left, GPU vs CPU) and peak device memory (right). The dense $N \times N$ Jacobian makes the GPU worthwhile only at larger N .

From term to merged mechanism.

- derivation of the term and its continuous operator (§1);
- a residual and an *analytic* Jacobian, assembled by hand (§2);
- a finite-difference derivative test — the non-negotiable gate — plus an autograd cross-check (§2);
- a limiting case with a known closed-form answer (the CH dispersion, §3);
- an MMS spatial-convergence study at the design order, and the temporal order (§3);
- a conservation check (§3);
- a GPU profile with an honest cost verdict (§4);
- a tolerance-checked scientific result (§5); and PR-quality docs (README.md + this chapter).

13.7 Explore on your own

Q13.7.1 Replace the sigmoid arrest with a power-law degenerate mobility $M(\phi) = M_0 \phi(1 - \phi)$ and re-derive $M'(\phi)$. Update `Mmob` and `Mprime`, then confirm the finite-difference Jacobian test still passes — the derivative test catches a wrong M' immediately.

Q13.7.2 Make the arrest width w smaller and re-run the coarsening study. How sharp must the glass transition be before the arrested domain scale stops depending on w ? Relate this to the interface width $\sqrt{\kappa}$.

Q13.7.3 The GPU profile shows the dense solve losing to the CPU at small N . Sketch what a sparse (tridiagonal-plus-mobility) assembly would change, and at what N you would expect the crossover to move. Connect to Chapter 21.

Q13.7.4 Extend the mechanism to two dimensions (the field becomes $N \times N$). Which of the verification steps (derivative test, MMS, dispersion) transfer unchanged, and which need a 2-D manufactured solution?

Scientific result: vitrification freezes the drying-film morphology (the P10 clock stops)

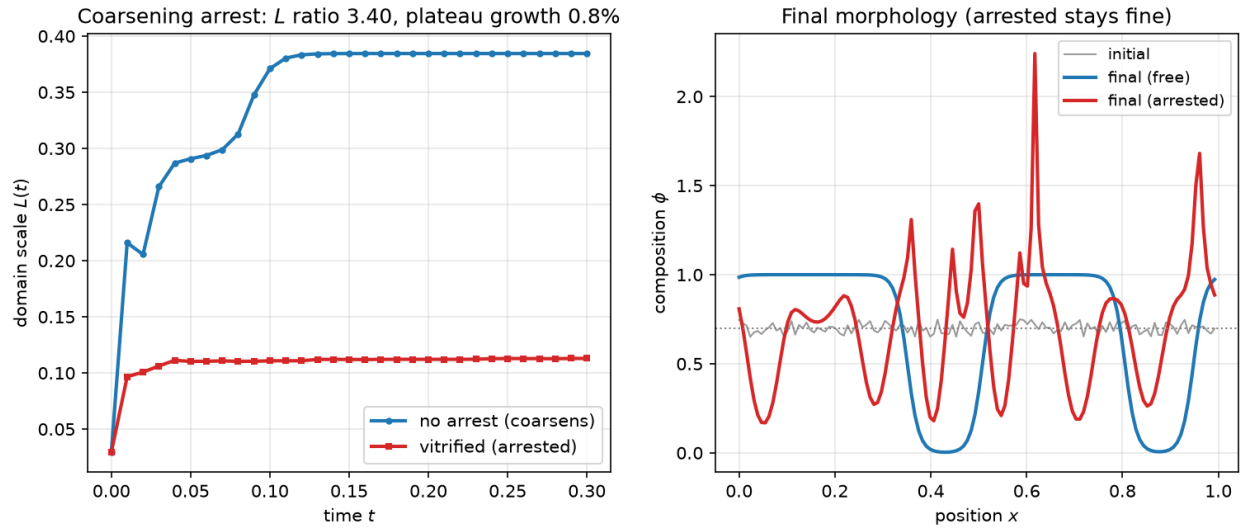


Figure 13.5: Scientific result: without arrest the domains coarsen (L grows); with the polymer-rich matrix vitrified, coarsening halts and a fine morphology is frozen in.

Part II

Computation

Chapter 14

From weak form to CUDA: one scalar problem, every stage

Key idea. Every solver in this course is the same assembly line: a strong PDE becomes a weak form, the weak form becomes an element residual and its Jacobian, quadrature turns those into numbers, a local-to-global map scatters them into a sparse matrix, constraints and boundary conditions are imposed, Newton linearizes, a linear solve runs on the device, and the field updates. This chapter walks *one* small scalar problem down that line and names the *exact* DiffSim file and function at each stage — so the phase-field bricks in the rest of the course read as variations on a path you have already traced end to end. Then you **do** the load-bearing step yourself: add a coefficient to the weak form, derive its analytic Jacobian contribution, and watch a regression test flip from red to green.

14.1 The model

We use the smallest problem that still has every ingredient of the phase-field solvers: a *nonlinear*, scalar, steady reaction–diffusion equation on the unit square with Dirichlet data,

$$-\nabla^2 u + \alpha u^3 = f \quad \text{in } \Omega = (0, 1)^2, \quad u = g \quad \text{on } \partial\Omega. \quad (14.1)$$

The Laplacian is the *diffusion* (the same operator as the interface term of Cahn–Hilliard); the cubic αu^3 is a *reaction* (a stand-in for the nonlinear $f'(c)$ of the bulk free energy). One scalar unknown per node keeps the bookkeeping visible; everything generalizes to the mixed (c, μ) and multi-field blocks by carrying more degrees of freedom per node.

Strong $\rightarrow$ weak (integration by parts). Multiply (14.1) by a test function w that vanishes on the Dirichlet boundary, integrate over Ω , and move one derivative off the Laplacian:

$$-\int_{\Omega} w \nabla^2 u \, dV = \int_{\Omega} \nabla w \cdot \nabla u \, dV - \int_{\partial\Omega} w (\nabla u \cdot n) \, dS. \quad (14.2)$$

With $w = 0$ on $\partial\Omega$ (strong Dirichlet) the surface term drops, and the problem becomes: find u such that for every admissible w ,

$$R(w; u) = \underbrace{\int_{\Omega} \nabla w \cdot \nabla u \, dV}_{\text{diffusion}} + \underbrace{\alpha \int_{\Omega} w u^3 \, dV}_{\text{reaction}} - \underbrace{\int_{\Omega} w f \, dV}_{\text{load}} = 0. \quad (14.3)$$

Element residual and its Jacobian. Expand $u_h = \sum_b N_b u_b$ and take $w = N_a$ (Galerkin). The residual entry for node a on element e is (14.3) evaluated with $w = N_a$. Because the reaction is nonlinear we solve by Newton, which needs the *tangent* $J = \frac{\partial R}{\partial u}$. Differentiating (14.3) in a trial direction N_b ,

$$J_{ab} = \underbrace{\int_{\Omega} \nabla N_a \cdot \nabla N_b dV}_{\text{stiffness (constant)}} + \underbrace{\alpha \int_{\Omega} N_a (3u^2) N_b dV}_{\text{reaction Jacobian}}. \quad (14.4)$$

The derivative of αu^3 is $3\alpha u^2$, so the reaction contributes a *u-weighted mass matrix* to the Jacobian. This second term is the one you will add by hand in §14.3; the stiffness term is *u*-independent and is assembled once.

Quadrature and the octree chain rule. Every integral is a Gauss sum $\int(\cdot) dV = \sum_q(\cdot)|J|w_q$. On the octree’s cube elements the reference-to-physical map is a scaling by $h/2$, so $|J| = (h/2)^{\text{dim}}$ and physical gradients are reference gradients times $\text{dscale} (= 2/h)$. The tables $N_a(x_q)$, $\nabla N_a(x_q)$ and w_q come from `mesh/basis.py` (`basis_tables`, `gauss_1d`); inside a kernel they are read through the `assembly/femelm.py` accessors `fe_N`, `fe_dN_s`, `fe_detJxW_s`. The tutorial’s readable NumPy assembly uses exactly these tables — and a test proves its stiffness matrix equals the production Warp-kernel assembly to the last bit.

14.2 The pipeline, stage by stage

Figure 14.1 is the map: the left column is the stage, the right column is the DiffSim file and function that owns it. The tutorial file `scalar_reaction.py` implements the whole path in transparent NumPy/SciPy (one function per stage, each annotated with its production counterpart), so you can read a stage here, then open the one line of code that does it, then open the production kernel it mirrors.

1. **Mesh + basis.** `build_uniform` (`octree/build.py`) $\rightarrow$ `build_mesh` (`mesh/nodes.py`) $\rightarrow$ `build_constraints` (`mesh/constraints.py`) $\rightarrow$ `basis_tables` (`mesh/basis.py`) $\rightarrow$ `DeviceMesh`. `from_mesh` (`assembly/operators.py`). The `DeviceMesh` holds the per-degree connectivity, the basis tables, and the constraint matrix T , all as device arrays.
2. **Element residual + Jacobian.** The physics of one integration point. The production phase-field kernel is `make_ch_newton` (`physics/cahn_hilliard.py`); the linear model brick is `PoissonBrick` (`api/example_bricks.py`), whose `Integrands_Ae/ Integrands_be` are the annotated “write the weak form, get the assembly” API.
3. **Quadrature.** `basis_tables / gauss_points` (`physics/poisson.py`) supply x_q , N , ∇N , w_q .
4. **Local $\rightarrow$ global (scatter).** The element block `conn[e, a]` $\rightarrow$ global dof; `assemble_brick_csr` (`api/equation.py`) or the device scatter kernels of `assembly/device_assembly.py`.
5. **CSR assembly.** COO triplets $\rightarrow$ CSR, then the constrained $T^{\top} K T$; `assemble_csr / volume_triplets / CSROperator` (`assembly/operators.py`).
6. **Constraints / BCs.** Hanging-node interpolation is the matrix T from `build_constraints`; strong Dirichlet is row replacement (`_BCOperator`, `assembly/dirichlet.py`; and the CH stepper’s own row replacement in `step`).
7. **Newton.** `NonlinearSolver` (`solvers/newton.py`), or `CahnHilliardStepper.step`’s monolithic loop: assemble J, R ; solve $J \delta u = -R$; line-search; update.
8. **Linear solve.** `solve_linear` (`solvers/linsolve.py`) dispatches `splu` (host direct), `cudss` (GPU direct), `blockch` (the mixed-CH Schur preconditioner), returning a `LinearSolveResult` (`solvers/result.py`).

9. **Device update.** The solution vector updates the field state; host/device transfers are the `wp.array(...)` uploads and `.numpy()` pulls around `wp.launch`.

Where the tutorial meets production. For a *uniform* box there are no hanging nodes, so the constraint matrix T is the identity and the free-node system is just the node system — which is why `scalar_reaction.py` can impose Dirichlet data by row replacement and skip T . The chapter’s cross-check test asserts that its hand-assembled stiffness equals `assemble_brick_csr(dm, PoissonBrick())` (the production Warp kernel through $T^T K T$) to 0 — the transparent NumPy *is* the device math.

14.3 Hands-on: add a coefficient and its analytic Jacobian

This is the exercise. The starter file `scalar_reaction_starter.py` already has the reaction term αu^3 in the *residual*, so it solves the right PDE — but its *Jacobian* is missing the reaction contribution (14.4). A Newton solve with a wrong Jacobian may still limp to an answer, so a convergence check alone would not catch the bug. The sharp, hardware-independent test is a **derivative check**: the analytic Jacobian must match a finite difference of the residual,

$$\frac{\|J(u)v - (R(u + \varepsilon v) - R(u))/\varepsilon\|}{\|(R(u + \varepsilon v) - R(u))/\varepsilon\|} \ll 1. \quad (14.5)$$

Do it (red → green).

```
cd computational/00_weak_form_to_cuda
# 1. RED: the starter's Jacobian omits the reaction term
RD_MODULE=scalar_reaction_starter pytest -q test_reaction_jacobian.py
# 2. Add the analytic reaction Jacobian to jacobian() in the module
#    you are editing (the completed reference is scalar_reaction.py):
#    Ke = prob._Ke + reaction_jacobian_elem(prob, gp_value(prob, u))
# 3. GREEN:
pytest -q test_reaction_jacobian.py
```

With the reaction Jacobian *missing*, (14.5) is 4.9×10^{-3} — the test fails. With the analytic term $3\alpha u^2$ *added*, it is 7.9×10^{-9} (finite-difference truncation only) — the test passes. The same edit restores Newton’s *quadratic* convergence (Fig. 14.2, right): the residual norm is squared each step once the tangent is exact.

Why a derivative test, not just “it converged”. A missing or wrong Jacobian term is the single most common bug when adding physics to an implicit solver, and it hides well: the residual (the equation you solve) can be perfectly correct while the tangent (how you get there) is wrong, so the run still lands near an answer, just slowly or fragilely. The finite-difference check in (14.5) tests the tangent *directly* and deterministically — it is the same check the differentiable track and any new-term workflow (residual → analytic Jacobian → derivative test) must pass before an order or a gradient is trusted.

14.4 Run it: verify the whole pipeline

Run it.

```
cd computational/00_weak_form_to_cuda
python run.py --config configs/c0.yaml --device cpu --solver splu \
  --mode reference --output outputs/c0 --overwrite
python gen_figures.py --run-dir outputs/c0
```

The harness runs the manufactured-solution convergence study ($p=1$ and $p=2$), the Newton-convergence trace, the analytic-Jacobian check (against both the completed and the starter modules), and the production-kernel cross-check; it writes `results.json`, checks it against `baseline.yaml` (tolerances, not bit-identity), and records provenance in `metadata.json`. `gen_figures.py` renders the figures and `numbers/c0.tex` from the saved run.

What you should see. We verify the assembly is correct with the *method of manufactured solutions*: pick $u^* = \sin(\pi x)\sin(\pi y)$ (zero on the boundary), feed the source $f = 2\pi^2 u^* + \alpha(u^*)^3$ that makes it exact, and measure the L^2 error under mesh refinement. Degree- p elements converge at h^{p+1} — the single most diagnostic correctness check for a discretization. The measured orders are 2.01 ($p=1$) and 2.98 ($p=2$), on the predicted lines (Fig. 14.2, left).

Measured self-check (reference mode, `splu`; reproduced within `baseline.yaml` tolerances, not bit-for-bit).

	$p = 1$	$p = 2$
L^2 error: coarse $\rightarrow$ fine	$3.3 \times 10^{-3} \rightarrow 5.1 \times 10^{-5}$	$1.6 \times 10^{-3} \rightarrow 3.2 \times 10^{-6}$
observed order (expect $p+1$)	2.01 (2)	2.98 (3)
Newton (level 5): residual $4.2 \times 10^{-1} \rightarrow 8.9 \times 10^{-13}$ in 6 iterations		
analytic Jacobian vs FD: 7.9×10^{-9} (complete) <i>vs</i> 4.9×10^{-3} (starter, reaction Jac missing)		
transparent stiffness vs production kernel: 0		

14.5 Code walk-through

`scalar_reaction.py` is organized one function per pipeline stage.

1. Mesh + tables (the real path).

```
tree = build_uniform(level, dim=2)          # octree/build.py
mesh = build_mesh(tree, p)                  # mesh/nodes.py
cons = build_constraints(mesh)               # mesh/constraints.py
dm    = DeviceMesh.from_mesh(mesh, cons, basis_tables(p, 2), device)
```

The same four calls every tutorial makes; `basis_tables` gives N , ∇N (reference) and w_q , and the octree factors `dscale` ($= 2/h$), $|J| = (h/2)^{\dim}$ convert to physical space.

2. Element residual + Jacobian (quadrature by `einsum`).

```
def reaction_jacobian_elem(prob, uq):      # the hands-on term
    wq = 3.0 * prob.alpha * uq**2          # d/du (alpha u^3)
    return einsum("qa,eq,qb,q->eab", N, wq, N, detJxW) # INT N_a (3 a u^2)
    N_b
```

The `einsum` is the Gauss sum of (14.4); the diffusion stiffness is the same pattern with $\nabla N_a \cdot \nabla N_b$.

3. Local-to-global + Dirichlet, then the real solve.

```
J = coo_matrix((Ke.ravel(), (rows, cols))).tocsr()    # scatter -> CSR
J.rows[i] = [i]; J.data[i] = [1.0]                 # Dirichlet row = I
du = solve_linear(J, -R, solver="splu")              # solvers/linsolve.py
```

`rows/cols` are the connectivity outer-product — the exact triplet pattern `assemble_brick_csr` builds — and the linear solve is the production dispatcher, so swapping `"splu"` for `"cudss"` runs the same Newton step on the GPU.

14.6 Explore on your own

Q14.6.1 Change the reaction to αu^5 . Write its residual and the analytic Jacobian weight ($5\alpha u^4$), update both, and confirm the derivative check (14.5) passes and Newton stays quadratic. What happens to the convergence if you update the residual but *not* the Jacobian?

Q14.6.2 Switch the linear solve from `splu` to `cudss` (`--solver cudss`, GPU). The Newton iterates and the L^2 error should be identical to solver tolerance — verify that the *discretization* answer does not depend on the *linear-algebra* backend.

Q14.6.3 Make the reaction coefficient a field, $\alpha(x)$. Which pipeline stage changes — the residual, the Jacobian, the assembly, or the solve? (Hint: only the integrand; the scatter and solve are untouched. This is why the brick API asks you for one integration point.)

Q14.6.4 Push the $p=1$ study to level 7. Does the order stay 2, or does round-off floor the error? Predict the level from the coarse-grid error and machine epsilon, then confirm.

Q14.6.5 Read `make_ch_newton` in `physics/cahn_hilliard.py` and match its four 2×2 Jacobian blocks ($\frac{\partial R_c}{\partial c}, \frac{\partial R_c}{\partial \mu}, \frac{\partial R_\mu}{\partial c}, \frac{\partial R_\mu}{\partial \mu}$) to the single scalar J_{ab} of this chapter. Which block is the analogue of the reaction Jacobian you added here?

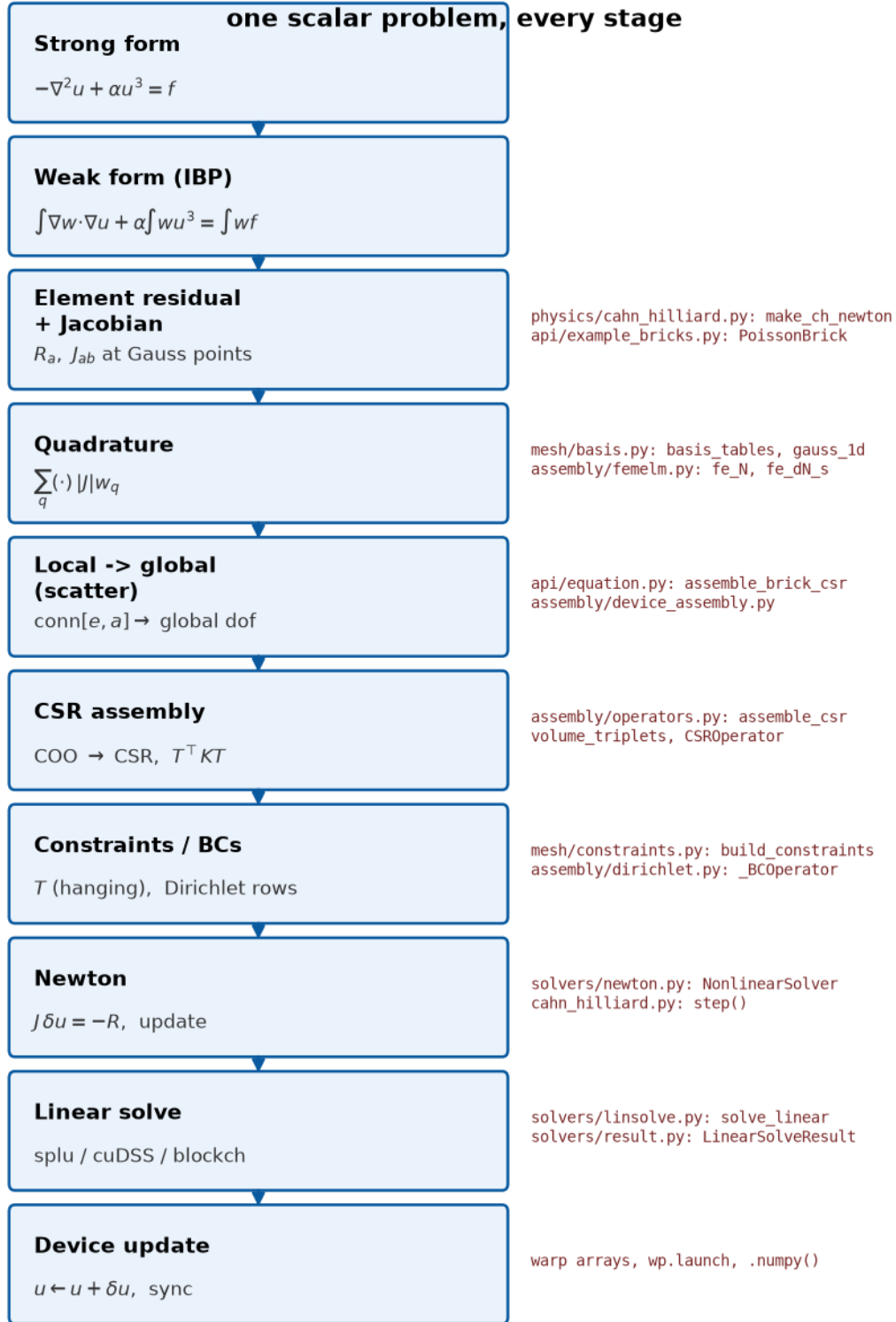


Figure 14.1: The weak-form-to-CUDA pipeline for one scalar problem. Each stage (left) maps to the exact DiffSim source (right). The tutorial’s `scalar_reaction.py` walks the whole path in readable NumPy; the phase-field bricks (`physics/cahn_hilliard.py` and siblings) are the same path with more fields per node and the arithmetic in Warp kernels.

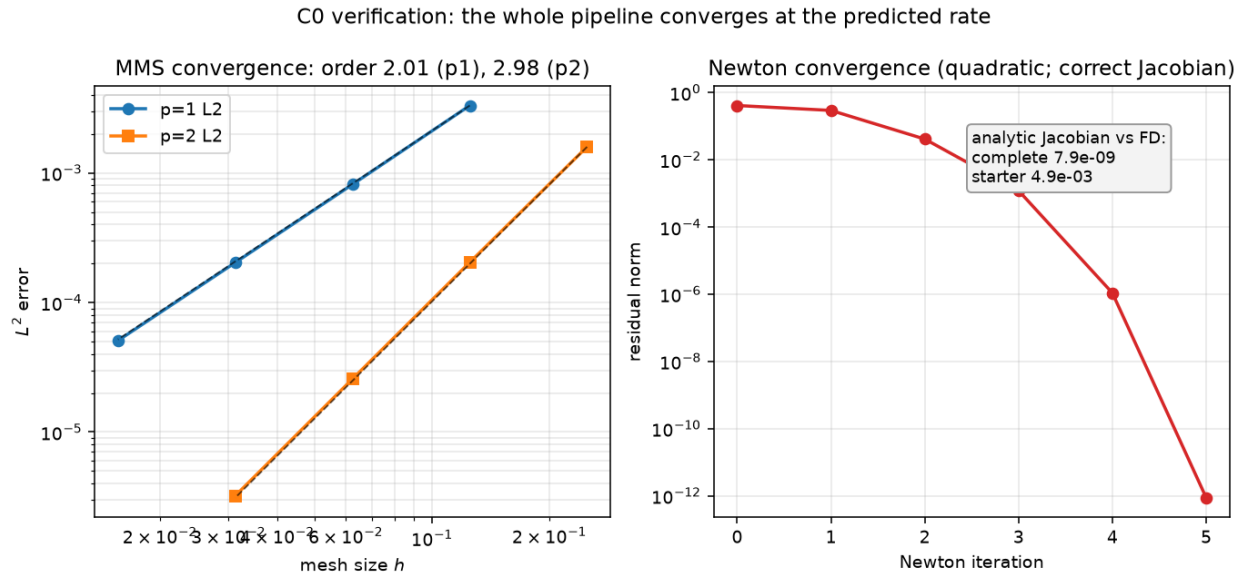


Figure 14.2: Verification of the whole pipeline. *Left:* manufactured- solution L^2 error falls at the predicted h^{p+1} rate (order 2.01 for $p=1$, 2.98 for $p=2$), proof every stage is assembled correctly. *Right:* with the correct analytic Jacobian, Newton converges quadratically (residual $4.2 \times 10^{-1} \rightarrow 8.9 \times 10^{-13}$ in 6 iterations); the inset contrasts the analytic-vs-finite-difference Jacobian error for the completed brick and the hands-on starter.

Chapter 15

Convergence: basis order and time integration

Key idea. A discretization is only trustworthy if it *converges* at the rate the theory predicts — but only if you measure the *right* error. This chapter separates the two discretization errors so each is measured alone, controls the *algebraic* (solver) error so it cannot contaminate the plot, verifies the reference before trusting it, and shows how the study can lie to you.

Learning objectives. By the end you can: separate spatial, temporal, and algebraic error and measure each alone; report the observed spatial order ($L^2 \sim h^{p+1}$, $H^1 \sim h^p$) for both $p = 1$ and $p = 2$, for both fields c and μ ; report the observed temporal order (BDF1 $\rightarrow 1$, BDF2 $\rightarrow 2$) against a reference verified by halving its Δt and a Richardson bound; and diagnose the four deliberate failures (wrong BC, wrong source, under-resolved feature, under-resolved reference) that each produce a plausible but wrong order.

Prerequisites. The weak (FE) form and Gauss quadrature, the method of manufactured solutions, BDF scheme order, Richardson extrapolation; Physics Chapter 3 (the same Cahn–Hilliard brick discretized here); Chapter 00 (environment green).

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory (an 8 GB card is ample); solver `splu` on the mixed (c, μ) saddle, exact to round-off. `run.py` runs all four studies in ≈ 1 – 2 min on the GPU — comparable to physics P1’s quick mode, measured ≈ 75 s wall on an RTX 6000 Ada (mostly Warp kernel compile + Python start-up).

This concept walks through the Cahn–Hilliard convergence gates that ship with the solver: $p = 1$ vs $p = 2$ spatial order, and BDF1 vs BDF2 temporal order, each measured against a manufactured or self-convergent reference. It connects to Physics Chapter 3 (the same brick) and to the adaptive-stepping concept (Chapter 17: why variable-coefficient BDF2 matters once Δt changes).

15.1 Three errors, not one

Every result from this simulator is wrong. The question is only *how* wrong, and whether the error shrinks — at the rate the theory promises — as you refine. A time-dependent finite-element computation carries *three* errors, and a convergence study is only meaningful when you measure

one at a time:

- a **spatial** error from representing $c(\mathbf{x})$ in a finite basis on a mesh of size h ; for degree- p elements the L^2 error scales as h^{p+1} and the H^1 seminorm (the gradient error) as h^p ;
- a **temporal** error from advancing in finite steps Δt ; for a scheme of order q it scales as Δt^q ;
- an **algebraic** error from solving the nonlinear system only approximately (Newton stopped at some tolerance, the linear solve at another).

The exponents $p + 1$, p and q are *predictions*. Measuring them is the single most important correctness check in scientific computing: a code can run, conserve mass, and look physical while quietly converging one order too slow because a term is missing from the weak form. The catch is that a spatial study run with a too-large Δt measures the *time* error, and a study where Newton stopped too early measures the *solver* error — either way you get a wrong-but-plausible number. So we isolate each.

15.2 Spatial order: a steady manufactured solution

The trouble with checking a solver against a “true” answer is that PDEs rarely have closed-form solutions. The *method of manufactured solutions* (MMS) turns the problem around: *pick* a smooth field, compute what source term would make it an exact solution, and feed that source to the solver. Now you know the exact answer everywhere, so the error is measurable.

The key correction over a naive MMS is to pick a **steady** (time *independent*) manufactured field. Then $c_t^* = 0$, the time term contributes nothing at the discrete steady state, and the measured error is pure spatial error — *independent of Δt* . We manufacture composition and chemical potential independently (the cleanest choice — it avoids taking a Laplacian of the cubic $f'(c)$ by hand):

$$c^* = \cos(\pi x) \cos(\pi y), \quad \mu^* = \sin(\pi x) \sin(\pi y). \quad (15.1)$$

Substituting into the mixed Cahn–Hilliard system (3.3) leaves a residual; we *add it back* as two steady source terms so the pair is an exact solution:

$$f_c = -M \nabla^2 \mu^* = 2\pi^2 M \mu^*, \quad (15.2)$$

$$f_m = \mu^* - ((c^*)^3 - c^*) - 2\pi^2 \kappa c^*. \quad (15.3)$$

We impose c^*, μ^* as strong (Dirichlet) data on the boundary (the manufactured field is not no-flux), then take a few very large Δt steps from the exact field: the time term $\sigma \sim 1/\Delta t$ vanishes and each Newton solve lands on the discrete *steady* FE system — staying in the correct branch of the non-monotone cubic $f'(c)$ (small- Δt marching can drift to a spurious steady branch, a real Cahn–Hilliard pitfall).

We report L^2 and H^1 errors for *both* fields c and μ , plus the discrete mass error, over 4 mesh levels at $p = 1$ and 4 at $p = 2$ (two points is not a study). The observed orders match theory to two figures: L^2 order $p+1$, H^1 order p , for each field.

Reading a convergence order. If the error is $E(h) \approx C h^k$, then between two meshes with h and $h/2$, $\log_2(E(h)/E(h/2)) = k$. That ratio — the *observed order* — is what you compare against the theoretical value. It is far more diagnostic than any single error value: the *slope* is the physics of the method; the *intercept* C is just problem-dependent. We fit the slope by least squares across all levels.

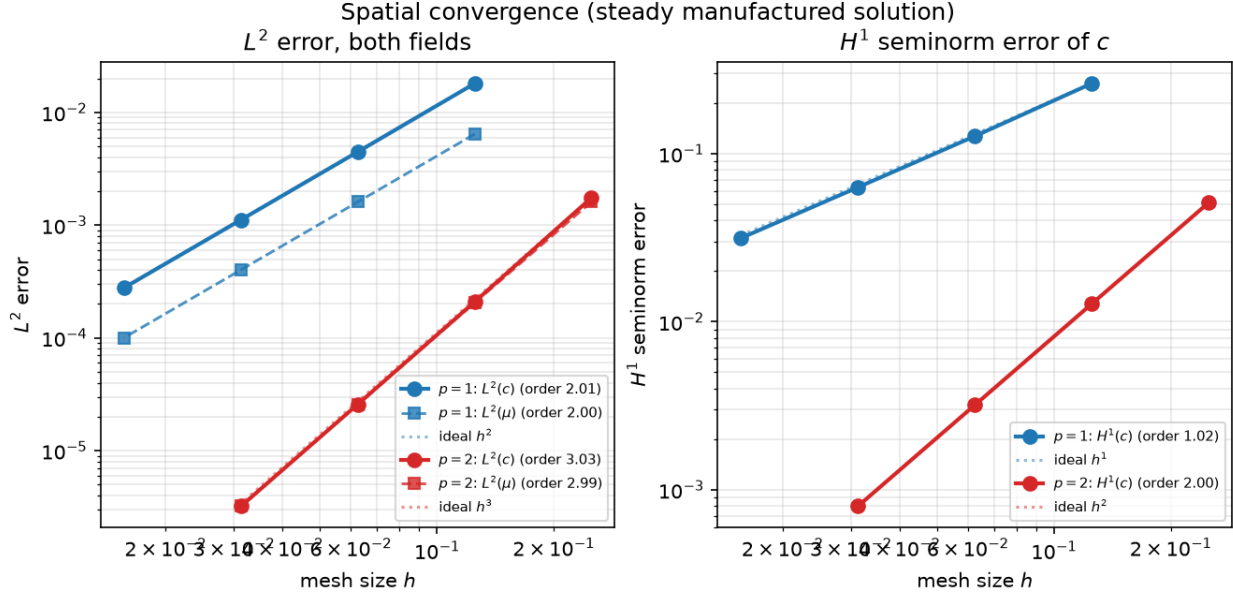


Figure 15.1: Spatial convergence under mesh refinement, steady MMS. *Left:* L^2 error of both c and μ falls along the ideal slope $p+1$ (order 2.01 for $p = 1$, 3.03 for $p = 2$). *Right:* the H^1 seminorm of c falls one order slower, along h^p (order 1.02 and 2.00). Both fields, both norms, at the predicted rate.

15.3 Algebraic error: control it, or the plot is worthless

A convergence plot contaminated by solver error measures the solver, not the discretization. The check is direct: hold the mesh fixed and *tighten* the Newton tolerance from 10^{-4} to 10^{-12} . If the measured L^2 error moves, the plot was solver-limited. Here the linear solve is a direct sparse LU (`splu`) — exact to round-off, so Newton is the only algebraic knob — and the discretization error moves by just 3.4×10^{-13} of itself (Fig. 15.3, left). The discretization error is what we claim to measure. *A study in which tightening the tolerance changes the answer must fail this check before any order is quoted.*

15.4 Temporal order: a *verified* self-convergence reference

For the time error we do not need a manufactured source. We freeze an *over-resolved* mesh, march the same smooth initial blend with a shrinking Δt over 4 levels, and compare each run against a very fine- Δt reference march on the identical mesh — so the spatial error cancels and only the time error remains. Backward Euler (**BDF1**) is first order; the two-step **BDF2** is second order. At constant Δt , BDF2 runs its textbook coefficients $\frac{3}{2}c^{n+1} = 2c^n - \frac{1}{2}c^{n-1} + \Delta t(\cdots)$; Chapter 17 returns to what happens when Δt is allowed to *vary* (and why the coefficients must then change).

Self-convergence has one pitfall: if the reference is not fine enough it is not “truth,” and the measured order is meaningless. So we *verify* it: halving the reference Δt leaves the BDF2 order unchanged ($2.06 \rightarrow 2.04$, a shift of 0.021), and a Richardson estimate bounds the reference’s own error at 3.2×10^{-9} — orders below the coarsest sampled error. Only then do we trust it.

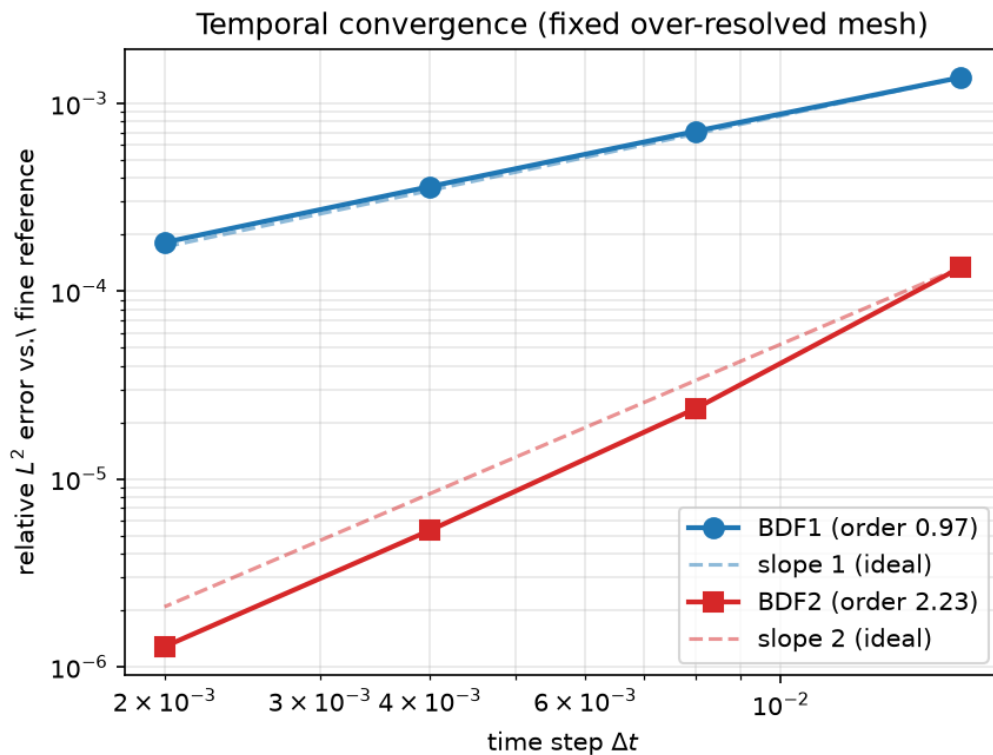


Figure 15.2: Temporal convergence at fixed mesh. Relative L^2 error against the verified fine- Δt reference: BDF1 tracks slope 1 (order 0.97), BDF2 slope 2 (order 2.23). At the same Δt , BDF2 is over an order of magnitude more accurate.

15.5 Ways to get a wrong answer

An honest workflow includes the failure modes. Each of the following produces a plausible-looking but *wrong* order; learning to read the signature is the skill.

failure	measured order	signature to diagnose
wrong BC (natural, not pinned)	0.00	error flat and huge — boundary dominates
wrong source (κ term dropped)	0.00	converges to the <i>wrong</i> steady state
under-resolved feature ($k=6$)	1.18	pre-asymptotic — mesh cannot resolve it
under-resolved reference	-0.01	error saturates at the reference's own error
loose Newton (algebraic)	—	caught by the invariance check above

The last is subtle: on this smooth, well-conditioned problem Newton converges in one or two iterations, so even a loose tolerance does *not* contaminate the order — which is exactly why you must *verify* algebraic control rather than assume it. The lesson is not “Newton is always fine” but “check, do not assume.”

15.6 Run it

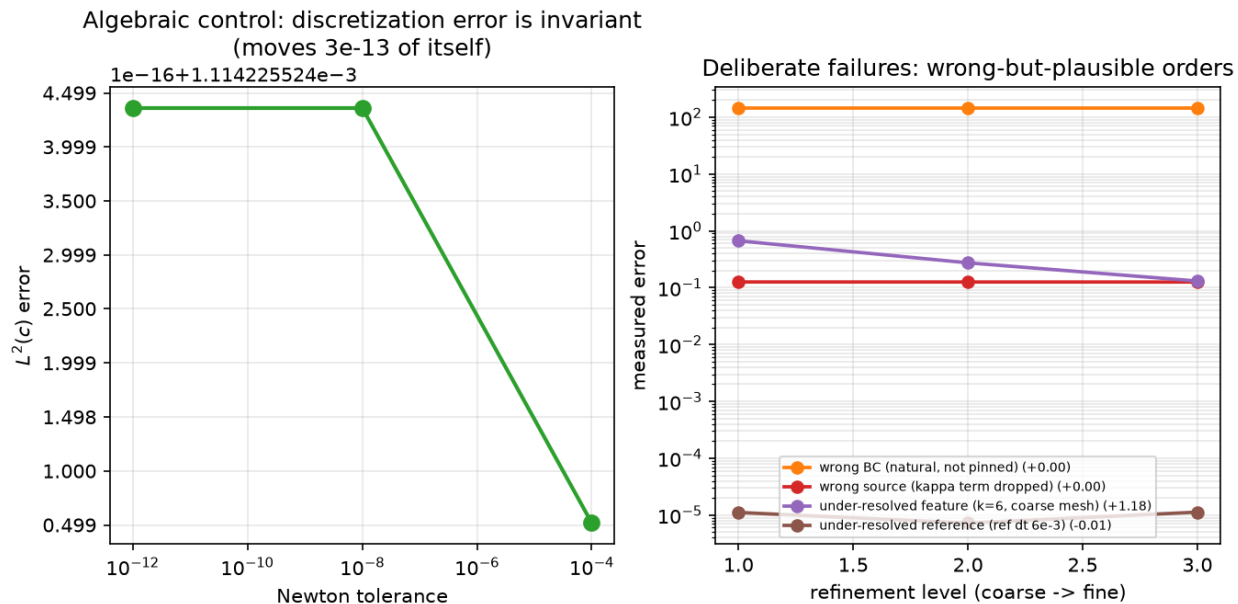


Figure 15.3: *Left*: algebraic control — the L^2 error is flat as the Newton tolerance sweeps $10^{-4} \rightarrow 10^{-12}$ (discretization error invariant). *Right*: four deliberate failures, each a wrong or erratic order the student must diagnose.

Run it.

```
cd computational/01_convergence
python run.py          # spatial + algebraic + temporal + failures
```

It runs the steady-MMS spatial study (L^2/H^1 for c and μ), the algebraic-control sweep, the verified temporal study for BDF1 and BDF2, and the four deliberate failures, printing eight PASS/FAIL gate checks. Compare with `EXPECTED.md` and the figures above.

Measured self-check (your `run.py` should reproduce these to a couple of significant figures; the observed *orders* are the load-bearing quantities). Spatial rows list $L^2(c)$ error coarse $\rightarrow$ fine and the three fitted orders $L^2(c) / H^1(c) / L^2(\mu)$.

study	discretization	error (coarse $\rightarrow$ fine)	orders (expect)
spatial	$p = 1$	$1.8 \times 10^{-2} \rightarrow 2.8 \times 10^{-4}$	2.01/1.02/2.00 (2/1/2)
	$p = 2$	$1.7 \times 10^{-3} \rightarrow 3.2 \times 10^{-6}$	3.03/2.00/2.99 (3/2/3)
temporal	BDF1	$1.4 \times 10^{-3} \rightarrow 1.8 \times 10^{-4}$	0.97 (1)
	BDF2	$1.3 \times 10^{-4} \rightarrow 1.3 \times 10^{-6}$	2.23 (2)

Algebraic control: the error moves 3.4×10^{-13} of itself under a Newton-tolerance sweep $10^{-4} \rightarrow 10^{-12}$.

15.7 Code walk-through

The study lives in `convergence.py`, which drives the production brick `src/diffsim/physics/cahn_hilliard.py` — the same brick as Physics Chapter 3.

1. Steady manufactured solution and its sources.

```
def c_star(x): return cos(pi x) cos(pi y) # time-independent
def f_c(x, t): return 2*pi^2 * M * mu_star(x) # residual, added back
```

`c_star`, `mu_star` are the exact fields; `f_c`, `f_m` the sources that make them exact. Because they are steady, the measured error is pure spatial error.

2. Errors on the solver's quadrature.

```
cv, cg = st._gp_scalar(c_free) # value + gradient at Gauss points
l2c += sum(w * (cv - c_star(xq))**2) # L2
h1c += sum(w * ||cg - grad_c_star(xq)||**2) # H1 seminorm
```

Integrated with the *same* quadrature the solver uses — no nodal proxy. Both fields, both norms, plus $\int c$ for the mass error.

3. Algebraic control and verified reference.

```
algebraic_control(p=1, level=5, tols=(1e-4, 1e-8, 1e-12)) # error invariant
verify_reference(ref_dts=(2e-4, 1e-4)) # order stable + Richardson bound
```

15.8 Explore on your own

Q15.8.1 Extend the $p=1$ spatial sweep to level 7 (128×128). Does the observed order stay at 2? Because the study is steady the answer is Δt -independent — but round-off eventually floors the error. Predict at which level the L^2 error stops halving-squared and confirm it.

Q15.8.2 Change the manufactured wavenumber to $k = 2$ ($\cos(2\pi x) \cos(2\pi y)$). The error *intercept* should jump (the field is harder to resolve) while the *order* stays $p + 1$. Verify both, and explain why the slope is invariant but the constant is not.

Q15.8.3 Reproduce the *wrong-source* failure: drop the $\kappa \nabla^2 c$ term from `f_m`. You measured order 0.00 — the scheme converges, but to the wrong steady state. Why is a wrong-but-nonzero order more dangerous than an obvious crash?

Q15.8.4 The under-resolved-feature failure used $k = 6$ on levels 2, 3, 4 and measured order 1.18. Push the same field to levels 5, 6, 7 — at what resolution does the order recover to 2? This is the boundary of the asymptotic range.

Q15.8.5 The $p2$ spatial study uses coarser levels (2–5) than $p1$ (3–6). Count the degrees of freedom at $p=2$ level 4 versus $p=1$ level 6 and compare the two L^2 errors per DOF — which discretization is more accurate for the same cost here?

Chapter 16

Boundary conditions, numerically

Key idea. The mixed Cahn–Hilliard system has *two* boundary conditions, not one — and the *way* each is imposed decides what the simulation conserves and whether its matrix stays symmetric. A no-flux (natural) boundary costs nothing and conserves mass exactly; a Dirichlet boundary is imposed by editing rows of the linear system and *deliberately* breaks conservation; a weakly imposed (penalty) boundary trades exactness for a symmetric matrix.

Learning objectives. By the end you can: derive the two boundary terms of the mixed (c, μ) weak form (the mass flux $M \nabla \mu \cdot \mathbf{n}$ and the interfacial term $\kappa \nabla c \cdot \mathbf{n}$) and say which BCs make each vanish; compare natural, wall-energy (wetting/Robin), Dirichlet, and manufactured (MMS) conditions on whether they conserve mass; verify the flux balance $d/dt \int_{\Omega} c dV = - \oint_{\partial\Omega} \mathbf{J} \cdot \mathbf{n} dS$ directly from a run; and contrast weak (penalty) versus strong (row-replace / symmetric-elimination) imposition, including the penalty parameter’s exactness-vs-conditioning trade-off.

Prerequisites. Integration by parts / the weak form, symmetric vs. non-symmetric linear systems; Chapter 15 (this track’s convergence study) and Physics Chapter 5 (the physical wall energy); Chapter 00 (environment green).

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory (an 8 GB card is ample); solver `splu` on the (c, μ) saddle (the tiny weak-vs-strong demo is a dense NumPy solve, no GPU). `run.py` completes in ≈ 30 – 60 s, comparable to Physics P1’s quick mode (≈ 75 s wall, measured, RTX 6000 Ada; mostly kernel compile); expect the same first-run Warp-compile overhead.

This concept shows, with measured numbers, the full boundary structure of the mixed system, the taxonomy of physical boundary conditions, how each constraint is implemented in the matrix, and the flux balance that ties mass conservation to the wall. It connects to Physics Chapter 3 (whose spinodal runs are all no-flux) and to the manufactured-solution study of Chapter 15 (which pinned both fields as an MMS device).

16.1 Two boundary conditions live in the weak form

The mixed system solves *two* equations, so integrating each by parts produces *two* surface integrals. For the conserved transport equation, multiply by a test function w and integrate by parts:

$$\int_{\Omega} w c_t + M \int_{\Omega} \nabla w \cdot \nabla \mu = M \oint_{\partial\Omega} w (\nabla \mu \cdot \mathbf{n}) dS. \quad (16.1)$$

For the chemical-potential equation $\mu = f'(c) - \kappa \nabla^2 c$, the same treatment of the $\nabla^2 c$ term gives a *second* surface integral:

$$\int_{\Omega} q \mu = \int_{\Omega} q f'(c) + \kappa \int_{\Omega} \nabla q \cdot \nabla c - \kappa \oint_{\partial\Omega} q (\nabla c \cdot \mathbf{n}) dS. \quad (16.2)$$

So the boundary enters in *two* places: the mass flux $M \nabla \mu \cdot \mathbf{n}$ in (16.1), and the interfacial term $\kappa \nabla c \cdot \mathbf{n}$ in (16.2). A complete boundary condition must say what happens to *both*.

Natural (no-flux + no-wetting). Setting $\nabla \mu \cdot \mathbf{n} = 0$ and $\nabla c \cdot \mathbf{n} = 0$ makes both surface terms vanish, and you impose them by *doing nothing* — simply omit the integrals. This is why they are called *natural*. Physically: no mass crosses the wall (zero flux) and the interface meets the wall at 90° (no wetting preference). Total mass $\int_{\Omega} c dV$ is then conserved to solver tolerance; every spinodal run in Physics Chapter 3 is a natural run, which is why its mass drift was machine epsilon.

Wall free energy (wetting / Robin). A real thin film often *wets* its substrate: one component prefers the wall. Adding a surface free energy $\oint g_s(c) dS$ to (3.1) makes the $\kappa \nabla c \cdot \mathbf{n}$ term *nonzero* — the condition becomes $\kappa \nabla c \cdot \mathbf{n} + g'_s(c) = 0$, a Robin-type wetting condition that still keeps $\nabla \mu \cdot \mathbf{n} = 0$ (no mass crosses, so mass is *still* conserved). It is genuine physics but lives in the multiphase film brick, not this binary brick (see docs/dev/2026-07-13-m5-device-assembly.md, the A2 wall term); we name it here for completeness and use it in the device track.

16.2 The taxonomy

condition	imposes	conserves mass?
natural (no-flux)	$\nabla \mu \cdot \mathbf{n} = 0, \nabla c \cdot \mathbf{n} = 0$	yes (exactly)
wall energy (wetting)	$\kappa \nabla c \cdot \mathbf{n} + g'_s(c) = 0$	yes
prescribed composition	$c = g$ (Dirichlet on c)	no (reservoir)
prescribed potential	$\mu = g$ (Dirichlet on μ)	no
periodic	c, μ identified across faces	yes
manufactured (MMS)	$c = g_c$ and $\mu = g_\mu$	no (device)

Manufactured vs. physical Dirichlet. Chapter 15 pinned *both* c and μ on the boundary. That is an **MMS device**, not a physical reservoir: it forces the discrete solution to match a known field so the error is measurable. A *physical* composition-controlled contact pins only $c = g$; what μ should be there follows from the interior physics and is *not* ours to set. Pinning both over-determines the boundary — perfectly fine for manufacturing an exact solution, wrong as a model of a real contact. This tutorial’s “Dirichlet” runs pin $c = g$ (and, for the binary brick’s convenience, $\mu = 0$); read that $\mu = 0$ as a modelling shortcut, not a law (Exercise).

16.3 How a constraint is implemented

A prescribed value $u_i = g$ is a linear-algebra edit of the assembled Newton system $A \delta x = r$. There are three ways, and they differ in what they do to the matrix (Fig. 16.1), shown on a tiny $-u'' = 0$ system whose exact solution is the straight line $u = a + (b - a)x$:

1. **Row replacement.** Overwrite row i with the identity row, $A_{i,:} \leftarrow e_i^\top$, $r_i \leftarrow g - x_i$. Fast and exact, but it *breaks symmetry* (the column is left intact) — measured `symmetric` =asymmetric. A non-symmetric matrix rules out a Cholesky/CG path.
2. **Symmetric elimination.** Also zero column i , moving its known contribution to the right-hand side. Keeps the matrix symmetric-positive-definite (`symmetric` =symmetric) at the cost of touching the RHS. This is what a Jacobian assembly should do so the tangent stays SPD.
3. **Weak (penalty / Nitsche).** Add a large penalty to the diagonal, $A_{ii} += \beta$, $r_i += \beta g$. The value is imposed only *approximately*, to $\mathcal{O}(1/\beta)$, but symmetry is preserved and no row is destroyed — convenient on hanging-node or immersed boundaries where a clean row does not exist.

On a hanging-node octree, a Dirichlet node that is also *constrained* (a hanging node interpolated from its parents) cannot simply have its row replaced — the constraint operator T must be applied first, and the boundary value distributed to the parent nodes. The brick’s constraint matrix T handles this; the row edits above act in the *reduced* (free-node) space $A = T^\top K T$.

Strong Dirichlet: row replacement vs symmetric elimination on a tiny $-u'' = 0$ system

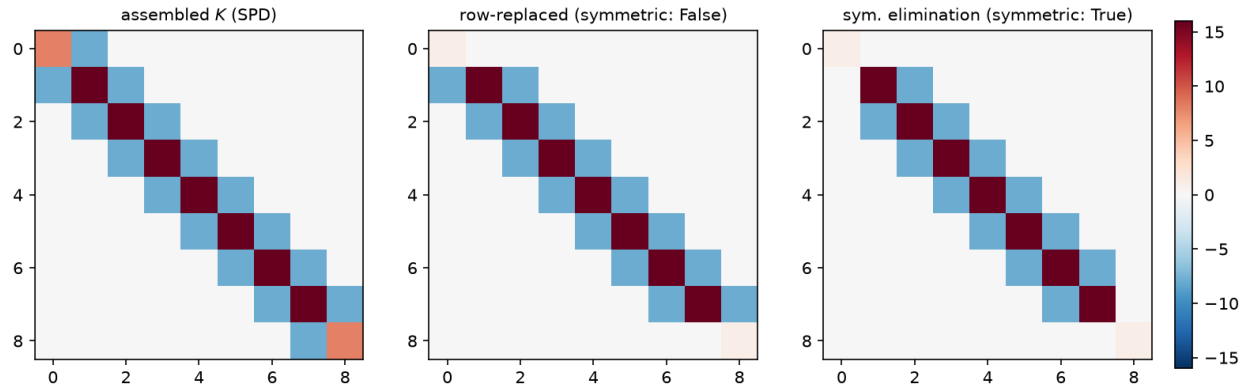


Figure 16.1: Strong Dirichlet on a tiny $-u'' = 0$ system. Left: the assembled SPD stiffness. Middle: row replacement pins the value but leaves the matrix asymmetric (the column is untouched). Right: symmetric elimination pins the value *and* keeps the matrix symmetric.

16.4 The flux balance ties conservation to the wall

Integrating the conserved dynamics over the whole domain and using the divergence theorem gives an exact statement:

$$\frac{d}{dt} \int_{\Omega} c dV = M \oint_{\partial\Omega} \nabla \mu \cdot \mathbf{n} dS = - \oint_{\partial\Omega} \mathbf{J} \cdot \mathbf{n} dS, \quad \mathbf{J} = -M \nabla \mu. \quad (16.3)$$

The mass changes at *exactly* the rate material crosses the wall. In the discrete conserved scheme these two are the *same* quantity, so we measure the net flux directly as dm/dt from the quadrature-mass series (Fig. 16.3). Under no-flux the boundary integral is zero and $dm/dt = 5.5 \times 10^{-15}$

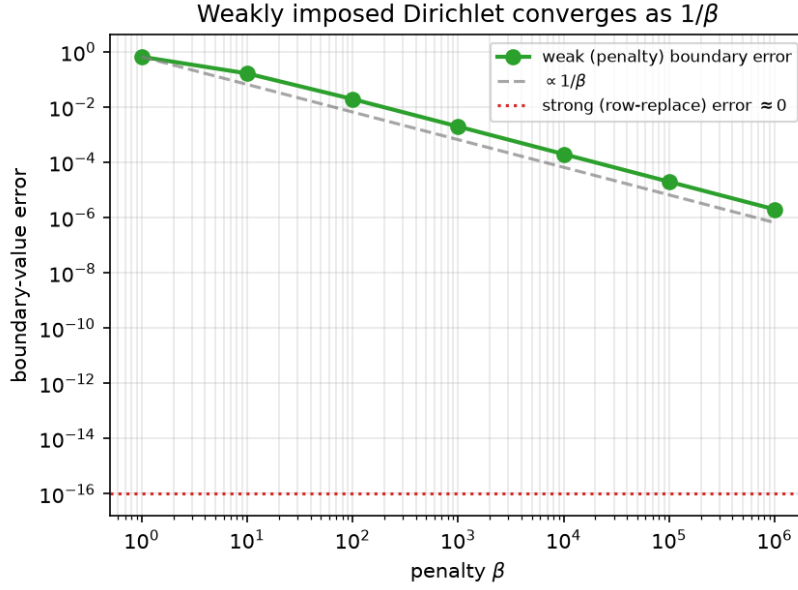


Figure 16.2: A *weakly* imposed Dirichlet value: the penalty method’s boundary error falls as $1/\beta$ (measured 2.0×10^{-4} at $\beta = 10^4$), approaching the strong (row-replace) value whose error is $\approx 1.0 \times 10^{-16}$. The trade is exactness for a symmetric matrix and no destroyed row.

(machine precision — mass exactly conserved). Under Dirichlet the wall is a reservoir: the influx is +4.15 early and decays to +0.54 as the interior equilibrates to the wall value and the reservoir stops pumping. Watching the flux shut off is the measurable signature of (16.3).

16.5 Run it

Run it.

```
cd computational/02_boundary_conditions
python run.py          # fields + flux + BC matrix + weak/strong
```

It marches the *identical* spinodal blend (same fixed seed) under the boundary taxonomy, measures the flux balance, and runs the weak-vs-strong Dirichlet demo, printing nine PASS/FAIL checks. Compare with EXPECTED.md and Figs. 16.4–16.3.

Measured self-check (fixed seed in bc.py; field values are seed-dependent, the conservation/pinning contrast is not).

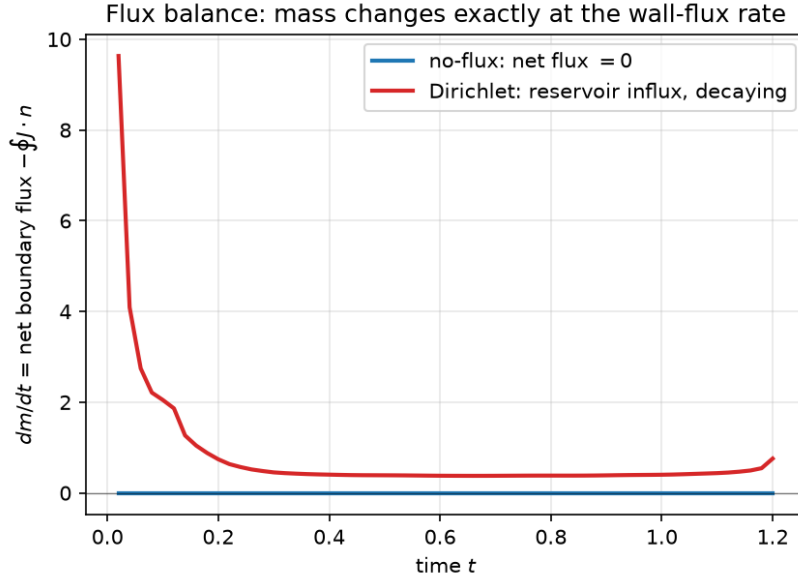


Figure 16.3: Flux balance (16.3). dm/dt (the net boundary flux) is flat at zero for no-flux, and a decaying reservoir influx for Dirichlet.

	natural (no-flux)	Dirichlet ($c = 0.9$)
mass drift $ \Delta m $	1.3×10^{-16}	0.95
net flux dm/dt	5.5×10^{-15} (zero)	$+4.15 \rightarrow +0.54$
edge composition	-0.13 (free)	+0.90 (pinned)
field difference	2.00 (full range)	
mixed (L/R pinned)	drift 0.29, edge +0.63	
weak BC error ($\beta=10^4$)	2.0×10^{-4} ($\propto 1/\beta$)	

16.6 Code walk-through

The study lives in `bc.py`, driving the same production brick.

Selecting the boundary.

```
# natural: pass nothing -- the default weak form is no-flux + no-wetting
st = CahnHilliardStepper(dm, M, kap, dt, order=1)
# Dirichlet: pass the boundary node indices + the data functions
st = CahnHilliardStepper(dm, M, kap, dt, order=1, dirichlet=bidx,
                        gc_fn=lambda x,t: wall, gm_fn=lambda x,t: 0)
```

The three impositions (tiny system).

```
dirichlet_strong(K, b, i, g) # row replace (asymmetric)
dirichlet_strong(K, b, i, g, symmetric=True) # + column elim (SPD kept)
dirichlet_penalty(K, b, i, g, beta) # weak,  $O(1/\beta)$ 
```

Flux balance and the test matrix.

```
flux_balance("noflux"); flux_balance("dirichlet") # dm/dt = net flux
bc_test_matrix() # natural / +0.9 / -0.9 / 0.0 / mixed, one blend
```

Same blend, two boundary conditions -- the wall pins the Dirichlet field

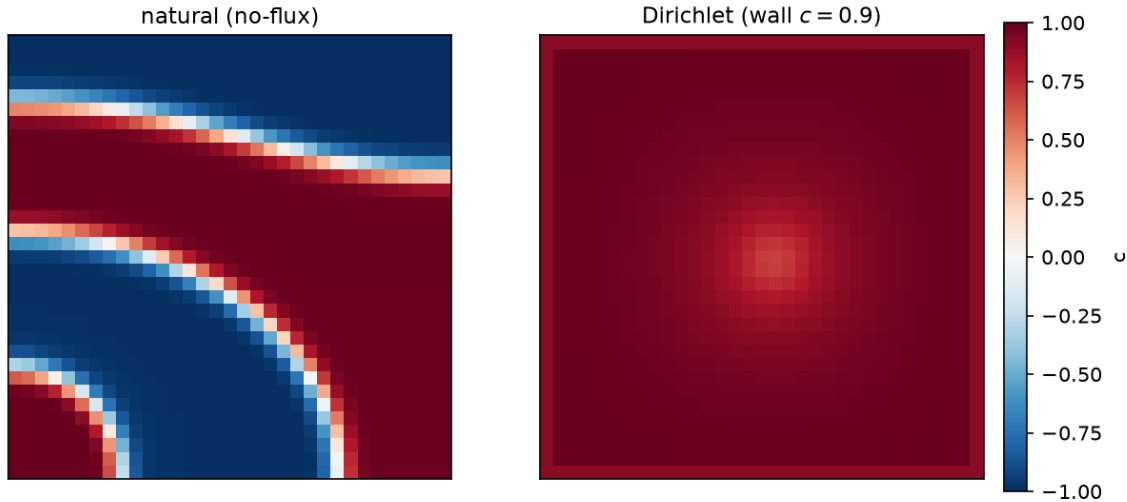


Figure 16.4: The same blend under two boundary conditions. Left: no-flux — domains meet the wall freely. Right: Dirichlet — the edge is pinned to $c = 0.9$ and that phase is drawn to the boundary. The measured field difference is 2.00 (the full order-parameter range).

16.7 Explore on your own

Q16.7.1 Sweep the Dirichlet wall value $g \in \{-0.9, 0, +0.9\}$ in the test matrix. You measured drifts 0.94, 0.09, 0.95 — explain why $g = 0$ nearly conserves mass (hint: the initial mean) and why that is a coincidence of the IC, not a property of the boundary condition.

Q16.7.2 Pin only two opposite edges (the mixed row: drift 0.29, edge +0.63). How do the boundary layer and the mass drift compare to the all-edges case? This mixed condition is common in device geometries (contacts on two sides, sealed on the others).

Q16.7.3 Under our Dirichlet runs the boundary μ is pinned to 0. Argue what μ at a composition-pinned contact *should* be, and why pinning both c and μ is a modelling choice (an MMS device) rather than a law. What would a c -only Dirichlet require of the solver?

Q16.7.4 Raise the penalty β past 10^6 in the weak-BC demo. The boundary error keeps falling as $1/\beta$ — but at some β the matrix condition number explodes and the solve loses accuracy. Find that crossover and explain the exactness-vs-conditioning trade.

Q16.7.5 The no-flux mass drift is $\sim 10^{-16}$ — machine epsilon, not solver tolerance. Refine the mesh one level and re-measure: does the drift grow with the number of nodes (accumulated round-off) or stay at machine level? What does that tell you about how conservation is enforced here?

Chapter 17

Octree refinement, dynamic AMR, and temporal adaptivity

Key idea. Resolution should follow the physics. In space, an *octree* puts small elements only where the field is sharp. We build up in three honest steps: first a *static* octree refined to a fixed geometric criterion (not yet adaptive); then the full *solution-adaptive* cycle — estimate, mark, refine/coarsen, 2:1 balance, rebuild, conservatively transfer the field *and the BDF history*, continue — driving the production Cahn–Hilliard brick so that refinement tracks a moving interface while mass is conserved to machine precision across every remesh. In time, an *LTE-controlled* step ladder shrinks Δt during the violent onset and grows it through slow coarsening; we report its *real* cost, not a fictional saving, and show why growing Δt forces BDF2 into its variable-coefficient form — the same scheme that must survive a remesh.

Learning objectives. By the end you can: distinguish octree refinement to a *fixed geometric criterion* from solution-adaptive AMR; build the full dynamic-AMR cycle (estimate, mark, refine/coarsen, 2:1 balance, rebuild T , transfer state *and* BDF history, continue) and verify it (mass conserved across remeshes, free energy continuous, the refined region tracking the interface); show why a conservative L2 projection on the common refinement conserves $\int_{\Omega} c$ exactly while naive nodal injection does not; measure whether dynamic AMR *pays* (error-vs-dofs against a uniform-fine reference); build a *real* cost accounting for adaptive time-stepping against a matched-accuracy fixed- Δt sweep; explain why a growing Δt forces BDF2's variable-coefficient form; and analyze the *space-time interaction* of a remesh during a variable-step BDF2 march — why both history levels must transfer for order 2 to survive.

Prerequisites. Chapter 15 (the BDF1/BDF2 order this chapter must preserve under a varying step *and* across a remesh); Chapter 16 (the constraint operator T , $A = T^{\top}KT$, that hanging-node refinement reuses); Physics P2 (*Adaptive time stepping*). Chapter 00 (environment green).

Expected cost. Any CUDA GPU, 2-D meshes (max octree level 7 for the AMR reference, 16,641 nodes at most), < 1 GB device memory; solver `splu` (the CH saddle is indefinite, as elsewhere in this track). The static octree, conservative-transfer, dynamic-AMR cycle, and space-time studies are fast (each < 20s); the error-vs-dofs study pays for a uniform-level-7 reference (≈ 65 s), and the temporal-adaptivity cost accounting is comparable to physics P1's quick mode.

This concept measures spatial adaptivity — static octrees, then true dynamic AMR — and temporal adaptivity on the Cahn–Hilliard brick. It builds on the convergence study (Chapter 15: the BDF2 order adaptivity must preserve).

17.1 Octree refinement and hanging-node constraints

A spinodal field is sharp at its interfaces and smooth everywhere else. A uniform mesh fine enough to resolve the interfaces wastes almost all its elements on the flat bulk. An *octree* mesh refines only where a criterion says to. Here we refine any element within a band of a *static* circle and leave the rest coarse.

Honesty about “adaptive.” The refinement in this section is to a *fixed geometric criterion* (a circle we chose), not to the *running solution’s* error. That is octree refinement, not solution-adaptive AMR. It is the lead-in: the next sections turn it into the real thing. A reader must not mistake a static refinement demo for dynamic AMR.

Hanging nodes. When a fine element meets a coarse one, the fine element has a node in the middle of the coarse element’s edge — a *hanging node* that is not a true degree of freedom. It is *constrained* to its coarse neighbors, and that constraint lives in the matrix T (recall $A = T^\top K T$). The solver never sees the hanging nodes as unknowns; `build_constraints` builds T , and the *same* stepper runs unchanged — which is why the tutorial can hand the CH brick an adaptive mesh and step.

The measured result: refining along one circle at level 6 gives 1,173 nodes against 4,225 for the uniform mesh at the same finest h — a $3.6\times$ reduction in degrees of freedom (Fig. 17.1), and the CH brick steps on it.

17.2 What dynamic AMR additionally requires — and the conservative transfer that makes it work

Static refinement is the easy half. *Dynamic* AMR must, every few steps: **(a)** estimate the running solution’s error, **(b)** *mark* elements to refine or coarsen, **(c)** refine/coarsen, **(d)** enforce 2:1 balance (no element more than one level from its neighbor), **(e)** rebuild the mesh and the constraint operator T , **(f)** *transfer* the solution — *and the BDF history* — to the new mesh, and **(g)** continue the march. Step (f) is where conservation is won or lost.

Why naive transfer leaks mass. When a region is coarsened, the state must be moved to the new mesh without losing what it represents. A *naive* transfer (inject the value at the surviving nodes) throws away sub-cell structure and changes $\int_\Omega c$. On a field of droplets smaller than the coarse cell (Fig. 17.2), injection loses 20% of the mass while cell averaging conserves it to 1.0×10^{-17} . A dynamic-AMR implementation cannot use nodal injection for the restriction: mass would leak every time a region coarsens.

The finite-element conservative transfer. On an unstructured octree the “cell average” generalizes to the L^2 (Galerkin) projection with a consistent mass matrix. The subtlety is the quadrature: to integrate $\int N_a^{\text{new}} c_{\text{old}} dV$ *exactly*, we integrate on the *common refinement* of the two meshes

Octree refinement (static criterion): 1173 vs 4225 nodes
(3.6x fewer dofs, same finest h)

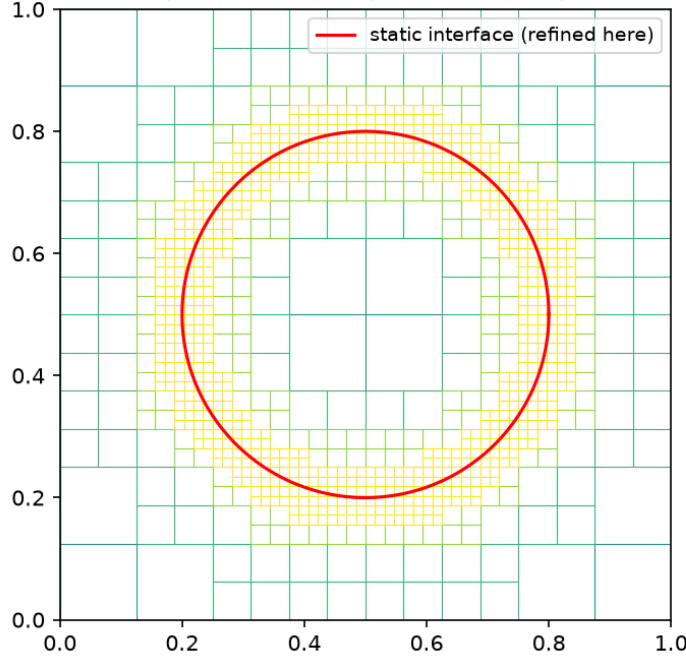


Figure 17.1: Octree refinement along a *static* circular interface: small elements hug the circle, large elements fill the bulk. The mesh has 1,173 nodes versus 4,225 uniform ($3.6\times$ fewer dofs). Element color is refinement level.

— the finer cell wins in every region — so both the new basis function and the old field are linear on each quadrature cell. Because the finite-element space contains the constant and its basis is a partition of unity, this preserves $\int_{\Omega} c$ *exactly*: with $A = T_{\text{new}}^{\top} M_{\text{new}} T_{\text{new}}$ and $b = T_{\text{new}}^{\top} \int N^{\text{new}} c_{\text{old}} dV$,

$$\int_{\Omega} c_{\text{new}} = \mathbf{1}_{\text{free}}^{\top} A c_{\text{new}} = \mathbf{1}_{\text{free}}^{\top} b = \int_{\Omega} c_{\text{old}}. \quad (17.1)$$

Measured on the CH brick’s meshes: the transfer conserves $\int_{\Omega} c$ to 1.1×10^{-16} under refinement and 5.6×10^{-17} under coarsening (machine precision, *both* directions), it converges at order 1.95 against an analytic field, and a refine-then-coarsen round-trip on a representable field is lossless. The same operator carries the two BDF history levels c^n, c^{n-1} , each conserved independently. This is the primitive the cycle below is built on.

17.3 Dynamic AMR: the full solution-adaptive cycle

We now run the whole cycle on a moving interface. The indicator is the per-element $|\nabla c|$ (large on the diffuse interface, ≈ 0 in the flat bulk); elements above a fraction of its maximum are marked to refine, elements below a smaller fraction are marked to coarsen (only complete sibling groups collapse). After marking we refine, coarsen, re-impose 2:1 balance, rebuild the mesh, constraints and device operators, conservatively transfer c, μ and both history levels, and continue. The initial condition is adapted *before* the march — otherwise the first few steps run on an under-resolved interface and inject error a uniform-fine run never pays.

Conservative transfer: naive restriction loses sub-cell mass; averaging is exact

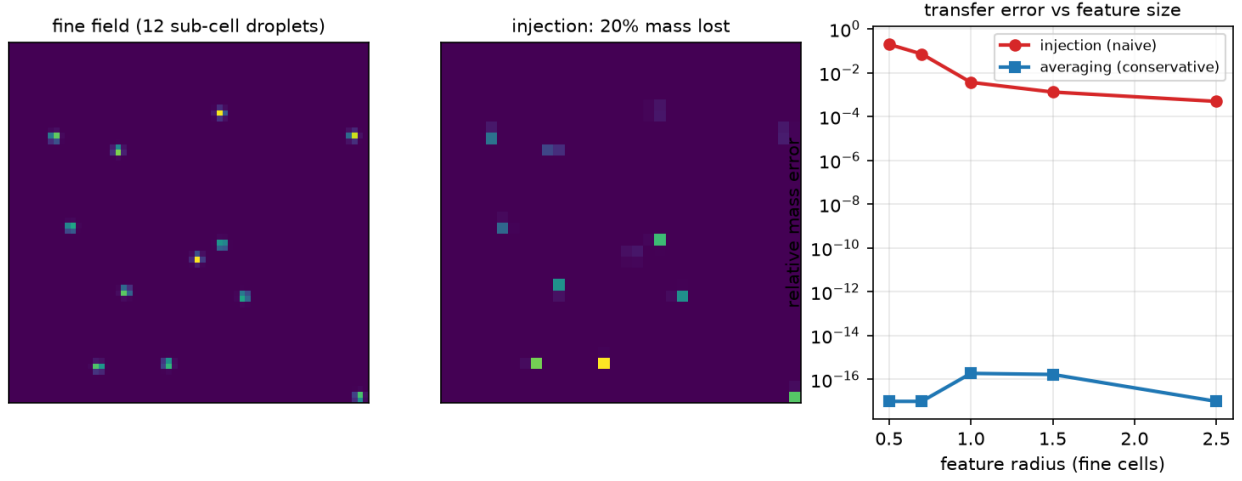


Figure 17.2: Conservative transfer, illustrated on a structured grid. Left: a fine field of sub-cell droplets. Middle: naive injection to the coarse grid misses droplets and loses 20% of the mass. Right: injection error grows sharply once the feature drops below the coarse cell, while cell averaging is exact at every size.

Measured cycle. Over the run the march remeshes 4 times. Across *every* remesh, $\int_{\Omega} c$ changes by at most 5.6×10^{-17} (machine precision), and the free energy F by at most 4.7×10^{-10} — the transfer projection error only, never a spurious jump (Fig. 17.4). The refined region tracks the interface: the overlap between the finest cells and the interface band is 1.00. The march stays stable (bounded, finite) at a peak of 1,413 dofs.

17.4 Does dynamic AMR pay? Error versus dofs

Adaptivity is only worth its machinery if it reaches a given accuracy at fewer degrees of freedom. We score the AMR final state against a uniform-level-7 reference (16,641 dofs) and compare with a uniform-mesh sweep (Fig. 17.5). Dynamic AMR reaches error 2.5×10^{-3} at 1,413 dofs — comparable to the uniform level-6 mesh (2.0×10^{-3} at 4,225 dofs) but at $2.5 \times$ fewer dofs than a uniform mesh would need for the same error, and $2.1 \times$ less wall time. The advantage grows with the finest level, because the uniform mesh pays for resolution everywhere while AMR pays only along the interface.

17.5 Space-time interaction: a remesh during a BDF2 march

Dynamic AMR and BDF2 are not independent. BDF2 is a two-step method: its current step reads c^n and c^{n-1} . When a remesh lands between two steps, *both* history levels must be transferred to the new mesh, or the scheme is fed an inconsistent history. We measure the temporal order of a variable-step BDF2 march that remeshes at its midpoint, two ways: **both** (conservatively transfer c^n and c^{n-1} , keeping the step-ratio $r = \Delta t / \Delta t_{\text{prev}}$ so the variable-coefficient BDF2 continues uninterrupted) and **drop** (transfer only c^n and restart as BDF1 for one step — the “only c^n is not enough” failure).

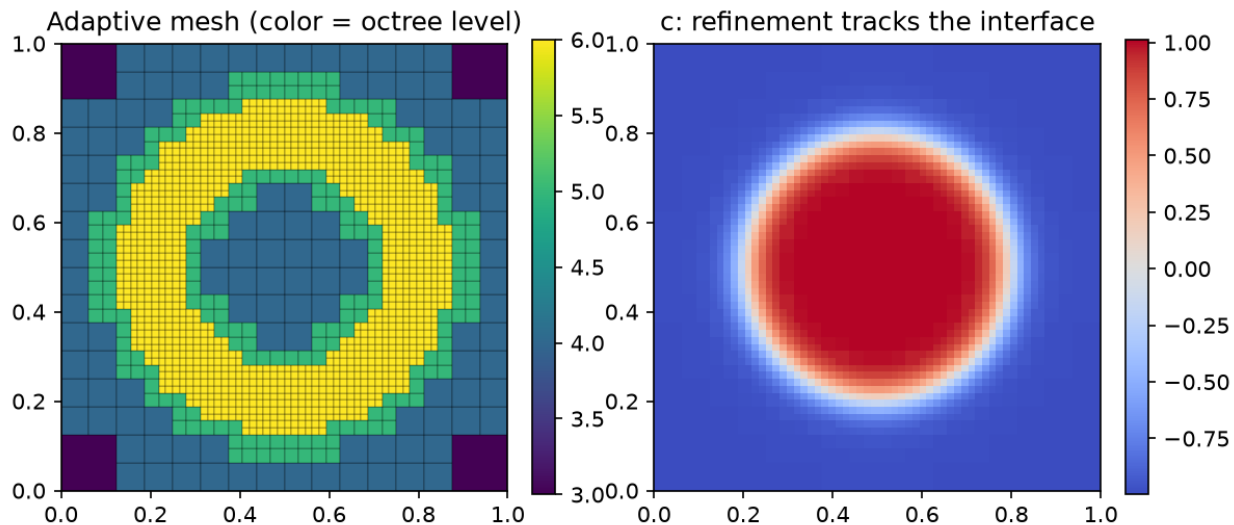


Figure 17.3: Dynamic AMR over a shrinking droplet. Left: the adaptive mesh, colored by octree level — the finest cells (level 6) hug the interface in a 2:1-balanced annulus grading down to a coarse bulk. Right: the field c ; refinement tracks exactly where the field varies.

Measured order recovery. Transferring both history levels recovers order 1.94 across the remesh — second order survives. Dropping the second level forces a BDF1 restart; a *single* restart still recovers order 2.11 (one first-order local error is $O(\Delta t^2)$, the same size as the global BDF2 error), but it inflates the error constant by $1.7\times$. The lesson: a remesh is order-preserving *only* when the full BDF history transfers; and because the step-doubling controller sees the inflated post-remesh error as a larger LTE, dropping history also makes the controller shrink Δt needlessly. This is why C1 and C3 are one story.

17.6 Temporal adaptivity: the LTE step ladder

Phase separation is violent at onset and glacial during coarsening. A fixed Δt must be small enough for the onset, wasting effort ever after. The *adaptive* march sizes each step to the physics using a *local truncation error* (LTE) estimate: it takes one step of size Δt and two of size $\Delta t/2$ from the same state, and the scaled difference estimates the step error. If it is below tolerance, accept and *grow* Δt ; otherwise reject, halve, and retry.

Over a quench the accepted step spans $1.0 \times 10^{-5} \rightarrow 6.6 \times 10^{-2}$ (a $6623\times$ range, Fig. 17.6): tiny during onset, large during coarsening.

Real cost accounting (not a fiction). The honest way to value adaptivity is to *count the work*, not to multiply the horizon by the smallest step. To reach $t = 0.60$ the adaptive controller took 138 accepted and 31 rejected steps — 507 Newton solves in all (each accepted step costs *three*: one full plus two half), 2,373 Newton iterations, at final error 2.4×10^{-2} . A matched-accuracy fixed-dt sweep finds $\Delta t = 5.0 \times 10^{-4}$ reaches the same accuracy at 4,380 Newton iterations — so adaptivity saves $1.8\times$ the Newton work and $2.0\times$ the wall time here (Fig. 17.7). The saving is real but *modest*, and it grows with the horizon. This is the honest picture the old “horizon / smallest-step” ratio badly overstated.

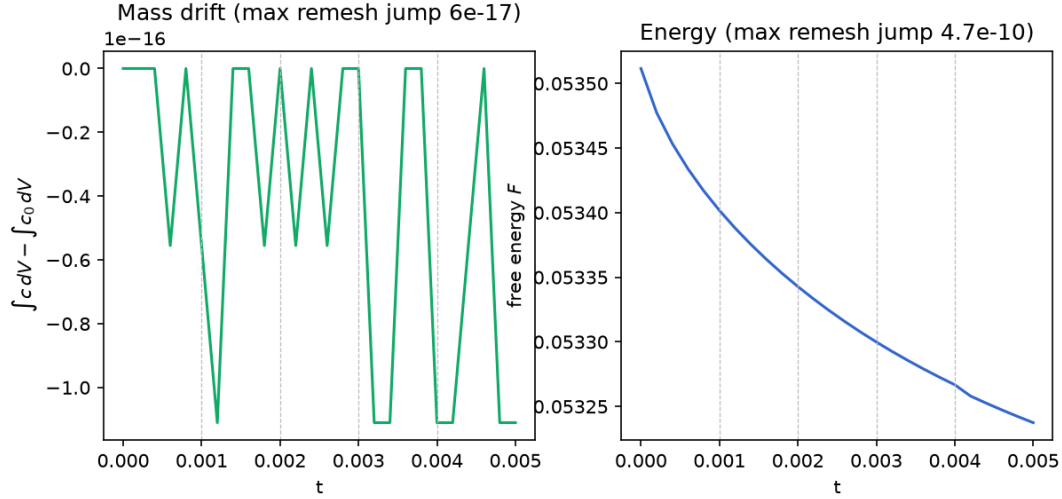


Figure 17.4: Conservation across remeshes (dashed lines mark remesh times). Left: the mass drift stays at round-off. Right: the free energy is continuous across each remesh — the conservative transfer moves the state without a spurious energy jump.

17.7 Why variable-coefficient BDF2

Growing Δt safely has a hidden cost. The textbook BDF2 coefficients $\frac{3}{2}, [2, -\frac{1}{2}]$ are derived assuming a *constant* step. Feed them a step that changed from the last one and the scheme is only *formally* second order — its measured order collapses toward 1. We measure this *here*, with a tutorial-local constant-coefficient march (we keep the real alternating history spacing but force the coefficient ratio $r = \Delta t / \Delta t_{\text{prev}}$ to 1 each step): order 0.93. The fix rebuilds the coefficients each step from the actual ratio,

$$c_0 = \frac{1 + 2r}{1 + r}, \quad \text{history weights } [1 + r, -\frac{r^2}{1+r}], \quad (17.2)$$

which reduces to the constant-step form at $r = 1$ and stays consistent when $r \neq 1$. The brick uses it, and the same alternating sequence now measures order 2.01 — second order restored. It is the *same* scheme, and the *same* order-preservation requirement, that the space-time section stressed across a remesh.

17.8 Run it

Run it.

```
cd computational/03_adaptivity
python run.py                # static octree + dynamic AMR + cost + BDF2
                             order
```

Prints the PASS/FAIL checks. Compare with EXPECTED.md and Figs. 17.1–17.7.

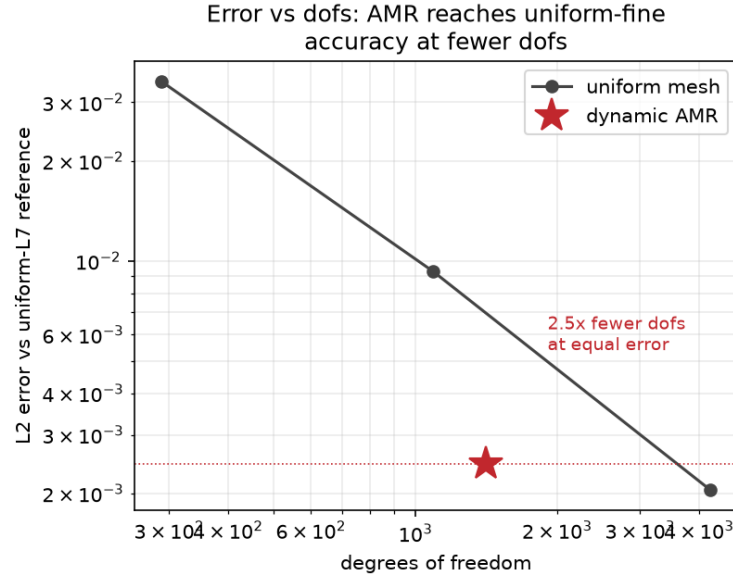


Figure 17.5: Error versus dofs. The uniform sweep (black) traces the accuracy–cost curve; dynamic AMR (red star) sits to its left — uniform-fine accuracy at $2.5\times$ fewer dofs.

Measured self-check (fixed seed; node counts are exact, step counts are seed/card-dependent).

study	measured	note
octree nodes (adaptive / uniform)	1,173 / 4,225	$3.6\times$ fewer
FE transfer mass (refine / coarsen)	1.1×10^{-16} / 5.6×10^{-17}	machine precision
FE transfer order	1.95	2nd-order projection
AMR remesh mass / energy jump	5.6×10^{-17} / 4.7×10^{-10}	conserved / continuous
AMR interface overlap	1.00	refinement tracks interface
AMR error vs dofs	$2.5\times$ fewer	at equal error
remesh order (both / drop history)	1.94 / 2.11	order 2 preserved
adaptive Newton saving	$1.8\times$	modest, grows with horizon
BDF2 order (variable / constant)	2.01 / 0.93	order preserved vs collapsed

17.9 Code walk-through

The dynamic-AMR cycle (the Phase-3 core).

```
ind = interface_indicator(steeper, mesh)          # |grad c| per element
refine_mask, coarse_mask = mark(tree, ind, ...)
new_tree = adapt_tree(tree, refine_mask, coarse_mask) # +2:1 balance
steeper, mesh, cons = remesh(steeper, mesh, cons, new_tree, ...)
# remesh conservatively transfers c, mu AND both BDF history levels
```

Conservative transfer (the piece AMR needs).

```
conservative_transfer(mesh_old, cons_old, tb, mesh_new, cons_new, tb,
                      [c, mu, hist0, hist1])      # L2 proj on common refinement
```

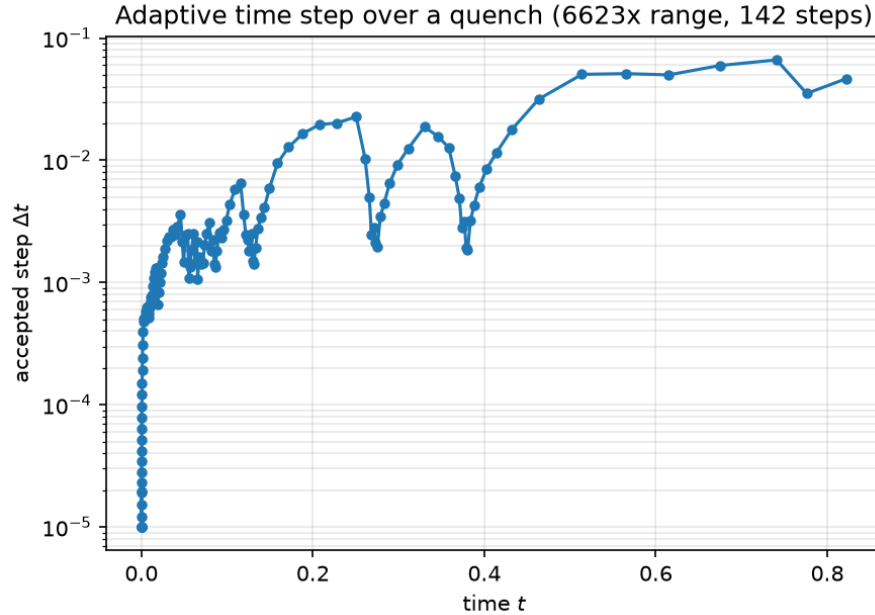


Figure 17.6: The accepted time step over a quench (log scale). Δt is tiny during the spinodal onset and grows orders of magnitude through coarsening — the property that makes long horizons affordable.

Real cost accounting.

```
instrumented_march(st, t_end, tol)    # counts acc/rej, solves, Newton, wall
adaptive_cost()                     # + matched-accuracy fixed-dt sweep
```

Constant- vs variable-coefficient BDF2 (both measured).

```
_var_march(dm, dt)                  # brick: coefficients from the real  $r = dt/dt_{prev}$ 
_const_march(dm, dt)                # forced  $r=1$  on a varying history -> the bug
```

17.10 Explore on your own

Q17.10.1 Change the refinement indicator from $|\nabla c|$ to a gradient-jump (the difference in ∇c across element faces). Does the refined region still track the interface? Which indicator marks fewer cells for the same final error?

Q17.10.2 Coarsen more aggressively (raise the coarsen fraction) and watch the free-energy jump across remeshes. At what point does the coarsening transfer error stop being negligible — and does 2:1 balance re-refine the region you tried to coarsen?

Q17.10.3 Extend the cost accounting to $t_{\text{end}} = 1.2$. Does the adaptive Newton saving grow (the coarsening tail lengthens)? Plot the saving versus horizon and find where adaptivity first pays for its step-doubling overhead.

Q17.10.4 In the space-time study, remesh *every few steps* instead of once. With “drop” (a BDF1 restart at every remesh) the number of restarts now grows as $\Delta t \rightarrow 0$ — predict and then measure what happens to the global order, and contrast with “both.”

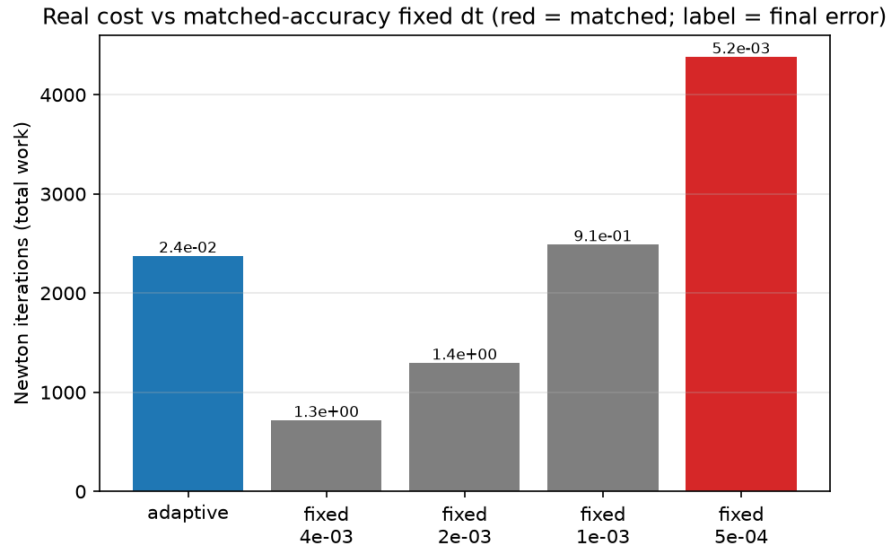


Figure 17.7: Real cost: total Newton iterations for the adaptive march versus the fixed-dt sweep (red = the matched-accuracy fixed run; labels are final error). Adaptivity's advantage is measured, not assumed.

Q17.10.5 Force the constant-coefficient bug back on a *fixed* Δt ($r \equiv 1$) and confirm you recover order 2 — then argue why the alternating sequence, not the fixed one, is the test that exposes the variable-coefficient issue.

Chapter 18

The solver ecosystem

Key idea. Every implicit step ends in a linear solve of the *indefinite* (c, μ) Cahn–Hilliard saddle, and the choice of solver is a *correctness* decision before it is a speed one. A general-purpose GPU direct solver (cuDSS) **diverges** on the polynomial CH saddle — it does no pivoting, and the polynomial well is indefinite in the spinodal band — so the field blows up. The pivoted CPU direct solver `splu` is safe at small scale; the CH-specific `blockch` preconditioner, which splits the saddle into symmetric-positive-definite sub-solves, is the choice at scale. *Not* cuDSS on the raw CH block.

Learning objectives. By the end you can: explain why the mixed (c, μ) Jacobian becomes an indefinite saddle once $f'' = 3c^2 - 1 < 0$ in the spinodal band; reproduce the measured divergence (cuDSS on the polynomial saddle: $[-552, 542]$ vs `splu`’s bounded $[-1.03, 1.01]$); recognize the one-iterate-residual trap (cuDSS’s residual $\sim 5 \times 10^{-13}$ is tiny even though its march diverges); describe how `blockch/blockch_dev` splits the saddle into SPD sub-solves instead of ever forming one indefinite factorization; and read `recommend_solver`’s measured rationale across the 2-D and 3-D regimes.

Prerequisites. Chapter 00 (environment green); Computational C1 (this track); Physics P1 (the (c, μ) Cahn–Hilliard model this chapter solves).

Expected cost. Any CUDA GPU; the live parts here are 2-D (largest live benchmark $\sim 33k$ dofs); the 3-D numbers are *cited* from the dev notes, not re-run. Solver headline: cuDSS diverges on the raw polynomial CH saddle, `splu` is safe. The live run is comparable to Physics P1’s quick mode (≈ 75 s wall, RTX 6000 Ada).

This concept teaches how to choose, using a real marched divergence test — the honest way to check a solver on the CH saddle — plus the live 2-D timing and the cited 3-D scaling tables.

18.1 The real test: march it and look at the solution

Each Newton iterate solves $A\delta = r$, where A is the mixed (c, μ) Cahn–Hilliard Jacobian — a *saddle*-structured, *indefinite* sparse matrix. The tempting shortcut is to solve one captured system and check its residual. That shortcut **lies**: on the polynomial saddle, cuDSS’s residual at a single Newton iterate is 4.6×10^{-13} (as small as `splu`’s 1.4×10^{-12}), yet *marching* the real 20-step spinodal with cuDSS blows the field up. A small residual does not imply a small error when the matrix is ill-conditioned. So we measure the *marched solution* (Table 18.1, Fig. 18.1).

energy	solver	final c range (20 steps)	verdict
polynomial	splu	$[-1.03, 1.01]$	ok
polynomial	cuDSS	$[-552, 542]$	DIVERGED
Flory–Huggins	splu	$[0.06, 0.94]$	ok
Flory–Huggins	cuDSS	$[0.001, 0.999]$	ok

Table 18.1: Marched-solution correctness (level 5, $\Delta t = 0.02$, μ -init consistent). cuDSS blows the polynomial field up by a factor ~ 500 ; splu is safe on both energies.

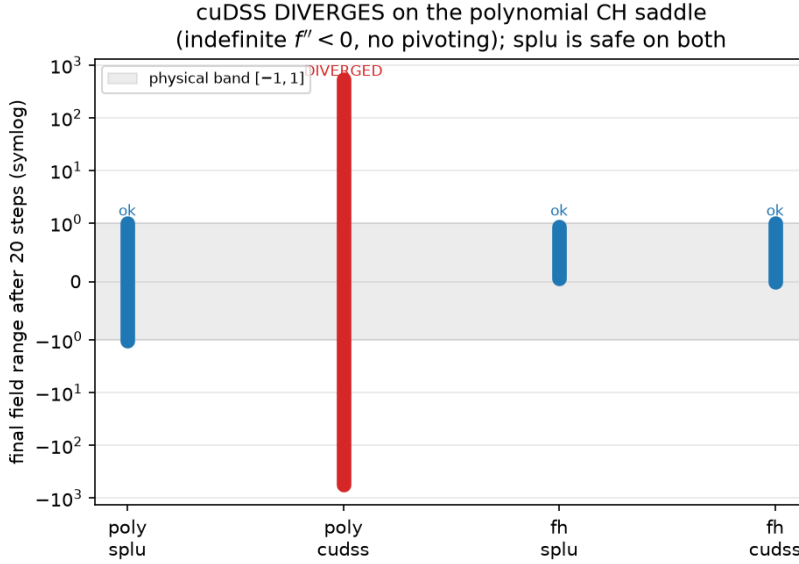


Figure 18.1: Final field range after 20 steps for each (energy, solver). cuDSS on the polynomial saddle escapes the physical band $[-1, 1]$ by orders of magnitude (symlog axis); the pivoted **splu** and cuDSS-on-Flory–Huggins stay inside.

18.2 Why: an indefinite saddle with no pivoting

The polynomial well $f(c) = \frac{1}{4}(c^2 - 1)^2$ has bulk curvature $f''(c) = 3c^2 - 1$, which is *negative* across the spinodal band $|c| < 1/\sqrt{3}$ — exactly the region the dynamics drives the field into. There the (c, μ) block is genuinely *indefinite*. cuDSS factorizes without the partial pivoting an indefinite system needs, so its solution is wrong, and the error compounds over the Newton and time iterations until the field escapes. Flory–Huggins survives because its entropic curvature $f'' = A\left(\frac{1}{c} + \frac{1}{1-c}\right) \geq 4A$ stays *positive* and better-conditioned — but you cannot count on that in general, and the production default must be safe for the polynomial well.

The recipe this forces.

- **small:** **splu** — pivoted CPU direct, safe on the indefinite saddle.
- **at scale:** **blockch** / **blockch_dev** — an FGMRES around a block preconditioner that splits the saddle into *SPD* sub-solves (mass, W_1 , W_2), so no monolithic indefinite factorization is ever formed.
- **not** cuDSS on the raw CH block. (cuDSS is fine on the well-conditioned SPD sub-blocks)

a preconditioner factors, and on block-masked 3-D film patterns.)
 The `--solver` auto default of `splu` for CH is correct.

18.3 The 2-D timing (context only)

With correctness settled, we still measure speed — but note that for CH the fastest 2-D solver is *disqualified*. Timing the linear solve (factorize + solve, cold and warm, CUDA-synchronised median) shows cuDSS overtaking splu as the 2-D problem grows past $\sim 10^4$ dofs (at 33,282 dofs it is $\sim 2.9\times$ faster; Fig. 18.2). But that speed is unusable on the CH saddle because cuDSS returns a wrong answer there. The lesson: *fastest* is not *correct*.

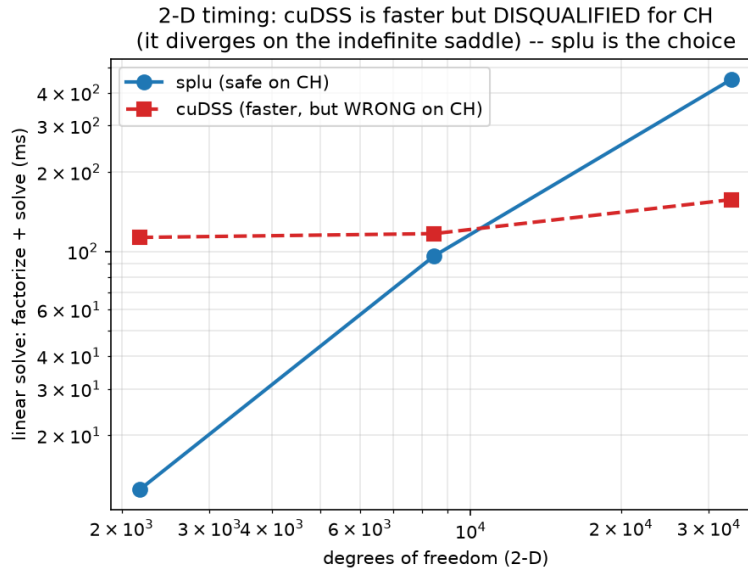


Figure 18.2: 2-D linear-solve timing on the real CH Jacobian. cuDSS is faster above $\sim 10^4$ dofs — but it *diverges* on the CH saddle (dashed, disqualified), so `splu` is the 2-D choice.

18.4 The 3-D story, cited

The 3-D numbers take minutes per step, so we cite the dev-note tables. At scale the *memory* of a direct factorization compounds the accuracy problem: cuDSS’s fill hits the 48 GB card at $\sim 811,008$ dofs (a > 16 min factorization that never returns). `blockch_dev` beats masked cuDSS $8.4\times$ on the slab64 solve and keeps marching where cuDSS ceilings; a block-masked cuDSS pattern cuts the fill ($17.7 \rightarrow 3.70$ s/call) below the ceiling; AMGX adds nothing at production Δt (mass-dominated inners); and matrix-free reaches 6,389,760 dofs on one card (Fig. 18.3).

18.5 Decision support, with a rationale

Rather than a one-line `choose_solver(dim, dofs)`, the tutorial’s `recommend_solver` weighs dofs, nnz, block structure, precision, memory, factorization reuse, tolerance, Δt , and hardware, and returns a *rationale* that begins with the correctness constraint:

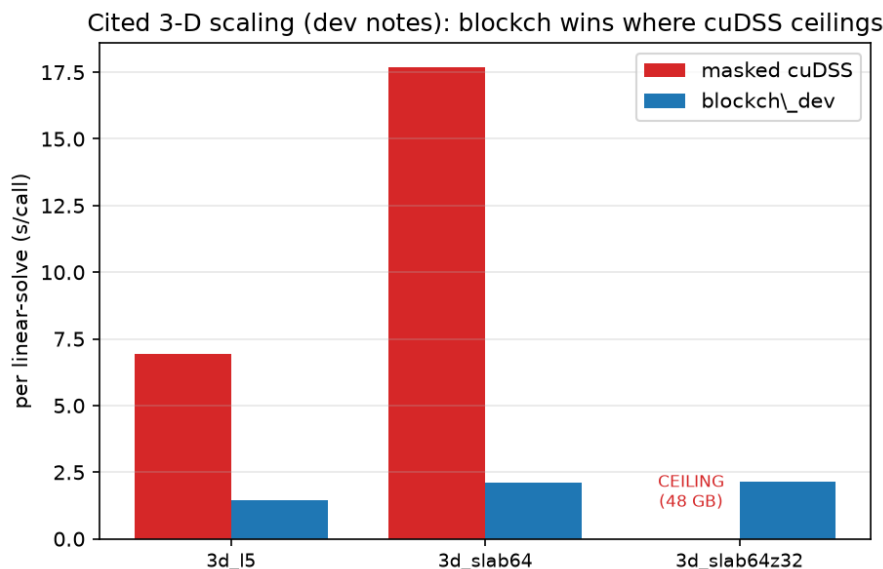


Figure 18.3: Cited 3-D per-solve cost (dev notes): masked cuDSS versus `blockch_dev`. `blockch` is $8.4\times$ cheaper at slab64 and marches where cuDSS’s factorization ceilings on 48 GB.

regime	dofs (order)	recommendation
2-D, small	$< 2 \times 10^4$	<code>splu</code> (pivoted; safe on saddle)
2-D, large	$< 5 \times 10^5$	<code>splu</code> / <code>blockch</code> (<i>not</i> cuDSS)
3-D, small	$< 5 \times 10^5$	masked cuDSS / <code>blockch_dev</code>
3-D, large	up to $\sim 1.5 \times 10^6$	<code>blockch_dev</code>
3-D, extreme	$> \text{int32}$ / memory	matrix-free (or multi-GPU)

Every recommendation names *why* — and never picks cuDSS on the raw indefinite CH block.

18.6 Code walk-through

The real correctness test (marched, not one-shot).

```
march_divergence() # marches 20 steps per (energy, solver); c.min/max
```

It marches the production stepper and reads the final field range — the only honest test on an indefinite saddle.

The trap, and the supported capture API.

```
solver_correctness() # one-iterate residual: cuDSS looks fine (~1e-13)!
st = CahnHilliardStepper(..., capture_system=True) # no monkeypatch
A, b = st.last_system
```

The one-iterate residual is captured through a supported stepper flag (not a `solve_linear` monkey-patch) — and it *lies*, which is the point.

18.7 Explore on your own

Q18.7.1 Reproduce the divergence: march the polynomial spinodal with cuDSS for 5, 10, 15, 20 steps and plot $\max |c|$ versus step. Is the blow-up gradual or sudden, and what does that say about

how the unpivoted-factorization error compounds?

Q18.7.2 cuDSS survives Flory–Huggins because $f'' \geq 4A > 0$. Lower the FH regularization floor (push c toward 0 or 1 so f'' spikes) and re-march. Does cuDSS still survive, and at what conditioning does it break?

Q18.7.3 The one-iterate residual for cuDSS was 4.6×10^{-13} — tiny — yet the march diverged. Construct the argument (condition number $\times$ residual bounds the error) that explains how a small residual can accompany a large error, and estimate the effective condition number needed.

Q18.7.4 `blockch` splits the saddle into SPD sub-solves. Read the design-laws in `src/diffsim/solvers/linsolve.py`: which naive block preconditioners *also* diverge on FH, and why does the two-factor signed- F form survive? Contrast that (a preconditioner failure) with the cuDSS direct-solve failure here.

Q18.7.5 Using the cited cuDSS memory ceiling ($\sim 811,008$ dofs on 48 GB), estimate it on an 80 GB A100 — and note that even where cuDSS *fits* in 3-D, it is still the raw indefinite block, so the masked pattern (SPD-friendlier) or `blockch_dev` is what you actually run.

Chapter 19

Two dimensions versus three

Key idea. Going from 2-D to 3-D is not a small constant factor. Every refinement multiplies the degrees of freedom by 4 in 2-D but 8 in 3-D; the matrix grows denser per row for reasons rooted in the finite- element *connectivity* (not a finite-difference stencil); the full memory footprint — far more than the CSR values — overruns the card and the 32-bit index arithmetic; and, crucially, the *physics* at equal resolution is different. 2-D is not a cheap 3-D.

Learning objectives. By the end you can: explain why dofs scale as 2^{dim} per refinement level ($\times 4$ in 2-D, $\times 8$ in 3-D); derive the sparsity count $(2p+1)^{\text{dim}} \times n_{\text{fields}}$ from element connectivity, basis order, fields, and dimension, and show why a naive “ $3^{\text{dim}} - 1$ ” finite-difference stencil undercounts it; produce a complete device-memory accounting (not just the CSR values) with `estimate_capacity.py`; explain why int32 CSR indices are an *implementation choice* with a real overflow risk once $\text{nnz} > 2^{31}$, not a hardware limit; and show that 2-D is not a cheap 3-D at equal resolution (interfacial-area density, $S(q)$ wavelength, and percolation differ).

Prerequisites. Finite-element connectivity and CSR sparsity patterns; Chapter 00 (environment green); Chapter 15 and Chapter 18 (this track).

Expected cost. Any CUDA GPU; the live 2-D/tiny-3-D runs use well under 1 GB device memory (an 8 GB card is ample). Solver `blockch` by default in `estimate_capacity.py` (or `splu/cudss`). The 128^3 and 256^3 memory figures below are *estimated* capacity-planning numbers, not runs: 128^3 fits a 48 GB card, 256^3 does not and is never actually launched here. Live-run cost is of the same order as physics P1’s quick mode (≈ 75 s wall, RTX 6000 Ada, mostly kernel compile).

This concept measures the 2-D-vs-3-D growth live on small meshes, accounts the memory completely, and shows that the morphology itself changes with dimension. It is the capstone of the computational track: it explains *why* everything in Chapter 18 (the device assembly path, the `blockch` solver, the matrix-free frontier) exists.

19.1 Where the sparsity really comes from

Refine a mesh by one level and the number of elements multiplies by 2^{dim} — 4 in 2-D, 8 in 3-D. That much is obvious. The density of the matrix is subtler, and it is easy to state wrong.

A row of the Jacobian is a degree of freedom. Its nonzeros are one per (coupled node) $\times$ (field), where two nodes couple iff they share a finite *element*. For a degree- p structured mesh the coupled

nodes form a $(2p+1)^{\text{dim}}$ block — for $p = 1$, 3^{dim} : *nine* nodes in 2-D, *twenty-seven* in 3-D (the node itself included). The mixed (c, μ) system carries 2 fields, and within a coupled node pair the 2×2 field block is dense, so

$$\frac{\text{nnz}}{\text{dof}} = \underbrace{(2p+1)^{\text{dim}}}_{\text{connectivity} \times \text{order} \times \text{dim}} \times \underbrace{n_{\text{fields}}}_{\text{block}} = 18 \text{ (2-D)}, \quad 54 \text{ (3-D)}. \quad (19.1)$$

So the density is set by five things: element **connectivity**, basis **order** p , number of **fields**, the field **block** structure, and **dimension** (hanging-node **constraints** add a few more couplings). A “ $3^{\text{dim}} - 1$ finite-difference stencil” is the wrong mental model: it omits the node itself and the field block, and undercounts. The *measured* averages (18 in 2-D, 47 in 3-D) sit just below the interior maxima 18 and 54 because boundary nodes have fewer neighbours.

The compounding. 2-D dofs grow $\times 3.9/\text{level}$ and carry 18 nnz/dof; 3-D dofs grow $\times 7.0/\text{level}$ and carry 47. Four levels is $256\times$ the dofs in 2-D but $4096\times$ in 3-D — and each 3-D dof is $\sim 3\times$ denser. This is why a code that is instant in 2-D needs a GPU, a device assembler, and a scalable solver in 3-D.

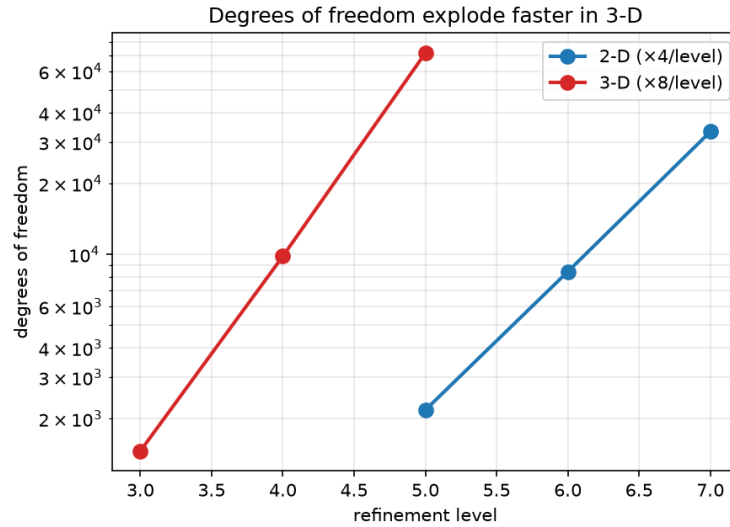


Figure 19.1: Degrees of freedom versus refinement level (log scale). Each 2-D level multiplies dofs by ~ 4 ; each 3-D level by ~ 8 .

19.2 Accounting the memory completely

The CSR *values* are only a fraction of what a real solve holds. The `estimate_capacity.py` CLI accounts every buffer: CSR values, column indices *and* row pointers, the device index mirror a resident assembler keeps, the Newton solution/residual and the BDF history vectors, the local-to-global connectivity map, quadrature/geometry tables, the solver workspace (direct *fill-in* for splu/cuDSS, or the extracted preconditioner blocks for `blockch`), and the saved output snapshot.

Estimate before you launch.

```
python estimate_capacity.py --dim 3 --n 128 --solver blockch
python estimate_capacity.py --dim 3 --n 256 # exceeds 48 GB
python estimate_capacity.py --dim 2 --level 8 --solver splu --idx-bytes 8
```

At 128^3 the total is ~ 8.6 GB — dominated by the *solver workspace*, not the matrix values — and it fits a 48 GB card. At 256^3 it is ~ 68.1 GB and does *not* (Fig. 19.2); only a matrix-free or multi-GPU path remains. The lesson is that “will it fit?” is answered by the *complete* footprint, and the dominant term is often the solver’s, not the matrix’s.

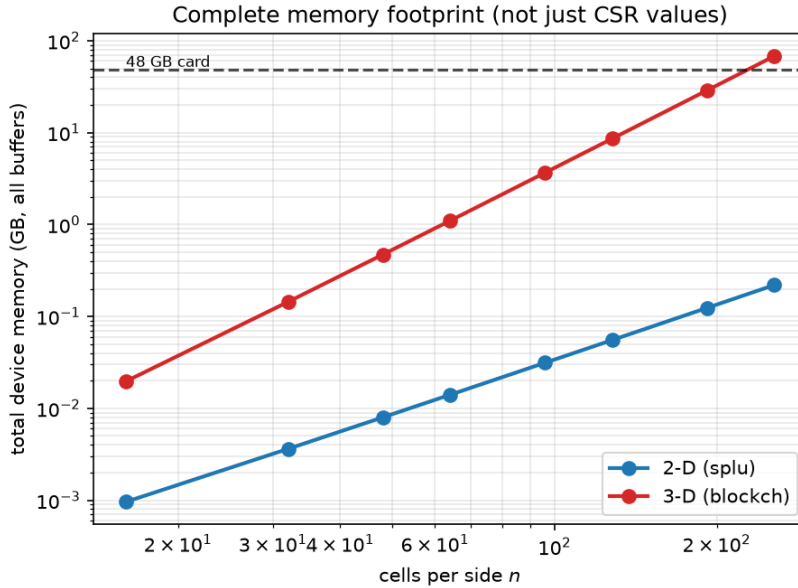


Figure 19.2: Complete device-memory footprint (all buffers) versus n , 2-D and 3-D, with the 48 GB card wall. The 3-D curve crosses it between 128^3 and 256^3 ; the 2-D curve stays far below.

The int32 ceiling is an implementation choice. The CSR index arithmetic here is 32-bit, so $\text{nnz} > 2^{31} \approx 2.1 \times 10^9$ overflows the slot arithmetic. That is a property of *this build*, not a law: an int64 index build lifts the wall (at $2\times$ the index memory), and a distributed (multi-rank) or block-masked pattern sidesteps it by never storing one monolithic CSR. The cited ($M=3, K=2$) films cross 2^{31} , which is why the block-masked pattern exists (it drops the structural zeros the superset stores).

19.3 The device-scale ladder

The live meshes stop at tens of thousands of dofs; production films run at millions (cited, dev notes): 3d_slab64 (417,792 dofs) is the largest cuDSS-stable case on 48 GB; 3d_film128 (6,389,760 dofs) runs on one card with blockch; ($M=3, K=2$) 128x128x48 (8,028,160 dofs) overflows int32; and 256x256x128 (84,541,440 dofs) needs matrix-free or multi-GPU.

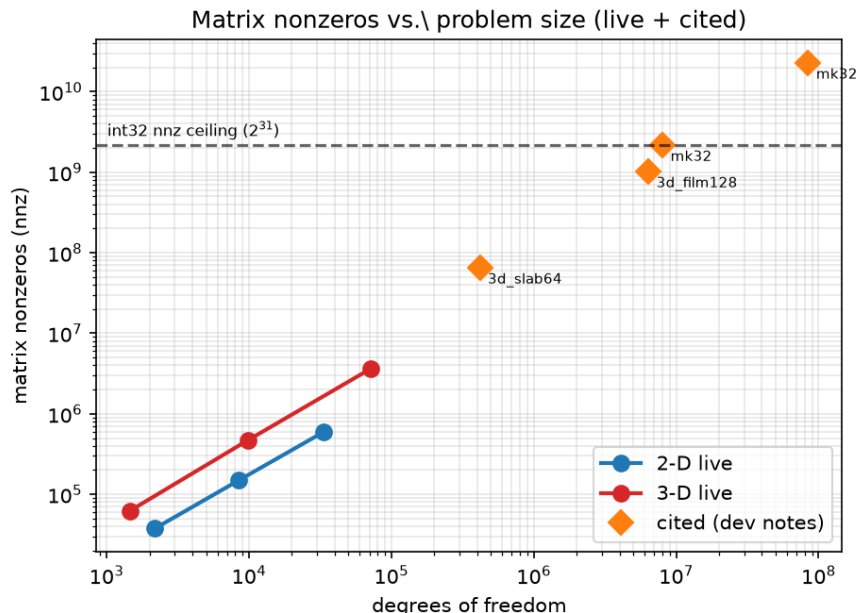


Figure 19.3: Matrix nonzeros versus problem size: live 2-D/3-D runs plus the cited device-scale points, with the 32-bit CSR ceiling (2^{31}). The $(M=3, K=2)$ 128-class films cross it; the Nova-class case (22.8B nnz) has no stored-CSR form at all.

19.4 Physics at equal resolution: 2-D is not cheap 3-D

The deepest reason to care about the cost of 3-D is that you cannot dodge it by working in 2-D: at the *same* resolution the two answer *different physical questions*. We quench the same spinodal blend in 2-D and 3-D at identical $h = 0.062$ and measure the morphology (Fig. 19.4):

- interfaces are *curves* in 2-D but *surfaces* in 3-D, so the interfacial-area density differs (12.3 vs 13.1 in length $^{d-1}$ per unit volume);
- the characteristic $S(q)$ wavelength differs (0.163 vs 0.163);
- connectivity differs in kind: a 3-D blend can be *bicontinuous* (both phases percolate simultaneously), which is topologically impossible for two phases in 2-D. Percolation, coarsening exponents, and stratification under a substrate are all dimension-specific.

So a 2-D study is a legitimate, cheaper *model problem* — excellent for convergence, BCs, solver behaviour — but its *morphology* is not a thin slice of the 3-D answer. Report which question you are asking.

Domain and aspect ratio. Two more knobs the cost hides: a larger *domain* at fixed h scales dofs as $(\text{box size})^{\text{dim}}$ — linearly in area (2-D) but as volume (3-D); and a thin-film *aspect ratio* (wide x, y , thin z , e.g. $64 \times 64 \times 16$) keeps 3-D tractable by shrinking the cheap direction, which is exactly the `slab64` class on the ladder above. Choosing the domain and aspect ratio is choosing where on the cost curve you sit.

Measured self-check (counts exact; step times card-dependent).

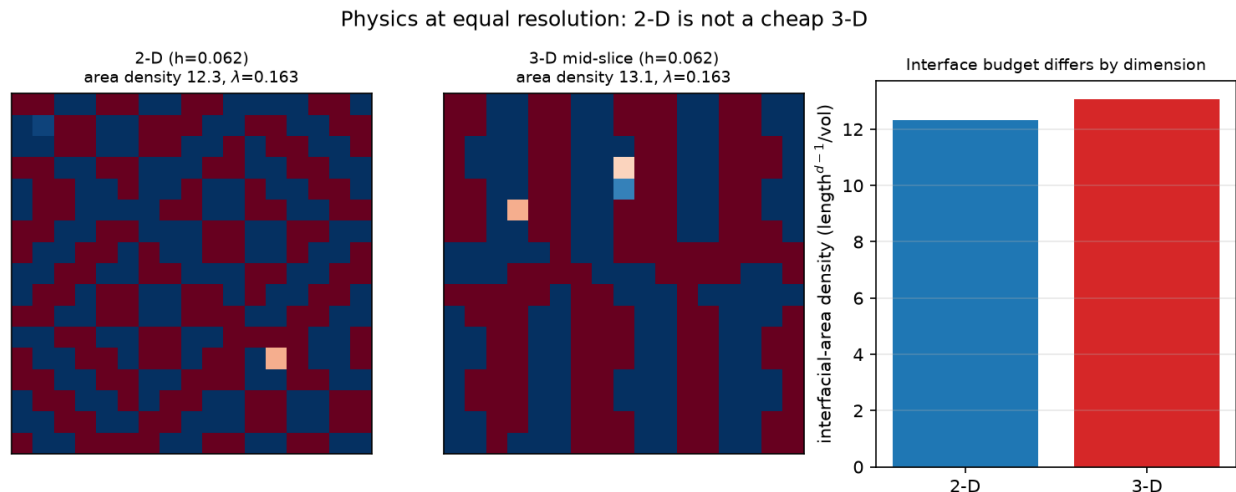


Figure 19.4: Same spinodal quench at equal resolution. Left: 2-D field. Middle: a 3-D mid-slice. Right: interfacial-area density by dimension. The topology, not just the cost, is different.

	2-D	3-D
dofs per level	×3.9	×7.0
nnz per dof (FE model)	18	47
interfacial-area density	12.3	13.1
$S(q)$ wavelength	0.163	0.163
memory (complete): $128^3 \rightarrow 8.6$ GB, $256^3 \rightarrow 68.1$ GB (> 48)		

19.5 Code walk-through

Measuring a real system's size (no monkeypatch).

```
dofs, nnz, step_s = _capture_nnz(dim, level)    # capture_system=True API
sparsity_breakdown(3)    # 27 coupled nodes x 2 fields = 54 nnz/dof
```

Complete memory + physics.

```
memory_accounting(3, 128)    # every buffer, not just CSR values
physics_comparison(level=..)  # 2-D vs 3-D morphology at equal h
```

19.6 Explore on your own

Q19.6.1 Run `estimate_capacity.py` with `--idx-bytes 8 (int64)` at 256^3 . How much does the total grow, and does int64 alone make it fit a 48 GB card, or is the solver workspace still the wall?

Q19.6.2 At equal resolution you measured different interfacial-area densities in 2-D and 3-D. Coarsen both further (more steps) and track the density over time — do the 2-D and 3-D coarsening rates differ? Which dimension coarsens faster, and why?

Q19.6.3 Fit a power law to the live 3-D step times ($\text{step} \sim \text{dofs}^\alpha$). Which stage — host assembly or the solve — drives α , and does that match the C4 story?

Q19.6.4 An octree mesh (Chapter 17) refined only at interfaces changes the 3-D growth law: a surface scales as area, not volume. Estimate the dofs of an interface-refined 3-D film versus the uniform 71,874, and argue why adaptivity matters far more in 3-D.

Q19.6.5 A bicontinuous 3-D morphology has both phases percolating. Sketch a diagnostic (from `diffsim.diagnostics.morphology`) that would distinguish a percolating from a droplet morphology, and explain why the same diagnostic is degenerate in 2-D.

Chapter 20

Nonlinear solver diagnostics

Key idea. Every implicit phase-field step is a *nonlinear* solve. Whether a run is trustworthy, merely slow, or silent garbage is decided inside the Newton loop — and that loop leaves a readable trail: residual and update norms, safeguard activity, a condition number. This chapter teaches you to read that trail, to recognize the handful of ways a nonlinear solve fails, and to act on each. The failures here are not hypothetical: we *run* them and they genuinely fail.

Learning objectives. By the end you can: read a Newton solve’s residual $\|r\|$ and update $\|\delta x\|_\infty$ per iteration and recognize quadratic convergence; choose between an absolute, relative, and residual stopping test and say when each misleads; explain what the trust clamp and the box projection do and see them act; and diagnose the four canonical failures — Newton stagnation, a min- Δt wall, an ill-conditioned Jacobian, and a diverging start — from their signatures, using the decision tree, and leaving a failure checkpoint behind.

Prerequisites. Newton’s method and its quadratic convergence; the mixed Cahn–Hilliard weak form and its analytic Jacobian (Chapter 14); adaptive time stepping and local truncation error (Chapter 17); Physics Chapter 3 (the brick solved here).

Expected cost. Any CUDA GPU, 2-D, < 0.5 GB device memory; solver `splu` (exact to round-off, so Newton is the only nonlinear knob). `run.py` runs every diagnostic in $\approx 20\text{--}30$ s on an RTX 6000 Ada (mostly Warp kernel compile + Python start-up); the studies are single- and few-step.

This chapter instruments the *production* Cahn–Hilliard Newton solve (`src/diffsim/physics/cahn_hilliard.py`, the brick of Chapters 3 and 15) into a repeatable diagnostic workflow. We do not modify the stepper. The per-iteration trajectory it does not log we reconstruct through its public API — re-solving the same step capped at 1, 2, 3, ... Newton iterations and reading its `last_newton` record and captured system — which is exact because a Newton solve from a fixed state is deterministic.

20.1 Anatomy of a healthy Newton solve

Each Cahn–Hilliard step solves the coupled nonlinear system $R(x) = 0$ for $x = (c, \mu)$ by Newton’s method: assemble the residual R and the tangent $J = \partial R / \partial x$, solve $J \delta x = -r$ (with $r = R$), update $x \leftarrow x + \delta x$, repeat. The single most informative thing you can watch is the pair of norms

per iteration: the residual $\|r\|$ (how far the equations are from satisfied) and the update $\|\delta x\|_\infty$ (how much the solution still moves). With a *correct analytic Jacobian* Newton is **quadratically** convergent: near the root the residual is roughly squared each step. On a gentle poly quench we measure

iteration	$\ r\ $	$\ \delta x\ _\infty$	behaviour
1	5.6×10^{-2}	3.1×10^{-3}	first step
2	2.0×10^{-8}	1.2×10^{-6}	residual squared
3	2.6×10^{-16}	1.6×10^{-15}	at round-off

Three iterations, the residual falling from 5.6×10^{-2} to 2.6×10^{-16} (Fig. 20.1, green). That squaring is the fingerprint of a correct tangent; if you see *linear* convergence instead, suspect the Jacobian (Chapter 14 shows how a missing Jacobian term degrades exactly this curve). This is also the honest way to separate *algebraic* error (an unconverged Newton solve) from *discretization* error (Chapter 15): only once $\|r\|$ is at tolerance is the step’s error purely the discretization’s.

20.2 When have you converged? Three stopping tests

“Converged” is a choice, not a fact. Three tests are in common use, all at the same tolerance τ :

- **residual** $\|r\| < \tau$: the equations are nearly satisfied — the most physical test, but its scale is the residual’s, not the solution’s;
- **absolute update** $\|\delta x\|_\infty < \tau$: the iterate has stopped moving — what the stepper actually uses;
- **relative update** $\|\delta x\|_\infty / \|x\| < \tau$: the iterate has stopped moving *relative to its own size*.

On this well-scaled problem, where c and μ are $O(1)$, all three fire at the same iteration 3. They part company the moment the field is rescaled: multiply the solution by 10^{-3} (a change of units, not of physics) and the *absolute* update test now trips $10^3\times$ sooner — it is blind to scale. The lesson: an absolute update tolerance is a statement about your units; prefer a residual or a relative test, or verify that your fields are $O(1)$ before trusting an absolute one.

20.3 Safeguards: what keeps a hard step alive

A raw Newton step off a bad iterate can be enormous or unphysical. The production stepper carries two safeguards, and a diagnostic run should confirm they are working rather than assume it.

Trust clamp. A composition increment beyond 2 units is always a diverging transient (poly’s physical range is $[-1, 1]$, Flory–Huggins’ is $(0, 1)$). When $\max |\delta c| > 2$ the stepper rescales the *whole* update to preserve its direction while capping its length. Unclamped, a quench-onset step at large Δt drives $|c| \sim 37$, where $f'' = 3c^2 - 1 \approx 4100$ poisons any linear solve.

Box projection (Flory–Huggins). The FH bulk energy $f(c) = A[c \ln c + (1 - c) \ln(1 - c)] + Bc(1 - c)$ has $f''(c) \sim A/c$ near the wall $c \rightarrow 0$; the kernel regularizes the logarithm with a cap $1/\varepsilon = 10^4$. To keep early Newton iterates off that cap, each iterate is clipped into $[10^{-3}, 1 - 10^{-3}]$, and the stepper records how many dofs it moved and by how much. On a deep FH quench ($B = 8$) the projection moves 27 dofs (largest correction 2.00) and the step still converges — exactly the intended behaviour: the safeguard absorbs the overshoot instead of letting it detonate the solve.

C6: anatomy of a healthy Newton solve, and a stagnating one

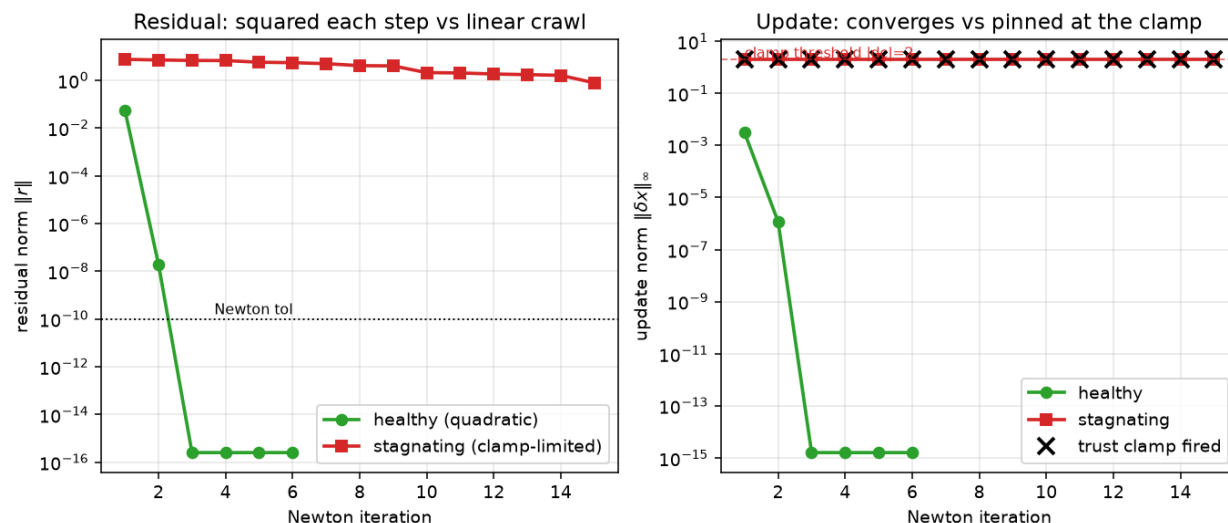


Figure 20.1: Anatomy of a Newton solve. *Left*: residual norm per iteration — a healthy poly step (green) squares its residual each step to round-off in 3 iterations; a stagnating step (red) only crawls down linearly. *Right*: update norm — the healthy step’s update collapses, while the stagnating step’s update is pinned at the trust clamp $|\delta c| = 2$ (x marks) every iteration, the signature of a diverging direction.

20.4 Four ways a nonlinear solve fails

An honest workflow rehearses its failures. Each case below is *run* and genuinely fails; the skill is reading the signature and taking the right action (Fig. 20.2).

1. Newton stagnation (the step size is too large). Take a deep poly quench — a high-amplitude random field — at $\Delta t = 0.05$. The raw Newton direction diverges, so the trust clamp fires on *every* iteration (clamp fraction 1.00); the residual crawls down only linearly and after the stepper’s full budget of 15 iterations is still 7.9×10^{-1} , many orders from tolerance (Fig. 20.1, red). The trap is that it *looks* like slow progress. The fix is not more iterations — it is a smaller step: the identical quench at $\Delta t = 2.0 \times 10^{-3}$ converges in 8 iterations. A persistently clamped update with sub-quadratic residual decay means “reduce Δt ,” full stop.

2. The min- Δt wall. An adaptive integrator shrinks Δt until its local-truncation-error (LTE) estimate meets a tolerance (Chapter 17). Ask for an LTE tolerance it cannot reach and set the floor $\Delta t_{\min}$ too high, and the controller is trapped: every step is a forced accept at the floor. We drive 8 steps, all at $\Delta t_{\min}$, and measure the LTE *at the floor* directly — 1.38, far above the requested tolerance 1.0×10^{-9} . The signature (constant $\Delta t = \Delta t_{\min}$ with $\text{LTE} > \text{tol}$) says the controller has run out of room: loosen the LTE tolerance, or accept — knowingly — the floor’s accuracy. Silently “succeeding” at the floor is the failure to avoid.

3. An ill-conditioned Jacobian. As a Flory–Huggins state approaches the wall, $f''(c) \sim A/c$ climbs toward the $1/\varepsilon$ cap and the tangent’s condition number climbs with it. Freezing a uniform state at each mean composition and forming its Jacobian, the exact 2-norm condition number rises

from 1.3×10^3 at $c = 0.5$ to 6.3×10^4 at $c = 0.003$ — $49\times$ worse (Fig. 20.3). An ill-conditioned tangent starves an iterative linear solver and erodes accuracy in a direct one; the cure is upstream — keep iterates off the wall (the projection above), or precondition. This is why the projection is not cosmetic.

4. A diverging start (wrong Jacobian or bad initial guess). If convergence is linear from the very first step, or the residual grows, suspect the tangent itself or an inadmissible initial condition. Verify the Jacobian against a finite difference (Chapter 14's red→green gate) and seed a physical initial state. This case is diagnosed, not with a number, but by ruling the others out.

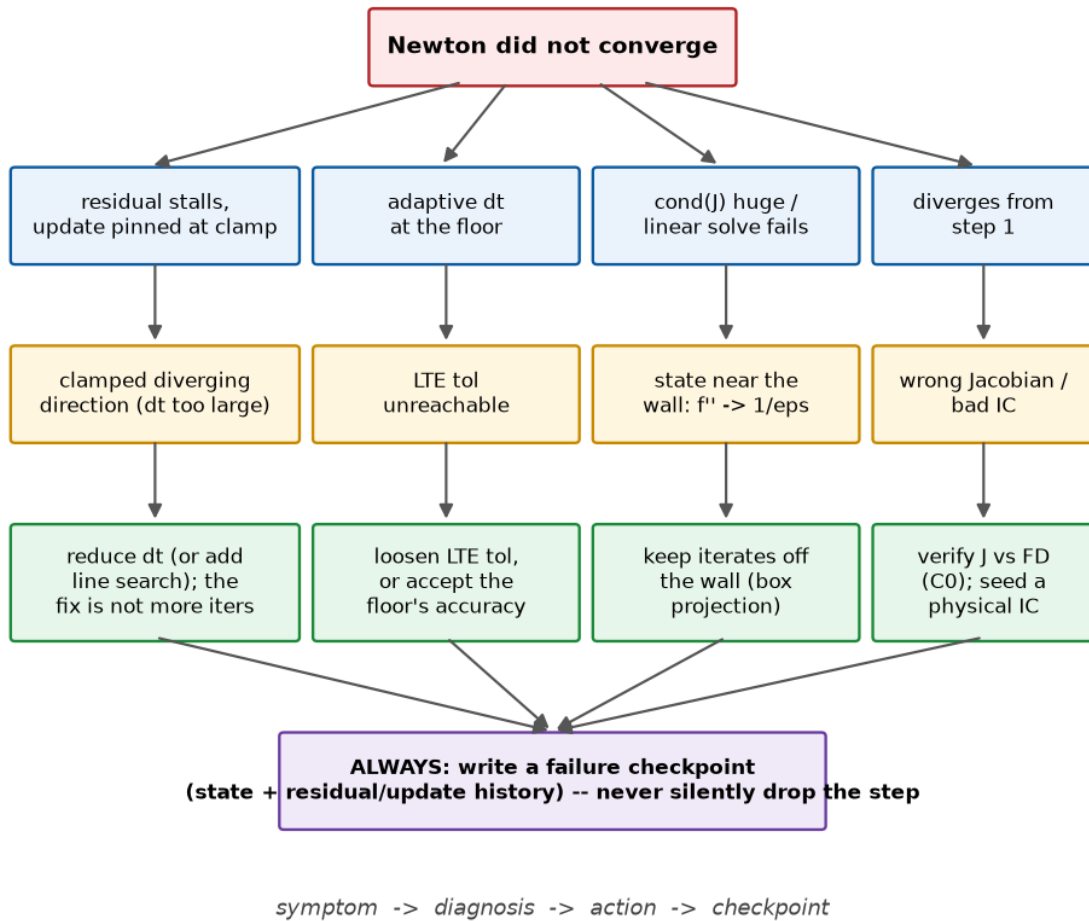


Figure 20.2: The nonlinear-failure decision tree: from the symptom you can see (residual/update/ Δt /condition), to the diagnosis, to the action — and always a failure checkpoint. A stalled residual with a clamped update means reduce Δt ; Δt stuck at the floor means the LTE tolerance is unreachable; a huge condition number means the state is at the wall; divergence from step 1 means the Jacobian or the initial guess.

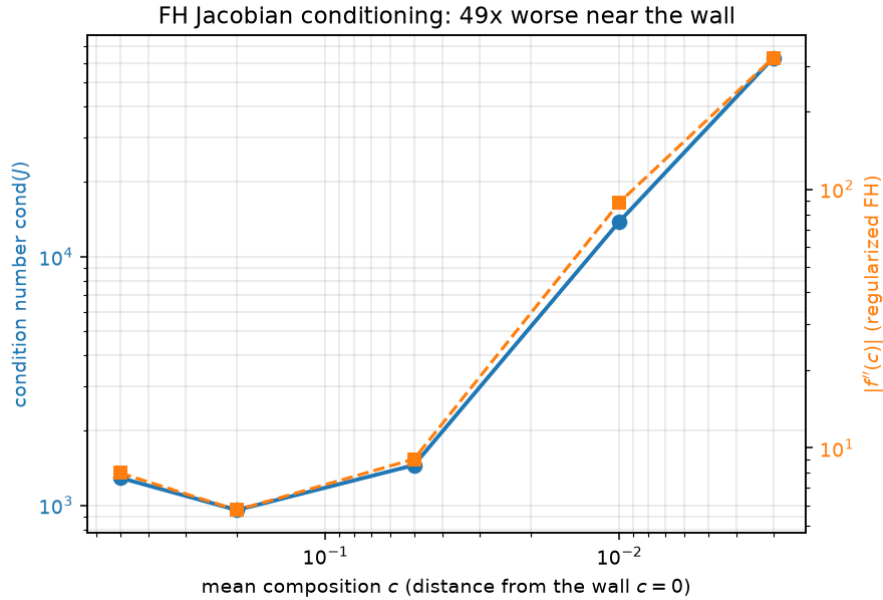


Figure 20.3: Ill-conditioning near the wall. As the Flory–Huggins state approaches $c = 0$, $|f''(c)|$ (orange) grows toward the regularization cap and the Jacobian condition number (blue) climbs 49× from bulk to wall. Keeping Newton iterates off the wall keeps the tangent solvable.

20.5 Never drop a failed step: checkpoints

A production campaign must never silently discard a step that failed to converge — that is how a bad result masquerades as a good one. When the stagnation case fails, the run writes a **failure checkpoint** to `checkpoints/stagnation_step.npz`: the initial and last Newton states, the full residual/update history, the convergence flag, and the exact config to reproduce it. A post-mortem then starts from data, not from a re-run that may not reproduce.

Run it.

```
cd computational/06_nonlinear_diagnostics
python run.py --config configs/c6.yaml --device cuda:0 \
    --solver splu --output outputs/c6 --overwrite --mode reference
python gen_figures.py --run-dir outputs/c6
```

It runs the healthy anatomy, the stopping-criterion comparison, the safeguard report, and the four failures, prints nine PASS/FAIL gates (several of which *assert a failure* — the stagnation solve must not converge, the controller must stay pinned), writes a failure checkpoint, and checks `baseline.yaml`. Compare with `EXPECTED.md`.

Measured self-check (reproduce to a couple of significant figures; the *behaviours* are the load-bearing quantities).

case	measured	signature
healthy poly	3 iters, $\text{res} \rightarrow 2.6 \times 10^{-16}$	quadratic
stagnation	<i>not converged</i> , clamp 1.00	reduce Δt
$\hookrightarrow$ fixed	8 iters at $\Delta t = 2.0 \times 10^{-3}$	smaller step converges
FH safeguards	27 dofs projected, converged	overshoot absorbed
min- Δt	8 steps at floor, LTE 1.38	tol unreachable
ill-conditioned	cond $1.3 \times 10^3 \rightarrow 6.3 \times 10^4$	wall blow-up

20.6 Code walk-through

The diagnostics live in `diagnostics_ch.py`; they drive the production stepper unchanged.

1. Reconstruct the per-iteration trajectory.

```
for k in range(1, kmax+1):
    # cap Newton at k iterations
    st = CahnHilliardStepper(..., newton_max=k, capture_system=True)
    st.set_initial(ic, mu_init="consistent"); st.step()
    A, r = st.last_system
    # the k-th assembled system
    res.append(norm(r)); dx.append(st.last_newton["dx_inf"])
```

Deterministic, so the k -th entry is exactly the k -th internal iterate — no stepper edits needed.

2. Detect the safeguards. A clamped iteration is $|\|\delta x\|_\infty - 2| < 10^{-9}$; the projection exposes `st.proj_dofs` and `st.proj_max` directly.

3. Measure the min- Δt LTE. One $\Delta t_{\min}$ step versus two $\Delta t_{\min}/2$ steps from the same state gives the step-doubling LTE (Chapter 17); at the floor it exceeds the requested tolerance.

20.7 Explore on your own

Q20.7.1 Reproduce the stagnation case, then bisection Δt between 0.05 and 2.0×10^{-3} to find the largest step that still converges within the 15-iteration budget. How does that critical Δt scale if you halve the mesh size h ?

Q20.7.2 The absolute update test tripped $10^3\times$ early when the field was rescaled by 10^{-3} . Implement a relative-residual test $\|r\|/\|r_0\| < \tau$ and show it is invariant to that rescaling. Which test would you make the default, and why?

Q20.7.3 Push the Flory–Huggins conditioning sweep to $c = 10^{-4}$ (closer to the wall). Does $\text{cond}(J)$ keep climbing, or does the $1/\varepsilon$ cap flatten it? Relate the plateau to the regularization constant $\varepsilon = 10^{-4}$.

Q20.7.4 Turn the stagnation failure into a *line search*: after the clamp, backtrack the step ($\delta x \leftarrow \alpha \delta x$, halving α until the residual decreases). Does the deep quench now converge at $\Delta t = 0.05$? What does that cost in iterations versus simply reducing Δt ?

Q20.7.5 Open the failure checkpoint `checkpoints/stagnation_step.npz` and plot `c_last`. Where on the domain is the unconverged residual largest — and does that location coincide with the sharpest composition gradient?

Chapter 21

CUDA profiling and memory

Key idea. “It’s slow” is not a diagnosis. A profile is: it tells you *where* a time step’s time and memory actually go, so you optimize the thing that matters instead of the thing you guessed. This chapter profiles one complete Cahn–Hilliard step with tools that work on any box, teaches the Nsight Systems commands for the device-level detail, and — crucially — shows that on the tutorial’s direct-solver path the GPU is barely the bottleneck at all.

Learning objectives. By the end you can: time one complete step with CUDA *synchronization* and explain why an unsynchronized timer lies; measure kernel-launch overhead, device memory bandwidth, and host↔device copy bandwidth, and place a kernel on the memory-bound side of a roofline; read a sparse-LU factor-vs-solve split and argue for factor reuse; account for a step’s device memory analytically when the mempool hides it; and drive `nsys profile/nsys stats` with NVTX ranges to capture one step’s timeline.

Prerequisites. The Newton step and its assembly/solve stages (Chapter 14); the solver landscape and cost accounting (Chapters 18, 19); basic GPU execution model (kernels, launches, host/device memory).

Expected cost. One CUDA GPU, 2-D, well under 1 GB of problem memory; solver `splu.run.py` completes in $\approx 30\text{--}40$ s on an RTX 6000 Ada, including one `nsys` capture.

This chapter profiles the production step (`src/diffsim/physics/cahn_hilliard.py`) at level 6 (8450 dofs, 148996 nonzeros). Every number here is measured with a CUDA-synchronized timer or a driver memory query, so it reproduces with or without Nsight.

21.1 Time one step — and synchronize, or the timer lies

CUDA kernels are launched *asynchronously*: the host queues the work and returns immediately, long before the GPU finishes. A timer wrapped around a launch with no synchronization therefore measures the *launch*, not the *work* — it can report a kernel as “instant” while the GPU is still busy. The fix is a `synchronize()` before you start and stop the clock; the course’s `diagnostics.profiling` timers do exactly this. Our kernel-launch micro-benchmark shows the host-side cost of a single launch is $\approx 41\mu\text{s}$ — small, but a step that launches thousands of tiny kernels pays it thousands of times.

Timed properly, one complete level-6 step takes about 2143 ms and runs 12 Newton iterations.

Where does that time go? Not where a GPU course would have you guess.

21.2 Data movement rules: bandwidth vs arithmetic intensity

Two numbers bound everything a kernel can do: how fast it can compute, and how fast it can move data. Their ratio for the *kernel* is its **arithmetic intensity** (flops per byte moved); the ratio for the *hardware* is where the roofline bends. A streaming triad $c = a + 2b$ touches three arrays and does two flops — arithmetic intensity 0.083 flop/byte, deep in the *memory-bound* regime — and on this card it sustains 706 GB/s, near the hardware peak. A finite element assembly is the same kind of animal: a little arithmetic per node, a lot of gather/scatter, memory-bound.

The lesson sharpens when the data has to cross the PCIe bus. A host↔device copy sustains only 26 GB/s — $27\times$ slower than on-device memory (Fig. 21.1). Every Newton iteration of the `splu` path copies the assembled element Jacobians back to the host to build the sparse matrix: 35 MB per step across that slow link. Data movement, not flops, is what you profile for.

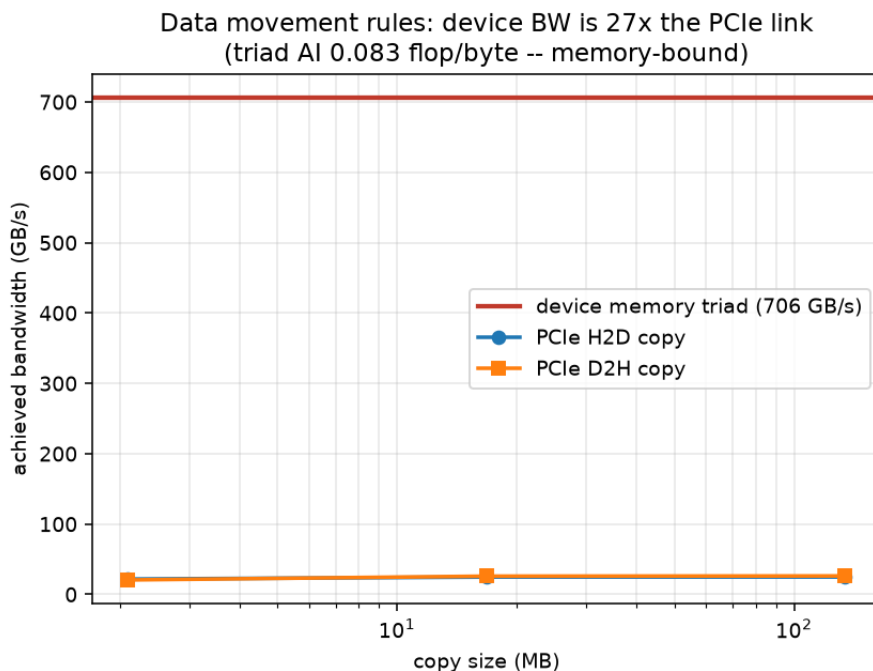


Figure 21.1: Bandwidth is the currency. The device memory triad (red) sustains 706 GB/s near the card’s peak; a host↔device PCIe copy saturates at 26 GB/s, $27\times$ slower. With arithmetic intensity 0.083 flop/byte the triad is memory-bound — moving the data, not doing the math, sets the time.

21.3 Factor vs solve: the case for reuse

On the tutorial’s direct-solver path the real cost is neither on the GPU nor on the bus — it is the host sparse-LU *factorization*. Factoring the level-6 saddle system costs 88 ms; a triangular *solve* with that factor costs only 3.6 ms — a $24\times$ ratio. Because the Cahn–Hilliard tangent changes every Newton iteration, the stepper refactors each time: 12 factorizations for one step. That is the single

biggest line in the step’s budget (Fig. 21.2, left), and it is why the GPU sits nearly idle on this path.

Two honest conclusions follow, both pointing forward. First, when a matrix (or its symbolic structure) is reused — a frozen tangent, a modified-Newton scheme, or a solver that caches its symbolic factorization — you pay the factor cost once and amortize it over many cheap solves. Second, a GPU direct solver (cuDSS) or an iterative device solver moves this dominant cost onto the hardware the tutorial has been provisioning all along (Chapter 18). Profiling did not just measure the step; it *redirected* the optimization.

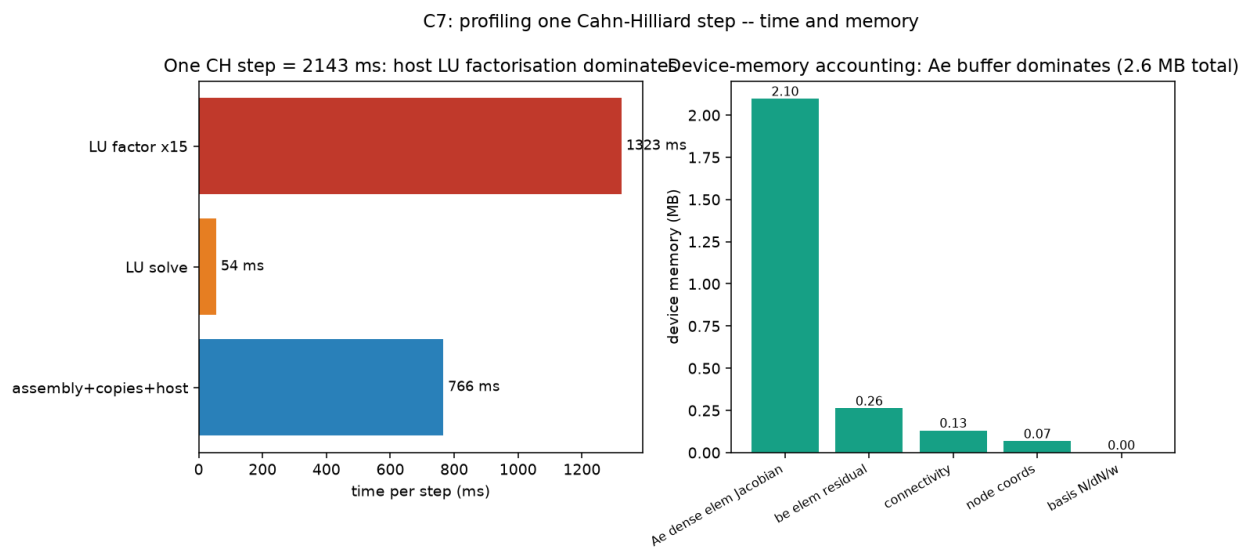


Figure 21.2: Profiling one step. *Left*: the step-time budget — host LU factorization (repeated once per Newton iteration) dominates; the triangular solves and the GPU assembly/copies are a sliver. *Right*: the device-memory accounting — the dense per-element Newton Jacobian buffer **Ae** (2.10 MB) dwarfs the mesh and basis arrays.

21.4 Device memory: measure it, or account for it

Peak device memory is the number that decides whether a problem fits. It is also the one a naive query gets wrong: Warp manages device memory through a *caching mempool*, so an in-process “used-memory” delta reads ≈ 0 (the pool recycles freed buffers) and a driver query is quantized to the pool’s reservation granularity. Measured in a *fresh* process, the level-6 step’s driver footprint is 2.1 MB; the honest, reproducible number is an **analytical accounting** of the arrays the code allocates (2.56 MB total), dominated by the dense element-Jacobian buffer **Ae** at 2.10 MB (Chapter 19’s memory discipline, on the device side). The two agree because **Ae** is the step’s dominant transient allocation. When the mempool hides the high-water mark, you account for it.

21.5 Nsight Systems: capture one step

For the device-level timeline — which kernels ran, how long, what memory copies moved — the tool is Nsight Systems. The discipline is to profile **one complete step**, not a whole run: JIT-warm the

kernels with a throwaway step, wrap the step you care about in an NVTX range, and profile that. `nsys_workload.py` does exactly this, marking `warmup` and `ch_step` ranges. Capture and summarize with:

```
nsys profile --trace=cuda,nvtx --sample=none --force-overwrite=true \
  -o outputs/c7/step_profile <repo>/venv/bin/python nsys_workload.py
nsys stats --report nvtx_sum --report cuda_gpu_kern_sum \
  outputs/c7/step_profile.nsys-rep
```

On the course box this capture succeeds (`nsys_captured = yes`) and the NVTX summary reports the two ranges directly — the profiled `ch_step` at ≈ 2005 ms, the compile-heavy `warmup` at ≈ 2744 ms.

On kernel-level rows. The per-kernel and per-memcpy tables (`cuda_gpu_kern_sum`, `cuda_gpu_mem_time_sum`) require a CUPTI that matches your CUDA *driver*. On a box whose driver is newer than the installed Nsight, `nsys stats` reports “does not contain CUDA kernel data” while still giving the NVTX timeline (as here). The commands are unchanged — on a matched toolchain the same two reports print the per-kernel breakdown. Run them on your box; the NVTX timing and the synchronized timers in this chapter carry the quantitative story either way.

Run it.

```
cd computational/07_cuda_profiling
python run.py --config configs/c7.yaml --device cuda:0 \
  --solver splu --output outputs/c7 --overwrite --mode reference
python gen_figures.py --run-dir outputs/c7
```

It times one synchronized step, measures launch overhead / bandwidth / the factor-vs-solve split / the device-memory footprint, and runs one `nsys` capture; it prints PASS/FAIL gates for the invariants (bandwidth $\gg$ PCIe, memory-bound triad, factor $\gg$ solve) and checks `baseline.yaml`. Compare with `EXPECTED.md`.

Measured self-check (hardware-dependent; the *ratios* are the load-bearing quantities).

quantity	measured	invariant
one step	2143 ms, 12 its	—
launch overhead	41 μ s	per launch
device / PCIe BW	706 / 26 GB/s	27 $\times$ gap
triad intensity	0.083 flop/byte	memory-bound
factor / solve	88 / 3.6 ms	24 $\times$
device memory	2.56 MB (Ae 2.10)	accounted

21.6 Explore on your own

Q21.6.1 Re-time the step *without* the CUDA synchronization (drop the `sync` in the timer). By how much does the reported time change, and in which direction? Explain the sign from the async launch model.

Q21.6.2 Sweep the mesh level $5 \rightarrow 7$ and plot step time, factor time, and the analytical device memory against the dof count. Which grows fastest, and does the factor-vs-solve ratio hold?

Q21.6.3 Switch the solver to a device path (Chapter 18) and re-profile. Does the host factorization line in Fig. 21.2 shrink? What becomes the new dominant cost, and is it now on the GPU?

Q21.6.4 The triad reaches 706 GB/s. Estimate the card’s peak memory bandwidth from its spec and compute the fraction achieved. What would a *compute*-bound kernel (high arithmetic intensity) look like on the same plot?

Q21.6.5 On your own box, run the two `nsys stats` reports. If the per-kernel table prints, which kernel dominates `ch_step`, and is it the assembly or something else? If it reports “no CUDA kernel data,” what does that tell you about your Nsight/driver versions?

Chapter 22

Extending DiffSim safely

Key idea. The bridge from *using* a solver to *contributing* to one is a discipline, not a leap. Adding a new physics term is easy; adding it *safely* — so the next person can trust it — means carrying it through a fixed sequence of gates: equations and units, a config knob, the residual, the *analytic* Jacobian, a finite-difference check, a limiting-case check, a convergence study, the conservation and energy invariants, a profile, and documentation. This chapter walks that entire sequence on one small real term, and every gate is *run* and passes.

Learning objectives. By the end you can: add a new free-energy term to a Cahn–Hilliard model with correct units; write its residual and its *analytic* tangent contribution; verify that tangent against a finite difference; prove the extension reduces to the base model in the limiting case; confirm the discretization still converges at the theoretical order; check that mass and energy invariants still hold; profile the new term’s cost; and write the docs a reviewer needs. In short: turn a good idea into a contribution someone else can build on.

Prerequisites. The weak form $\rightarrow$ CUDA pipeline and the red $\rightarrow$ green Jacobian gate (Chapter 14); convergence and MMS (Chapter 15); nonlinear diagnostics (Chapter 20); profiling (Chapter 21); the Cahn–Hilliard energy (Physics Chapter 3).

Expected cost. No GPU required — the worked reference is a transparent readable-NumPy solver on the diffsim mesh infrastructure (exactly as Chapter 14), so every stage is visible and cheap. `run.py` and the `pytest` battery each finish in a few seconds on a laptop.

This chapter extends Chapter 14’s hands-on (add a Jacobian term to a scalar problem) to a *full* contribution on a conserved, mixed system. The worked reference lives in `sextic_ch.py` (the solver) and `test_sextic_ch.py` (its gate); `run.py` walks the whole workflow and prints a PASS/FAIL for each stage (Fig. 22.2).

22.1 Step 1 — the term, its equations, and its units

The base Cahn–Hilliard bulk energy is the quartic double well $f_0(c) = \frac{1}{4}(c^2 - 1)^2$. We add a sixth-order Landau term — a higher-order stabilizer that steepens the well and bounds deep quenches:

$$f(c) = \frac{1}{4}(c^2 - 1)^2 + \frac{\beta}{6}c^6, \quad f'(c) = c^3 - c + \beta c^5, \quad f''(c) = 3c^2 - 1 + 5\beta c^4. \quad (22.1)$$

The new coefficient β is dimensionless (energy-scaled), and $\beta = 0$ recovers the base model *exactly* — the limiting case we will test. Only two derived quantities matter downstream: $f'(c)$, which enters the chemical-potential residual, and $f''(c)$, which enters the Jacobian. Getting the analytic f'' contribution $5\beta c^4$ right is the crux of the contribution.

22.2 Step 2 — a config knob

A new physical parameter is a new config field, validated and defaulted through the course schema (Chapter 00's harness), never a hard-coded constant:

```
"beta": Field(float, default=0.5, min=0.0),    # sixth-order Landau
        coefficient
```

The resolved value is written to `config.resolved.yaml`, so every run records exactly which β produced it.

22.3 Steps 3–4 — residual and *analytic* Jacobian

The mixed backward-Euler weak form is $R_c = 0$, $R_\mu = 0$ with

$$R_c = \int_{\Omega} w \frac{c - c^n}{\Delta t} dV + M \int_{\Omega} \nabla w \cdot \nabla \mu dV, \quad (22.2)$$

$$R_\mu = \int_{\Omega} q \mu dV - \int_{\Omega} q f'(c) dV - \kappa \int_{\Omega} \nabla q \cdot \nabla c dV. \quad (22.3)$$

The new term rides in through $f'(c)$: the only nonlinear assembly, $\int N_a f'(c)$, gains βc^5 . Newton then needs the tangent $J = \partial R / \partial x$, whose four blocks per node are

$$J_{cc} = \frac{M_e}{\Delta t}, \quad J_{c\mu} = M K_e, \quad J_{\mu c} = -M_e^{f''} - \kappa K_e, \quad J_{\mu\mu} = M_e, \quad (22.4)$$

with M_e the mass and K_e the stiffness element matrices and $M_e^{f''} = \int_{\Omega} N_a f''(c) N_b dV$ the f'' -weighted mass matrix. The new term enters exactly one block, $J_{\mu c}$, through $f''(c) = 3c^2 - 1 + 5\beta c^4$. This is the single most error-prone line in any contribution — so we never trust it by eye.

22.4 Steps 5–6 — verify the derivative, and the limiting case

Finite-difference check. The analytic tangent must match a finite difference of the residual, $Jv \approx [R(x + \varepsilon v) - R(x)]/\varepsilon$. On a random admissible state the relative error is 6.6×10^{-10} (and 5.0×10^{-10} at $\beta = 0$) — limited only by finite-difference truncation, the signature of a *correct* tangent (a wrong f'' would sit at $O(1)$, exactly Chapter 14's red→green contrast). The new term's own derivatives check too: $f'_6 = \beta c^5$ and $f''_6 = 5\beta c^4$ agree with a finite difference to 3.6×10^{-6} and 1.1×10^{-5} .

Limiting case. A safe extension must *not* change the base model where it shouldn't. At $\beta = 0$ the new-term value, first, and second derivatives vanish identically and f' , f'' reduce to $c^3 - c$ and $3c^2 - 1$ — so the extended solver reproduces the base double well bit-for-bit. This guards against the classic regression: a “harmless” addition that quietly perturbs existing results.

22.5 Step 7 — convergence still holds

Adding a term must not break the discretization’s order of accuracy. Using a steady manufactured solution with the term active (Chapter 15’s method: manufacture c^* , μ^* , add the residual they leave as sources, pin them on the boundary), the L^2 error of c falls from 1.8×10^{-2} to 1.1×10^{-3} over three mesh levels at observed order 2.01 — the theoretical $p+1 = 2$ for linear elements (Fig. 22.1, left). The manufactured source carries the new βc^5 term, so this is a genuine test of the extended residual, not the base one.

22.6 Step 8 — conservation and energy still hold

The two physical invariants of Cahn–Hilliard must survive the change. A no-flux march conserves the quadrature mass $\int c dV$ to 1.2×10^{-15} (relative) — Model-B transport moves material, never creates it. And the Ginzburg–Landau energy $F = \int [f(c) + \frac{\kappa}{2} |\nabla c|^2] dV$ decreases monotonically, $0.3144 \rightarrow 0.2500$, with largest positive step increment 0 — the discrete Lyapunov property intact (Fig. 22.1, middle). A term that broke either invariant would be physically wrong no matter how well it converged.

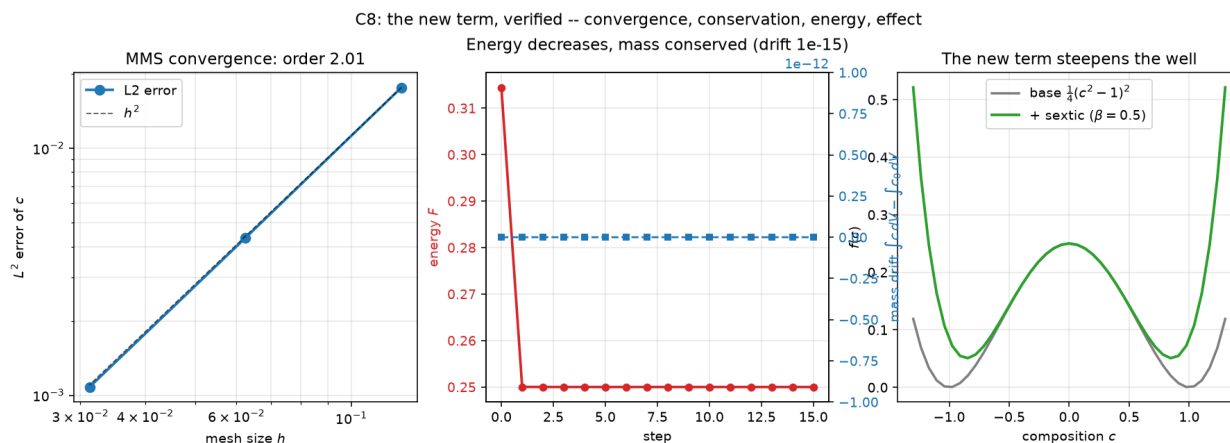


Figure 22.1: The new term, verified. *Left*: steady-MMS convergence at order 2.01 (the predicted h^2) with the term active. *Middle*: a no-flux march — the energy F decreases monotonically while the mass drift stays at round-off (1.2×10^{-15}). *Right*: the new term steepens the free-energy well, the physical effect it was added for.

22.7 Step 9 — profile the cost

A contribution should know its own cost (Chapter 21). Timed with CUDA-synchronized stage timers, the new term adds only a fifth power evaluation to the residual and a fourth power to the Jacobian — its assembly overhead is negligible, within timing noise of the base assembly. The term buys a real physical effect (a steeper well, bounded deep quenches; Fig. 22.1, right) for essentially no cost.

22.8 Step 10 — documentation and changelog

The last gate is the one that makes the work *reusable*: docstrings stating the equations and units, the schema field, an EXPECTED.md of the passing gates, and a CHANGELOG.md entry naming the term, its parameter, and the tests that cover it. An undocumented correct term is a landmine for the next contributor.

Run it.

```
cd computational/08_extending_diffsim
python run.py --config configs/c8.yaml --device cpu \
    --solver splu --output outputs/c8 --overwrite --mode reference
python -m pytest -q test_sextic_ch.py          # the contributor's gate
python gen_figures.py --run-dir outputs/c8
```

run.py walks all ten steps and prints eight PASS/FAIL gates; pytest runs the 7-check battery (test_sextic_ch.py); results are checked against baseline.yaml. Compare with EXPECTED.md.

Measured self-check (deterministic on CPU; the FD-consistency and order are the load-bearing quantities).

gate	measured	meaning
new-term f', f'' vs FD	$3.6 \times 10^{-6} / 1.1 \times 10^{-5}$	term derivatives correct
analytic Jacobian vs FD	6.6×10^{-10}	tangent correct
limiting case $\beta \rightarrow 0$	exact	no regression
MMS order	2.01	convergence intact ($p+1$)
mass drift (rel)	1.2×10^{-15}	conserved
energy start $\rightarrow$ end	0.3144 $\rightarrow$ 0.2500	Lyapunov

22.9 Code walk-through

The new term, isolated. sextic_ch.py keeps the term as three one-liners so the derivative test targets it directly:

```
def sextic_term(c, beta):
    return dict(f=(beta/6)*c**6, fp=beta*c**5, fpp=5*beta*c**4)
```

Where it enters. The residual gains βc^5 in one place (fprime), the Jacobian gains $5\beta c^4$ in one block (fdoubleprime inside $J_{\mu c}$). Every other line is the base model — the smallest possible diff is the safest.

22.10 Explore on your own

Q22.10.1 Add a *different* term — a cubic tilt $\frac{\gamma}{3}c^3$ (which breaks the well's symmetry). Write f', f'' , add them, and run the full gate. Which invariant is most sensitive to getting the tilt's Jacobian wrong?

Q22.10.2 Deliberately break the tangent: drop the $5\beta c^4$ from `fdoubleprime`. Which gates fail, and which still pass? Explain why the MMS study can still look plausible while the FD check screams.

Q22.10.3 Make β a *field* $\beta(\mathbf{x})$ (spatially varying). What changes in the residual, the Jacobian, and the units? Re-run the FD check.

Q22.10.4 Push β to 2 and re-quench from a large-amplitude state. Does the steeper well keep $|c|$ more tightly bounded than the base model? Relate the bound to the balance $c^3 \sim \beta c^5$.

Q22.10.5 Port the term to the production Warp kernel (`cahn_hilliard.py: make_ch_newton`). What must you add for the device path that the transparent NumPy reference did not need, and how would you re-run this exact gate against the device implementation?

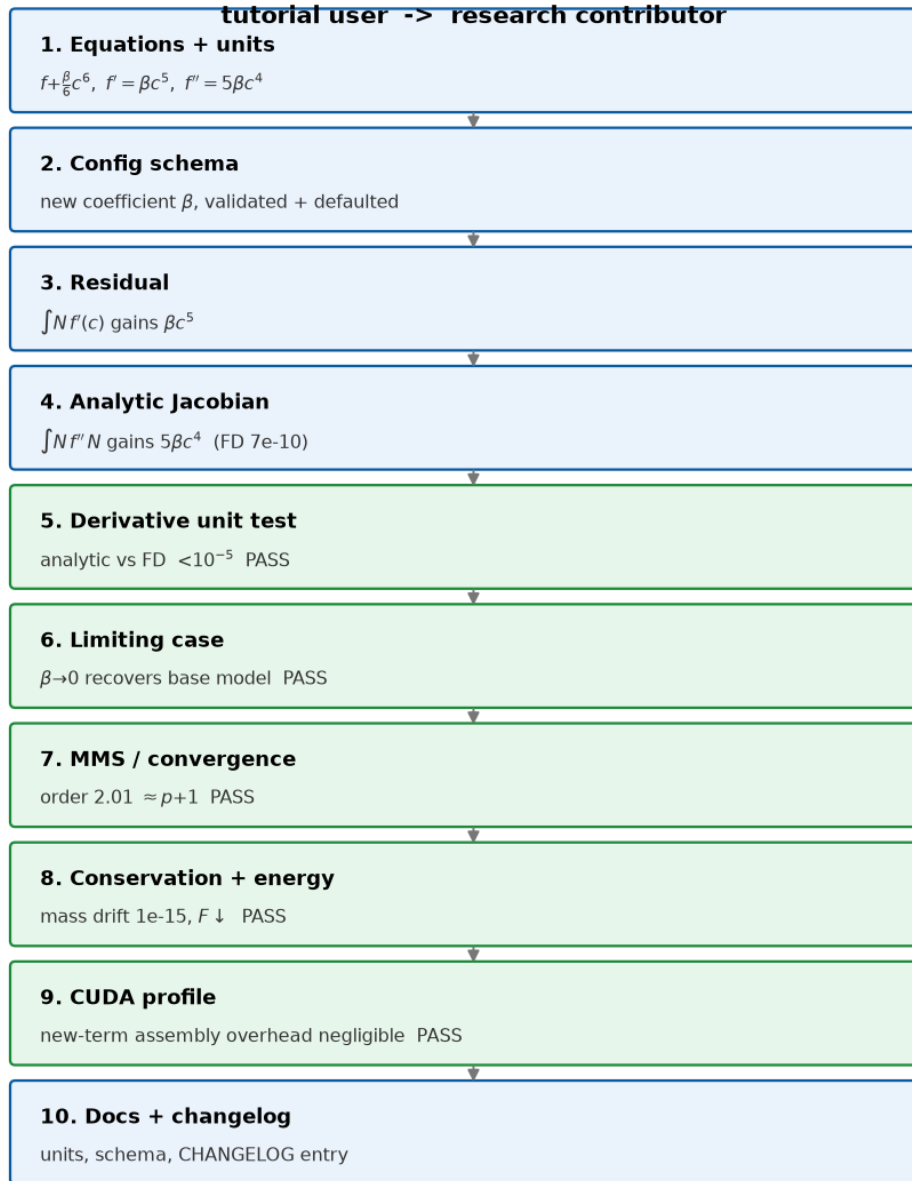


Figure 22.2: The contributor workflow, end to end: from equations and units to docs, each stage a gate the worked sextic term passes. Steps 5–9 (green) are the verification core — the finite-difference tangent check, the limiting case, the convergence study, the conservation/energy invariants, and the profile.

Chapter 23

Reproducible campaigns

Key idea. A single verified run is a data point, not a result. A *campaign* is the unit of morphology science: one frozen model swept over parameters and seeds, aggregated with confidence intervals, and reported *without silently dropping the runs that failed*. The skill is bookkeeping — immutable configs, deterministic run ids, checkpoint/restart, and an aggregation step that surfaces failures rather than averaging around them.

Learning objectives. By the end you can: freeze a campaign to an immutable resolved manifest (the way the Phase-0 harness freezes one run to `config.resolved.yaml`); expand a parameter sweep $\times$ seed ensemble into a deterministic run grid; checkpoint and restart idempotently (skip completed, retry failed); aggregate a seed ensemble into mean, standard deviation and a bootstrap 95% confidence interval with the shared `diffsim.diagnostics.stochastic`; enforce the **no-silent-drop** rule (a crashed run and a tolerance-violating run are both *surfaced*); separate **exploratory** scans from **confirmatory** replicates; and scale the identical manifest to a SLURM job array.

Prerequisites. Physics Chapter 3 (the binary Cahn–Hilliard spinodal brick that is the campaign engine here), Computational Chapter 15 (a campaign is only meaningful over a *converged* model), the Phase-0 harness, and the stochastic diagnostics.

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory. The 9-run demo (8 real runs + 1 injected fault) finishes in under a minute on an RTX 6000 Ada; a restart with everything completed returns in seconds.

23.1 The unit of work is the campaign

Chapters 15 and 3 taught you to *trust one run*: converge it, control the algebraic error, verify the reference. But a research claim — “deeper quenches coarsen to larger domains” — is never one run. It is a *sweep* over the physical parameter, replicated across a *seed ensemble* because the initial noise is random, and it must survive the messy reality that some runs crash and some run but lie. The difference between a defensible campaign and a directory full of scripts is entirely bookkeeping:

- an **immutable resolved config** so “what ran” is never in doubt — the manifest is frozen to `campaign.resolved.yaml`, exactly as a single run is frozen to `config.resolved.yaml`;
- **deterministic run ids** so a restart is idempotent (skip the completed, retry the failed) rather than a re-run or a collision;

- an **aggregation** that reports the mean *and* every failure, and refuses to present a partial ensemble as whole.

23.2 The model, and why we sweep χ

The engine is the fast 2-D binary Cahn–Hilliard spinodal march of Physics 3, with the Flory–Huggins log free energy

$$f(c) = A[c \ln c + (1 - c) \ln(1 - c)] + B c(1 - c), \quad (23.1)$$

whose enthalpic parameter B is the Flory interaction χ . We sweep χ (the code’s `fh_B`) with the entropic scale $A = 0.15$ held fixed so the whole band is spinodally unstable. Two reasons to sweep χ rather than the evaporation rate k_e : first, χ is the one *production* parameter in the materials database that carries a real measured uncertainty (the P3HT:PCBM value 0.86, order-of-magnitude 0.5), which Chapter 24 then propagates — the two chapters share one physical control; second, the spinodal march is an order of magnitude cheaper than the drying-film k_e driver, so the campaign runs locally on a shared GPU. The cluster-scale k_e variant is exactly the same manifest submitted as a job array (§23.6). The morphology observable is the interfacial-area domain length L_{area} (box-fraction units).

23.3 Sweep, ensemble, and the two intents

The manifest declares a frozen `base` model and a list of `sweeps`; the runner takes the Cartesian product of each sweep’s values with its seed list. We run two sweeps with *different intent*:

- an **exploratory** scan of $\chi \in \{0.36, 0.86, 1.36\}$ (the material’s mean and $\pm 1\sigma$), two seeds each — a hypothesis generator whose confidence intervals are wide and *not* a measurement;
- a **confirmatory** replicate at the nominal $\chi = 0.86$ with a fresh, dedicated seed pair — the pre-registered headline number, so it is never cherry-picked from the scan.

The measured response is monotone in χ (Fig. 23.1): L_{area} rises from 0.261 at $\chi = 0.36$ through 0.416 to 0.517 at $\chi = 1.36$, a parametric sensitivity $dL_{\text{area}}/d\chi \approx 0.256$. Each point carries a bootstrap 95% interval from `diffsim.diagnostics.stochastic`; the confirmatory replicate (0.374) sits a little below the exploratory point at the same χ (0.416) — that gap between two seed sets *is* the stochastic variability Chapter 24 budgets.

23.4 Checkpoint, restart, and failed-run recovery

Every cell writes `status.json` into its own `runs` sub-directory. Re-invoking the campaign is idempotent: a cell already `completed` is skipped, a cell that `failed` is retried. A crashed campaign therefore resumes where it stopped, and fixing a manifest typo re-runs only the broken cell — no wasted GPU-hours, no manual bookkeeping.

23.5 The no-silent-drop rule

This is the whole point of the chapter. An aggregation that quietly drops the runs that failed reports a clean mean over the survivors and hides that a corner of parameter space is unreliable. We demonstrate two failure classes, both *surfaced* (Fig. 23.2):

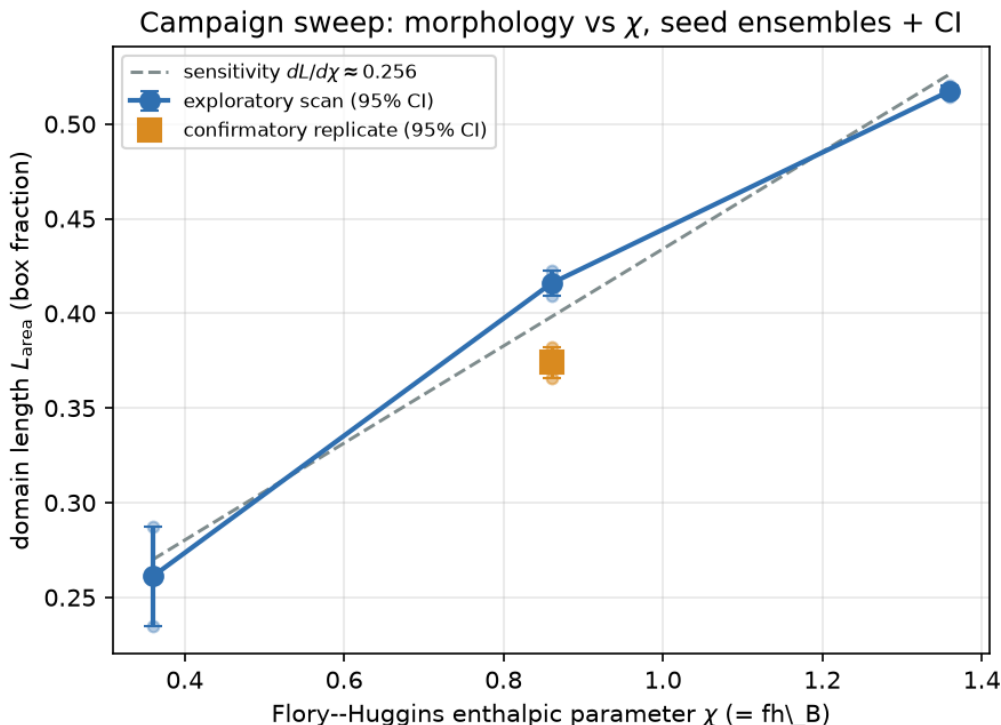


Figure 23.1: Campaign sweep. Domain length L_{area} vs the Flory–Huggins parameter χ : the exploratory scan (circles, 95% CI) is monotone with slope 0.256; faint dots are the per-seed runs; the confirmatory replicate (square) is the pre-registered headline at nominal χ . The seed-set gap at $\chi = 0.86$ is real stochastic spread, not error.

1. **A crashed run.** The manifest injects a cell with a mistyped solver; it raises (unknown solver). The runner catches it, records the cell `failed`, and the campaign continues — 1 of 9 runs, surfaced by id and reason.
2. **A tolerance-violating run.** Under a tightened mass-drift gate (1.00×10^{-3}) the deepest quench “completes” numerically but drifts 3.17×10^{-2} — finite, yet physically unreliable. It is marked `failed` (validation), which flips `campaign_complete` to false and the exit code to a nonzero CI gate. A silent nan-mean would have blessed it.

The aggregate reports the mean *and* the failures, with how many seeds actually contributed at each point; the production gate uses a mass bound of 5.00×10^{-2} , which all demo cells clear.

23.6 At cluster scale: a SLURM job array

Locally the runner sweeps every cell in one process. At research scale each cell is a multi-hour GPU job, so the campaign becomes a SLURM job *array*: one array task per cell, each resolving its (χ, seed) from the array task index via `--only k`. The provided `job_array.sbatch` mirrors the real kits in `cluster/wodo_campaign` — the same header shape, the same bootstrap, the same discipline of streaming per-task JSON to a log so even a preempted task is a complete measurement. Re-submitting is safe (the checkpoint skips completed cells); a dependent aggregate job re-reads every `status.json`, crashed tasks included, so a cluster failure is surfaced exactly as the local one is.

Campaign ledger: every run accounted for, failures surfaced not dropped

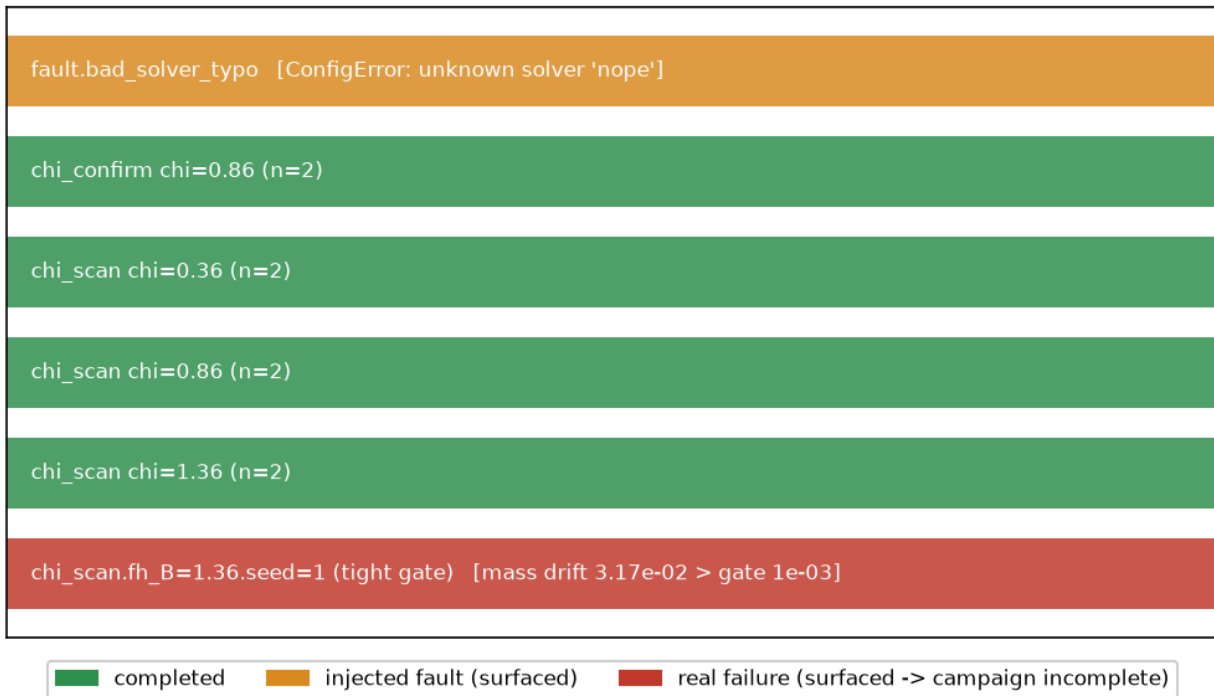


Figure 23.2: The campaign ledger: every run is accounted for. Completed cells (green), the injected fault (orange, surfaced), and a real tolerance-violating run under a tightened gate (red, which marks the campaign incomplete). No failure is silently dropped.

23.7 Run it

Run it.

```
cd computational/09_reproducible_campaigns
python campaign.py --manifest manifest.yaml \
    --output outputs/demo --overwrite      # the tiny real sweep
python campaign.py --manifest manifest.yaml \
    --output outputs/demo                  # RESTART: skips completed
python gen_figures.py                     # figures + numbers/c9.tex
```

It runs the 8 real cells plus the injected fault, writes `campaign.resolved.yaml`, `metadata.json` and `summary.json`, and gates on `baseline.yaml`. Compare with `EXPECTED.md`.

Measured self-check (reproduce to a couple of significant figures; the *trend* and the *book-keeping invariants* are load-bearing).

sweep	intent	χ	mean L_{area} ($\pm$ sd)
chi_scan	exploratory	0.36	0.261 (± 0.037)
	exploratory	0.86	0.416 (± 0.010)
	exploratory	1.36	0.517 (± 0.004)
chi_confirm	confirmatory	0.86	0.374 (± 0.011)

Sensitivity $dL_{\text{area}}/d\chi \approx 0.256$; 8 completed, 1 injected fault surfaced, campaign complete.

23.8 Code walk-through

The runner is `campaign.py`; the campaign is `manifest.yaml`.

1. Freeze the config, enumerate the grid.

```
resolved = resolve_manifest("manifest.yaml") # defaults made explicit
save      -> campaign.resolved.yaml         # the immutable record
specs     = enumerate_runs(resolved)        # deterministic run ids
```

2. One cell, and its hard admissibility gate.

```
rec = run_spinodal(..., fh_B=chi, seed=seed) # a real 2-D CH march
obs = observable(rec)                         # L_area + mass_drift
if not finite(obs) or obs.mass_drift > gate:   # -> FAILED (surfaced)
    status = failed(validation)
```

A raised exception is caught the same way (failure class `exception`) so one bad cell never aborts the campaign.

3. Aggregate without dropping.

```
agg = ensemble_aggregate(vals) # mean, sd, sem
ci   = bootstrap_ci(vals)      # 95% percentile CI
campaign_complete = (n_failed_real == 0) # injected faults excepted
```

23.9 Explore on your own

Q23.9.1 Add a third seed to the exploratory scan. Does the bootstrap CI narrow? Predict the $\sqrt{n}$ scaling of the standard error and check it against the reported `sem`.

Q23.9.2 Point the sweep at `kappa` instead of `fh_B` (the gradient-energy coefficient sets the interface width). Predict the sign of the domain-length response to κ and confirm it. Which control is the campaign more sensitive to?

Q23.9.3 Inject a *second* fault: a cell whose `level` is so large it exhausts device memory. Show it is surfaced (class `exception`), not dropped, and that the completed cells still aggregate.

Q23.9.4 Tighten the mass gate until a *real* exploratory cell fails. Confirm `campaign_complete` flips to false and the process exits nonzero — the CI gate that refuses to bless an incomplete campaign.

Q23.9.5 Turn the exploratory scan into a confirmatory one: choose a single χ , run five seeds, and report the mean with its bootstrap CI. Argue why this is now a measurement and the three-point scan was not.

Chapter 24

Validation and uncertainty

Key idea. “Is the answer right?” is not one question but six: code verification, solution verification, model validation, parameter uncertainty, stochastic variability, and model–experiment discrepancy. Conflating them is the most common way a computational study misleads — a converged run can still be swamped because a material input is known only to order of magnitude. This chapter separates the six and closes with the deliverable that forces the separation: an *uncertainty budget* that names the dominant source of error.

Learning objectives. By the end you can name and separately measure all six error questions; propagate a material parameter uncertainty from `materials.yaml` to a morphology observable by a finite-difference sensitivity; combine parametric, stochastic and numerical standard deviations in quadrature into one budget; and read off the dominant source — and therefore where the next effort should go (a better χ , more seeds, or a finer mesh).

Prerequisites. Computational Chapter 15 (code and solution verification — the MMS order study cited here), Chapter 23 (the seed ensemble and the χ sweep whose sensitivity is reused), the `materials.yaml` schema and loader (the parameter-uncertainty source), and the stochastic diagnostics.

Expected cost. Any CUDA GPU, 2-D, < 1 GB device memory. The budget campaign is ≈ 13 real runs (a 5-seed ensemble, a six-run sensitivity, a three-level mesh refinement, a linear-stability probe); $\approx 2\text{--}3$ min on an RTX 6000 Ada.

24.1 Six questions, not one

A number from this simulator can be wrong in six independent ways, and a defensible study answers each separately:

1. **Code verification** — am I solving the equations *right*? Order/MMS, conservation, energy monotonicity: a bug check against *mathematics*.
2. **Solution verification** — is the *discretization* error of *this* run small enough? Mesh refinement of the observable.
3. **Model validation** — is it the *right model of reality*? Comparison against an independent benchmark.
4. **Parameter uncertainty** — how much does the observable move because a material input has a *spread*?

5. **Stochastic variability** — how much does it move *seed to seed* at fixed parameters?

6. **Model–experiment discrepancy** — the residual gap once the rest are accounted.

The trap is to answer one and imply the others: “it converged, so it is right” confuses (2) with (1) and (3); “mean $\pm$ seed-sd” reports (5) alone and hides (4). We use one material case and one observable to keep all six commensurate.

24.2 One material case, one observable

The case is P3HT:PCBM, read from `materials.yaml`: the Flory interaction $\chi_{\text{pf}} = 0.86$ with a reported spread 0.50 (order of magnitude — the OSC literature has no single “true” χ for this blend). We take that spread as the 1σ parametric uncertainty on χ . The parameter and its uncertainty are pulled through the validated loader, never hand-copied. χ enters the binary Cahn–Hilliard free energy as the enthalpic B (Chapter 23’s engine); the observable is the interfacial-area domain length L_{area} , with nominal value 0.408.

24.3 Verification: code, then solution

Code verification (1). Two invariants a real bug would break: Model-B dynamics conserve mass (measured drift 2.61×10^{-4}) and the free energy is a Lyapunov functional (monotone decrease, largest positive increment 0). The full spatial/temporal order verification is Chapter 15’s manufactured-solution study — cited, not repeated.

Solution verification (2). Refining the mesh from 16^2 to 64^2 moves L_{area} from 0.428 through 0.423 to 0.403 (Fig. 24.2, left). The change over the last refinement, 0.0201, is our *numerical* uncertainty on the observable — the discretization contribution to the budget.

24.4 Model validation (3)

Verification asks whether we solved the equations right; validation asks whether they are the right equations. Absent lab data we validate against an *independent physical benchmark*: Cahn–Hilliard linear stability predicts the fastest-growing *onset* wavelength $\lambda^* = 2\pi/k^*$. A tiny- Δt growth-spectrum probe measures the early-time fastest mode; its peak lands at $\lambda_{\text{meas}} = 0.1818$ against the predicted $\lambda^* = 0.1877$, a 3.2% discrepancy. Note the subtlety: the budgeted observable L_{area} is the *coarsened* end state, many times λ^* — so the *onset* probe, not the end field, is the correct benchmark (comparing the coarsened length to λ^* gives a spurious two-hundred-percent “discrepancy”). Experimental validation would need measured $S(q)$; that is the honest gap in facet (6).

24.5 Uncertainty: parametric and stochastic

Parameter uncertainty (4). A central finite-difference sensitivity (each side averaged over seeds to strip the noise) gives $dL_{\text{area}}/d\chi \approx 0.111$. Times the material’s 1σ spread 0.50 this is a parametric standard deviation $\sigma_{\text{param}} = 0.0557$.

Stochastic variability (5). A 5-seed ensemble at fixed χ gives a seed-to-seed standard deviation $\sigma_{\text{stoch}} = 0.0337$ — the same kind of ensemble spread Chapter 23 aggregated.

24.6 The uncertainty budget

Independent sources add in *variance*, so we combine them in quadrature (Fig. 24.1):

$$\sigma_{\text{total}} = \sqrt{\sigma_{\text{param}}^2 + \sigma_{\text{stoch}}^2 + \sigma_{\text{num}}^2} = 0.0681, \quad (24.1)$$

with $\sigma_{\text{num}} = 0.0201$ the discretization term. The result is unambiguous: the **parameter uncertainty dominates**, 67% of the variance — $1.7\times$ the next largest — against 24% stochastic and 9% numerical. The combined uncertainty is 16.7% of the observable. Because χ is known only to order of magnitude, the morphology’s uncertainty is set by the *material input*, not the seed noise or the mesh. The actionable conclusion: to sharpen this prediction, measure χ better — do *not* buy more GPU seeds or refine the mesh.

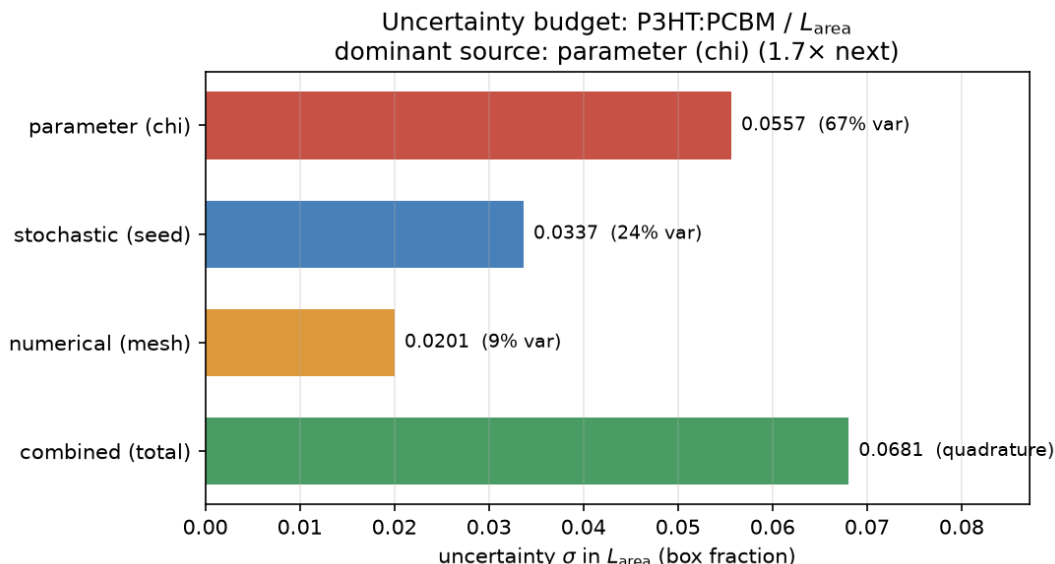


Figure 24.1: The uncertainty budget for L_{area} . The parametric (χ) contribution dominates at 67% of the variance; the seed and mesh contributions are smaller. The combined bar is the quadrature sum — not the arithmetic sum of the three.

24.7 Run it

Run it.

```
cd computational/10_validation_and_uncertainty
python uncertainty.py          # print the six facets + the budget
python gen_figures.py         # figures + numbers/c10.tex + outputs/budget.
                                json
```

It reads P3HT:PCBM and its χ uncertainty from `materials.yaml`, runs the 5-seed ensemble, the sensitivity, the mesh refinement and the linear-stability probe, and writes `budget.json`. Gate with `baseline.yaml`; compare with `EXPECTED.md`.

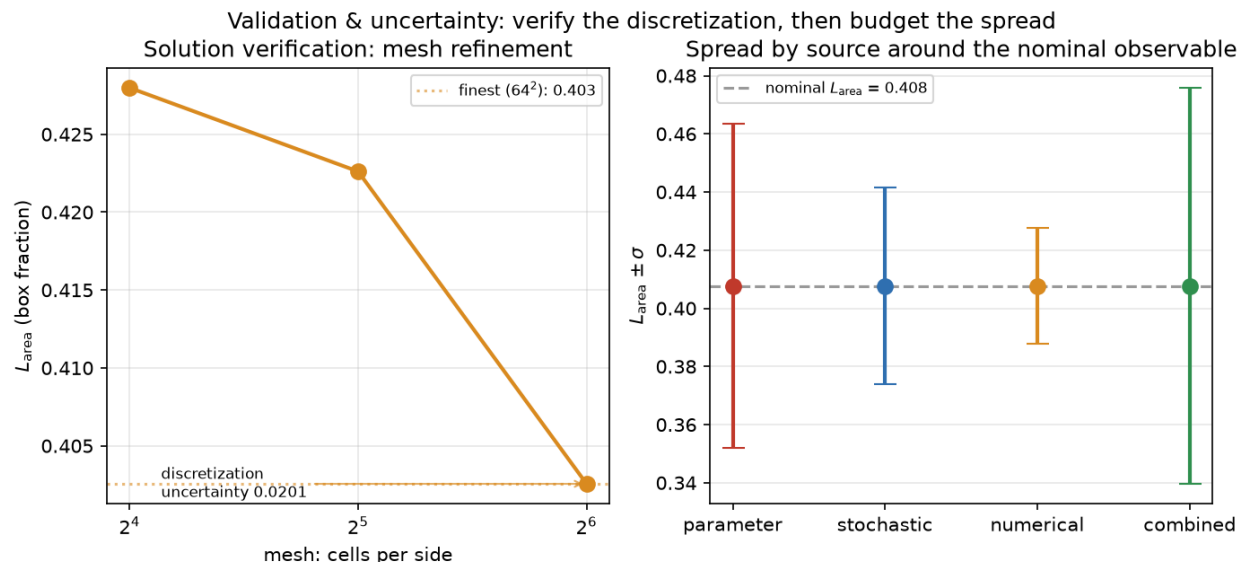


Figure 24.2: *Left*: solution verification — L_{area} under mesh refinement, the last step defining the discretization uncertainty 0.0201. *Right*: the spread by source around the nominal observable; the parametric error bar is the widest, the combined (quadrature) bar wider than any single one.

Measured self-check (the *dominant source* and the *ordering* are load-bearing; last digits vary by card).

source	σ in L_{area}	% of variance
parameter (χ)	0.0557	67
stochastic (seed)	0.0337	24
numerical (mesh)	0.0201	9
combined (quadrature)	0.0681	—

Dominant source: parameter ($1.7\times$ the next). Code verification: mass drift 2.61×10^{-4} , energy monotone. Model validation: λ^* 0.1877 vs measured 0.1818 (3.2%).

24.8 Code walk-through

The six facets and the budget live in `uncertainty.py`.

1. Read the material and its uncertainty.

```
mat = load_system("P3HT_PCBM") # materials.yaml, validated
chi0 = mat.param("chi_polymer_fullerene").value # 0.86
sig = mat.param("chi_polymer_fullerene").uncertainty # order_of_mag 0.5
```

2. Propagate: sensitivity times spread.

```
dLdchi = (L(chi0+d) - L(chi0-d)) / (2 d) # noise-averaged over seeds
sigma_param = abs(dLdchi) * sig # facet 4
sigma_stoch = sd(L over seeds at chi0) # facet 5
```

3. Combine in quadrature, name the dominant source.

```
sigma_total = sqrt(param^2 + stoch^2 + num^2)
dominant     = argmax(variance contributions)      # -> parameter (chi)
```

24.9 Explore on your own

Q24.9.1 Build the budget for a *different* material in `materials.yaml` (e.g. PDPP5T:PCBM). Does the parametric source still dominate? For which materials would the seed noise win, and why?

Q24.9.2 Reproduce the onset-vs-coarsened trap: compare the end-state L_{area} wavelength to λ^* and watch the “discrepancy” balloon past 200%. Explain why the early-time probe is the only fair model-validation benchmark here.

Q24.9.3 Add a fourth budget term: the temporal (time-step) uncertainty, by refining Δt the way facet 2 refines the mesh. Where does it rank?

Q24.9.4 Interpret “order of magnitude” as a factor (log-normal) rather than an additive ± 0.5 . Recompute σ_{param} . Does the *dominance* survive the reinterpretation?

Q24.9.5 The combined σ is 0.0681, not the arithmetic sum $0.0557 + 0.0337 + 0.0201$. Explain why independent sources add in variance, and by how much the arithmetic sum would overstate the true uncertainty.

Part III

Differentiation

Chapter 25

What differentiable simulation is

Key idea. A classical solver answers “*what happens?*”. A differentiable solver also answers “*what should I change?*” — it returns the gradient of any scalar output J with respect to any input parameter p (material coefficients, a process schedule, geometry). That single object dJ/dp turns the simulator from a *predictor* into an *instrument for design and inference*. And because a gradient you cannot trust is worse than none, DiffSim computes it *three independent ways* and checks that they agree.

25.1 Why a gradient

Every question we ask of a morphology simulator downstream — fit this material parameter to a measured morphology, choose a quench schedule that hits a target crystallinity, learn a free-energy functional from snapshots — is an *optimization*. Optimization needs a gradient: dJ/dp says how a scalar objective J (a domain size, an interfacial area, a crystalline fraction) moves when you nudge a parameter p .

The naive way to get dJ/dp is finite differences: run the simulation at $p+\epsilon$ and $p-\epsilon$ and subtract. That costs *two forward solves per parameter* and forces you to tune ϵ (too big: truncation error; too small: roundoff). With hundreds of parameters — a temperature schedule sampled every step, a free energy with many coefficients — it is hopeless. The **adjoint method** returns dJ/dp for *all* parameters at once, at the cost of a single extra (linear) solve backward through the trajectory, exact to machine precision. That $O(1)$ -in-parameters cost is what makes design possible.

25.2 The three ways to a gradient

DiffSim’s differentiable core (`src/diffsim/adjoint/`) computes the same gradient three ways, and the tutorial runs all three live:

1. **Adjoint** — a hand-derived reverse sweep. We do *not* tape the Newton iterations; we differentiate only the converged discrete relation $R_n(x_n; x_{\text{hist}}, p) = 0$ (the implicit-function theorem). For $J = \sum_n j(x_n)$,

$$J_n^\top \lambda_n = \frac{\partial j}{\partial x_n} - \sum_{k \geq 1} \left(\frac{\partial R_{n+k}}{\partial x_n} \right)^\top \lambda_{n+k}, \quad \frac{\partial J}{\partial p} = - \sum_n \lambda_n^\top \frac{\partial R_n}{\partial p}, \quad (25.1)$$

where $J_n = \partial R_n / \partial x_n$ is *exactly* the Jacobian the forward Newton already assembled at convergence, and $\partial R_n / \partial p$ is analytic. This is the production capability.

2. **Autograd twin** — a faithful PyTorch reimplementation of the *same* discrete solver, differentiated by `torch.autograd` through the converged Newton loop. Different code, a different differentiation engine, the same math: an *independent* reference.
3. **Finite differences** — central differences $(J(p + \epsilon) - J(p - \epsilon))/2\epsilon$. No calculus at all; the ground truth every gradient must match, down to the FD error floor.

The adjoint differentiates the *real* brick. The discrete forward in `adjoint/phasefield.py` is a vectorised re-implementation of the production Cahn–Hilliard residual and Jacobian that is *bit-parity* with `CahnHilliardStepper` (measured $\max |\Delta c| \sim 10^{-16}$). So the gradient is taken through the same operator the research code marches — not a look-alike toy.

25.3 Run it

Run it.

```
cd differentiable/01_three_way_gradient
python run.py # 9x9 nodes, 3 steps, three-way check
```

It marches a short Flory–Huggins Cahn–Hilliard rollout, forms the scalar objective $J = \frac{1}{2}\|c_N - \frac{1}{2}\|^2$, and computes $dJ/d\{M, \kappa, \chi, A\}$ all three ways. It prints the table and **self-checks** that the methods agree — that agreement is the gate. Compare with `EXPECTED.md`.

Measured self-check (three-way gradient of $J = \frac{1}{2}\|c_N - \frac{1}{2}\|^2$ through a 3-step march, 9×9 nodes).

parameter	adjoint dJ/dp
M (mobility)	3.077×10^{-1}
κ (gradient)	-7.357×10^0
$\chi = B$ (Flory)	8.196×10^{-1}
A (entropic)	-1.668×10^0

Adjoint and autograd twin agree to $\sim$ machine precision (worst $\leq 1.8 \times 10^{-16}$): the *same* discrete math, two independent differentiation engines. Adjoint and finite differences agree to worst $\leq 1.5 \times 10^{-9}$: the FD truncation/roundoff floor.

25.4 Code walk-through

The tutorial’s core is `three_way.py`; it drives `src/diffsim/adjoint/phasefield.py`.

1. One reverse sweep, all gradients.

```
fwd = CHForward(dm, FHEnergy(A, B), M=M, kappa=kappa, dt=dt, order=order)
fwd.set_initial(ic); fwd.run(n_steps) # record the trajectory
dJdx = [zeros] * n_steps
dJdx[-1][0::2] = fwd.steps[-1]["c"] - target # dj/dc on the final
step
g_adj = CHAdjoint(fwd).gradient(dJdx, ["M", "kappa", "B", "A"])
```

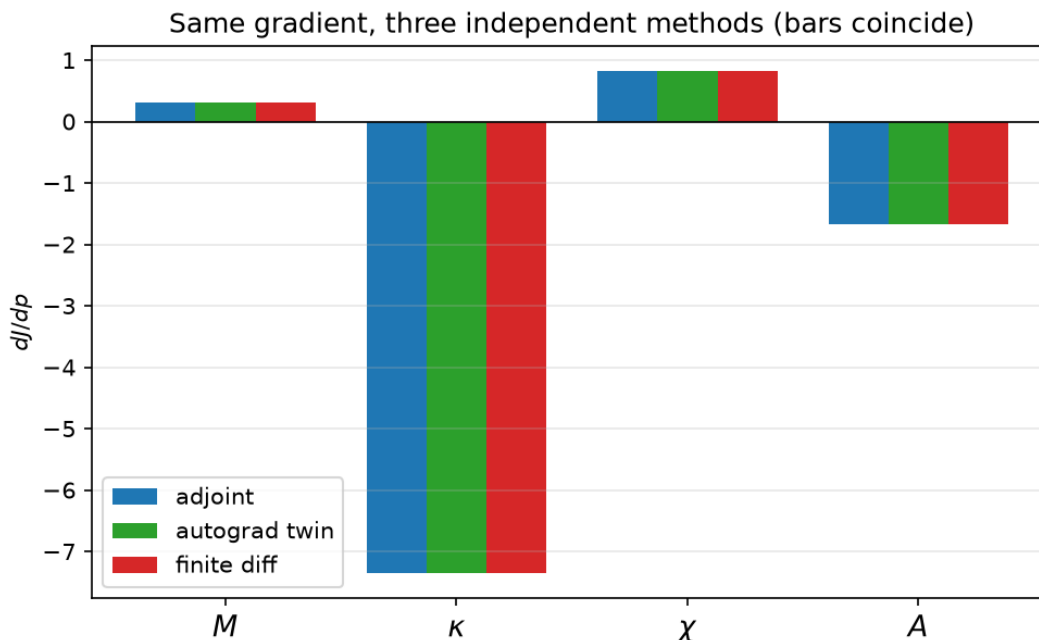


Figure 25.1: The same gradient, three ways. For each parameter the adjoint, autograd-twin, and finite-difference bars are *visually coincident* — that overlap is the message: three independent methods return one number. The signs are physical (raising χ drives the field away from the uniform $c = \frac{1}{2}$, so J grows; $dJ/d\chi > 0$).

gradient runs (25.1) backward through the recorded steps and returns *every* dJ/dp from that one sweep.

2. The independent twin.

```
twin = CHTwin(dm, energy="fh", dt=dt, order=order, device="cpu")
out = twin.march(c0, None, M, kappa, dict(A=A, B=B), n_steps) # leaf
      tensors
loss = 0.5 * ((out[-1][0] - target)**2).sum(); loss.backward()
```

Autograd differentiates through the converged Newton iterations; because the loop is driven to a tight residual, that returns the same implicit-function-theorem gradient the hand adjoint derives.

3. The ground truth. Central differences on each parameter — two forward solves apiece — with no calculus, is what pins the other two to reality.

25.5 Explore on your own

Q25.5.1 Switch to BDF2 (--order 2 --steps 4). The variable-coefficient history now couples *two* previous steps into each residual; does the three-way agreement survive? (It must — the adjoint's history cotangent carries both slots.)

Q25.5.2 Add a second objective, e.g. the interfacial energy $\frac{\kappa}{2} \|\nabla c_N\|^2$. Re-derive its seed dj/dc_N and confirm the adjoint still matches finite differences.

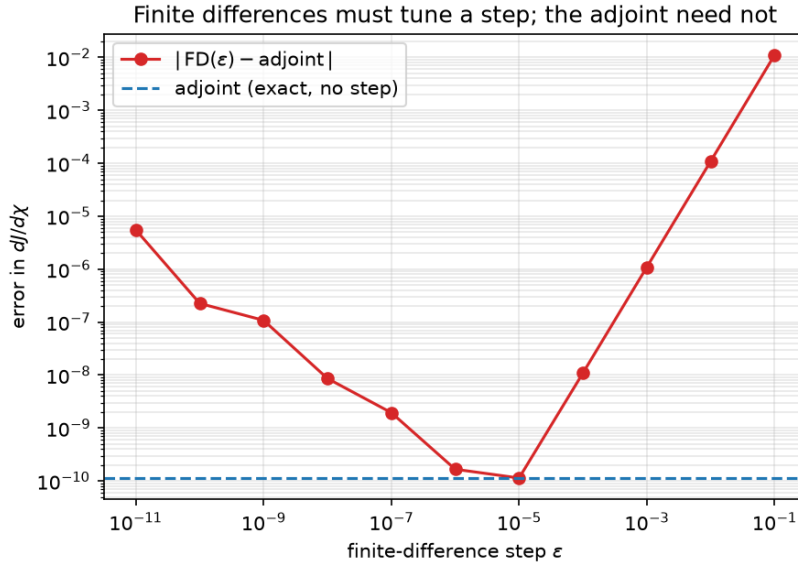


Figure 25.2: Why the adjoint, not finite differences. The error of the central difference for $dJ/d\chi$ is a V-shaped function of the step ϵ : on the right, $O(\epsilon^2)$ truncation error falls as ϵ shrinks; on the left, catastrophic cancellation (roundoff) takes over and it rises. The valley floor (best FD $\approx 1.1 \times 10^{-10}$) is the most FD can achieve — and it sits far above the adjoint, which is exact and needs no step to tune.

Q25.5.3 Time the adjoint vs. finite differences as you add parameters (say, a per-step schedule of length N). The adjoint cost is flat in N ; the FD cost grows linearly. At what N does the crossover make FD impractical?

Q25.5.4 Shrink the FD step ϵ toward 10^{-12} and watch the error in Fig. 25.2 *rise*. Explain why, and why the adjoint has no analogous floor.

Chapter 26

Morphology sensitivity

Key idea. With the gradient now trustworthy, ask a physical question: how sensitive is the *morphology* to the material parameters? Pick a scalar summary of the pattern — here the *demixing amplitude* $J = \frac{1}{2}\|c_N - \bar{c}\|^2$, the spatial variance of the composition — and the adjoint returns $dJ/d\chi$ and $dJ/d\kappa$ in one sweep. The *signs* are the physics: $dJ/d\chi > 0$ (interaction sharpens the pattern), $dJ/d\kappa < 0$ (gradient penalty blurs it).

26.1 A scalar handle on the morphology

A morphology is a whole field; to differentiate we need a scalar. Any morphology metric will do — domain size, interfacial area, a structure factor — and the adjoint machinery is identical: pick $J(c_N)$, hand it the seed dj/dc_N , get every dJ/dp . We use the demixing amplitude

$$J(c_N) = \frac{1}{2} \sum_i (c_{N,i} - \bar{c})^2, \quad \bar{c} = \text{mean}(c_0), \quad (26.1)$$

which is small for a nearly uniform blend and grows as the two phases separate — a scalar proxy for “how demixed is the film”. Because the Cahn–Hilliard dynamics conserve mass, $\bar{c}$ is fixed by the initial condition, so the seed $dj/dc_N = (c_N - \bar{c})$ is exact and clean.

The two sensitivities have opposite, and physical, signs. Above the spinodal ($\chi > 2A$) a larger interaction χ deepens the enthalpic well and drives demixing harder, so J grows: $dJ/d\chi > 0$. The gradient penalty $\frac{\kappa}{2}|\nabla c|^2$ instead *opposes* composition variation — a larger κ thickens interfaces and smooths the field — so $dJ/d\kappa < 0$. The adjoint recovers both, and finite differences confirm them.

26.2 Run it

Run it.

```
cd differentiable/02_morphology_sensitivity
python run.py # dJ/dchi and dJ/dkappa, FD-verified
```

It marches a short Flory–Huggins rollout at $\chi = 2.5$, $\kappa = 0.01$ (interior data, so the logs stay smooth), forms (26.1), and prints $dJ/d\chi$, $dJ/d\kappa$ with their finite-difference checks and sign assertions. Compare with EXPECTED.md.

Measured self-check (6-step march, $\chi = 2.5$, $\kappa = 0.01$).

	value	meaning
demixing amplitude J	3.095×10^{-1}	how separated the blend is
$dJ/d\chi$	7.433×10^{-1}	> 0 : interaction sharpens
$dJ/d\kappa$	-6.691×10^0	< 0 : gradient penalty blurs
worst adjoint-vs-FD	1.0×10^{-9}	the FD-verified gate

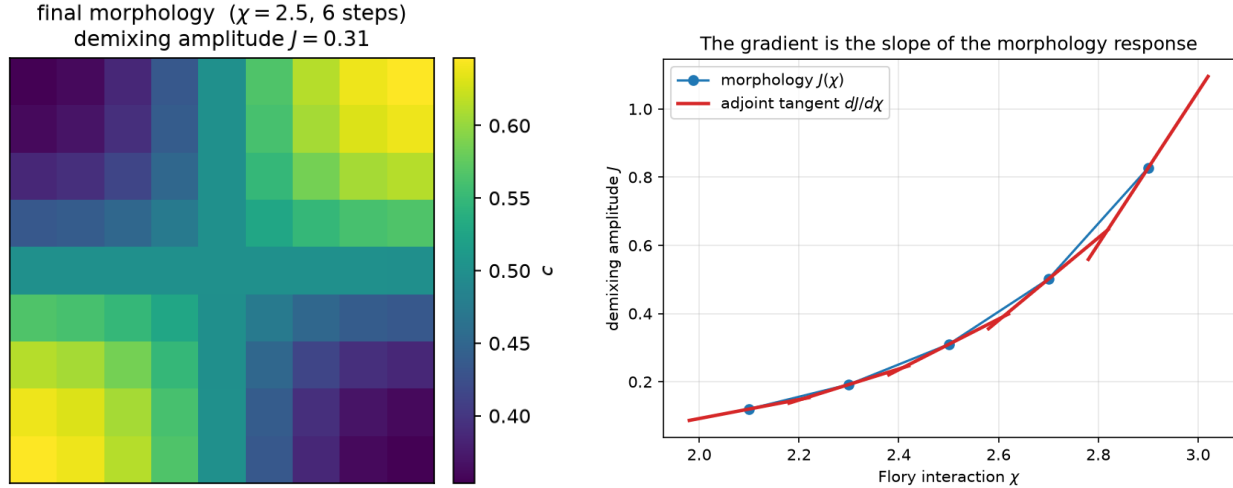


Figure 26.1: Left: the final morphology whose scalar summary J we differentiate. Right: the demixing amplitude $J(\chi)$ sampled across interaction strengths (blue), with the adjoint tangents $dJ/d\chi$ (red) drawn at each point. The tangents lie *along* the response curve — the gradient is the local slope of the morphology’s answer to a change in χ .

26.3 Code walk-through

The seed is the only thing that changes from D1; the adjoint sweep is identical.

```
fwd, cN = march(dm, coords, chi, kappa, n_steps) # short CH rollout
dJdx[-1][0::2] = cN - c_bar # dj/dc_N for eq. (D2)
g = CHAdjoint(fwd).gradient(dJdx, ["B", "kappa"]) # -> dJ/dchi, dJ/dkappa
```

Swap in any other differentiable morphology metric — redefine $J(c_N)$ and its seed — and the same call returns its sensitivities. That substitutability is the point: the adjoint does not care what scalar you put on top of the field.

26.4 Explore on your own

Q26.4.1 Replace the demixing amplitude with a genuine interfacial-area proxy, $J = \frac{1}{2}\|\nabla c_N\|^2$. Derive its seed and check the sign of $dJ/d\kappa$ — does a thicker interface raise or lower this metric?

Q26.4.2 Sweep χ from below the spinodal ($\chi < 2$) to well above. Where does $dJ/d\chi$ turn on, and does the onset line up with the spinodal threshold from Physics P1?

Q26.4.3 Push the rollout longer (`--steps 20`) and watch the Flory–Huggins logs approach the walls. At what point does the forward solve fail, and what does that tell you about differentiating stiff free energies near the pure phases?

Chapter 27

Recovering parameters from a morphology

Key idea. Turn the arrow around. Instead of “given the material, what morphology?”, ask “given the morphology, what material?”. This is an *inverse problem*: minimise the misfit between the simulated and the target field over the material parameters, using the adjoint gradient. It is the tutorial-scale version of *learning the material from the morphology* — the loop that, at research scale, fits a free-energy functional to measured micrographs.

27.1 The inverse problem

Given a target composition field c_N^{target} (from an experiment, or — here — from a known ground-truth material so we can check the answer), find the parameters $\theta = (\chi, \kappa)$ that reproduce it. The objective is the field misfit

$$L(\theta) = \frac{1}{2} \|c_N(\theta) - c_N^{\text{target}}\|^2, \quad (27.1)$$

and we minimise it by quasi-Newton descent (L-BFGS-B). Every optimiser step needs L and its gradient $dL/d\theta$; the adjoint delivers both from one forward rollout plus one reverse sweep — the seed is $dj/dc_N = c_N - c_N^{\text{target}}$, exactly the D2 machinery with a different scalar on top.

Identifiability. The recovery works because χ and κ leave *distinguishable* fingerprints: the interaction sets *how much* the blend demixes, the gradient penalty sets the *interface width*. When two parameters move the morphology the same way, no gradient can separate them and the loss stalls on a valley floor — a real hazard of inverse problems, and one of the questions below.

27.2 Run it

Run it.

```
cd differentiable/03_recover_parameters
python run.py                # FD-check the gradient, then recover (chi,
                             kappa)
```

It generates a target from truth $(\chi, \kappa) = (2.5, 0.01)$, checks the loss gradient against finite

differences, then recovers the material from the wrong initial guess. Compare with `EXPECTED.md`.

Measured self-check (6-step rollout, L-BFGS-B on the adjoint gradient).

parameter	truth	recovered	error
χ (interaction)	2.5	2.50000	2.2×10^{-11}
κ (gradient pen.)	0.01	0.01000	1.8×10^{-11}

Field misfit L : 6.17×10^{-2} (initial guess) $\rightarrow 1.6 \times 10^{-20}$ (recovered), in 13 optimiser evaluations.

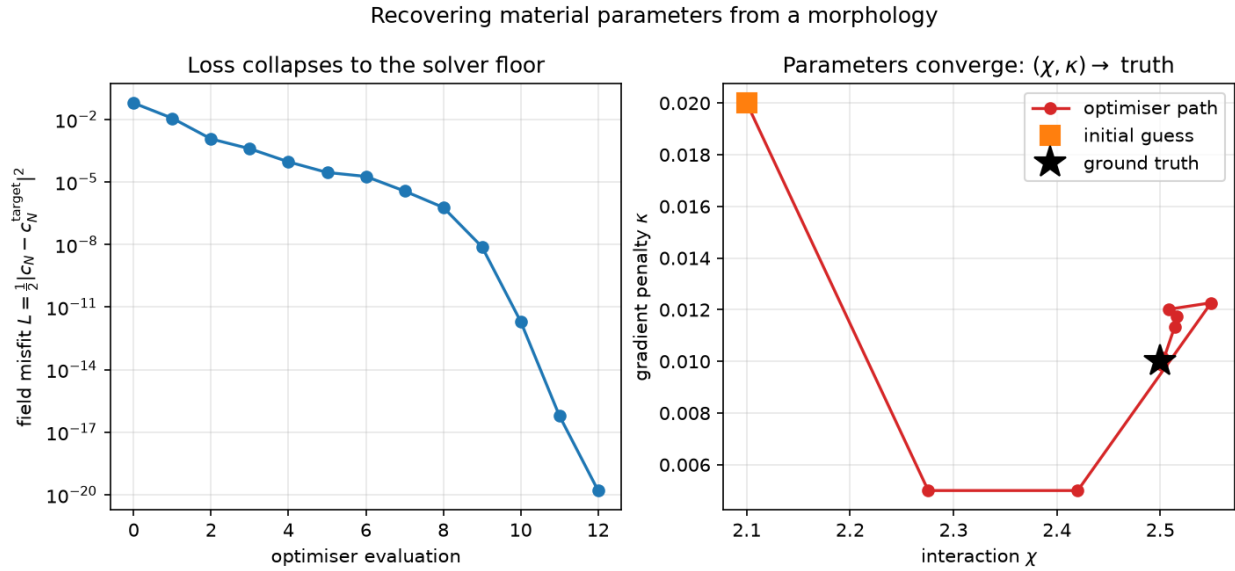


Figure 27.1: Recovering the material from the morphology. Left: the field misfit (27.1) collapses to the solver floor — the target was produced by the forward model, so a perfect fit exists and gradient descent finds it. Right: the optimiser’s path in (χ, κ) space, from the wrong initial guess (square) to the ground truth (star).

27.3 Code walk-through

The objective-and-gradient the optimiser calls is the whole method:

```
def loss_and_grad(theta, target):
    fwd, cN = march(dm, coords, chi=theta[0], kappa=theta[1], ...)
    r = cN - target; L = 0.5 * (r @ r) # eq. (D3)
    dJdx[-1][0::2] = r # dL/dc_N
    g = CHAdjoint(fwd).gradient(dJdx, ["B", "kappa"])
    return L, np.array([g["B"], g["kappa"]])

minimize(loss_and_grad, theta0, jac=True, method="L-BFGS-B", bounds=...)
```

scipy’s L-BFGS-B does the rest. Bounds keep the trial parameters in the interior region where the Flory–Huggins logs stay smooth — a reminder that inverse problems inherit the forward model’s

domain of validity.

27.4 Explore on your own

Q27.4.1 Fit *only* χ with κ held at the truth, then fit both. Does the joint problem converge as cleanly, or do the two parameters trade off early in the descent?

Q27.4.2 Corrupt the target with noise ($c^{\text{target}} \rightarrow c^{\text{target}} + \sigma \xi$). How does the recovered (χ, κ) degrade with σ , and does the loss floor rise to $\sim \sigma^2$ times the number of nodes?

Q27.4.3 Start the guess on the *wrong side* of the spinodal ($\chi_0 < 2$). Does the optimiser still find the demixing truth, or does a vanishing early gradient trap it in the mixed state?

Chapter 28

Process gradients

Key idea. Material parameters are what the blend *is*; *process* parameters are what we *do* to it. The headline process control is the temperature schedule $T(t)$ — the quench — applied while the film crystallises. The adjoint returns the gradient of a morphology objective with respect to the *whole schedule at once*, as a *time series* dJ/dT_n : one number per step, from a single reverse sweep. That is what makes designing a process (a long schedule) tractable.

28.1 Temperature enters everywhere

The physics is the coupled Cahn–Hilliard $\times$ Allen–Cahn crystallisation system (Physics P6–P7): a conserved composition ϕ and a non-conserved crystallinity ψ . Temperature enters each step’s residual through *two* channels:

$$\text{drive} = \Delta h \left(\frac{T}{T_m} - 1 \right) \quad (\text{Turnbull crystallisation}), \quad B(T) = B_0 + b_T (T - T_{\text{ref}}) \quad (\text{Flory interaction}). \quad (28.1)$$

So a change in T at step n ripples through both the crystallisation thermodynamics and the demixing thermodynamics of that step, and then forward through every later step via the coupled history. The adjoint sweep runs that dependency backward and returns dJ/dT_n for all n simultaneously.

One sweep, N gradients. A finite-difference schedule gradient costs $2N$ extra forward solves (perturb each T_n up and down); the adjoint costs *one* backward solve, independent of the schedule length N . For a schedule sampled every time step — hundreds of controls — only the adjoint is affordable. This is the same $O(1)$ -in- parameters advantage as D1, now paying off on a genuinely high- dimensional control.

28.2 Run it

Run it.

```
cd differentiable/04_process_gradients
python run.py                # dJ/dT_n time series, FD-verified per step
```

It marches a 4-step crystallisation under the schedule $T = [0.5, 0.6, 0.7, 0.55]$, forms an objective on the final ϕ, ψ , and returns the whole dJ/dT_n series, checking each entry against finite

differences. Compare with EXPECTED.md.

Measured self-check (schedule gradient of a 4-step BDF2 crystallisation, target $\langle\psi\rangle = 0.45$).

step n	T_n	dJ/dT_n (adjoint)
0	0.50	3.826×10^{-1}
1	0.60	2.998×10^{-1}
2	0.70	3.328×10^{-1}
3	0.55	2.822×10^{-1}

Worst adjoint-vs-FD agreement over the schedule: 6.8×10^{-10} (N gradients checked, not one).

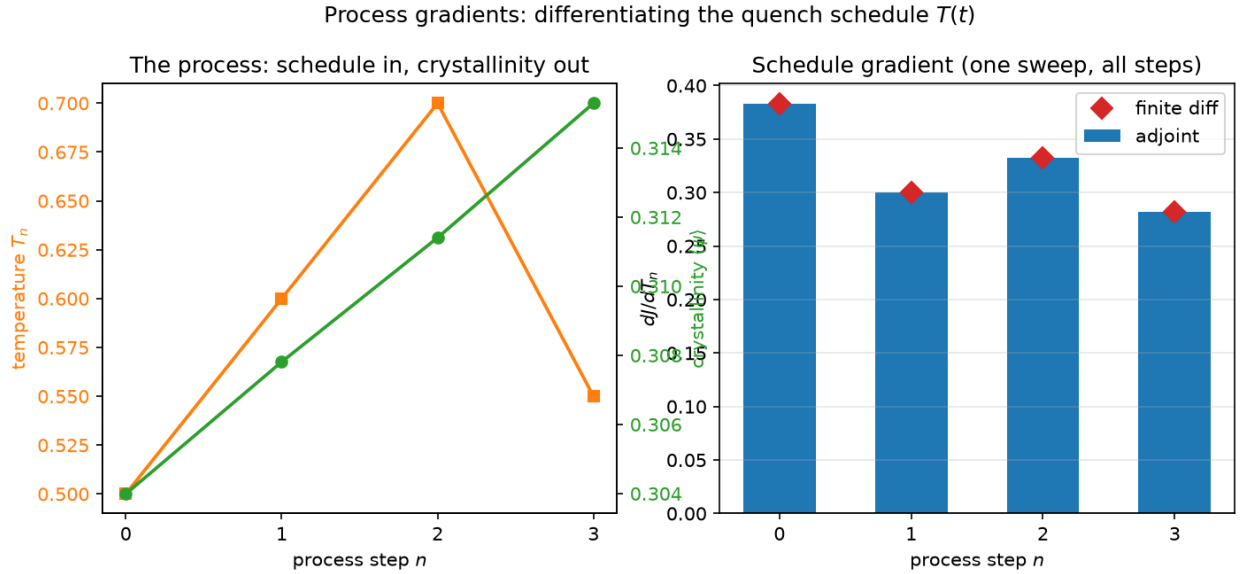


Figure 28.1: Differentiating the quench. Left: the imposed schedule T_n (orange) and the crystallinity $\langle\psi\rangle$ it produces (green) — the process, input and output. Right: the schedule gradient dJ/dT_n (adjoint bars, finite-difference diamonds), one value per step from a single reverse sweep. The bars say how much the temperature at each moment of the process matters to the final morphology.

28.3 Code walk-through

```
fwd = CACHForward(dm, FHEnergy(A, B), ..., dt=dt, order=2)
fwd.set_initial(phi0, psi0)
fwd.set_schedule(Tsched, bT=bT, Tref=Tref) # per-step temperature control
fwd.run(len(Tsched))
...
gT = CACHAdjoint(fwd).temperature_gradient(dJdx) # the whole time series
```

`set_schedule` makes each step read its temperature from `Tsched[n]`; `temperature_gradient` runs the reverse sweep and, at each step, contracts the adjoint state against $\partial R_n / \partial T$ (both the drive

and the b_T chi channel) — returning the length- N vector dJ/dT_n .

28.4 Explore on your own

Q28.4.1 Which step's temperature matters most, and why? Relate the ranking of $|dJ/dT_n|$ to when crystallisation is most active in the $\langle\psi\rangle$ trace.

Q28.4.2 Set $b_T = 0$ (decouple T from the Flory interaction). How much of each dJ/dT_n came from the demixing channel versus the crystallisation drive?

Q28.4.3 Lengthen the schedule to $N = 12$ and time the adjoint against a finite-difference gradient. Confirm the adjoint cost is flat while the FD cost grows like N — the crossover D1 anticipated, now on a real process control.

Chapter 29

Crystallinity sensitivity

Key idea. Device performance often hinges on *crystallinity* — how much of the film has ordered. This concept differentiates a crystallinity metric through the coupled Cahn–Hilliard $\times$ Allen–Cahn rollout with respect to the thermodynamic crystallisation parameters $(\Delta h, T_m, \Delta\sigma, \varepsilon^2, L)$ — all five from one adjoint sweep, each finite-difference verified. Now the sensitivities flow through the *non-conserved* crystallinity field ψ , not just the composition.

29.1 Differentiating the crystalline order

The crystallisation free energy (Physics P6) adds to the demixing energy a driven Allen–Cahn field ψ :

$$f = f_{\text{FH}}(\phi) + \phi[q(\psi)\Delta\sigma + p(\psi)\text{drive}], \quad \text{drive} = \Delta h\left(\frac{T}{T_m} - 1\right), \quad (29.1)$$

with $q(\psi) = \psi^2(1 - \psi)^2$ the double-well barrier (height $\Delta\sigma$) and $p(\psi) = \psi^2(3 - 2\psi)$ the monotone interpolation. The crystallinity relaxes by $\dot{\psi} = -L[\partial f/\partial\psi - \varepsilon^2\nabla^2\psi]$. Five parameters shape it: the latent heat Δh and melting point T_m set the *driving force*, the barrier $\Delta\sigma$ *opposes* ordering, the gradient penalty ε^2 sets *interface stiffness*, and L sets the *rate*. We take a crystallinity metric $J = \frac{1}{2}\|\psi_N - \psi^*\|^2$ and let the adjoint report how each moves it.

Reading the signs. At the tutorial setting the mean crystallinity $\langle\psi\rangle \approx 0.3122$ sits *below* the target $\psi^* = 0.4$. So any parameter that *raises* crystallinity lowers J : raising the latent heat Δh deepens the drive ($dJ/d\Delta h < 0$), while raising the barrier $\Delta\sigma$ resists ordering ($dJ/d\Delta\sigma > 0$). The gradient’s sign is a statement of physics, and it is exactly what finite differences confirm.

29.2 Run it

Run it.

```
cd differentiable/05_crystallinity_sensitivity
python run.py # dJ/d{dh, Tm, dsig, eps2, L}, FD-verified
```

It marches a 3-step crystallisation, forms $J = \frac{1}{2}\|\psi_N - 0.4\|^2$, and prints the five sensitivities with their finite-difference checks. Compare with `EXPECTED.md`.

Measured self-check (3-step crystallisation, $\langle\psi\rangle = 0.3122$, target $\psi^* = 0.4$).

parameter	adjoint dJ/dp	role
Δh (latent heat)	-8.325×10^{-2}	driving force
T_m (melting temp)	-1.388×10^{-2}	drive sign
$\Delta\sigma$ (barrier)	1.640×10^{-2}	opposes ordering
ε^2 (stiffness)	-1.673×10^{-1}	interface width
L (AC kinetics)	-7.191×10^{-2}	relaxation rate

Worst adjoint-vs-FD agreement: 9.9×10^{-10} .

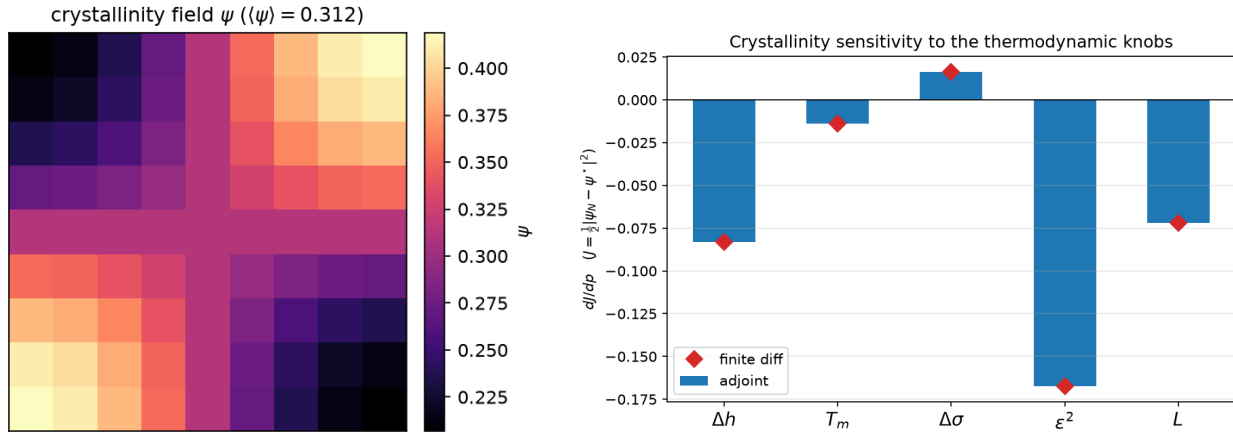


Figure 29.1: Left: the crystallinity field ψ whose scalar summary we differentiate. Right: the five crystallisation sensitivities (adjoint bars, finite-difference diamonds). Their relative sizes rank the knobs: the interface stiffness ε^2 and the latent heat Δh move the crystallinity most at this setting.

29.3 A frontier, named honestly

Noise is a different derivative. Nucleation is seeded by a *stochastic* term (Physics P8). The adjoint here differentiates the deterministic crystallisation *drift*. A sensitivity to the *noise amplitude* is a derivative of an *expectation* — a pathwise or score-function estimator, not a single-trajectory adjoint — and is a documented frontier. The tutorial differentiates the drift parameters the merged adjoint covers, and stops there rather than present a stochastic gradient it has not verified.

29.4 Explore on your own

Q29.4.1 Push T below and above T_m . The drive $\Delta h(T/T_m - 1)$ changes sign at $T = T_m$; does $dJ/d\Delta h$ flip with it, and does $\langle\psi\rangle$ melt back down above T_m ?

Q29.4.2 Rank the five sensitivities by magnitude and design the *cheapest* intervention: which single parameter, moved a little, changes the crystallinity most? Does the ranking survive a longer rollout?

Q29.4.3 Add the composition target back (differentiate $J = \frac{1}{2}(\|\phi_N - \phi^*\|^2 + \|\psi_N - \psi^*\|^2)$) and watch the demixing parameters M, κ, χ re-enter the gradient. Which parameters couple demixing and crystallisation?

Chapter 30

Learning the free energy from snapshots

Key idea. D3 recovered scalar *parameters* of a known free energy. Now recover the free-energy *functional* itself: given a time series of morphology snapshots, learn the bulk energy $f(c)$ that produced them. This is *learning the thermodynamics from the morphology* — the Wodo–Ganapathysubramanian programme — at tutorial scale, and it runs on exactly the same adjoint.

30.1 A learnable free energy

Parametrise the derivative of the bulk energy on a monomial basis,

$$f'(c) = \sum_i a_i c^{p_i}, \quad p = [3, 1], \quad \frac{\partial f'}{\partial a_i} = c^{p_i}, \quad (30.1)$$

so the coefficient sensitivity $\partial f'/\partial a_i$ is just the basis function — it drops straight into the adjoint's $\partial R_\mu/\partial p = -\int N(\partial f'/\partial p)$. The ground truth is the canonical double well $f'(c) = c^3 - c$, i.e. $a = [1, -1]$. We fit the a_i by minimising the misfit over a *trajectory* of snapshots,

$$L(a) = \sum_{n \in \mathcal{S}} \frac{1}{2} \|c_n(a) - c_n^{\text{data}}\|^2, \quad (30.2)$$

with $\mathcal{S}$ a chosen subset of steps; the gradient dL/da comes from one CHAdjoint sweep that seeds every snapshot step.

What a morphology cannot tell you. A constant term in $f'(c)$ merely shifts the chemical potential μ by a constant; the conserved c -dynamics ($c_t = \nabla \cdot M \nabla \mu$) are invariant to it. So a constant is *unidentifiable* from composition data, and the basis (30.1) excludes it on purpose. Knowing what your data *cannot* constrain is as important as the fit itself.

30.2 Run it

Run it.

```
cd differentiable/06_learn_free_energy
python run.py # learn  $f'(c)=c^3-c$  from snapshots
```

It generates a truth trajectory, FD-checks the trajectory-loss gradient, recovers the coefficients from clean data, and shows the noisy recovery sharpen as more snapshots are used. Compare with EXPECTED.md.

Measured self-check (learn $f'(c) = c^3 - c$ from 10 snapshots).

- **Clean recovery:** $a = [1.00000, -1.00000]$, $|a - \text{truth}|_\infty = 1.4 \times 10^{-11}$, loss 6.6×10^{-21} .
- **Noise beats down with data** ($\sigma = 0.01$, averaged over 12 seeds): recovery error 0.105 at $K=1$ snapshot $\rightarrow 7.0 \times 10^{-3}$ at $K=10$ — about $15\times$ better.

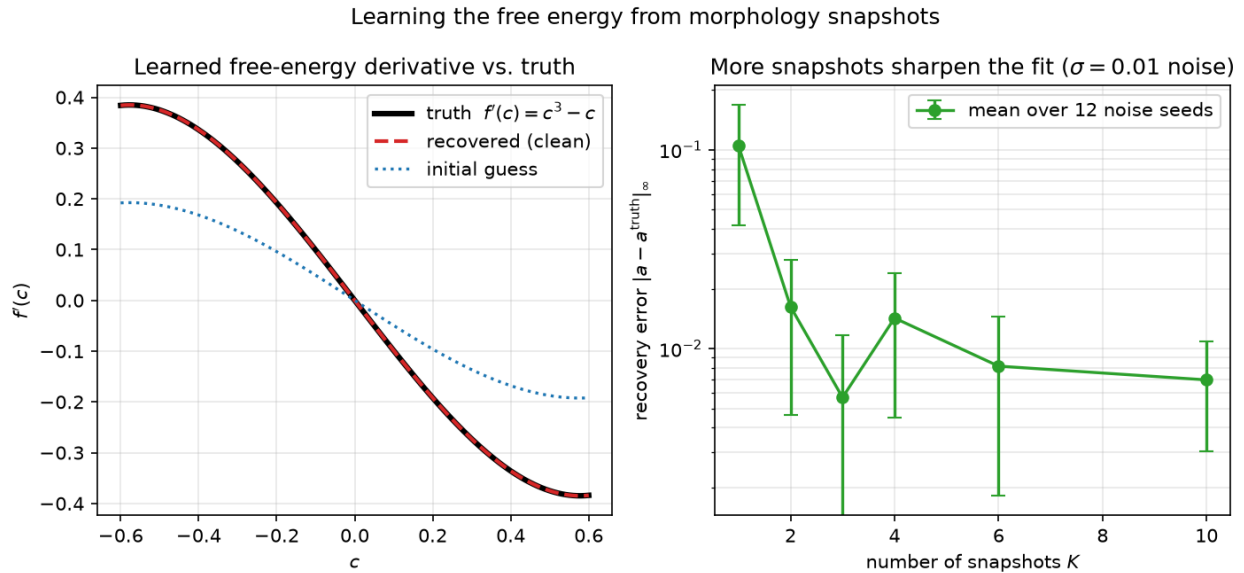


Figure 30.1: Learning the free energy from morphology. Left: the recovered $f'(c)$ (dashed) lies exactly on the truth $c^3 - c$ (black), starting from a wrong initial guess (dotted). Right: with noisy data the recovery error falls as more snapshots K are used — each snapshot is an independent constraint on the same coefficients, so a trajectory beats a single frame.

30.3 Code walk-through

```
f = CHForward(dm, PolyBasisEnergy([3,1], a), M=M, kappa=kappa, ...) #  $f'=a_0 c^3 + a_1 c$ 
f.set_initial(c0); f.run(nst)
for n in snaps: # trajectory misfit
    r = f.steps[n]["c"] - data[n]; L += 0.5 * (r @ r); dJ[n][0::2] = r
g = CHAdjoint(f).gradient(dJ, ["a0", "a1"]) #  $dL/da_i$  via  $df'/da_i = c^{\{p_i$ 
}
minimize(loss_and_grad, a0, jac=True, method="L-BFGS-B")
```

The only new ingredient over D3 is the *basis* energy: because $\partial f'/\partial a_i$ is analytic, the same adjoint that returned material-parameter gradients now returns *functional* gradients. An MLP free energy would slot into the identical loop (the autograd twin already differentiates an arbitrary f') — the documented next step.

30.4 Explore on your own

Q30.4.1 Add a spurious basis term (e.g. c^5) the truth does not use. Does the fit drive its coefficient to zero, and how much data does that take?

Q30.4.2 Try to also recover a constant term $a c^0$. Watch it stay unidentified no matter how much data you add, and connect that to the conservation argument above.

Q30.4.3 Increase the noise σ and re-plot error-vs- K . Does the floor scale like σ , and does the number of snapshots needed for a given accuracy grow as you would expect from averaging?

Chapter 31

Capstone: inverse-designing a process

Key idea. The capstone closes the loop. Everything before this *computed* a gradient; here we *use* it to design. This is PDE-constrained optimisation: find a temperature schedule $T(t)$ that drives the crystallisation to a *target crystalline fraction*, by minimising an objective over the whole schedule with the D4 schedule gradient. The simulator, run backward, returns the recipe.

31.1 Design as optimisation

Specify what you want — here a target crystalline fraction ϕ^* — as an objective over the process control $T(t)$:

$$\min_{T \in [T_{\min}, T_{\max}]^N} J(T) = \frac{1}{2} (\langle \psi_N \rangle(T) - \phi^*)^2, \quad (31.1)$$

where $\langle \psi_N \rangle$ is the mean crystallinity the schedule produces. Each optimiser step needs J and its gradient dJ/dT_n ; the adjoint returns the whole time-series gradient (D4) from one reverse sweep, and a bounded quasi-Newton method (L-BFGS-B) walks the schedule downhill. The bounds keep the temperatures physical.

A reachable target, so we can check. The target is generated by a ground-truth cold ramp, so a schedule that hits it provably exists — which lets the tutorial *verify* that the designed process reaches the specification, not merely that the optimiser stopped. Point the same machinery at a real specification and it returns a candidate recipe.

31.2 Run it

Run it.

```
cd differentiable/07_inverse_design_capstone
python run.py # design T(t) to hit a target crystallinity
```

It sets a target from a ground-truth ramp, FD-checks the design gradient, then optimises the schedule from a flat guess and confirms the achieved fraction reaches the target. Compare with EXPECTED.md.

Measured self-check (design a 6-step schedule to a target crystalline fraction).

quantity	value
target crystalline fraction	0.23877
flat guess $T = 1$ gives	0.21909
designed schedule achieves	0.23877
achieved – target	4.2×10^{-15}
objective J : initial → final	$1.94 \times 10^{-4} \rightarrow 8.7 \times 10^{-30}$
optimiser evaluations	7

The designed schedule is interior (no step pinned at the bounds).

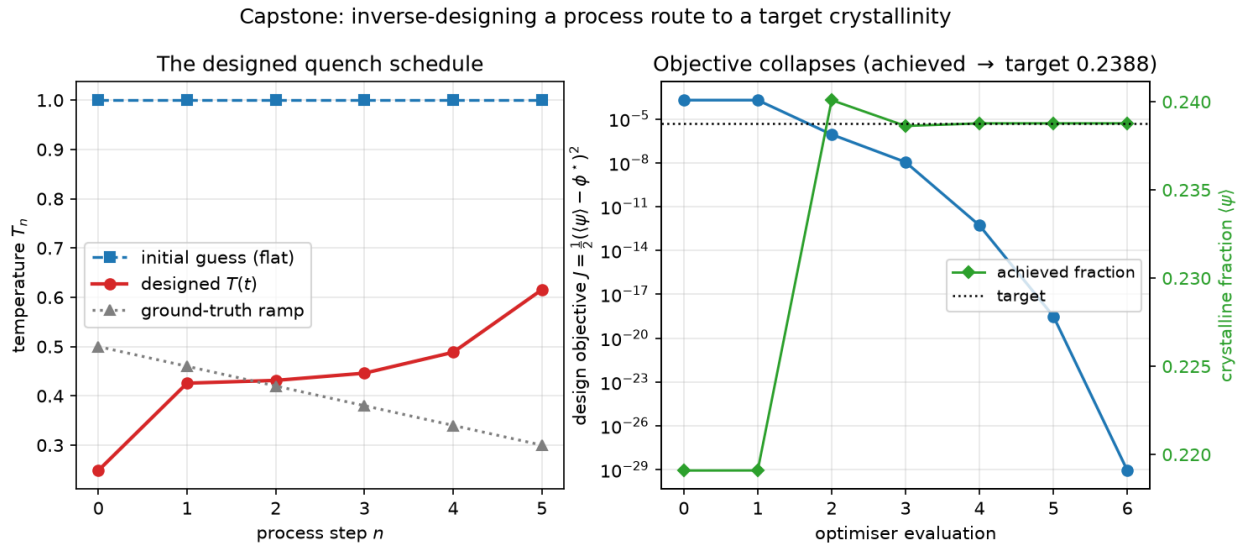


Figure 31.1: Inverse-designing the process. Left: the schedules — the flat initial guess, the designed $T(t)$, and the ground-truth ramp that set the target. Right: the design objective (31.1) collapses as the achieved crystalline fraction (green) climbs to the target (dotted). A handful of adjoint gradients suffice.

31.3 Code walk-through

The objective-and-gradient is the D4 schedule gradient wrapped in an optimiser:

```
def obj_and_grad(T, target):
    f = march(dm, coords, T, dt)
    m = mean(f.steps[-1]["x"][2::3])
    J = 0.5 * (m - target)**2
    dJ[-1][2::3] = (m - target) / nn
    return J, CACHAdjoint(f).temperature_gradient(dJ)

minimize(obj_and_grad, T0, jac=True, method="L-BFGS-B", bounds=[(0.2, 2.0)]*NS)
```

That is the whole capstone: a forward crystallisation, a reverse sweep for the schedule gradient, and `scipy` to descend. Every earlier concept — the verified gradient (D1), the sensitivities (D2, D4, D5), the inverse fits (D3, D6) — was a rehearsal for this.

31.4 Explore on your own

Q31.4.1 Many schedules hit the *same* scalar fraction. Add a second term to (31.1) — e.g. a target demixing amplitude or domain size — and watch the designed schedule become more constrained. Does it move toward the ground-truth ramp?

Q31.4.2 Tighten the bounds until the target becomes *unreachable*. How does the optimiser report failure, and what does the residual objective tell you about the closest achievable crystallinity?

Q31.4.3 Add a smoothness penalty $\lambda \sum_n (T_{n+1} - T_n)^2$ to (31.1) (real quenches cannot jump). How does the designed schedule change as λ grows, and what does it cost in achieved accuracy?

Q31.4.4 Swap the objective for a target on the *composition* field ϕ instead of the crystallinity ψ . Which process controls matter now, and does the coupled system still admit a schedule that hits it?

Bibliography

- [1] Samuel M. Allen and John W. Cahn. A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening. *Acta Metallurgica*, 27(6):1085–1095, 1979. doi: 10.1016/0001-6160(79)90196-2.
- [2] Melvin Avrami. Kinetics of phase change. I. general theory. *The Journal of Chemical Physics*, 7(12):1103–1112, 1939. doi: 10.1063/1.1750380.
- [3] John W. Cahn and John E. Hilliard. Free energy of a nonuniform system. I. interfacial free energy. *The Journal of Chemical Physics*, 28(2):258–267, 1958. doi: 10.1063/1.1744102.
- [4] Paul J. Flory. Thermodynamics of high polymer solutions. *The Journal of Chemical Physics*, 10(1):51–61, 1942. doi: 10.1063/1.1723621.
- [5] Kevin L. Gu, Yan Zhou, Xiaodan Gu, Hongping Yan, Ying Diao, Tadanori Kurosawa, Baskar Ganapathysubramanian, Michael F. Toney, and Zhenan Bao. Tuning domain size and crystallinity in isoindigo/PCBM organic solar cells via solution shearing. *Organic Electronics*, 40: 79–87, 2017. doi: 10.1016/j.orgel.2016.10.033.
- [6] Maurice L. Huggins. Solutions of long chain compounds. *The Journal of Chemical Physics*, 9(5):440, 1941. doi: 10.1063/1.1750930.
- [7] Vikas Negi, Olga Wodo, Jacobus J. van Franeker, René A. J. Janssen, and Peter A. Bobbert. Simulating phase separation during spin coating of a polymer–fullerene blend: A joint computational and experimental investigation. *ACS Applied Energy Materials*, 1(2):725–735, 2018. doi: 10.1021/acsaem.7b00189.
- [8] Olivier J. J. Ronsin and Jens Harting. Phase-field simulations of the morphology formation in evaporating crystalline multicomponent films. *arXiv preprint*, 2022. arXiv:2204.11628.
- [9] Maxime Siber, Olivier J. J. Ronsin, and Jens Harting. Crystalline morphology formation in phase-field simulations of binary mixtures. *Journal of Materials Chemistry C*, 11:14276–14294, 2023. doi: 10.1039/D3TC03047D. arXiv:2310.11844.
- [10] Maxime Siber, Olivier J. J. Ronsin, Gitti L. Frey, and Jens Harting. Interplay of crystallization and amorphous spinodal decomposition during thermal annealing of organic photoactive layers. *arXiv preprint*, 2025. arXiv:2512.16390.
- [11] Maxime Siber, Olivier J. J. Ronsin, and Jens Harting. Extending the Flory–Huggins theory for crystalline multicomponent mixtures. *Macromolecules*, 2025. doi: 10.1021/acs.macromol.5c02297. arXiv:2512.16370.

- [12] James A. Warren, Ryo Kobayashi, Alexander E. Lobkovsky, and W. Craig Carter. Extending phase field models of solidification to polycrystalline materials. *Acta Materialia*, 51(20):6035–6058, 2003. doi: 10.1016/S1359-6454(03)00388-4.
- [13] Olga Wodo and Baskar Ganapathysubramanian. Computationally efficient solution to the Cahn–Hilliard equation: Adaptive implicit time schemes, mesh sensitivity analysis and the 3D isoperimetric problem. *Journal of Computational Physics*, 230(15):6037–6060, 2011. doi: 10.1016/j.jcp.2011.04.012.
- [14] Olga Wodo and Baskar Ganapathysubramanian. Modeling morphology evolution during solvent-based fabrication of organic solar cells. *Computational Materials Science*, 55:113–126, 2012. doi: 10.1016/j.commatsci.2011.12.012.